RaschPy

Version 1.4.0

Mark Elliott

October 2026

Abstract

RaschPy (Elliott, 2026) is a Python package for Rasch analysis which can estimate parameters for a variety of Rasch models, generate a range of model fit statistics and output tables and graphical plots. *RaschPy* also contains simulation functionality. *RaschPy* is open source and free to download. Specifications are subject to change as the software is developed.

RaschPy is capable of estimating parameters and generating tables of estimates and fit statistics plus assorted plotting and simulation functionaility, for the following Rasch models:

- Simple Logistic Model (SLM), aka dichotomous Rasch model (Rasch, 1960, 1968)
- Partial Credit Model (PCM) (Masters, 1982)
- Rating Scale Model (RSM) (Andrich, 1978)
- Many-Facet Rasch Model (MFRM) (Linacre, 1994), RSM formulation
- Extended Many-Facet Rasch Models (Extended MFRM) (Elliott & Buttery, 2022b), RSM formulation

Parameter estimation uses conditional pairwise methods: PAIR (Choppin, 1968, 1985) and the Eigenvector Method (EVM) (Garner & Engelhard, 2002, 2009) for SLM and PCM, and the Conditional Pairwise Adjacent Thresholds (CPAT) method (Elliott & Buttery, 2022a) for the RSM and MFRM models. The priority vector is by default derived from the pairwise matrix by a Bradley–Terry maximum-likelihood fixed point (Bradley & Terry, 1952); closed-form alternatives – cosine maximisation (Kou & Lin, 2014), least squares, and EVM – are selectable via the `method` argument.

For each model, methods are grouped under the following sections:

- Preliminaries: Loading data and editing labels.
- Core functions: Functions for category probabilities, expected scores, variance and kurtosis.
- Parameter estimation: Estimating item, person and (where applicable) rater parameters and generating bootstrapped standard errors of estimation.
- Statistical output: Generating tables of estimates and fit statistics for items, persons and (where applicable) raters, plus overall test statistics, residual correlation analysis and category counts.
- Plotting functionality: Plots for a variety of item-level and test-level functions, showing theoretical curves with the option to overlay relevant features such as observed class category responses and threshold lines.

Contents

1 About RaschPy **10**

2 Loading data **12**

2.1 SLM . . . 12

2.1.1 loadup_slm . . . 12

2.2 PCM . . . 14

2.2.1 loadup_pcm . . . 14

2.3 RSM . . . 16

2.3.1 loadup_rsm . . . 16

2.4 MFRM . . . 18

2.4.1 loadup_mfrm_single . . . 18

2.4.2 loadup_mfrm_xlsx_tabs . . . 20

2.4.3 loadup_mfrm_multiple . . . 22

2.5 Exogenous variables (for DIF analysis) . . . 25

2.5.1 loadup_exogenous . . . 25

3 class SLM **26**

3.1 Preliminaries . . . 26

3.1.1 SLM . . . 26

3.1.2 check_data_connectivity . . . 29

3.1.3 rename_item . . . 31

3.1.4 rename_items_all . . . 31

3.1.5 rename_person . . . 32

3.1.6 rename_persons_all . . . 32

3.2 Core functions . . . 33

3.2.1 cat_prob . . . 33

3.2.2 exp_score . . . 34

3.2.3 variance . . . 35

3.2.4 kurtosis . . . 36

3.3 Parameter estimation . . . 37

3.3.1 calibrate . . . 37

3.3.2 std_errors . . . 40

3.3.3 calibrate_anchor . . . 42

3.3.4 person . . . 46

3.3.5 person_estimates . . . 49

3.3.6 score_lookup . . . 51

3.3.7 score_lookup_table . . . 53

3.3.8 csem . . . 55

3.4 Statistical output . . . 56

3.4.1 item_stats_df . . . 56

3.4.2 person_stats_df . . . 60

3.4.3 test_stats_df . . . 63

3.4.4 global_fit . . . 66
3.4.5 res_corr_analysis . . . 68
3.4.6 save_stats . . . 70
3.4.7 save_residuals . . . 71
3.4.8 category_counts_df . . . 72
3.4.9 dif_test . . . 74
3.4.10 recovery_stats . . . 79
3.5 Plotting functionality . . . 82
3.5.1 Shared plotting arguments . . . 82
3.5.2 icc . . . 83
3.5.3 crcs . . . 85
3.5.4 iic . . . 87
3.5.5 tcc . . . 89
3.5.6 test_info . . . 91
3.5.7 test_csem . . . 93
3.5.8 std_residuals_plot . . . 94
3.5.9 wright_map . . . 95
3.5.10 plot_anchor_selection . . . 99

4 class PCM **101**
4.1 Preliminaries . . . 101
4.1.1 PCM . . . 101
4.1.2 check_data_connectivity . . . 104
4.1.3 rename_item . . . 105
4.1.4 rename_items_all . . . 105
4.1.5 rename_person . . . 106
4.1.6 rename_persons_all . . . 106
4.2 Core functions . . . 107
4.2.1 cat_prob_centred . . . 107
4.2.2 cat_prob_uncentred . . . 108
4.2.3 exp_score_centred . . . 109
4.2.4 exp_score_uncentred . . . 110
4.2.5 variance_centred . . . 111
4.2.6 variance_uncentred . . . 112
4.2.7 kurtosis_centred . . . 113
4.2.8 kurtosis_uncentred . . . 114
4.3 Parameter estimation . . . 115
4.3.1 calibrate . . . 115
4.3.2 std_errors . . . 118
4.3.3 calibrate_anchor . . . 121
4.3.4 person . . . 126
4.3.5 person_estimates . . . 129
4.3.6 score_lookup . . . 131
4.3.7 score_lookup_table . . . 133

4.3.8 csem . . . 135
4.4 Statistical output . . . 136
4.4.1 item_stats_df . . . 136
4.4.2 threshold_stats_df . . . 140
4.4.3 category_stats_df . . . 143
4.4.4 person_stats_df . . . 145
4.4.5 test_stats_df . . . 149
4.4.6 model_selection . . . 152
4.4.7 global_fit . . . 154
4.4.8 res_corr_analysis . . . 156
4.4.9 save_stats . . . 159
4.4.10 save_residuals . . . 160
4.4.11 category_counts_df . . . 161
4.4.12 dif_test . . . 163
4.4.13 recovery_stats . . . 169
4.5 Plotting functionality . . . 171
4.5.1 Shared plotting arguments . . . 171
4.5.2 icc . . . 172
4.5.3 crcs . . . 174
4.5.4 threshold_ccs . . . 176
4.5.5 iic . . . 178
4.5.6 tcc . . . 180
4.5.7 test_info . . . 182
4.5.8 test_csem . . . 183
4.5.9 std_residuals_plot . . . 184
4.5.10 wright_map . . . 185
4.5.11 plot_anchor_selection . . . 191

5 class RSM **193**
5.1 Preliminaries . . . 193
5.1.1 RSM . . . 193
5.1.2 check_data_connectivity . . . 195
5.1.3 rename_item . . . 196
5.1.4 rename_items_all . . . 196
5.1.5 rename_person . . . 197
5.1.6 rename_persons_all . . . 197
5.2 Core functions . . . 198
5.2.1 cat_prob . . . 198
5.2.2 exp_score . . . 199
5.2.3 variance . . . 200
5.2.4 kurtosis . . . 201
5.3 Parameter estimation . . . 202
5.3.1 calibrate . . . 202
5.3.2 std_errors . . . 207

5.3.3 calibrate_anchor . . . 210
5.3.4 person . . . 215
5.3.5 person_estimates . . . 217
5.3.6 score_lookup . . . 219
5.3.7 score_lookup_table . . . 221
5.3.8 csem . . . 223
5.4 Statistical output . . . 224
5.4.1 item_stats_df . . . 224
5.4.2 threshold_stats_df . . . 228
5.4.3 category_stats_df . . . 231
5.4.4 person_stats_df . . . 233
5.4.5 test_stats_df . . . 236
5.4.6 model_selection . . . 239
5.4.7 global_fit . . . 241
5.4.8 res_corr_analysis . . . 243
5.4.9 save_stats . . . 246
5.4.10 save_residuals . . . 247
5.4.11 category_counts_df . . . 248
5.4.12 dif_test . . . 250
5.4.13 recovery_stats . . . 257
5.5 Plotting functionality . . . 259
5.5.1 Shared plotting arguments . . . 259
5.5.2 icc . . . 260
5.5.3 crcs . . . 262
5.5.4 threshold_ccs . . . 264
5.5.5 iic . . . 266
5.5.6 tcc . . . 268
5.5.7 test_info . . . 270
5.5.8 test_csem . . . 271
5.5.9 std_residuals_plot . . . 272
5.5.10 wright_map . . . 273
5.5.11 plot_anchor_selection . . . 279

6 class MFRM **281**
6.1 Preliminaries . . . 281
6.1.1 MFRM . . . 281
6.1.2 check_data_connectivity . . . 285
6.1.3 rename_item . . . 286
6.1.4 rename_items_all . . . 286
6.1.5 rename_person . . . 287
6.1.6 rename_persons_all . . . 287
6.1.7 rename_facet_element . . . 288
6.1.8 rename_facet_elements_all . . . 288
6.1.9 drop_facet_elements . . . 289

6.2 Core functions . . . 290
6.2.1 cat_prob . . . 290
6.2.2 exp_score . . . 296
6.2.3 variance . . . 300
6.2.4 kurtosis . . . 304
6.3 Parameter estimation . . . 308
6.3.1 calibrate . . . 308
6.3.2 calibrate_anchor . . . 321
6.3.3 std_errors . . . 328
6.3.4 anchor_std_errors . . . 336
6.3.5 person . . . 338
6.3.6 person_estimates . . . 340
6.3.7 score_lookup . . . 343
6.3.8 score_lookup_table . . . 345
6.3.9 csem . . . 349
6.3.10 model_selection . . . 351
6.3.11 per_rater_model_selection . . . 354
6.4 Statistical output . . . 358
6.4.1 item_stats_df . . . 358
6.4.2 threshold_stats_df . . . 364
6.4.3 category_stats_df . . . 369
6.4.4 rater_stats_df . . . 372
6.4.5 check_anchor_homogeneity . . . 378
6.4.6 person_stats_df . . . 381
6.4.7 test_stats_df . . . 385
6.4.8 item_res_corr_analysis . . . 388
6.4.9 rater_res_corr_analysis . . . 392
6.4.10 save_stats . . . 396
6.4.11 save_residuals_items / save_residuals_raters . . . 398
6.4.12 category_counts_df . . . 399
6.4.13 dif_test . . . 401
6.4.14 drf_test . . . 407
6.4.15 recovery_stats . . . 412
6.5 Plotting functionality . . . 415
6.5.1 Shared plotting arguments . . . 415
6.5.2 icc . . . 416
6.5.3 crcs . . . 418
6.5.4 threshold_ccs . . . 420
6.5.5 iic . . . 422
6.5.6 tcc . . . 424
6.5.7 test_info . . . 426
6.5.8 test_csem . . . 428
6.5.9 std_residuals_plot . . . 430

6.5.10 wright_map . . . 432

7 class SLM_Sim 439

7.1 Generating an SLM_Sim simulation . . . 439

7.1.1 SLM_Sim . . . 439

7.2 Customising an SLM_Sim simulation . . . 442

7.2.1 rename_item . . . 442

7.2.2 rename_items_all . . . 442

7.2.3 rename_person . . . 443

7.2.4 rename_persons_all . . . 443

8 class PCM_Sim 444

8.1 Generating a PCM_Sim simulation . . . 444

8.1.1 PCM_Sim . . . 444

8.2 Customising an PCM_Sim simulation . . . 448

8.2.1 rename_item . . . 448

8.2.2 rename_items_all . . . 448

8.2.3 rename_person . . . 449

8.2.4 rename_persons_all . . . 449

9 class RSM_Sim 450

9.1 Generating an RSM_Sim simulation . . . 450

9.1.1 RSM_Sim . . . 450

9.2 Customising an RSM_Sim simulation . . . 455

9.2.1 rename_item . . . 455

9.2.2 rename_items_all . . . 455

9.2.3 rename_person . . . 456

9.2.4 rename_persons_all . . . 456

10 class MFRM_Sim_Global 457

10.1 Generating an MFRM_Sim_Global simulation . . . 457

10.1.1 MFRM_Sim_Global . . . 457

10.2 Customising an MFRM_Sim_Global simulation . . . 462

10.2.1 rename_item . . . 462

10.2.2 rename_items_all . . . 462

10.2.3 rename_person . . . 463

10.2.4 rename_persons_all . . . 463

10.2.5 rename_rater . . . 464

10.2.6 rename_raters_all . . . 464

10.2.7 drop_facet_elements . . . 465

11 class MFRM_Sim_Items 466

11.1 Generating an MFRM_Sim_Items simulation . . . 466

11.1.1 MFRM_Sim_Items . . . 466

11.2 Customising an MFRM_Sim_Items simulation . . . 472

11.2.1 rename_item . . . 472
11.2.2 rename_items_all . . . 472
11.2.3 rename_person . . . 473
11.2.4 rename_persons_all . . . 473
11.2.5 rename_rater . . . 474
11.2.6 rename_raters_all . . . 474
11.2.7 drop_facet_elements . . . 475

12 class MFRM_Sim_Thresholds 476
12.1 Generating an MFRM_Sim_Thresholds simulation . . . 476
12.1.1 MFRM_Sim_Thresholds . . . 476
12.2 Customising an MFRM_Sim_Thresholds simulation . . . 482
12.2.1 rename_item . . . 482
12.2.2 rename_items_all . . . 482
12.2.3 rename_person . . . 483
12.2.4 rename_persons_all . . . 483
12.2.5 rename_rater . . . 484
12.2.6 rename_raters_all . . . 484
12.2.7 drop_facet_elements . . . 485

13 class MFRM_Sim_Matrix 486
13.1 Generating an MFRM_Sim_Matrix simulation . . . 486
13.1.1 MFRM_Sim_Matrix . . . 486
13.2 Customising an MFRM_Sim_Matrix simulation . . . 492
13.2.1 rename_item . . . 492
13.2.2 rename_items_all . . . 492
13.2.3 rename_person . . . 493
13.2.4 rename_persons_all . . . 493
13.2.5 rename_rater . . . 494
13.2.6 rename_raters_all . . . 494
13.2.7 drop_facet_elements . . . 495

14 class MFRM_Sim_Bivector 496
14.1 Generating an MFRM_Sim_Bivector simulation . . . 496
14.1.1 MFRM_Sim_Bivector . . . 496
14.2 Customising an MFRM_Sim_Bivector simulation . . . 502
14.2.1 rename_item . . . 502
14.2.2 rename_items_all . . . 502
14.2.3 rename_person . . . 503
14.2.4 rename_persons_all . . . 503
14.2.5 rename_rater . . . 504
14.2.6 rename_raters_all . . . 504
14.2.7 drop_facet_elements . . . 505

15 class MFRM_Sim_Centrality 506

15.1 Generating an MFRM_Sim_Centrality simulation . . . 506
15.1.1 MFRM_Sim_Centrality . . . 506
15.2 Customising an MFRM_Sim_Centrality simulation . . . 510
15.2.1 rename_item . . . 510
15.2.2 rename_items_all . . . 510
15.2.3 rename_person . . . 511
15.2.4 rename_persons_all . . . 511
15.2.5 rename_rater . . . 512
15.2.6 rename_raters_all . . . 512
15.2.7 drop_facet_elements . . . 513

16 class MFRM_Sim_PseudoHalo 514
16.1 Generating an MFRM_Sim_PseudoHalo simulation . . . 514
16.1.1 MFRM_Sim_PseudoHalo . . . 514
16.2 Customising an MFRM_Sim_PseudoHalo simulation . . . 518
16.2.1 rename_item . . . 518
16.2.2 rename_items_all . . . 518
16.2.3 rename_person . . . 519
16.2.4 rename_persons_all . . . 519
16.2.5 rename_rater . . . 520
16.2.6 rename_raters_all . . . 520
16.2.7 drop_facet_elements . . . 521

17 class MFRM_Sim_Bistretch 522
17.1 Generating an MFRM_Sim_Bistretch simulation . . . 522
17.1.1 MFRM_Sim_Bistretch . . . 522
17.2 Customising an MFRM_Sim_Bistretch simulation . . . 527
17.2.1 rename_item . . . 527
17.2.2 rename_items_all . . . 527
17.2.3 rename_person . . . 528
17.2.4 rename_persons_all . . . 528
17.2.5 rename_rater . . . 529
17.2.6 rename_raters_all . . . 529
17.2.7 drop_facet_elements . . . 531

References 532

1 About RaschPy

Built-in dependencies: `math`, `string`, `warnings`. Non-built-in dependencies: `numpy`, `pandas`, `matplotlib`, `scipy`, `sklearn`, `seaborn`, `openpyxl`. Not part of the core `anaconda` installation: `seaborn` and `openpyxl`.

Maintainer: Dr Mark Elliott `<markelliott@cantab.net>`

URL: `https://github.com/MarkElliott999/RaschPy`

Issues: `https://github.com/MarkElliott999/RaschPy/issues`

RaschPy is offered free under an Apache 2.0 licence, but please cite when used: Elliott, M. (2026). *RaschPy Rasch analysis in Python*. URL: `https://github.com/MarkElliott999/RaschPy`

Model and simulation class hierarchy

Table 1: Model and simulation classes in *RaschPy*. All four analysis classes inherit from a common `Rasch` base class; the simulation classes from `Rasch_Sim`.

Model class	**Description**	**Simulation class**
`SLM`	Simple Logistic Model (dichotomous)	`SLM_Sim`
`PCM`	Partial Credit Model (polytomous, item-varying thresholds)	`PCM_Sim`
`RSM`	Rating Scale Model (polytomous, shared thresholds)	`RSM_Sim`
`MFRM`	MFRM — global rater representation	`MFRM_Sim_Global`
	MFRM — by-item rater representation	`MFRM_Sim_Items`
	MFRM — by-threshold rater representation	`MFRM_Sim_Thresholds`
	MFRM — full facet-effect matrix representation	`MFRM_Sim_Matrix`
	MFRM — bivector representation	`MFRM_Sim_Bivector`
	MFRM — centrality representation	`MFRM_Sim_Centrality`
	MFRM — pseudo-halo representation	`MFRM_Sim_PseudoHalo`
	MFRM — bistretch representation	`MFRM_Sim_Bistretch`

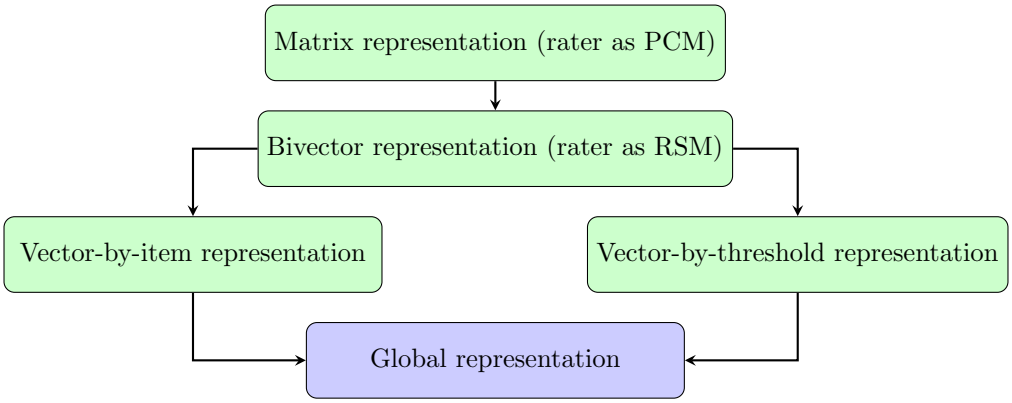


Figure 1: Hierarchy of extended many-facet Rasch models: an arrow flows to a restricted form of the more general form. Green boxes indicate extended models. From Elliott (2025).

Reproducibility

Every stochastic operation in *RaschPy* – simulation (all `*_Sim` classes), bootstrap standard errors (`std_errors`, `anchor_std_errors`), and subsampling for large-sample log-likelihood evaluation (`model_selection`, `per_rater_model_selection`) – accepts a `seed` argument. Each stochastic method draws its own independent generator via `numpy.random.default_rng(seed)`, rather than relying on `numpy`'s shared global random state. This means:

- `seed=None` (the default everywhere): fresh entropy is drawn each call, exactly as in earlier versions of *RaschPy* – no change to existing default behaviour.
- `seed=`*an integer*: the call becomes fully deterministic – repeating it with the same seed and the same data reproduces identical results, useful for regression testing, reviewer replication, and reporting a fixed simulation seed alongside published results.

Any method that auto-triggers one of the above internally – `fit_statistics`, `item_stats_df`, `test_stats_df`, `threshold_stats_df` and `rater_stats_df` (MFRM), `calibrate_anchor`, `dif_test`, and `check_anchor_homogeneity` (MFRM) – also accepts `seed`, which it forwards to whichever internal bootstrap/subsampling step it triggers. The seed is only consulted if that internal step runs, so if for example, standard errors have already been computed, `item_stats_df(seed=42)` has nothing left to seed). `calibrate` itself is deterministic (PAIR/CPAT involve no randomness) and takes no `seed` argument.

Example – a fully reproducible SLM simulation and bootstrap:

```
import raschpy as rp
sim = rp.SLM_Sim(no_of_items=10, no_of_persons=500, seed=42)
slm = rp.SLM(sim)
slm.calibrate()
slm.std_errors(no_of_samples=200, seed=42)
```

These five lines of code will always produce identical `slm.items` and `slm.item_se`.

2 Loading data

2.1 SLM

2.1.1 loadup_slm

Description

Function to load dichotomously scored data for use with class SLM.

Usage

```
loadup_slm(filename, item_names=True, person_names=True, long=False)
```

Arguments

filename	The full filename, including suffix .csv, .xlsx or .json.
item_names	Boolean. If True, the first row of data will be read as the item names. If False, item names will be allocated following the format Item_1 etc. Default value is True.
person_names	Boolean. If True, the first column of data will be read as the person names. If False, person names will be allocated following the format Person_1 etc. Default value is True.
long	Boolean. If value is True, data will be expected in long format; if value is False, data will be expected in wide format, as per toy example below. Default value is False.

Input file format

	Item_1	Item_2
Person_1	1	0
Person_2	0	1
Person_3	1	

Table 3: Wide format SLM data

Person	Item	Score
Person_1	Item_1	1
Person_1	Item_2	0
Person_2	Item_1	0
Person_2	Item_2	1
Person_3	Item_1	1
Person_3	Item_2	

Table 4: Long format SLM data

Input files can be in wide format, with `long=False` (the default) or long format, with `long=True`. It is not necessary to specify whether the file is a csv, xlsx or json file; this will be inferred from the suffix `.csv`, `.xlsx` or `.json`. Examples of the required formats can be found in Table 3, which shows a toy example with three persons and two items in wide format, and Table 4, which shows the same data in long format. Missing data should be left blank, as shown for Person_3 and Item_2. Person and item names are optional, but if they are omitted, arguments `person_names=False` and/or `item_names=False` must be passed.

Returns

- pandas dataframe of responses in required format for class `SLM`
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load a csv file in wide format with item and person labels:

```
my_slm_df, my_invalid_responses = loadup_slm('my_slm_data.csv')
```

To load a csv file in wide format with no item or person labels:

```
my_slm_df, my_invalid_responses = loadup_slm('my_slm_data.csv', item_names=False,
                                             person_names=False)
```

To load an xlsx file in long format:

```
my_slm_df, my_invalid_responses = loadup_slm('my_slm_data.xlsx', long=True)
```

2.2 PCM

2.2.1 loadup_pcm

Description

Function to load polytomously scored data for use with class PCM.

Usage

```
loadup_pcm(filename, max_score_vector=None, item_names=True, person_names=True,
           long=False)
```

Arguments

filename	The full filename, including suffix .csv, .xlsx or .json.
max_score_vector	A vector of the maximum score for each item, as a list, numpy array or pandas Series. If omitted or max_score_vector=None, the maximum scores will be inferred from the data (passing a vector is recommended, however).
item_names	Boolean. If True, the first row of data will be read as the item names. If False, item names will be allocated following the format Item_1 etc. Default value is True.
person_names	Boolean. If True, the first column of data will be read as the person names. If False, person names will be allocated following the format Person_1 etc. Default value is True.
long	Boolean. If value is True, data will be expected in long format; if value is False, data will be expected in wide format, as per toy example below. Default value is False.

Input file format

	Item_1	Item_2
Person_1	1	3
Person_2	2	2
Person_3	2	

Table 6: Wide format PCM data

Person	Item	Score
Person_1	Item_1	1
Person_1	Item_2	3
Person_2	Item_1	2
Person_2	Item_2	2
Person_3	Item_1	2
Person_3	Item_2	

Table 7: Long format PCM data

Input files can be in wide format, with `long=False` (the default) or long format, with `long=True`. It is not necessary to specify whether the file is a csv, xlsx or json file; this will be inferred from the suffix `.csv`, `.xlsx` or `.json`. Examples of the required formats can be found in Table 6, which shows a toy example with three persons and two items in wide format, and Table 7, which shows the same data in long format. Missing data should be left blank, as shown for Person_3 and Item_2. Person and item names are optional, but if they are omitted, arguments `person_names=False` and/or `item_names=False` must be passed.

Returns

- pandas dataframe of responses in required format for class `PCM`
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load a csv file in wide format for six items with item and person labels, passing a list of maximum scores:

```
my_pcm_df, my_invalid_responses = loadup_pcm('my_pcm_data.csv',
                                             max_score_vector=[2, 2, 3, 3, 5, 5])
```

To load a csv file in wide format with no item or person labels, passing a the variable name for a pre-existing vectors of maximum scores:

```
my_pcm_df, my_invalid_responses = loadup_pcm('my_pcm_data.csv',
                                             max_score_vector=my_max_score_vector,
                                             item_names=False, person_names=False)
```

To load an .xlsx file in long format, inferring the maximum scores from the data:

```
my_pcm_df, my_invalid_responses = loadup_pcm('my_pcm_data.xlsx', long=True)
```

2.3 RSM

2.3.1 loadup_rsm

Description

Function to load polytomously scored data for use with class RSM.

Usage

```
loadup_rsm(filename, max_score=None, item_names=True, person_names=True, long=False)
```

Arguments

filename	The full filename, including suffix .csv, .xlsx or .json.
max_score	The maximum score available. If omitted or max_score=None, the maximum scores will be inferred from the data (passing a maximum score is recommended, however).
item_names	Boolean. If True, the first row of data will be read as the item names. If False, item names will be allocated following the format Item_1 etc. Default value is True.
person_names	Boolean. If True, the first column of data will be read as the person names. If False, person names will be allocated following the format Person_1 etc. Default value is True.
long	Boolean. If value is True, data will be expected in long format; if value is False, data will be expected in wide format. Default value is False.

Input file format

	Item_1	Item_2
Person_1	1	3
Person_2	3	2
Person_3	2	

Table 9: Wide format RSM data

Person	Item	Score
Person_1	Item_1	1
Person_1	Item_2	3
Person_2	Item_1	3
Person_2	Item_2	2
Person_3	Item_1	2
Person_3	Item_2	

Table 10: Long format RSM data

Input files can be in wide format, with `long=False` – the default – or long format, with `long=True`. Data loaded in long format will be converted to wide format for analysis. It is not necessary to specify whether the file is a csv, xlsx or json file; this will be inferred from the suffix `.csv`, `.xlsx` or `.json`. Examples of the required formats can be found in Table 9, which shows a toy example with three persons and two items in wide format, and Table 10, which shows the same data in long format. Missing data should be left blank, as shown for Person_3 and Item_2. Person and item names are optional, but if they are omitted, arguments `person_names=False` and/or `item_names=False` must be passed.

Returns

- pandas dataframe of responses in required format for class `RSM`
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load a csv file in wide format with item and person labels, with a maximum score of 5:

```
my_rsm_df, my_invalid_responses = loadup_rsm('my_rsm_data.csv', max_score=5)
```

To load a csv file in wide format with no item or person labels, passing a maximum scores stored as a variable `my_max_score`:

```
my_rsm_df, my_invalid_responses = loadup_rsm('my_rsm_data.csv',
                                             max_score=my_max_score,
                                             item_names=False, person_names=False)
```

To load an .xlsx file in long format, inferring the maximum score from the data:

```
my_rsm_df, my_invalid_responses = loadup_rsm('my_rsm_data.xlsx', long=True)
```

2.4 MFRM

2.4.1 loadup_mfrm_single

Description

Function to load polytomously scored data for use with class MFRM from a single csv file, xlsx or json tab.

Usage

```
loadup_mfrm_single(filename, max_score=None, item_names=True, long=False)
```

Arguments

filename	The full filename, including suffix .csv, .xlsx or .json.
max_score	The maximum score available. If omitted or max_score=None, the maximum scores will be inferred from the data (passing a maximum score is recommended, however).
item_names	Boolean. If True, the first row of data will be read as the item names. If False, item names will be allocated following the format Item_1 etc. Default value is True.
long	Boolean. If value is True, data will be expected in long format; if value is False, data will be expected in wide format. Default value is False.

Input file format

		Item_1	Item_2
Rater_1	Person_1	1	3
Rater_1	Person_2	3	2
Rater_2	Person_1	2	2
Rater_2	Person_2	4	

Table 12: Wide format MFRM data, single sheet

Rater	Person	Item	Score
Rater_1	Person_1	Item_1	1
Rater_1	Person_1	Item_2	3
Rater_1	Person_2	Item_1	3
Rater_1	Person_2	Item_2	2
Rater_2	Person_1	Item_1	2
Rater_2	Person_1	Item_2	2
Rater_2	Person_2	Item_1	4
Rater_2	Person_2	Item_2	

Table 13: Long format MFRM data, single sheet

Input files can be in wide format, with long=False – the default – or long format, with long=True. Data loaded in long format will be converted to wide format for analysis. It is not necessary to specify whether the file is a csv, xlsx or json file; this will be inferred from the suffix .csv, .xlsx or .json. Examples of the required formats can be found in Table 12, which shows a toy example with two persons and two items

rated by two raters in wide format, and Table 13, which shows the same data in long format. Missing data should either be left blank, as shown for Person_2 and Item_2, rated by Rater_2, or the row may be omitted entirely if there are no observations (for any individual observation in the case of long form). Item names are optional, but if they are omitted, the argument `item_names=False` must be passed. Unlike for `SLM`, `PCM` and `RSM`, person names are mandatory for `MFRM` data.

Returns

- pandas dataframe of responses in required format for class `MFRM`
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load a csv file in wide format with item labels, with a maximum score of 5:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_single('my_mfrm_data.csv', max_score=5)
```

To load a json file in wide format with no item labels, passing a maximum scores stored as a variable `my_max_score` :

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_single('my_mfrm_data.json',
                                                      max_score=my_max_score,
                                                      item_names=False)
```

To load an xlsx file in long format, inferring the maximum score from the data:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_single('my_mfrm_data.xlsx', long=True)
```

2.4.2 loadup_mfrm_xlsx_tabs

Description

Function to load polytomously scored data for use with class MFRM from an xlsx file with multiple tabs: one tab for each rater.

Usage

```
loadup_mfrm_xlsx_tabs(filename, max_score, item_names=True, missing=None, long=False)
```

Arguments

filename	The full filename, including suffix .xlsx .
max_score	The maximum score available. If omitted or max_score=None, the maximum scores will be inferred from the data (passing a maximum score is recommended, however).
item_names	Boolean. If True, the first row of data will be read as the item names. If False, item names will be allocated following the format Item_1 etc. Default value is True.
missing	String, int, float, list or None: value(s) in the raw data to treat as missing (NaN) before coercion to scores, e.g. a code such as -99 or 'M'. Default is missing=None.
long	Boolean. If value is True, data will be expected in long format; if value is False, data will be expected in wide format. Default value is False.

Input file format

	Item_1	Item_2
Person_1	1	3
Person_3	5	4

Table 15: Wide format MFRM data, multiple xlsx tabs: Rater_1

	Item_1	Item_2
Person_2	3	4
Person_3	5	

Table 16: Wide format MFRM data, multiple xlsx tabs: Rater_2

Person	Item	Score
Person_1	Item_1	1
Person_1	Item_2	3
Person_3	Item_1	5
Person_3	Item_2	4

Table 17: Long format MFRM data, multiple xlsx tabs: Rater_1

Person	Item	Score
Person_2	Item_1	3
Person_2	Item_2	4
Person_3	Item_1	5

Table 18: Long format MFRM data, multiple xlsx tabs: Rater_2

Input files can be in wide format, with long=False – the default – or long format, with long=True. Data loaded in long format will be converted to wide format for analysis. Examples of the required formats can be found in Tables 15 and 16, which show a toy example with three persons and two items rated by two raters in wide format, and Tables 17 and 18, which show the same data in long format. The names of the tabs should match the raters, and will automatically be processed as the rater names (in particular, they should be unique). Missing data should either be left blank, as shown for Person_3 and Item_2, rated by Rater_2, or the row may be omitted entirely if there are no observations (or for any individual observation in the case of long form). Item names are optional, but if they are omitted, the argument item_names=False must be passed. Unlike for SLM, PCM and RSM, person names are mandatory for MFRM data.

Returns

- pandas dataframe of responses in required format for class MFRM
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load an xlsx file in wide format with item labels, with a maximum score of 5:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_xlsx_tabs('my_mfrm_data.xlsx', max_score=5)
```

To load an xlsx file in wide format with no item labels, passing a maximum scores stored as a variable my_max_score :

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_xlsx_tabs('my_mfrm_data.xlsx',
                                                         max_score=my_max_score,
                                                         item_names=False)
```

To load an xlsx file in long format, inferring the maximum score from the data:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_xlsx_tabs('my_mfrm_data.xlsx', long=True)
```

2.4.3 loadup_mfrm_multiple

Description

Function to load polytomously scored data for use with class MFRM from multiple csv, xlsx or json files, with one file for each rater.

Usage

```
loadup_mfrm_multiple(filename_dict, max_score, item_names=True, missing=None, long=False)
```

Arguments

filename_dict	A dictionary in the format {'Rater_1': filename_1, 'Rater_2': filename_2, ... } with the full filenames, including suffixes .csv or .xlsx. An equivalent pandas Series may also be passed, as my_series.to_dict().
max_score	The maximum score available. If omitted or max_score=None, the maximum scores will be inferred from the data (passing a maximum score is recommended, however).
item_names	Boolean. If True, the first row of data will be read as the item names. If False, item names will be allocated following the format Item_1 etc. Default value is True.
missing	String, int, float, list or None: value(s) in the raw data to treat as missing (NaN) before coercion to scores, e.g. a code such as -99 or 'M'. Default is missing=None.
long	Boolean. If value is True, data will be expected in long format; if value is False, data will be expected in wide format. Default value is False.

Input file format

	Item_1	Item_2
Person_1	1	3
Person_3	5	4

Table 20: Wide format MFRM data, multiple files: Rater_1

	Item_1	Item_2
Person_2	3	4
Person_3	5	

Table 21: Wide format MFRM data, multiple files: Rater_2

Person	Item	Score
Person_1	Item_1	1
Person_1	Item_2	3
Person_3	Item_1	5
Person_3	Item_2	4

Table 22: Long format MFRM data, multiple files: Rater_1

Person	Item	Score
Person_2	Item_1	3
Person_2	Item_2	4
Person_3	Item_1	5

Table 23: Long format MFRM data, multiple files: Rater_2

Input files can be in wide format, with `long=False` – the default – or long format, with `long=True`. Data loaded in long format will be converted to wide format for analysis. Examples of the required formats can be found in Tables 20 and 21, which show a toy example with three persons and two items rated by two raters in wide format, and Tables 22 and 23, which show the same data in long format. Missing data should either be left blank, as shown for Person_3 and Item_2, rated by Rater_2, or the row may be omitted entirely if there are no observations (or for any individual observation in the case of long form). Item names are optional, but if they are omitted, the argument `item_names=False` must be passed. Unlike for `SLM`, `PCM` and `RSM`, person names are mandatory for `MFRM` data.

Returns

- pandas dataframe of responses in required format for class `MFRM`
- pandas dataframe of invalid responses, where a person has not responded to any items; these responses are removed from the response dataframe.

Examples

To load data from two csv files in wide format with item labels, with a maximum score of 5:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_multiple({'Rater_1':'my_mfrm_data_1.csv',
                                                         'Rater_2':'my_mfrm_data_2.csv'},
                                                         max_score=5)
```

To load data from two csv files in wide format with no item labels, passing a maximum score stored as a variable `my_max_score` :

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_multiple({'Rater_1':'my_mfrm_data_1.csv',
                                                          'Rater_2':'my_mfrm_data_2.csv'},
                                                         max_score=my_max_score,
                                                         item_names=False)
```

To load data from a csv file and an xlsx file in long format, inferring the maximum score from the data:

```
my_mfrm_df, my_invalid_responses = loadup_mfrm_multiple({'Rater_1':'my_mfrm_data_1.csv',
                                                         'Rater_2':'my_mfrm_data_2.csv'},
                                                        long=True)
```

2.5 Exogenous variables (for DIF analysis)

2.5.1 loadup_exogenous

Description

Function to load person-level exogenous variables (e.g. gender, age or first language) for use in Differential Item Functioning (DIF) analysis. Unlike the response loaders for the main objects, values are read as raw category labels rather than numeric scores, since such variables are typically categorical (e.g. `'Female'`, `'21 '` or `'Mandarin'`). Loaded independently of the response file, rather than as extra columns within it, so that an exogenous variable can never accidentally be parsed as an item score.

Usage

```
loadup_exogenous(filename, person_names=True)
```

Arguments

`filename`	The full filename, including suffix `.csv`, `.xlsx` or `.json`. Format is inferred from the suffix; anything other than `.xlsx` or `.json` is read as csv.
`person_names`	Boolean. If `True`, the first column of data is read as the person names and used as the index. If `False`, persons are labelled `Person_1`, `Person_2` etc., in file row order. Default value is `True`.

Input file format

A person-indexed table of covariate columns, e.g. a first column of person names followed by columns such as `Gender`, `L1`. Persons need not appear in the same order as (or even be fully identical to) the response data — see the constructor documentation for `SLM` (Section 3.1.1) for how mismatches between the response data and the exogenous data are handled.

Returns

pandas dataframe: person-indexed covariate table. Pass directly as the `exogenous` argument to the `SLM`, `PCM` or `RSM` constructor.

Examples

To load a csv file of exogenous variables with person labels:

```
my_exogenous_df = loadup_exogenous('my_exogenous_data.csv')
```

To load an xlsx file of exogenous variables with no person labels:

```
my_exogenous_df = loadup_exogenous('my_exogenous_data.xlsx', person_names=False)
```

3 class SLM

3.1 Preliminaries

3.1.1 SLM

Description

Creates an object of the class SLM from a pandas dataframe of dichotomously scored data for analysis. No analysis can be run until an object is created.

Usage

```
SLM(dataframe, extreme_persons=True, no_of_classes=5, validate=True, exogenous=None)
```

Arguments

dataframe	pandas dataframe with items as columns (item names as column names) and persons as index (person names as row names).
extreme_persons	Boolean: if False, all persons with extreme scores (all responses correct or all responses incorrect) are removed from the response dataframe. Default is extreme_persons=True.
no_of_classes	Integer: the number of classes of persons grouped by location for overplotting observed responses on theoretical curves. Default is no_of_classes=5
validate	Boolean: if True, automatically calls check_data_connectivity (Section 3.1.2) on creation and issues a UserWarning if the item response network is disconnected, or if any item has a structurally unresolvable zero in the pairwise calibration matrix, or the directed comparison network is connected but not strongly connected (which prevents a finite method='log-lik' solution on an unpowered matrix). Default is validate=True.
exogenous	pandas dataframe or None: optional person-level covariates (e.g. gender, age or first language) for Differential Item Functioning (DIF) analysis (Section 3.4.9), indexed by person identifier – typically produced by loadup_exogenous (Section 2.5.1). Values are kept as raw category labels, not pre-dummy-coded, so multi-level covariates work without rework. Persons present in dataframe without a matching exogenous record, and vice versa, are permitted rather than raising an error – exogenous data (e.g. demographic covariates) is often optional to supply, for instance for GDPR reasons – and any such gaps are reported via a UserWarning and the self.no_exogenous_persons/self.exogenous_only_persons attributes below. Default is exogenous=None.

Returns

Object of class SLM. Analyses are run using methods defined on the SLM object, with results stored as

attributes of the SLM object.

Several attributes of object SLM are automatically generated on its creation:

self.responses	pandas dataframe: Dataframe of valid responses.
self.invalid_responses	pandas dataframe: Dataframe of invalid responses (persons with no responses to any items, i.e. all missing data).
self.extreme_persons	pandas dataframe: Persons with extreme scores (all-correct or all-incorrect). When True (the default), extreme scorers are retained in self.responses and this attribute is empty; when False, they are removed and stored here.
self.no_of_items	Integer: Number of items.
self.item_names self.item_ids	List: List of item names. self.item_ids is an alias for users who find *_ids more natural than *_names.
self.no_of_persons	Integer: Number of persons.
self.person_names self.person_ids	List: List of person names. self.person_ids is an alias for users who find *_ids more natural than *_names.
self.generating	Namespace object (_SimParams): Only present when the model is instantiated by passing a simulation object (an SLM_Sim object for class SLM) as the dataframe argument. Contains all attributes of the simulation object (e.g. self.generating.persons, self.generating.items), enabling direct comparison of generating and estimated parameters.
self.connectivity_status	Dictionary: result of check_data_connectivity (Section 3.1.2), present only when validate=True. Always contains at least 'connected' (Boolean), 'components_count' (Integer) and 'strongly_connected' (Boolean).
self.exogenous	pandas dataframe or None: person-level covariates passed via the exogenous argument, re-indexed onto self.person_names. None if not supplied.
self.no_exogenous_persons	pandas index: Persons present in the response data with no matching exogenous record. Empty if exogenous was not supplied or every person has a record.
self.exogenous_only_persons	pandas index: Persons present in the exogenous data but not in the response data (and so ignored). Empty if exogenous was not supplied or every exogenous person is also in the response data.

Example

To create an object from a dataframe my_slm_dataframe, with 10 observed classes:

```
my_slm = SLM(my_slm_dataframe, no_of_classes=10)
```

To create an object from a dataframe `my_slm_dataframe`, additionally carrying exogenous variables for DIF analysis loaded into `my_exogenous_df`:

```
my_slm = SLM(my_slm_dataframe, exogenous=my_exogenous_df)
```

3.1.2 check_data_connectivity

Description

Validates whether the item response network is fully connected, in the specific sense required for PAIR/CPAT pairwise calibration: two items are connected if at least one person's response differentiates between them (e.g. scores higher on one than the other) – not merely that some person answered both, since a person who scores identically on two items contributes nothing to their relative calibration. Builds the same directed pairwise comparison matrix that `calibrate` (Section 3.3.1) itself uses, then runs a breadth-first search over item-level adjacency to find connected components. A disconnected network means item locations are estimated independently per component and cannot be placed on a common scale.

Also flags a subtler 'directionally isolated' case: an item with a structurally zero row or column in the directed matrix (no 1s or no 0s for `SLM`) can still appear connected under the undirected view – it has an edge in at least one direction – but breaks `calibrate`'s directed matrix with a zero that matrix powering cannot resolve, which can silently produce `NaN`/overflow rather than a clear error.

Also verifies *strong* connectivity of the directed win-graph – the network in which an edge runs from item i to item j whenever at least one person scored higher on i than on j. The Bradley–Terry log-likelihood estimator (`calibrate(method='log-lik')`) has a finite, unique solution only when this directed graph is strongly connected (**Ford1957**). When the network is connected but not strongly connected – for example with near-Guttman data or a very short test – a `UserWarning` names the strongly connected components and notes that `method='log-lik'` on its default unpowered matrix will not converge for the affected items. The other methods (`'cos'`, `'ls'`, `'evm'`), and `'log-lik'` with `matrix_power` ≥ 1, are unaffected.

Called automatically on object creation when `validate=True` (the default; see Section 3.1.1), storing its result as `self.connectivity_status` and issuing a `UserWarning` for either condition above, but can also be re-run manually, e.g. after removing items or persons from `self.responses` by hand.

Usage

```
self.check_data_connectivity()
```

Returns

Dictionary. Always contains:

`'connected'`	Boolean: `True` if the item response network is fully connected.
`'components_count'`	Integer: number of connected components (1 if fully connected).
`'directionally_isolated_items'`	List: items with a structurally zero row or column in the directed pairwise matrix. These pass the undirected connectivity check but will break `calibrate`.
`'strongly_connected'`	Boolean: `True` if the directed win-graph is strongly connected – the condition for a finite `method='log-lik'` solution on an unpowered matrix (**Ford1957**).

If disconnected, additionally contains:

`'isolated_items'`	list: items forming singleton components.
`'all_sub_groups'`	List of lists: all components.

Example

To manually re-check connectivity on object `my_slm` after editing `self.responses` directly, and inspect the result:

```
my_slm.connectivity_status = my_slm.check_data_connectivity()
my_slm.connectivity_status['connected']
```

3.1.3 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of `self.responses` with new name.

Example
To rename an item in object `my_slm` from `Item_1` to `my_new_item_name`:

```
my_slm.rename_item('Item_1', 'my_new_item_name')
```

3.1.4 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of `self.responses` with new names.

Example
To rename all items in object `my_slm` with item names in a list stored as a variable `my_new_item_names`:

```
my_slm.rename_items_all(my_new_item_names)
```

3.1.5 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the index of self.responses with new name. Also replaces person name in self.exogenous or self.no_exogenous_persons as appropriate if self.exogenous exists.

Example
To rename a person in object my_slm from Person_1 to my_new_person_name:

```
my_slm.rename_person('Person_1', 'my_new_person_name')
```

3.1.6 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the index of self.responses with new names. Also replaces all person names in self.exogenous and self.no_exogenous_persons if self.exogenous exists.

Example
To rename all persons in object my_slm with person names in a list stored as a variable my_new_person_names:

```
my_slm.rename_persons_all(my_new_person_names)
```

3.2 Core functions

3.2.1 cat_prob

Description

Category probability function which calculates the probability $P(X_{ni} = k)$ of scoring k, with $k \in \{0, 1\}$ from person location and item location. For a person n with location β_n attempting an item i with location δ_i, the probability of obtaining a score of k is given by:

$$P(X_{ni} = k) = \frac{e^{k(\beta_n - \delta_i)}}{1 + e^{\beta_n - \delta_i}} \tag{1}$$

Usage

```
self.cat_prob(person_location, item_location, category)
```

Arguments

person_location	Float: Person location
item_location	Float: Item location
category	Integer: Response category k, with $k \in \{0, 1\}$.

Returns

Float: probability of obtaining score k.

Example

To obtain the probability of a person with location 0.5 scoring 0 on an item with location 0 and store the result as a variable my_cat_prob:

```
my_cat_prob = self.cat_prob(0.5, 0, 0)
```

3.2.2 exp_score

Description

Expected score function which calculates the expected score $E(X_{ni})$ from person location and item location. The expected score is given by:

$$E(X_{ni}) = \sum_{k=0}^{1} kP(X_{ni} = k) \tag{2}$$

where $P(X_{ni} = k)$ is as described in Section 3.2.1.

In the dichotomous case, this is also the probability of obtaining a correct response; for a person n with location β_n attempting an item i with location δ_i,the equation reduces to the most familiar formulation of the SLM:

$$E(X_{ni}) = \frac{e^{\beta_n - \delta_i}}{1 + e^{\beta_n - \delta_i}} \tag{3}$$

Usage

```
self.exp_score(person_location, item_location)
```

Arguments

person_location	Float: Person location
item_location	Float: Item location

Returns

Float: expected score (also the probability of obtaining a correct response).

Example

To obtain the expected score for a person of location 0.5 attempting an item of location 0 and store the result as a variable my_exp_score:

```
my_exp_score = self.exp_score(0.5, 0)
```

3.2.3 variance

Description

Variance function which calculates the variance of the score $V(X_{ni})$ from person location and item location. The variance is given by:

$$V(X_{ni}) = \sum_{k=0}^{1} P(X_{ni} = k)(k - E(X_{ni}))^2 \tag{4}$$

where $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 3.2.1 and 3.2.2 respectively.

In the dichotomous case, since it is a Bernoulli variable, the variance reduces to $p(1 - p)$, where p is the probability of obtaining a correct response; for a person n with location β_n attempting an item i with location δ_i, the variance is therefore given by:

$$V(X_{ni}) = \frac{e^{\beta_n - \delta_i}}{(1 + e^{\beta_n - \delta_i})^2} \tag{5}$$

The variance is also both the Fisher information for the response and the first partial differential of the expected score function with respect to person location.

Usage

```
self.variance(person_location, item_location)
```

Arguments

person_location	Float: Person location
item_location	Float: Item location

Returns

Float: variance (also the Fisher information provided by the response).

Example

To obtain the variance for a person of location 0.5 attempting an item of location 0 and store the result as a variable my_variance:

```
my_variance = self.variance(0.5, 0)
```

3.2.4 kurtosis

Description

Kurtosis function which calculates the kurtosis of the score $\kappa(X_{ni})$ from person location and item location. The kurtosis is given by:

$$\kappa(X_{ni}) = \sum_{k=0}^{1} P(X_{ni} = k)(k - E(X_{ni}))^4 \tag{6}$$

where $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 3.2.1 and 3.2.2 respectively.

Usage

```
self.kurtosis(person_location, item_location)
```

Arguments

person_location	Float: Person location
item_location	Float: Item location

Returns

Float: kurtosis

Example

To obtain the kurtosis for a person of location 0.5 attempting an item of location 0 and store the result as a variable my_kurtosis:

```
my_kurtosis = self.kurtosis(0.5, 0)
```

3.3 Parameter estimation

3.3.1 `calibrate`

Description

Produces item location estimates using PAIR (Choppin, 1968, 1985), eigenvector method (Garner & Engelhard, 2002) or related conditional pairwise methods. These methods are all based on the production of a conditional category response frequency matrix $\mathbf{C}$ of pairwise comparison counts between all possible pairs of items – $(1, 0)$ versus $(0, 1)$ – from which the final vector of item locations is derived; it is this final derivation step that differs between methods.

Three methods are entirely non-iterative and are derived from the positive pairwise reciprocal matrix $\mathbf{D} : \mathbf{D}_{ij} = \mathbf{C}_{ji}^n / \mathbf{C}_{ij}^n$, the ratios of the counts of the $\mathbf{C}_{ij}$ after raising $\mathbf{C}$ to a power n, which introduces indirect comparisons each time $\mathbf{C}$ is raised to a higher power. From the matrix $\mathbf{D}$, the final vector is derived in different ways:

- `'ls'` is the least-squares method (Choppin, 1968, 1985), which is the logarithm of the simple arithmetic row-wise mean of $\mathbf{D}$.
- `'evm'` is the eigenvector method (Garner & Engelhard, 2002), which uses the logarithm of the eigenvector corresponding to the principal eigenvalue of $\mathbf{D}$.
- `'cos'` is the cosine maximisation method (Kou & Lin, 2014), which first normalises the columns of $\mathbf{D}$ before taking the logarithms of the row-wise means.

In all these cases, the matrix power n is user-definable.

`'log-lik'` is the iterative maximum likelihood estimator (Bradley & Terry, 1952; Zermelo, 1929), which iteratively maximises the log-likelihood until the change is less than the set tolerance, using `'cos'` for a warm start. `'log-lik'` is the default method because although usually only trivially more accurate, it is highly robust to extremely sparse data, even at small sample sizes, provided the comparison graph is strongly connected (which a non-zero `constant` ensures; see below). `'cos'` and the other non-iterative methods can sometimes produce noticeably worse estimates under such conditions. For well conditioned data, `'cos'` in particular produces practically identical results at a faster speed. `'log-lik'` does not involve raising matrix powers, since it degrades accuracy for `'log-lik'` rather than improving it.

The additive smoothing constant `constant` is a form of regularisation, and also ensures connectivity (no zeros in the matrix): zeros or very low counts in comparisons resulting from sampling error can have an outsized influence on resulting estimates, leading to bias; the constant mitigates this effect for small sample sizes with diminishing impact on larger sample sizes. By default `constant` is `None`, which resolves to a data-adaptive value, $\min(1, 0.548\, I^{-0.2 \log_{10} N})$, where N is the number of persons with at least one response and I is the number of items. `robust` is deprecated and has no effect.

Usage

```
self.calibrate(constant=None, method='log-lik', matrix_power=None, log_lik_tol=0.000001,
               robust=False)
```

Arguments

`constant`	Float or `None`: additive smoothing constant (Elliott & Buttery, 2022b). Default is `constant=None`, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} N})$, where N is the number of persons with at least one response and I is the number of items.
`method`	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are `'ls'` for least squares (Choppin, 1968, 1985), `'evm'` for the eigenvector method(Garner & Engelhard, 2002), `'cos'` for cosine maximisation (Kou & Lin, 2014) or `'log-lik'` for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952; Zermelo, 1929). Default is `method='log-lik'`.
`matrix_power`	Integer or `None`: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) for central item location estimates is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is `matrix_power=None`, which resolves to `0` (no matrix powering) when `method='log-lik'` or `5` for other methods. Passing `matrix_power=0` with a method other than `'log-lik'` issues a `UserWarning`.
`log_lik_tol`	Float: convergence criterion for log-likelihood method. Ignored for other methods.
`robust`	Boolean: deprecated and has no effect; passing `robust=True` raises a `FutureWarning`. It will be removed in a future release. Default is `robust=False`.

Returns

Attribute `self.items`: pandas series of item estimates with item names as keys and estimates as values.
Attribute `self.constant`: the additive smoothing constant actually used, after resolving `None`.

Examples

To generate a set of estimates using the default Bradley–Terry log-likelihood method, with the default data-adaptive additive smoothing constant:

```
self.calibrate()
```

To generate a set of estimates using the log-likelihood method with the pairwise matrix raised to power 7 (overriding the unpowered default) and a tighter convergence criterion:

```
self.calibrate(method='log-lik', matrix_power=7, log_lik_tol=0.00000001)
```

To use the closed-form cosine maximisation method (Kou & Lin, 2014) instead of the default:

```
self.calibrate(method='cos')
```

3.3.2 std_errors

Description

Produces standard errors of item locations. `se_method='sandwich'` (default) computes analytic Huber–White sandwich standard errors (Huber, 1967), with persons as the independent units; `se_method='bootstrap'` draws `no_of_samples` bootstrap resamples of persons.

Usage

```
self.std_errors(interval=None, no_of_samples=500, constant=None, robust=False,
                method='log-lik', matrix_power=None, log_lik_tol=0.000001,
                seed=None, se_method='sandwich')
```

Arguments

`interval`	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval. Defines a central interval of proportion p, e.g. `interval=0.95` defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is `interval=None`.
`no_of_samples`	Integer: Number of bootstrap samples to generate. More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is `no_of_samples=500`.
`constant`	Float: additive smoothing constant (Elliott & Buttery, 2022b). Default is `constant=None`, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} N})$, where N is the number of persons with at least one response and I is the number of items.
`robust`	Boolean: deprecated and has no effect; passing `robust=True` raises a `FutureWarning`. It will be removed in a future release. Default is `robust=False`.
`method`	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are `'ls'` for least squares (Choppin, 1968, 1985), `'evm'` for the eigenvector method(Garner & Engelhard, 2002), `'cos'` for cosine maximisation (Kou & Lin, 2014) or `'log-lik'` for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is `method='log-lik'`

Continued on next page...

matrix_power	Integer or None: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method. Ignored for other methods.
seed	Integer or None: seed for the bootstrap resampling random number generator. Pass an integer for fully reproducible standard errors; None (default) draws fresh entropy each call. See Section 1.
se_method	String: 'sandwich' or 'bootstrap'. Default is se_method='sandwich'.

Returns

self.se_method	String: 'sandwich' or 'bootstrap', the method used.
self.se_covariance	pandas dataframe or None: sandwich covariance matrix of the item locations (sandwich only; None for the bootstrap).
self.item_se	pandas series: tem names as keys and item standard errors as values.
self.item_bootstrap	pandas dataframe: Contains the full bootstrap results, with a row for each bootstrap sample and a column for each item estimate.

If interval is specified, also returns:

self.item_low	Float: The lower bound of the specified interval.
self.item_high	Float: The upper bound of the specified interval.

Example

To generate item standard errors with a 95% interval from 200 samples:

```
self.std_errors(interval=0.95, no_of_samples=200)
```

Modifications to the estimation method are discussed in Section 3.3.1.

3.3.3 calibrate_anchor

Description

Anchors item location estimates onto externally-supplied reference values, supporting item banking: after running a standard calibration (Section 3.3.1), the whole scale is shifted by a translation constant so that a subset of common ("anchor") items line up with externally-supplied reference locations, e.g. from a bank of previously-calibrated items. Since all items have parallel item response functions under the SLM, this is a simple mean-shift.

By default (`selection_method='robust_z'`), the translation constant is computed via a robust, iterative outlier-trimming procedure based on the modified z-score (Iglewicz & Hoaglin, 1993): anchor items whose estimated values have drifted relative to the other anchor items when compared to the externally-supplied reference values are excluded before the shift is computed, so that a handful of stale or otherwise misbehaving anchor items cannot distort the shift applied to the rest of the scale. `selection_method='wald'` is an alternative that uses a formal significance test per anchor item instead of a descriptive-statistics trim, and requires bootstrap item standard errors (auto-triggers `std_errors`, Section 3.3.2, if not already computed). `selection_method='none'` takes a plain mean shift over all supplied anchors, with no trimming. Diagnostics from either selection procedure can be visualised with `plot_anchor_selection` (Section 3.5.10).

Usage

```
self.calibrate_anchor(anchors, calibrate=False, selection_method='robust_z',
                      corr_tol=0.95, sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05,
                      no_of_samples=500, adj=None, overwrite_anchors='none',
                      plot=True, plot_kwargs=None, constant=None, robust=False, method='log-lik',
                      matrix_power=None, log_lik_tol=0.000001, seed=None)
```

Arguments

`anchors`	Dictionary or pandas series: reference locations for a subset of items, keyed/indexed by item name.
`calibrate`	Boolean: if `True`, (re-)runs `calibrate` before anchoring. If `False`, `calibrate` is still auto-triggered if it has not already been run. Default is `calibrate=False`.
`selection_method`	String, one of `'robust_z'`, `'wald'` or `'none'`. `'robust_z'` (default): excludes outlying anchor items via the modified z-score procedure (Iglewicz & Hoaglin, 1993); uses `corr_tol`, `sd_ratio_tol`, `min_anchors`. `'wald'`: a formal per-item sequential significance test based on z-scores computed from item standard errors – needs bootstrap standard errors, so auto-triggers `std_errors` if not already computed; uses `wald_alpha` and `min_anchors`, not `corr_tol`/`sd_ratio_tol`. `'none'`: plain mean shift over every supplied anchor, no trimming.

Continued on next page...

corr_tol	Float: minimum acceptable Pearson correlation between anchor and observed values on the surviving (non-outlier) item set (selection_method='robust_z' only). Default is corr_tol=0.95.
sd_ratio_tol	Float: the ratio of observed to anchor standard deviations on the surviving item set must fall within $[1/\texttt{sd_ratio_tol}, \texttt{sd_ratio_tol}]$ (selection_method='robust_z' only). Default is sd_ratio_tol=1.1, i.e. the ratio must lie in $[0.909, 1.1]$.
min_anchors	Integer: floor on the number of surviving anchor items – selection stops (with a warning) once this floor is reached, even if the active method's tolerance/significance criterion is not yet satisfied, and is skipped altogether if the initial common item set is at or below this floor (selection_method in 'robust_z'/'wald' only). Default is min_anchors=6.
wald_alpha	Float: significance level for each anchor item's Wald test (selection_method='wald' only). Default is wald_alpha=0.05.
no_of_samples	Integer: bootstrap samples for the auto-triggered std_errors call, only used if item standard errors are not already computed (selection_method='wald' only). Default is no_of_samples=500.
adj	Float or None: if provided, applied as translation constant and selection step skipped. Intended for reuse across bootstrap resamples (Section 3.3.2), where recomputing anchor selection every time would inflate standard errors with anchor-item sampling variance. Default adj=None.
overwrite_anchors	String, one of 'none', 'rejected' or 'all': controls what self.anchor_items holds for the anchor items themselves, as opposed to non-anchor items, which always receive the shifted (calibrated + translation constant) value. 'none': every anchor item keeps exactly its externally-supplied value, unchanged – the usual convention, since an anchor's value is fixed by definition, not re-estimated. 'rejected': retained anchors keep their supplied values, but rejected anchors have their values overwritten with new, freshly-calibrated values; requires selection_method in 'robust_z'/'wald' and no adj override (otherwise falls back to 'none' with a warning). 'all': every anchor item has its value overwritten with a new, freshly-calibrated value like any other item. Default is overwrite_anchors='none'.
plot	Boolean: if True and selection_method in 'robust_z'/'wald' (so a selection table exists), calls plot_anchor_selection (Section 3.5.10). Has no effect when selection_method='none' or adj is supplied. Default plot=True.

Continued on next page...

plot_kwargs	Dictionary or None: extra keyword arguments forwarded to plot_anchor_selection when plot=True (e.g. filename, x_min/x_max, graph_title). Default is plot_kwargs=None.
constant, method, matrix_power, log_lik_tol	Calibration/bootstrap arguments, used only if calibrate or std_errors (selection_method='wald') triggered (Section 3.3.1).
seed	Integer or None: seed passed through to the internal std_errors call (only used if item standard errors are not already computed, selection_method='wald' only). None (default) draws fresh entropy each call. See Section 1.

Returns

self.anchor_items	pandas series: item locations shifted onto the anchor scale.
self.anchor_item_names	pandas index: names of the items supplied as anchors.
self.anchor_adj	Float: the translation constant actually applied.
self.anchor_selection	pandas dataframe or None: per-item diagnostics from whichever selection method was used – Anchor, Observed, Deviation, Robust z, Selected ('robust_z'), or Anchor, Observed, SE, Deviation, z, p, Selected ('wald'). None if selection_method='none' or adj supplied.
self.anchor_selected_items, self.anchor_dropped_items	pandas index or None: items retained / excluded as outliers. None if selection_method='none' or adj supplied.
self.anchor_original_corr, self.anchor_original_sd_ratio	Float or None: correlation / SD ratio before trimming.
self.anchor_corr self.anchor_sd_ratio	Float or None: correlation / SD ratio after trimming.
self.anchor_summary	pandas series: one-line summary – anchors supplied/common/selected/dropped, correlation and SD ratio before/after trimming, translation constant applied.
self.anchor_plot	matplotlib figure or None: figure from the auto-triggered plot_anchor_selection call. None if plot=False, selection_method='none', or adj was supplied directly.
self.calibrate_anchor_runs	Dictionary: every call's results above, keyed by the sorted anchor items supplied, so results from an earlier call survive a later call with a different anchor set instead of being overwritten in place.

Examples

To anchor item locations onto a bank of reference values stored in a dictionary `my_bank`, with automatic outlier trimming and diagnostic plot:

```
self.calibrate_anchor(my_bank)
```

To anchor without outlier trimming, taking a plain mean shift over all common items, with no plot:

```
self.calibrate_anchor(my_bank, selection_method='none', plot=False)
```

To anchor using the Wald significance test for trimming, with a fully reproducible bootstrap:

```
self.calibrate_anchor(my_bank, selection_method='wald', wald_alpha=0.05, seed=42)
```

To anchor with a stricter correlation tolerance and smaller floor on the number of retained anchors, overwriting rejected anchors' values with freshly-calibrated estimates:

```
self.calibrate_anchor(my_bank, corr_tol=0.98, min_anchors=4, overwrite_anchors='rejected')
```

3.3.4 person

Description

Generates a location estimate for a person using the Newton-Raphson method to produce a maximum likelihood estimate, or, by default, Warm's weighted likelihood estimate (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.person(persons, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
            ext_score_adjustment=0.5, missing_as_incorrect=False)
```

Arguments

persons	String or list: person name(s) for the location(s) being estimated. Pass 'all' to estimate for all persons in self.responses.
items	List: List of names of a subset of items, from which to generate estimate. Default items=None, which generates a location based on full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations. Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the person has an extreme score (all items correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
missing_as_incorrect	Boolean: if True, treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is missing_as_incorrect=False.

Returns

pandas series: person location estimate(s), in logits, indexed by person identifier. Returns NaN for any person

for whom estimation fails.

Examples

To generate a person location estimate for `Person_1` using the default settings and store the result as a variable, `my_person_location`:

```
my_person_location = self.person('Person_1')
```

To generate an MLE person location estimate without Warm bias correction for `Person_1` based on the first three items and store the result as a variable, `my_person_location`:

```
my_person_location = self.person('Person_1', ['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

3.3.5 person_estimates

Description

Generates location estimates for all persons using the Newton-Raphson method to produce maximum likelihood estimates, or, by default, Warm's weighted likelihood estimates (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.person_estimates(items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
                      ext_score_adjustment=0.5, missing_as_incorrect=False)
```

Arguments

items	List: List of names of a subset of items, from which to generate estimate. Default items=None, which generates a location based on full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations. Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
missing_as_incorrect	Boolean: if True, treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is missing_as_incorrect=False.

Returns

Attribute self.persons: pandas series with person names as keys and location estimates as values.

Example

To generate a set of person location estimates with Warm bias correction:

```
self.person_estimates()
```

To generate a set of person location estimates without Warm bias correction, on a subset of the first three items only:

```
self.person_estimates(items=['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

3.3.6 score_lookup

Description

Generates location estimate for a given raw score on a given set of items using the Newton-Raphson method to produce a maximum likelihood estimate, or, by default, Warm's weighted likelihood estimate (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.score_lookup(score, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
                  ext_score_adjustment=0.5)
```

Arguments

score	Integer: The raw score for which location is being estimated.
items	List: List of names of a subset of items, from which to generate estimate. Default items=None, which generates a location based on full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations. Default max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.

Returns

pandas series with raw scores as keys and person location estimates as values.

Examples

To generate a location estimate for a score of 10 on all items, with Warm bias correction, and store the result as a variable, my_score_location:

```
my_score_location = self.score_lookup(10)
```

To generate a location estimate for a score of 10 on a subset of items saved as a variable `my_items`, without Warm bias correction, and store the result as a variable, `my_score_location`:

```
my_score_location = self.score_lookup(10, items=my_items, warm_corr=False)
```

3.3.7 score_lookup_table

Description

Generates a score-to-location estimate correspondence for all raw scores on a set of items, using the Newton-Raphson method to produce maximum likelihood estimates, or, by default, Warm's weighted likelihood estimates (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.score_lookup_table(items=None, ext_scores=True, warm_corr=True, tolerance=0.00001,
                        max_iters=100, ext_score_adjustment=0.5,
                        filename=None, file_format='csv')
```

Arguments

items	List: List of names of a subset of items, from which to generate estimate. Default items=None, which generates a location based on full set of items. Only use when an estimate based on a subset of items is required.
ext_scores	Boolean: If True, location estimates for extreme scores (all correct/all incorrect) will be generated using the ext_score_adjustment argument. Default is ext_scores=True.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations. Default max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute `self.score_table`: pandas series with raw scores as keys and location estimates as values.

Examples

To generate a location lookup table for all items, including extreme scores, with Warm bias correction:

```
self.score_lookup_table()
```

To generate a location lookup table for a subset of items saved as a variable `my_items)`, without extreme scores and without Warm bias correction:

```
self.score_lookup_table(items=my_items, ext_scores=False)
```

3.3.8 csem

Description

Calculates conditional standard error of measurement for one or more persons.

Usage

```
self.csem(persons=None, person_locations=None, items=None)
```

Arguments

persons	List or string: person name(s). Overrides person_locations if provided. Default is persons=None, which uses all persons.
person_locations	pandas series, float, list, numpy array, or dictionary, with person names as keys (where applicable) and locations as values. If None, uses self.persons, automatically generating via self.person_estimates() if necessary. A raw location value (or values not indexed by a real person, e.g. a raw score lookup) is treated as hypothetical: since there is no observed response data for it, all items in items are treated as answered. Default is person_locations=None.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only for when an estimate based on a subset of items is required.

Returns

pandas series: conditional standard error of measurement in logits for each person (or location), indexed by person name (or by a location label, for hypothetical locations).

Examples

To generate the CSEM for all persons on all items and save the result as a variable, my_csem:

```
my_csem = self.csem()
```

To generate the CSEM for Person_1 on all items and save the result as a variable, my_csem:

```
my_csem = self.csem(persons='Person_1')
```

To generate the CSEM for a raw score of 3 on a subset of items saved to a variable my_items and save the result as a variable, my_csem:

```
my_csem = self.csem(persons=3, person_locations=self.score_table, items=my_items)
```

where self.score_table is the output from running self.score_lookup_table(items=my_items), as described in Section 3.3.7.

3.4 Statistical output

3.4.1 item_stats_df

Description

Produces a table of item statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

```
self.item_stats_df(full=False, zstd=False, disc=False, point_measure_corr=False, dp=3,
                   se=True, warm_corr=True, tolerance=1e-05, max_iters=100,
                   ext_score_adjustment=0.5, method='log-lik', constant=None,
                   robust=False, matrix_power=None, log_lik_tol=0.000001,
                   no_of_samples=500, interval=None, seed=None, filename=None,
                   file_format='csv')
```

Arguments

full	Boolean: If True, a full table with all statistics is produced. Default is full=False for a table with the most commonly reported statistics.
zstd	Boolean: If True, infit/outfit standardised z-scores are reported with mean square statistics. Default is zstd=False. Automatically True if full=True.
disc	Boolean: If True, item discrimination is reported. The discrimination of the empirical item slope relative to the ideal logistic ogive, with 1 perfect, > 1 showing overfit and < 1 showing underfit; discrimination is similar to the 2PL IRT discrimination parameter (Linacre, 2023), but is a descriptive statistic in the SLM rather than an item parameter.
point_measure_corr	Boolean: If True, point-biserial correlation between person location and item score is reported, with its expected value for an ideal item. Default is point_measure_corr=False. Automatically True if full=True.
dp	Integer: Number of decimal places reported in output.
se	Boolean: If True, computes and reports the SE column (and CI if interval=True). If False, skips the bootstrap (faster when only Infit MS/Outfit MS needed, e.g. repeated simulations, since those do not depend on the bootstrap). Forces interval=None when False. Default se=True.
warm_corr	Boolean: If True, Warm's (1989) bias correction is applied during generation of person estimates (see Section 3.3.4). Default is warm_corr=True.
tolerance	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is tolerance=0.00001.

Continued on next page...

max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 3.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 3.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 3.3.1). Default is constant=None, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} N})$, where N is the number of persons with at least one response and I is the number of items.
robust	Boolean: deprecated and has no effect; passing robust=True raises a FutureWarning. It will be removed in a future release. Default is robust=False.
matrix_power	Integer or None: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of item location estimates. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 3.3.1). Ignored for other methods.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 3.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Unused if se=False. Default is no_of_samples=500.

Continued on next page...

interval	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 3.3.2). Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Ignored if se=False. Default is interval=None.
seed	Integer or None: seed passed through to the internal std_errors/fit_statistics calls (only used if not already computed). None (default) draws fresh entropy each call. See Section 1.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.item_stats, a pandas dataframe with one row for each item and the following columns:

Estimate	Item location estimate.
SE	Bootstrapped standard error of item estimate. Only produced if se=True.
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 2.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 5%. Only produced if se=True and interval is set.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 97.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 95%. Only produced if se=True and interval is set.
Count	Count of responses.
Facility	Item facility: proportion of correct responses.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if zstd=True.
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if zstd=True.
Discrim	Item discrimination. Only produced if disc=True.
PM corr	Point-biserial correlation. Only produced if point_measure_corr=True.
Exp PM corr	Expected value of point-measure biserial correlation. Only produced if point_measure_corr=True.

Examples

To produce a summary `self.item_stats` table with the most commonly reported statistics:

```
self.item_stats_df()
```

To produce a full `self.item_stats` table with all statistics:

```
self.item_stats_df(full=True)
```

To produce a `self.item_stats` table with with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
self.item_stats_df(zstd=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

3.4.2 person_stats_df

Description

Produces a table of person statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

```
self.person_stats_df(full=False, rsem=False, dp=3, se=True, warm_corr=True,
                     tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                     method='log-lik', constant=None, robust=False,
                     filename=None, file_format='csv')
```

Arguments

full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a summary table with the most commonly reported statistics.
rsem	Boolean: If True, realistic standard error of measurement (RSEM), which takes into account item fit (Wright, 1996), is reported alongside the conditional standard error of measurement (CSEM). Default is rsem=False. Automatically True if full=True.
dp	Integer: Number of decimal places reported in output.
se	Boolean: If True, computes and reports the SE column (and CI if interval=True). If False, skips the bootstrap (faster when only Infit MS/Outfit MS needed, e.g. repeated simulations, since those do not depend on the bootstrap). Forces interval=None when False. Default se=True.
warm_corr	Boolean: If True, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 3.3.4). Default is warm_corr=True.
tolerance	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is max_iters=100.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 3.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 3.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 3.3.1). Default is constant=None, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} N})$, where N is the number of persons with at least one response and I is the number of items.
robust	Boolean: deprecated and has no effect; passing robust=True raises a FutureWarning. It will be removed in a future release. Default is robust=False.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 3.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is no_of_samples=500.
interval	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 3.3.2). Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is interval=None.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute `self.person_stats`, a pandas dataframe with one row for each person and the following columns:

`Estimate`	Item location estimate.
`CSEM`	Conditional standard error of measurement for person location estimate.
`RSEM`	Realistic standard error of measurement for person location estimate. Only produced if `rsem=True`
`Score`	Number of correct responses.
`Max score`	Maximum available score (number of items attempted).
`p`	Proportion of correct repsonses.
`Infit MS`	Infit mean square.
`Infit Z`	Standardised infit z-score.
`Outfit MS`	Outfit mean square.
`Outfit Z`	Standardised outfit z-score.

Examples

To produce a summary `self.person_stats` table with the most commonly reported statistics:
`self.person_stats_df()`

To produce a full `self.person_stats` table with all statistics:
`self.person_stats_df(full=True)`

Other arguments may be used to alter parameters of item location and/or person location estimation.

3.4.3 test_stats_df

Description

Produces a table of test-level statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX:

`Separation ratio` (Wright, 1996; Wright & Masters, 1982) is the SD of person locations reported as a ratio of SE units. For persons, $G_p = \sigma_p / \sqrt{\sum_n SE_n^2}$, where σ_p is the variance of the person estimates and SE_n is the RSEM (see Section 6.4.6) for person n. The formula is symmetrical for items, substituting the SE of estimation for RSEM.

`Strata` is the number of statistically distinct levels of either person or item locations as strata with centers three measurement errors apart (Wright & Masters, 1982:106). For persons, $H_p = (4G_p + 1)/3$, with symmetrical results for items.

`Reliability` is Rasch-specific reliability statistic (Wright, 1996), derived from PSI. reliability is similar to Cronbach's Alpha (Cronbach, 1951), and may be interpreted the same way: the proportion of variance of the estimates which stems from variation in location or location rather than estimation error. For persons, $R_p = G_p^2/(1 + G_p^2)$, with symmetrical results for items.

`Cronbach alpha` (Cronbach, 1951) (optional) is the most common classical test theory reliability statistic, computed directly from raw response data rather than derived from Rasch estimates.

Usage

```
self.test_stats_df(dp=3, warm_corr=True, tolerance=0.00001, max_iters=100,
                   ext_score_adjustment=0.5, method='log-lik', constant=None, robust=False,
                   alpha=False,seed=None, filename=None, file_format='csv')
```

Arguments

`dp`	Integer: Number of decimal places reported in output.
`warm_corr`	Boolean: If `True`, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 3.3.4). Default is `warm_corr=True`.
`tolerance`	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is `tolerance=0.00001`.
`max_iters`	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is `max_iters=100`.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 3.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 3.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 3.3.1). Default is constant=None, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} N})$, where N is the number of persons with at least one response and I is the number of items.
robust	Boolean: deprecated and has no effect; passing robust=True raises a FutureWarning. It will be removed in a future release. Default is robust=False.
alpha	Boolean: if True, adds a Cronbach alpha row to the Persons column of the returned table (the Items column is left blank, since Cronbach's (1951) alpha is a person-side reliability statistic). Computed on complete cases; with missing responses, a warning is raised and alpha computed after listwise deletion (may underestimate true value). Default alpha=False.
seed	Integer or None: seed passed through to the internal fit_statistics call (only used if not already computed). None (default) draws fresh entropy each call. See Section 1.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.test_stats, a pandas dataframe with two columns, Items and Persons and rows for a range of descriptive statistics describing the distributions of the estimates and different statistics related to reliability – these statistics describe the suitability of the data for estimating and differentiating the parameters, rather than properties of the parameters themselves. The statistics are:

`Mean`	Mean of estimates.
`SD`	Standard deviation of estimates.
`Separation ratio`	Separation ratio
`Strata`	Number of strata
`Reliability`	Reliability
`Cronbach alpha`	Cronbach's alpha

Example

To produce a `self.test_stats` table:

```
self.test_stats_df()
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

3.4.4 global_fit

Description

Wald test of invariance of item locations between two groups of persons calibrated separately, using Huber-White Sandwich covariances (Huber, 1967). `method='andersen-lr'` is not yet available.

Usage

```
self.global_fit(method='wald', split='score', covariate=None, welch=True, alpha=0.05,
                    seed=None, split_prop=0.5, se_correction='kauermann-carroll')
```

Arguments

`method`	String: `'wald'` or `'andersen-lr'` (raises `NotImplementedError`). Default is `method='wald'`.
`split`	String: `'score'` (median mean score), `'person_location'`, `'random'` or `'exogenous'`. Default is `split='score'`.
`covariate`	String or `None`: column of `self.exogenous` for `split='exogenous'`. Default is `covariate=None`.
`split_prop`	Float: proportion of persons in each outer group for `'score'` or `'person_location'` splits (0.5 = median split). Default is `split_prop=0.5`.
`welch`	Boolean: refer W/df to an F distribution with denominator degrees of freedom from the smaller group; otherwise W to χ^2. Default is `welch=True`.
`alpha`	Float: significance level. Default is `alpha=0.05`.
`seed`	Integer or `None`: seed for `split='random'`. Default is `seed=None`.
`se_correction`	String or `None`: small-sample correction of the sandwich covariances, `'kauermann-carroll'` (Kauermann & Carroll, 2001) or `None`. Default is `se_correction='kauermann-carroll'`.

Returns

Attributes:

`self.global_fit_summary`	pandas series: statistic, degrees of freedom, p-value and decision.
`self.global_fit_table`	pandas dataframe or `None`: per-parameter difference, SE and z (Wald only).
`self.global_fit_excluded`	List: parameters not compared (not estimable in both groups).
`self.global_fit_groups`	Dictionary: {`group name: calibrated model`}.

Examples

To test invariance between low and high scorers (the default median score split):

```
self.global_fit()
```

To test invariance between two groups defined by an exogenous variable `'Gender'`, stored in `self.exogenous`:

```
self.global_fit(split='exogenous', covariate='Gender')
```

3.4.5 res_corr_analysis

Description

Analysis of correlations of residuals to tests for violations of local item interdependence and unidimensionality requirements.

Usage

```
self.res_corr_analysis(warm_corr=True, tolerance=0.00001, max_iters=100,
                       ext_score_adjustment=0.5, constant=None, robust=False,
                       method='log-lik', matrix_power=None, log_lik_tol=0.000001, se=True)
```

Arguments

warm_corr	Boolean: If True, Warm's (1989) bias correction is applied during generation of person location estimates (see Section 3.3.4). Default warm_corr=True.
tolerance	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 3.3.4). Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 3.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for generation of item location estimates (see Section 3.3.1). Default is constant=None, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} N})$, where N is the number of persons with at least one response and I is the number of items.
robust	Boolean: deprecated and has no effect; passing robust=True raises a FutureWarning. It will be removed in a future release. Default is robust=False.

Continued on next page...

`method`	String: method for derivation of vector of item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) (see Section 3.3.1). Options are `'ls'` for least squares (Choppin, 1968, 1985), `'evm'` for the eigenvector method(Garner & Engelhard, 2002), `'cos'` for cosine maximisation (Kou & Lin, 2014) or `'log-lik'` for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is `method='log-lik'`
`matrix_power`	Integer or `None`: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of item location estimates. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is `matrix_power=None`, which resolves to `0` (no matrix powering) when `method='log-lik'` and to `5` for every other method. Passing `matrix_power=0` with a method other than `'log-lik'` issues a `UserWarning`.
`log_lik_tol`	Float: convergence criterion for log-likelihood method for generation of item location estimates (see Section 3.3.1). Ignored for other methods.
`se`	Boolean: passed through to the internal `fit_statistics` call (used if not already computed). If `False`, skips the bootstrap. Residual correlations and PCA do not depend on SE. Default `se=True`.

Returns

For tests of violation of the requirement for local item independence (Andrich & Kreiner, 2010; Marais, 2012):

`self.residual_correlations`	A pandas dataframe of pairwise correlations between item residuals.

For tests of violation of the requirement for unidimensionality based on principal component analysis of the standardised residual correlations (Pallant & Tennant, 2007; Smith, 2002):

`self.eigenvectors`	The eigenvectors of the standardised residual correlations matrix.
`self.eigenvalues`	The eigenvalues corresponding to the eigenvectors.
`self.variance_explained`	The variance explained by each principal component.
`self.loadings`	The loading of each item onto each of the principal components, for the the first of which large loadings ('large' typically interpreted as > 0.4 or < -0.4) may be interpreted as representing the presence of significant dimensionality, in analogy to factor analysis.

Example

To produce a residual correlation analysis:

```
self.res_corr_analysis()
```

Arguments may be used to alter parameters of item location and/or person location estimation.

3.4.6 save_stats

Description

Exports item_stats, person_stats and test_stats to file in a single call. Auto-triggers item_stats_df, person_stats_df and test_stats_df (Sections 3.4.1, 3.4.2, 3.4.3) if any have not already been run – with their default arguments plus whichever of dp/warm_corr/etc. below are passed, so call those methods directly first (e.g. with full=True) if non-default table columns are wanted in the saved output.

Usage

```
self.save_stats(filename, format='csv', dp=3, warm_corr=True, tolerance=0.00001,
                max_iters=100, ext_score_adjustment=0.5, method='log-lik', constant=None,
                robust=False, no_of_samples=500, interval=None)
```

Arguments

filename	String: output filename or path. For format='csv', used as a prefix (any trailing .csv is stripped first) for three files, _item_stats.csv, _person_stats.csv and _test_stats.csv. For format='xlsx', used as a single workbook filename (.xlsx appended automatically if not already present), with the three tables on separate sheets.
format	String: 'csv' or 'xlsx'. Default is format='csv'.
dp, warm_corr, tolerance, max_iters, ext_score_adjustment, method, constant, no_of_samples, interval	Passed through to item_stats_df/person_stats_df/test_stats_df (Sections 3.4.1, 3.4.2, 3.4.3), used only for whichever of those tables has not already been computed. See those sections for details of each.

Returns

None. Writes the three tables to file as described above.

Example

To save item, person and test statistics as three CSV files prefixed my_slm_results:

```
self.save_stats('my_slm_results')
```

To save all three tables to separate sheets of a single Excel workbook:

```
self.save_stats('my_slm_results', format='xlsx')
```

3.4.7 save_residuals

Description

Exports the residual correlation analysis (self.eigenvectors, self.eigenvalues, self.variance_explained and self.loadings) to file. Auto-triggers res_corr_analysis (Section 3.4.5) if not already run.

Usage

```
self.save_residuals(filename, format='csv', single=True, dp=3, warm_corr=True,
                    tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                    method='log-lik', constant=None, robust=False)
```

Arguments

filename	String: output filename or path (extension appended automatically if not already present).
format	String: 'csv' or 'xlsx'. Default is format='csv'.
single	Boolean: if True, writes all four tables (eigenvectors, eigenvalues, variance explained, loadings) into a single file – sequentially, separated by blank lines, for format='csv'; at successive row offsets on one sheet ('Item residual analysis'), for format='xlsx'. If False, writes each table to its own file (csv) or sheet (xlsx). Default is single=True.
dp, warm_corr, tolerance, max_iters, ext_score_adjustment, method, constant, robust	dp controls rounding of the saved values; the rest are passed through to res_corr_analysis (Section 3.4.5), used only if it has not already been run. See that section for details of each.

Returns

None. Writes the residual correlation analysis tables to file as described above.

Example

To save the residual correlation analysis as a single CSV file:

```
self.save_residuals('my_slm_residuals')
```

To save each table to its own sheet of a single Excel workbook:

```
self.save_residuals('my_slm_residuals', format='xlsx', single=False)
```

3.4.8 category_counts_df

Description

Produces a table of counts of scores in each category, plus total responses and missing responses.

Usage

```
self.category_counts_df(persons=None, items=None, counts_name=None,
                        filename=None, file_format='csv')
```

Arguments

persons	List or String: Person or subset of persons over which to count responses. Default persons=None, which returns counts for all persons.
items	List or String: Item or subset of items over which to count responses. Default items=None, which returns counts for all items.
counts_name	String or Int: Name under which to store counts on a subset of persons or items so that multiple sets of counts can be made available. Useful if e.g. the user wants to compare counts between different subgroups of persons according to a criterion such as an exogenous variable. If passed, stores counts within a namespace, self.counts. Default counts_name=None, when counts are returned as a simple attribute, self.category_counts_table.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

If counts_name=None:

Returned and stored as attribute self.category_counts_table, a pandas dataframe of category counts with one row per item and one column per response category, plus total and missing responses per item, and grand total row.

If counts_name=my_counts_name:

Returned and stored as attribute self.counts.my_counts_name stored in the namespace self.counts, a pandas dataframe of category counts with one row per item and one column per response category, plus total and missing responses per item and grand total row.

Example

To produce a dataframe of category counts for all persons and items:

```
self.category_counts_df()
```

To produce a dataframe of category counts for a subset of persons saved as a list `my_persons`, with a matching attribute name within the namespace `self.counts`:

```
self.category_counts_df(persons=my_persons, counts_name=my_persons)
```

3.4.9 dif_test

Description

Differential Item Functioning (DIF) test to test whether any items function differently for different subgroups of persons as defined by an exogenous person covariate (e.g. gender, first language) (Holland & Wainer, 1993). Splits non-extreme persons into groups by the named column in `self.exogenous`, designates one group as the reference (by default the largest), and compares each of the other ("focal") groups against it individually – a reference-group design, not all-pairwise, so covariates with more than two levels are supported directly.

For each focal group, both groups are calibrated independently (each self-centred), then purified onto a common scale – by default via a formal per-item Wald significance test (`selection_method='wald'`), with optional Welch correction and small-reference-group size adjustment (pass `selection_welch='True'`), or optionally via the robust modified z-score outlier-trimming procedure used by `calibrate_anchor` (Section 3.3.3; Iglewicz & Hoaglin, 1993) – the reference group's item location estimates supply the "anchor" values, the focal group's supply the "observed" values – so that genuine DIF items cannot contaminate the common scale used to test for DIF in the first place. A per-item test is then run on the purified scale for every item (not only the ones used to define the scale), using bootstrap standard errors from `std_errors` (Section 3.3.2). An item is flagged only if it clears both a (corrected) p-value threshold and an absolute logit-difference threshold so that a statistically significant but practically negligible difference is not flagged.

`dif_test` supports further options: – a per-item likelihood-ratio test alongside or instead of the Wald test (`test='lr'/'both'`; this mildly liberal, so `test='wald'` (the default) is preferred), an omnibus LR test of whether *any* item shows DIF (`omnibus`), and ETS-style A/B/C severity-category classification (Zwick et al., 1997) (`category`, `category_thresholds`, `category_alpha`), as described in the Table 62:

Table 62: ETS-style DIF category classification. Sign: + indicates that an item is harder for the focal group; - indicates that an item is easier for the focal group. The 0.43/0.64 logit thresholds are the Rasch-logit equivalents of the classic ETS Mantel-Haenszel delta-scale B/C boundaries (Tristan, 2006); the significance level (default `category_alpha=0.05`) is independent of `alpha`, which governs the separate `Flagged`/`Flagged_LR` columns.

Category	Criteria	Interpretation
A	|Difference| < 0.43, or not statistically significant	Negligible DIF
B+ / B-	Both |Difference| ≥ 0.43 *and* $p(\text{DIF} = 0) <$ `category_alpha` (two-sided Wald/Welch test), but not C	Slight to moderate DIF
C+ / C-	Both |Difference| ≥ 0.64 *and* $p(\lvert\text{Difference}\rvert \leq 0.43) <$ `category_alpha` (one-sided test that difference significantly exceeds B boundary)	Moderate to large DIF

Optionally, standard errors may be adjusted for large samples by setting `size_adjust=True`: at very large N, standard errors shrink enough that trivial differences become significant. Each group's item-location SE is multiplied by $\sqrt{n/\texttt{reference_n}}$ (Tristan, 2006), where n is the group size. The adjustment applies to the Wald/Welch test, the category tests and, if `welch=True`, the Welch–Satterthwaite degrees of freedom, which then use `reference_n` in place of n. Default is `size_adjust=False`.

Usage

```
self.dif_test(covariate, reference=None, selection_method='wald', corr_tol=0.95,
              sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05, test='wald',
              omnibus=True, welch=False, size_adjust=False, reference_n=100,
              correction='bh', alpha=0.05, logit_threshold=0.43, category=False,
              category_thresholds=(0.43, 0.64), category_alpha=0.05, no_of_samples=500,
              plot=False, plot_kwargs=None, warm_corr=True, tolerance=0.00001,
              max_iters=100, ext_score_adjustment=0.5, constant=None, robust=False, method='log-lik',
              matrix_power=None, log_lik_tol=0.000001, seed=None)
```

Optionally, standard errors may be adjusted for large samples by setting `size_adjust=True`: at very large N, standard errors shrink enough that trivial differences become significant. Each group's item-location SE is multiplied by $\sqrt{n/\texttt{reference_n}}$ (Tristan, 2006), where n is the group size. The adjustment applies to the Wald/Welch test, the category tests and, if `welch=True`, the Welch–Satterthwaite degrees of freedom, which then use `reference_n` in place of n. Default is `size_adjust=False`.

Arguments

`covariate`	String: column name in `self.exogenous` to group persons by.
`reference`	String or `None`: value of `covariate` to use as the reference group. If `None`, the largest group is used. Default is `reference=None`.
`selection_method`	String, one of `'wald'`, `'robust_z'` or `'none'`: how the common scale is purified before testing – see `calibrate_anchor` (Section 3.3.3) for what each option does. Default is `selection_method='wald'`.
`corr_tol`	Float: minimum correlation between the two groups' item locations on the retained items (`'robust_z'` only). Default is `corr_tol=0.95`.
`sd_ratio_tol`	Float: the ratio of the two groups' item-location standard deviations on the retained items must lie in $[1/\texttt{sd_ratio_tol}, \texttt{sd_ratio_tol}]$ (`'robust_z'` only). Default is `sd_ratio_tol=1.1`.
`min_anchors`	Integer: minimum number of items retained by purification (`'wald'` and `'robust_z'`). Default is `min_anchors=6`.
`wald_alpha`	Float: significance level for each item's purification test (`'wald'` only). Default is `wald_alpha=0.05`.
`test`	String, one of `'wald'`, `'lr'` or `'both'`: per-item DIF test(s) to compute. The existing `Flagged` column always reflects the Wald test; a separate `Flagged_LR` column is added when `test` is `'lr'` or `'both'`. Default is `test='wald'`.

Continued on next page...

omnibus	Boolean: if True (default), also runs an Andersen-style omnibus LR test (reference vs. focal, every common item jointly) per focal group, stored in self.dif_omnibus_table – answering the question "is there any DIF at all?"
welch	Boolean: if True, the per-item Wald test (test='wald' or 'both') becomes a Welch's t-test: anchor selection and statistic are unchanged (Difference / SE), but p-value computed against a t-distribution with Welch-Satterthwaite degrees of freedom. Adds a df column to self.dif_table. Only used for the Wald test. Default is welch=False.
size_adjust	Boolean: if True, rescales each group's item SE to what it would be at a standard reference sample size as per Tristan (2006). Default is size_adjust=False.
reference_n	Integer: reference sample size for size_adjust=True. 100 (default) is Tristán's (2006) recommendation; 60 is suggested there for closer alignment with ETS Category B boundary. Unused unless size_adjust=True.
correction	String or None, one of 'bh', 'bonferroni' or None: multiple-comparison correction applied across items within each focal-group comparison – Benjamini-Hochberg (1995) false discovery rate control or Bonferroni – applied separately to the Wald and per-item LR p-values if both computed. Not applied to omnibus test (one test per focal group, not a family of item-level comparisons). Default correction='bh'.
alpha	Float: p-value threshold for flagging, compared against the corrected p-value if correction is not None. Default is alpha=0.05.
logit_threshold	Float: absolute logit-difference threshold for flagging (ETS-style convention). An item is flagged only if both this and the p-value threshold are met. Default is logit_threshold=0.43.
category	Boolean: if True, adds an ETS-style Category column to self.dif_table, as described in Table 62 above, following Zwick et al. (1997). Independent of test/logit_threshold/Flagged – this reproduces the ETS scheme exactly, not a repackaging of the existing Wald flag. Default is category=False.
category_thresholds	Tuple of two floats: (B boundary, C boundary) in logits, per Zwick et al. (1997). Override with your own values if working from a different source or convention. Default is category_thresholds=(0.43, 0.64).
category_alpha	Float: significance level for both category tests – ETS standard 0.05, independent of alpha (which governs Flagged/Flagged_LR) so overriding one does not change the other. Default category_alpha=0.05.

Continued on next page...

no_of_samples	Integer: bootstrap resamples for each group's std_errors call (Section 3.3.2). Default is no_of_samples=500.
plot	Boolean: if True, calls plot_anchor_selection (Section 3.5.10) for each focal group that has a selection table (selection_method in 'wald'/'robust_z' only) and stores the resulting figures in self.dif_plots. Defaults plot=False – unlike calibrate_anchor's plot=True default – since dif_test can compare multiple focal groups against the reference in one call.
plot_kwargs	Dictionary or None: extra keyword arguments for plot_anchor_selection for every focal group when plot=True. Default plot_kwargs=None.
warm_corr, tolerance, max_iters, ext_score_adjustment	Person-estimation arguments, passed through to the group models; person estimates are only used if test is 'lr'/'both' or omnibus=True.
constant, method, matrix_power, log_lik_tol	Calibration arguments passed to the group models – see Section 3.3.1.
seed	Integer or None: seed passed through to each group's internal std_errors call. None (default) draws fresh entropy each call. See Section 1.

Returns

self.dif_table	pandas dataframe: one row per (item, focal group) pair, indexed by item. Columns: Group (focal group value), Reference/Focal/Focal (purified) (item location estimates), Difference (purified focal minus reference), SE, z, p, p (corrected), Selected (used to define the purified scale) and Flagged (both flagging thresholds met). If welch=True, also df (Welch-Satterthwaite), and p/p (corrected) are t-based rather than normal-based (z is unchanged either way). If test is 'lr' or 'both', also LR, p_LR, p_LR (corrected), Flagged_LR. If category=True, also Category ('A', 'B+', 'B-', 'C+', or 'C-').
self.dif_omnibus_table	pandas dataframe or None: one row per focal group: LR, df, p, Flagged. Andersen-style joint test of every common item at once. None if omnibus=False.
self.dif_reference	The reference group value.
self.dif_covariate	The covariate column name used.

Continued on next page...

self.dif_reference_model	SLM object: fitted model for the reference group.
self.dif_focal_models	Dictionary: {focal_value: SLM}, fitted models for each focal group.
self.dif_tc	Dictionary: {focal_value: float}, the translation constant applied to each focal group.
self.dif_anchor_selection	Dictionary: {focal_value: dataframe or None}, per-item selection diagnostics for each focal group (None if selection_method='none').
self.dif_plots	Dictionary: {focal_value: figure or None}, the plot_anchor_selection figure for each focal group, if plot=True and a selection table exists for that group.
self.dif_group_sizes	Dictionary: {group_value: int}, non-extreme, non-missing-covariate N used for each group.

Examples

To test for DIF by an exogenous variable 'L1' (which may have any number of levels), having supplied exogenous to the constructor, using the largest level as the reference group by default:

```
self.dif_test('L1')
```

To test for DIF against a specific reference level 'English', with 1000 bootstrap samples and diagnostic plots for every focal group:

```
self.dif_test('L1', reference='English', no_of_samples=1000, plot=True)
```

To additionally compute the per-item likelihood-ratio test and ETS-style DIF categories:

```
self.dif_test('L1', test='both', category=True)
```

3.4.10 recovery_stats

Description

Compares recovered parameter estimates of an object created from a simulation against the generating values (self.generating, Section 3.1.1). Issues a UserWarning and returns None unless the object was built from a simulation. The generating model may be SLM_Sim, PCM_Sim or RSM_Sim. With PCM_Sim or RSM_Sim, the generating max scores must all be 1, otherwise the method returns None.

Person estimates are compared against *pseudo-estimates* (Elliott & Buttery, 2022b): the person estimate from self.responses using the true generating item difficulties. The persons error column is (recovered estimate − pseudo-estimate), and the persons generating column holds the pseudo-estimate. The error combines parameter estimation error and model misspecification.

Usage

```
self.recovery_stats(anchor=False, alpha=0.05, correction='bh', divergence_tol=0.1,
                        plot=True, **plot_kwargs)
```

Arguments

`anchor`	Boolean: if `True`, compares `self.anchor_items` (from `calibrate_anchor`, Section 3.3.3) against `self.generating.items`, instead of `self.items`. Raises `ValueError` if `calibrate_anchor` has not been run yet. Persons are unaffected by this flag. Default `anchor=False`.
`alpha`	Float: significance threshold for the per-parameter z-test (`Flagged`/`Flagged_corrected` columns, below). Default `alpha=0.05`.
`correction`	String or `None`, one of `'bh'`, `'bonferroni'` or `None`: multiple-comparison correction applied within each parameter type's family of tests (matches `dif_test`'s convention, Section 3.4.9). Default `correction='bh'`.
`divergence_tol`	Float: threshold on the absolute difference between the OLS and Huber regression slopes (below) for the `Divergence_flag` summary row. Default `divergence_tol=0.1`.
`plot`	Boolean: if `True`, also produces a recovery plot (scatter, identity line, OLS and Huber fit lines) for each parameter type. Default `plot=True`.
`**plot_kwargs`	Forwarded to the internal plotting routine: `filename`, `file_format`, `dpi`, `plot_style`, `font`, `title_font_size`, `axis_font_size`, `labelsize`, `figsize` (same vocabulary as `wright_map`). A shared `filename` is suffixed per parameter type.

Returns

`self.recovery.items.table`, `self.recovery.persons.table`	pandas dataframes, one per parameter type (`items` and `persons`), each with columns `generating`, `estimated`, `SE`, `error` (estimated − generating), `abs_error`, `z`, `p`, `p_corrected`, `Flagged` and `Flagged_corrected`. For `persons`, `generating` holds the pseudo-estimate, and `SE` is `self.csem_vector` (the analytic conditional SEM), unaffected by `anchor`.
`self.recovery.items.summary`, `self.recovery.persons.summary`	pandas dataframes, one column (`Overall`) with rows `RMSE`, `MAE`, `bias` (mean signed error), `r` (Pearson correlation), `SD_ratio` (estimated SD over generating SD), `OLS_slope`, `OLS_intercept`, `Huber_slope`, `Huber_intercept`, `Divergence_flag` (`True` if the OLS and Huber slopes differ by more than `divergence_tol`) and `Prop_flagged` (proportion of parameters with `Flagged=True`). `r`, the two slopes and `SD_ratio` are set to `NaN` when fewer than 3 parameters are compared or the generating values have standard deviation below 0.05.

Examples

To compare recovered item and person estimates against the generating simulation, plotting both:

```
self.recovery_stats()
```

To compare anchored item estimates instead (having already run `calibrate_anchor`), without plotting:

```
self.recovery_stats(anchor=True, plot=False)
```

3.5 Plotting functionality

3.5.1 Shared plotting arguments

All the plotting methods described in this section share a set of arguments which may be used to customise the appearance of the plot or save the plot to file automatically. These arguments are:

`title`	String: Title for the plot, to appear in the image. Default is `title=None`.
`xmin`	Float: Minimum displayed point on x-axis, in logits. Default is `xmin=-5`.
`xmax`	Float: Maximum displayed point on x-axis, in logits. Default is `xmax=5`.
`plot_style`	String: Plot style to use. Available styles are `'white'`, `'dark'`, `'black'`, `'parchment'`, `'minimal'`, `'print'`, `'solarized-light'`, `'solarized-dark'`, `'slate'`, `'blueprint'`, `'newsprint'` and `'chalkboard'` – twelve Seaborn (Waskom, 2021)-based colour schemes. Default is `plot_style='white'`.
`palette`	String: Controls colours of lines and overplotted class intervals dots. Options are `'dark blue'`, `'light blue'`, `'dark red'`, `'light red'`, `'dark green'`, `'light green'`, `'dark grey'`, `'light grey'`, `'dark multi'`, `'light multi'` and `'colorblind multi'`. `'colorblind multi'` is the Okabe & Ito (2008) colour-vision-deficiency-safe qualitative palette. Default is `palette='colorblind multi'`. Not used for `std_residual_plot`.
`black`	Boolean: If `True`, the plot will be rendered in black and white. Default is `black=False`.
`marker_border`	Boolean: If `True`, observed data point markers are drawn with a black edge for contrast. Only used by `icc`, `crcs` and `tcc` (the methods with an `obs` argument). Default is `marker_border=True`.
`line_border`	Boolean: If `True`, curve lines are drawn with a contrasting outline stroke (white on dark `plot_style` backgrounds, black otherwise) to improve visibility. Only used by `icc`, `crcs` and `tcc`. Default is `line_border=False`.
`font`	String: The font to use in the plot. Default is `font='Times New Roman'`.
`title_font_size`	Float: The size of the title font in points. Default is `title_font_size=15`.
`axis_font_size`	Float: The size of the axis label font in points. Default is `axis_font_size=12`.
`labelsize`	Float: The size of the axis tick label font in points. Default is `labelsize=12`.
`filename`	String: The filename for the saved plot, with no suffix for format. If `None`, no file will be saved. Default is `filename=None`.
`file_format`	The format of the file: `png`, `jpg` or `svg`. Default is `file_format=png`.
`dpi`	The resolution of the plot in dpi (dots per inch) – higher resolution plots are better quality but have larger file sizes. Default is `dpi=300`.

3.5.2 icc

Description

Plots the item characteristic curves (or item response function) for an item: person location on the x-axis against expected score on the y-axis. Options to plot observed responses, item threshold line and lines showing locations corresponding to specified expected scores, and to highlight a specified response category.

Usage

```
self.icc(item, obs=False, no_of_classes=5, thresh_line=False, score_lines=None,
         score_labels=False, cat_highlight=None)
```

Arguments

item	String: The name of the item(s) to plot.
obs	Boolean: If True, mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is obs=False.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
thresh_line	Boolean: If True, a vertical line showing the threshold corresponding to the item location (the threshold between the location regions for which 0 or 1 are the most probable score) will be plotted. Default is thresh_line=False.
score_lines	List of floats between 0 and 1: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score. Default is score_lines=None.
score_labels	Boolean: If True, scores and locations corresponding to arguments passed to score_lines will be labelled on the plot. Default is score_labels=False.
cat_highlight	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is cat_highlight=None (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic item characteristic curve for `Item_1` and store the output as a variable `my_icc_plot` and save it to file as `my_icc_plot.png`:

```
my_icc_plot = self.icc('Item_1', filename=my_icc_plot)
```

To plot an item characteristic curve for `Item_1` with observed responses for 8 response classes and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc('Item_1', obs=True, no_of_classes=8)
```

To plot an item characteristic curve for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc('Item_1', thresh_line=True, cat_highlight=0)
```

To plot an item characteristic curve for `Item_1` with lines showing the locations corresponding to expected scores of 0.3 and 0.7, with the expected score and corresponding location labelled, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc('Item_1', score_lines=[0.3, 0.7], score_labels=True)
```

3.5.3 crcs

Description

Plots category response curves for an item: person location on the x-axis against the probability of obtaining a score in each category (0 or 1) on the y-axis. Options to plot observed proportions and item threshold line, and to highlight a specified response category.

Usage

```
self.crcs(item, obs=None, no_of_classes=5, thresh_line=False, cat_highlight=None)
```

Arguments

item	String: The name of the item to plot.
obs	List: List of integers (0 or 1). For each value, mean observed proportions in each ordered response category scoring in that category are plotted against the mean location of the corresponding response class. Default is obs=None.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
thresh_line	Boolean: If True, a vertical line showing the threshold corresponding to the item location (the threshold between the location regions for which 0 or 1 are the most probable score) will be plotted. Default is thresh_line=False.
cat_highlight	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is cat_highlight=None (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot basic category response curves for Item_1 and store the output as a variable my_crcs_plot and save it to file as my_crcs_plot.png:

```
my_crcs_plot = self.crcs('Item_1', filename=my_crcs_plot)
```

To plot category response curves for Item_1 with observed response proportions for category 0 for 8 response classes and store the output as a variable my_crcs_plot:

```
my_crcs_plot = self.crcs('Item_1', obs=[0], no_of_classes=8)
```

To plot category response curves for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = self.crcs('Item_1', thresh_line=True, cat_highlight=0)
```

3.5.4 iic

Description

Plots the item information curve for an item: person location on the x-axis against Fisher information on the y-axis. Options to plot item threshold line and lines showing Fisher information corresponding to specified locations, and to highlight a specified response category.

Usage

```
self.iic(item, ymax=None, thresh_line=False, point_info_lines=None, point_info_labels=False,
         cat_highlight=None)
```

Arguments

item	String: The name of the item(s) to plot.
ymax	Float: The maximum value to show on the y-axis. If None, will infer, plotting a maximum of 1.1 times the maximum item information. Default is ymax=None
thresh_line	Boolean: If True, a vertical line showing the threshold corresponding to the item location, which the location for which scores of 0 and 1 are equally probable, will be plotted. Default is thresh_line=False.
point_info_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the Fisher information corresponding to the location. Default is point_info_lines=None.
point_info_labels	Boolean: If True, locations and Fisher information corresponding to arguments passed to point_info_lines will be labelled on the plot. Default is point_info_labels=False.
cat_highlight	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is cat_highlight=None (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic item information curve for `Item_1` and store the output as a variable `my_iic_plot` and save it to file as `my_iic_plot.png`:

```
my_iic_plot = self.iic('Item_1', filename='my_iic_plot')
```

To plot an item information curve for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = self.iic('Item_1', thresh_line=True, cat_highlight=0)
```

To plot an item information curve for `Item_1` with lines showing the Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding Fisher information labelled, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = self.icc('Item_1', point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

3.5.5 tcc

Description

Plots the test characteristic curve (or test response function) for a set of items: person location on the x-axis against expected score on the y-axis. Options to plot observed responses and lines showing locations corresponding to specified expected scores.

Usage

```
self.tcc(items=None, obs=False, no_of_classes=5, score_lines=None, score_labels=False)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
obs	Boolean: If True, mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is obs=False.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
score_lines	List of floats between 0 and 1: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score. Default is score_lines=None.
score_labels	Boolean: If True, scores and locations corresponding to arguments passed to score_lines will be labelled on the plot. Default is score_labels=False.

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic test characteristic curve for all items and store the output as a variable my_tcc_plot and save it to file as my_tcc_plot.png:

```
my_tcc_plot = self.tcc(filename=my_tcc_plot)
```

To plot a test characteristic curve for Item_1 for a subset of items stored as a list **my_item_list**, with observed responses for 8 response classes and store the output as a variable my_tcc_plot:

```
my_tcc_plot = self.tcc(obs=True, no_of_classes=8)
```

To plot a test characteristic curve for `Item_1` for all items with lines showing the locations corresponding to expected scores of 13 and 20, with the expected score and corresponding location labelled, and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = self.tcc(score_lines=[13, 20], score_labels=True)
```

3.5.6 test_info

Description

Plots the test information curve: person location on the x-axis against total Fisher information on the y-axis. Option to plot lines showing Fisher information corresponding to specified locations.

Usage

```
self.test_info(items=None, ymax=None, point_info_lines=None, point_info_labels=False)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
ymax	Float: The maximum value to show on the y-axis. If None, will infer, plotting a maximum of 1.1 times the maximum test information. Default is ymax=None
point_info_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the total Fisher information corresponding to the location. Default is point_info_lines=None.
point_info_labels	Boolean: If True, locations and total Fisher information corresponding to arguments passed to point_info_lines will be labelled on the plot. Default is point_info_labels=False.

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic test information curve and store the output as a variable my_test_info_plot and save it to file as my_test_info_plot.png:

```
my_test_info_plot = self.test_info(filename='my_test_info_plot')
```

To plot a test information curve for a subset of items stored as a list **my_item_list** and store the output as a variable my_test_info_plot:

```
my_test_info_plot = self.test_info(items=my_item_list)
```

To plot a test information curve with lines showing the total Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding total Fisher information labelled, and store the output as a variable `my_test_info_plot`:

```
my_test_info_plot = self.test_info(point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

3.5.7 test_csem

Description

Plots the test conditional standard error of measurement (CSEM) curve: person location on the x-axis against CSEM (in logits) on the y-axis. Option to plot lines showing CSEM corresponding to specified locations.

Usage

```
self.test_csem(items=None, point_csem_lines=None, point_csem_labels=False, ymax=5)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
ymax	Float: The maximum value to show on the y-axis, in logits. Default is ymax=5
point_csem_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the CSEM curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the CSEM corresponding to the location. Default is point_csem_lines=None.
point_csem_labels	Boolean: If True, locations and CSEM corresponding to point_csem_lines labelled on the plot. Default point_csem_labels=False.

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic CSEM curve and store the output as a variable my_test_csem_plot and save it to file as my_test_csem_plot.png:

```
my_test_csem_plot = self.test_csem(filename='my_test_csem_plot')
```

To plot a CSEM curve for a subset of items stored as a list **my_item_list** and store the output as a variable my_test_csem_plot:

```
my_test_csem_plot = self.test_csem(items=my_item_list)
```

To plot a CSEM curve with lines showing the CSEM corresponding to locations of -0.3 and 0.7, with the location and corresponding CSEM labelled, and store the output as a variable my_test_csem_plot:

```
my_test_csem_plot = self.test_csem(point_csem_lines=[-0.3, 0.7], point_csem_labels=True)
```

3.5.8 std_residuals_plot

Description

Plots histogram of standardised residuals, with optional overplotting of standard Normal distribution.

Usage

```
self.std_residuals_plot(items=None, bin_width=0.5, normal=False, dpi=300)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
bin_width	Float: The width of the histogram bins along the x-axis. Default is bin_width=0.5.
dpi	Integer: resolution of the saved plot in dpi (dots per inch). Default is dpi=300.
normal	Boolean: If True, plots a standard normal distribution over the standardised residual histogram for comparison. Default is normal=False.

Additional arguments to customise the appearance of the plot are detailed in Section 3.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot and display a basic standardised residuals histogram and save it to file as my_std_residuals_plot.png:

```
self.std_residuals_plot(filename='my_std_residuals_plot')
```

To plot and display a standardised residuals histogram with bin width 1, with standard normal curve:

```
self.std_residuals_plot(bin_width=1, normal=True)
```

To plot and display a standardised residuals histogram on a subset of items stored as a list in a variable my_item_list:

```
self.std_residuals_plot(items=my_item_list)
```

3.5.9 wright_map

Description

Plots a Wright map (person-item map): back-to-back person and item distributions on a shared axis. Options for histograms or kernel density estimate (KDE) curves; items may also be presented by names in histogram format. wright_map does not use the shared plotting arguments described in Section 3.5.1.

Usage

```
self.wright_map(person_names=None, item_names=None, orientation='vertical',
                map_type='hist', item_labels=False, item_distribution=False,
                person_lim=None, item_lim=None, person_scaling=None,
                item_row_height=None, narrow_font=False, edge_padding=0.5,
                distribution_markers=False, group_by=None, group_colors=None,
                stack=True, blend=False, prop=False, no_of_bins=20, pad=False,
                plot_range=None, kde_points=500, bw_method='scott', line_width=1,
                figsize=None, title=None, plot_style='white', edge_color='black',
                person_color='skyblue', item_color='salmon',
                person_line_color='black', item_line_color='black',
                marker_color=None, alpha=0.6, black=False,
                font='Times New Roman', title_font_size=15, axis_font_size=12,
                labelsize=12, item_label_size=8, tick_interval=1, filename=None,
                file_format='png', dpi=300)
```

Arguments

person_names	String, list or None: Person subset to include. Default is person_names=None (all persons).
item_names	String, list or None: Item subset to include. Default is item_names=None (all items).
orientation	String: 'vertical' places location on the x-axis, with persons above the baseline; 'horizontal' places location on the y-axis, with persons to the left of the baseline. Default is orientation='vertical'.
map_type	String: 'hist' plots back-to-back histograms; 'kde' plots kernel density estimate curves instead. If item_labels=True, only governs the person side; the item side becomes a label panel rather than a distribution. Default is map_type='hist'.
item_labels	Boolean: If True, item names are listed instead of a bare histogram, grouped into no_of_bins histogram-style location bins and ordered by location. Default is item_labels=False.

Continued on next page...

item_distribution	Boolean: If True, plots a histogram/KDE plot as well as item labels. Forces item_labels=True. Default is item_distribution=False.
person_lim	Float or None: Controls axis limit for person distribution. If None, derived automatically from the distribution. Default is person_lim=None.
item_lim	Float or None: Controls axis limit for item distribution. If None, derived automatically from the distribution. Default is item_lim=None.
person_scaling	Float or None: Adjusts the scaling of the person distribution for clarity, particularly when the number of persons is very large compared to items. A scaling of 10 means that the person scale is 10 times the item scale, and so on. Default is person_scaling=None, which chooses the largest of 1, 2, 5, 10, 20, 50, ... that keeps the rescaled item distribution within half the height of the person distribution, and never below 1.
item_row_height	Float or None: Physical height, in inches, allocated per stacked item-label row. If None, derived automatically. Only used when item_labels=True. Default is item_row_height=None.
narrow_font	Boolean: If True, item labels are set in 'Arial Narrow' instead of font to save space. Falls back to Matplotlib's font substitution if 'Arial Narrow' is not installed. Default is narrow_font=False.
edge_padding	Float: Extra padding (in inches) added to the figure's right-hand margin in orientation='vertical'. No effect in orientation='horizontal'. Default is edge_padding=0.5.
distribution_markers	Boolean: If True, overlays mean and plus/minus 1 and 2 standard deviation reference markers, labelled μ, $\mu \pm \sigma$, $\mu \pm 2\sigma$ for persons and items. Default is distribution_markers=False.
group_by	pandas Series or None: Grouping of persons (e.g. a DIF group) used to split the person distribution into sub-distributions. Default group_by=None (persons as a single series).
group_colors	Dict or None: Controls colours for group_by. If None, colours are drawn from the 'tab10' colour map. Default is group_colors=None.
stack	Boolean: When group_by is set, for map_type='hist', True cumulatively stacks histogram bars while False draws separate columns per group: dodged, or alpha-blended if blend=True. For map_type='kde', True cumulatively stacks group KDE plots; False overlays group KDE plots. Default is stack=True.
blend	Boolean: When group_by is set, map_type='hist' and stack=False, if True, blends histogram bars using colour instead of dodging them (showing them side-by-side). Default is blend=False.

Continued on next page...

prop	Boolean: If True, plots distributions as proportions/density rather than counts. Default is prop=False.
no_of_bins	Integer: Number of histogram or label bins. Default is no_of_bins=20.
pad	Boolean: If True, adds 5% padding to both ends of the location axis limits. If False, the location axis is bound to the plotted range. Default is pad=False.
plot_range	Tuple of two floats, or None: (lo, hi) limits for the location axis. If None, derived automatically. Default is plot_range=None.
kde_points	Integer: Number of points at which each KDE curve is evaluated when map_type='kde'. Default is kde_points=500.
bw_method	String, scalar or callable: Bandwidth selection method passed to scipy.stats.gaussian_kde when map_type='kde'. Default is bw_method='scott'.
line_width	Float: Line width of KDE curves when map_type='kde'. Default is line_width=1.
figsize	Tuple of two floats, or None: Base figure size in inches for the person distribution. If None, defaults to (8, 6) for orientation='vertical' or (4, 8) for orientation='horizontal'. Default is figsize=None.
title	String or None: Plot title. Default is title=None (no title).
plot_style	String: Plot background style, as per plot_style in Section 3.5.1. Default is plot_style='white'.
edge_color	String: Edge colour of histogram bars when map_type='hist' and group_by is None. Default is edge_color='black'.
person_color	String: Fill colour for person distribution; ignored when group_by is set, overridden by a grey shade when black=True. Default is person_color='skyblue'.
item_color	String: Fill colour for item distribution when item_labels=False; overridden by a grey shade when black=True. Default is item_color='salmon'.
person_line_color	String: Line colour for person KDE plot when map_type='kde' and group_by is None. Default is person_line_color='black'.
item_line_color	String: Colour for item labels (item_labels=True) or item KDE plot line (item_labels=False, map_type='kde'). Default is item_line_color='black'.

Continued on next page...

marker_color	String, tuple or None: Colour of tick marks and labels drawn when distribution_markers=True. A single colour applies to both the person and item marks; a (person, item) pair sets them separately. Person marks use it only when group_by=None; grouped marks use the assigned group colours. Default is marker_color=None, which gives each set of marks a darker shade of its own distribution's colour (person_color or item_color).
alpha	Float: Fill transparency for distributions. Default is alpha=0.6.
black	Boolean: If True, renders person_color/item_color as grey shades instead. Has no effect on group_by colours. Default is black=False.
font	String: Plot font. Default font='Times New Roman'.
title_font_size	Float: Title font size in points. Default title_font_size=15.
axis_font_size	Float: Axis label font size in points. Default axis_font_size=12.
labelsize	Float: Axis tick label font size in points. Default labelsize=12.
item_label_size	Integer: Item name font size in points. Default item_label_size=8.
tick_interval	Float: Spacing between Location axis ticks. Default tick_interval=1.
filename	String or None: Filename for saved plot, with no suffix for format. If None, no file is saved. Default filename=None.
file_format	String: Format of saved file, 'png', 'jpg' or 'svg'. Default file_format='png'.
dpi	Integer: Resolution of saved plot in dpi (dots per inch). Default dpi=300.

Returns

matplotlib figure. Displays and closes the figure; if a filename argument is passed, also saves the image to file.

Examples

To plot a basic Wright map, with items and persons as back-to-back histograms:

```
self.wright_map()
```

To plot a Wright map with items listed by name in a dedicated panel, plus mean/±1SD/±2SD reference marks for both persons and items, and save it to file as my_wright_map.png:

```
self.wright_map(item_labels=True, distribution_markers=True, title='SLM Wright Map',
                filename='my_wright_map')
```

To plot a horizontally-oriented Wright map, split by an exogenous grouping variable stored in a pandas Series my_group (indexed like self.persons), with each group's histogram bars alpha-blended rather than stacked:

```
self.wright_map(orientation='horizontal', group_by=my_group, stack=False,
                blend=True)
```

3.5.10 plot_anchor_selection

Description

Scatter plot of anchor item locations (x-axis) against observed values for the same items (y-axis) to visualise the outlier-trimming diagnostics produced by `calibrate_anchor` (Section 3.3.3) or `dif_test` (Section 3.4.9). Selected and dropped items are plotted in different colours, with a solid $y = x$ identity line for reference and a dotted OLS best-fit line through the selected points only. `plot_anchor_selection` does not use the shared plotting arguments described in Section 3.5.1; its smaller set of arguments is described in full below.

Usage

```
self.plot_anchor_selection(result, xmin=-4, xmax=4, ymin=-3, ymax=3, figsize=(8, 6),
                           font='Times New Roman', title_font_size=15, axis_font_size=12,
                           labelsize=12, title='', dpi=300, filename=None,
                           file_format='png')
```

Arguments

`result`	dict or pandas dataframe: Either the full dictionary returned internally by the robust anchor-selection routine, or just its `'table'`, i.e. one of `self.anchor_selection` (from `calibrate_anchor`) or `self.dif_anchor_selection` (from `dif_test`), each a dataframe with `'Anchor'`, `'Observed'` and boolean `'Selected'` columns.
`xmin`, `xmax`, `ymin`, `ymax`	Floats: axis limits. Defaults are `xmin=-4`, `xmax=4`, `ymin=-3`, `ymax=3`.
`figsize`	Tuple of two floats: figure size in inches (width, height). Default is `figsize=(8, 6)`.
`font`	String: the font to use in the plot. Default is `font='Times New Roman'`.
`title_font_size`, `axis_font_size`, `labelsize`	Integers: font sizes in points for the title, axis labels and axis tick labels respectively. Defaults are `title_font_size=15`, `axis_font_size=12`, `labelsize=12`.
`title`	String: plot title. Default is `title=''` (no title).
`filename`	String or `None`: if provided, saves the plot to this path (with the suffix determined by `file_format`). Default is `filename=None` (not saved).
`file_format`	String: file format for saved plots, e.g. `'png'`, `'pdf'`, `'svg'`. Default is `file_format='png'`.
`dpi`	Integer: output resolution in dpi (dots per inch) when saving to file. Default is `dpi=300`.

Returns

matplotlib figure object. If a `filename` argument is passed, also saves the image to file. `calibrate_anchor` and `dif_test` both call this method automatically when their `plot` argument is `True` (see Sections 3.3.3 and 3.4.9), so it need not usually be called directly; it is documented separately here since the resulting figure objects (`self.anchor_plot`, `self.dif_plots`) can also be regenerated with different display arguments by calling it directly on the stored selection table.

Example

To regenerate the anchor-selection plot for an existing calibration's diagnostics, stored in `self.anchor_selection`, with wider axis limits and a custom title:

```
self.plot_anchor_selection(self.anchor_selection, xmin=-6, xmax=6, ymin=-6, ymax=6,
                           title='Anchor calibration')
```

4 class PCM

4.1 Preliminaries

4.1.1 PCM

Description

Creates an object of the class PCM from a pandas dataframe of polytomously scored data of items with individual Rasch-Andrich threshold structures, and potentially different maximum scores, for analysis. No analysis can be run until an object is created.

Usage

```
PCM(responses, max_score_vector=None, extreme_persons=True, no_of_classes=5,
    validate=True, exogenous=None)
```

Arguments

responses	pandas dataframe with items as columns (item names as column names) and persons as index (person names as row names).
max_score_vector	List or 1-dimensional numpy array of integers: List with the maximum possible score for each item. If no vector is passed, max_score_vector will be inferred from the data, although passing an argument is recommended. Default is max_score_vector=None.
extreme_persons	Boolean: if False, all persons with extreme scores (all responses correct or all responses incorrect) are removed from the response dataframe. Default is extreme_persons=True.
no_of_classes	Integer: the number of classes of persons grouped by location for overplotting observed responses on theoretical curves. Default is no_of_classes=5
validate	Boolean: if True, automatically calls check_data_connectivity (Section 4.1.2) on creation and issues a UserWarning if the item response network is disconnected, or if any item has a structurally unresolvable zero in the pairwise calibration matrix, or the directed comparison network is connected but not strongly connected (which prevents a finite method='log-lik' solution on an unpowered matrix). Default is validate=True.
exogenous	pandas dataframe or None: optional person-level covariates for Differential Item Functioning (DIF) analysis (Section 4.4.12). See the equivalent argument on the SLM constructor (Section 3.1.1) for full details – behaviour is identical. Default is exogenous=None.

Returns

Object of class PCM. Analyses are run using methods defined on the PCM object, with results stored as attributes of the PCM object.

Several attributes of object PCM are automatically generated on its creation:

self.responses	pandas dataframe: Dataframe of valid responses.
self.invalid_responses	pandas dataframe: Dataframe of invalid responses (persons with no responses to any items, i.e. all missing data).
self.extreme_persons	pandas dataframe: Dataframe of persons with extreme scores (all-correct or all-incorrect). When extreme_persons=True (the default), extreme scorers are retained in self.responses and this attribute is empty; when extreme_persons=False, they are removed and stored here.
self.max_score_vector	List or 1-dimensional numpy array of integers: List with the maximum possible score for each item.
self.no_of_items	Integer: Number of items.
self.item_names self.item_ids	List: List of item names. self.item_ids is an alias for users who find *_ids more natural than *_names.
self.no_of_persons	Integer: Number of persons.
self.person_names self.person_ids	List: List of person names. self.person_ids is an alias for users who find *_ids more natural than *_names.
self.generating	Namespace object (_SimParams): Only present when the model is instantiated by passing a simulation object (e.g. PCM_Sim) as the responses argument. Contains all attributes of the simulation object (self.generating.persons, self.generating.items, etc.), enabling direct comparison of generating and estimated parameters.
self.connectivity_status	Dictionary: result of check_data_connectivity (Section 4.1.2), present only when validate=True. Always contains at least 'connected' (Boolean), 'components_count' (Integer) and 'strongly_connected' (Boolean).
self.exogenous	pandas dataframe or None: person-level covariates passed via the exogenous argument, re-indexed onto self.person_names. None if not supplied.
self.no_exogenous_persons	pandas index: Persons present in the response data with no matching exogenous record.
self.exogenous_only_persons	pandas index: Persons present in the exogenous data but not in the response data.

Examples

To create an object from a dataframe `my_pcm_dataframe`, with four items with maximum scores of 3, 3, 5 and 5, and with 10 observed classes:

```
my_pcm = PCM(my_pcm_dataframe, max_score_vector=[3, 3, 5, 5], no_of_classes=10)
```

To create an object from a dataframe `my_pcm_dataframe`, additionally carrying exogenous variables for DIF analysis loaded into `my_exogenous_df`:

```
my_pcm = PCM(my_pcm_dataframe, max_score_vector=[3, 3, 5, 5], exogenous=my_exogenous_df)
```

4.1.2 check_data_connectivity

Description

Validates whether the item response network is fully connected: two items are connected if at least one person's response differentiates between them. For PCM, the underlying directed matrix is built at item-*category* granularity (one row per item-category, matching calibrate's own matrix – Section 4.3.1) rather than one row per item, since each item can have its own threshold structure. Category-rows are collapsed back to their owning item for the connectivity check, so two items are adjacent if any pair of their categories has a nonzero directed comparison in either direction.

The 'directionally isolated' check is correspondingly at category granularity: an item is flagged if *any* of its categories has a structurally zero row or column – most commonly a category that is never observed at all (e.g. no person ever scores the top category of an item). An item can therefore be flagged here even when it otherwise looks well-observed overall.

Also verifies *strong* connectivity of the directed win-graph – the network in which an edge runs from item i to item j whenever at least one person scored higher on i than on j. The Bradley–Terry log-likelihood estimator (calibrate(method='log-lik')) has a finite, unique solution only when this directed graph is strongly connected (**Ford1957**). When the network is connected but not strongly connected – for example with near-Guttman data or a very short test – a UserWarning names the strongly connected components and notes that method='log-lik' on its default unpowered matrix will not converge for the affected items. The other methods ('cos', 'ls', 'evm'), and 'log-lik' with matrix_power ≥ 1, are unaffected.

Called automatically on object creation when validate=True (the default; see Section 4.1.1), storing its result as self.connectivity_status.

Usage

```
self.check_data_connectivity()
```

Returns

Dictionary: 'connected', 'components_count', 'directionally_isolated_items', 'strongly_connected' always present; if disconnected, 'isolated_items'/'all_sub_groups' also present.

Example

To manually re-check connectivity on object my_pcm and inspect which items, if any, have an unobserved category:

```
my_pcm.connectivity_status = my_pcm.check_data_connectivity()
my_pcm.connectivity_status['directionally_isolated_items']
```

4.1.3 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of `self.responses` with new name.

Example
To rename an item in object `my_pcm` from `Item_1` to `my_new_item_name`:

```
my_pcm.rename_item('Item_1', 'my_new_item_name')
```

4.1.4 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of `self.responses` with new names.

Example
To rename all items in object `my_pcm` with item names in a list stored as a variable `my_new_item_names`:

```
my_pcm.rename_items_all(my_new_item_names)
```

4.1.5 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the index of self.responses with new name. Also replaces person name in self.exogenous or self.no_exogenous_persons as appropriate if self.exogenous exists.

Example
To rename a person in object my_pcm from Person_1 to my_new_person_name:
```
my_pcm.rename_person('Person_1', 'my_new_person_name')
```

4.1.6 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the index of self.responses with new names. Also replaces all person names in self.exogenous and self.no_exogenous_persons if self.exogenous exists.

Example
To rename all persons in object my_pcm with person names in a list stored as a variable my_new_person_names:
```
my_pcm.rename_persons_all(my_new_person_names)
```

4.2 Core functions

4.2.1 cat_prob_centred

Description

Category probability function which calculates the probability $P(X_{ni} = k)$ of scoring k, with $k \in \{0, m\}$, where m is the maximum score, from person location, central item location and Rasch-Andrich thresholds (see Section 4.2.2 below for the uncentred thresholds formulation). For a person n with location β_n attempting an item i with central item location δ_i and Rasch-Andrich thresholds $\{\tau_{i0}, ..., \tau_{im}\}$, the probability of obtaining a score of k is given by:

$$P(X_{ni} = k) = \frac{e^{k(\beta_n - \delta_i) - \sum_{t=0}^{k} \tau_{it}}}{\sum_{k=0}^{m} e^{k(\beta_n - \delta_i) - \sum_{t=0}^{k} \tau_{it}}} \tag{7}$$

In this formulation, an item is defined by a central item location, δ_i and a set of centred Rasch-Andrich thresholds $\{\tau_{ik}\}$, $k \in \{0, ..., m\}$ which sum to zero: an alternative formulation is to define the item solely by a set of uncentred thresholds, $\{\tau'_{ik}\}$, $k \in \{0, ..., m\}$, where $\tau'_{ik} = \delta_i + \tau_{ik}$ for $i = 1, ..., m$ and $\tau'_{ik} = 0$. Separate methods for all core functions are defined using uncentred thresholds.

Usage

```
self.cat_prob_centred(person_location, item_location, category, thresholds)
```

Arguments

person_location	Float: Person location
item_location	Float: Central item location
category	Integer: Response category k, with $k \in \{0, 1\}$.
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Returns

Float: probability of obtaining score k.

Example

To obtain the probability of a person of location 0.5 scoring 0 on an item of central location 0 with a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_cat_prob`:

```
my_cat_prob = self.cat_prob_centred(0.5, 0, 0, my_thresholds)
```

4.2.2 cat_prob_uncentred

Description

Category probability function which calculates the probability $P(X_{ni} = k)$ of scoring k, with $k \in \{0,\ m\}$, where m is the maximum score, from person location and uncentred thresholds (see Section 4.2.1 above for the centred thresholds formulation). For a person n with location β_n attempting an item i with thresholds $\{\tau_{i1}, ...,\ \tau_{im}\}$, the probability of obtaining a score of k is given by:

$$P(X_{ni} = k) = \frac{e^{k\beta_n - \sum_{t=0}^{k} \tau_{it}}}{\sum_{k=0}^{m} e^{k\beta_n - \sum_{t=0}^{k} \tau_{it}}} \tag{8}$$

In this formulation, an item is defined by a set of thresholds $\{\tau_{ik}\}$, $k \in \{1, ..., m\}$: an alternative formulation is to define the item a central item location, δ_i and a set of centred Rasch-Andrich thresholds $\{\tau'_k\}$, $k \in \{0, ..., m\}$ which sum to zero, where $\delta_i = \sum_i = 1^m \tau_{ik}/m$ and $\tau'_{ik} = \tau_{ik} - \delta_i$. Separate methods for all core functions are defined using centred thresholds.

Usage

```
self.cat_prob_uncentred(person_location, category, thresholds)
```

Arguments

person_location	Float: Person location
category	Integer: Response category k, with $k \in \{0,\ 1\}$.
thresholds	List or numpy array: Set of m uncentred thresholds for scores 1 to m, where m is the maximum score.

Returns

Float: probability of obtaining score k.

Example

To obtain the probability of a person of location 0.5 scoring 0 on an item with a set of uncentred thresholds stored as a variable my_thresholds, and store the result as a variable my_cat_prob:

```
my_cat_prob = self.cat_prob_uncentred(0.5, 0, my_thresholds)
```

4.2.3 exp_score_centred

Description

Expected score function which calculates the expected score $E(X_{ni})$ from person location, central item location and set of centred Rasch-Andrich thresholds. The expected score is given by:

$$E(X_{ni}) = \sum_{k=0}^{m} kP(X_{ni} = k) \tag{9}$$

where m is the maximum score for item i and $P(X_{ni} = k)$ is as described in Section 4.2.1.

Usage

```
self.exp_score_centred(person_location, item_location, thresholds)
```

Arguments

person_location	Float: Person location
item_location	Float: Central item location
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Returns

Float: expected score.

Example

To obtain the expected score for a person of location 0.5 attempting an item of location 0 with a set of Rasch-Andrich thresholds stored as a variable my_thresholds, and store the result as a variable my_exp_score:

```
my_exp_score = self.exp_score_centred(0.5, 0, my_thresholds)
```

4.2.4 exp_score_uncentred

Description

Expected score function which calculates the expected score $E(X_{ni})$ from person location and a set of uncentred thresholds. The expected score is given by:

$$E(X_{ni}) = \sum_{k=0}^{m} kP(X_{ni} = k) \tag{10}$$

where m is the maximum score for item i and $P(X_{ni} = k)$ is as described in Section 4.2.2.

Usage

```
self.exp_score_uncentred(person_location, thresholds)
```

Arguments

person_location	Float: Person location
thresholds	List or numpy array: Set of m thresholds, where m is the maximum score.

Returns

Float: expected score.

Example

To obtain the expected score for a person of location 0.5 attempting an item of location 0 with a set of thresholds stored as a variable my_thresholds, and store the result as a variable my_exp_score:

```
my_exp_score = self.exp_score_uncentred(0.5, my_thresholds)
```

4.2.5 `variance_centred`

Description

Variance function which calculates the variance of the score $V(X_{ni})$ from person location, central item location and set of Rasch-Andrich thresholds. The variance is given by:

$$V(X_{ni}) = \sum_{k=0}^{m} P(X_{ni} = k)(k - E(X_{ni}))^2 \tag{11}$$

where m is the maximum score for item i and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 4.2.1 and 4.2.3 respectively.

The variance is also both the Fisher information for the response and the first partial differential of the expected score function with respect to person location.

Usage

```
self.variance_centred(person_location, item_location, thresholds)
```

Arguments

`person_location`	Float: Person location
`item_location`	Float: Central item location
`thresholds`	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Returns

Float: variance (also the Fisher information provided by the response).

Example

To obtain the variance for a person of location 0.5 attempting an item with central location 0 and a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_variance`:

```
my_variance = self.variance_centred(0.5, 0, my_thresholds)
```

4.2.6 variance_uncentred

Description

Variance function which calculates the variance of the score $V(X_{ni})$ from person location and set of uncentred thresholds. The variance is given by:

$$V(X_{ni}) = \sum_{k=0}^{m} P(X_{ni} = k)(k - E(X_{ni}))^2 \tag{12}$$

where m is the maximum score for item i and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 4.2.2 and 4.2.4 respectively.

The variance is also both the Fisher information for the response and the first partial differential of the expected score function with respect to person location.

Usage

```
self.variance_uncentred(person_location, thresholds)
```

Arguments

person_location	Float: Person location
thresholds	List or numpy array: Set of m thresholds, where m is the maximum score.

Returns

Float: variance (also the Fisher information provided by the response).

Example

To obtain the variance for a person of location 0.5 attempting an item with a set of uncentred thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_variance`:

```
my_variance = self.variance_uncentred(0.5, my_thresholds)
```

4.2.7 kurtosis_centred

Description

Kurtosis function which calculates the kurtosis of the score $\kappa(X_{ni})$ from person location, central item location and Rasch-Andrich thresholds. The kurtosis is given by:

$$\kappa(X_{ni}) = \sum_{k=0}^{m} P(X_{ni} = k)(k - E(X_{ni}))^4 \tag{13}$$

where m is the maximum score for item i and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 4.2.1 and 4.2.3 respectively.

Usage

```
self.kurtosis_centred(person_location, item_location, thresholds)
```

Arguments

person_location	Float: Person location
item_location	Float: Central item location
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Returns

Float: kurtosis

Example

To obtain the kurtosis for a person of location 0.5 attempting an item with central location 0 and a set of Rasch-Andrich thresholds stored as a variable my_thresholds, and store the result as a variable my_kurtosis:

```
my_kurtosis = self.kurtosis_centred(0.5, 0, my_thresholds)
```

4.2.8 kurtosis_uncentred

Description

Kurtosis function which calculates the kurtosis of the score $\kappa(X_{ni})$ from person location, central item location and Rasch-Andrich thresholds. The kurtosis is given by:

$$\kappa(X_{ni}) = \sum_{k=0}^{m} P(X_{ni} = k)(k - E(X_{ni}))^4 \tag{14}$$

where m is the maximum score for item i and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 4.2.2 and 4.2.4 respectively.

Usage

```
self.kurtosis_uncentred(person_location, thresholds)
```

Arguments

person_location	Float: Person location
thresholds	List or numpy array: Set of m uncentred thresholds for scores 1 to m, where m is the maximum score.

Returns

Float: kurtosis

Example

To obtain the kurtosis for a person of location 0.5 attempting an item with a set of uncentred thresholds stored as a variable my_thresholds, and store the result as a variable my_kurtosis:

```
my_kurtosis = self.kurtosis_uncentred(0.5, my_thresholds)
```

4.3 Parameter estimation

4.3.1 `calibrate`

Description

Produces central item location estimates and, for each item, its own set of Rasch–Andrich threshold estimates. Uncentred thresholds are produced first, then split by item and mean-centred (Sections 4.2.1 and 4.2.2).

Both come from a single conditional pairwise extraction, with no separate threshold stage. The conditional category response frequency matrix $\mathbf{C}$ is built at threshold granularity – one row and column for every Rasch–Andrich threshold of every item – with $\mathbf{C}_{ij}$ counting persons who scored in the higher category of threshold i (on its item) and the lower category of threshold j (on its item), pooled across persons. Cells linking two thresholds of the same item are held at zero (PAIR compares distinct items).

Three methods are entirely non-iterative and are derived from the positive pairwise reciprocal matrix $\mathbf{D} : \mathbf{D}_{ij} = \mathbf{C}_{ji}^n / \mathbf{C}_{ij}^n$, the ratios of the counts of the $\mathbf{C}_{ij}$ after raising $\mathbf{C}$ to a power n, which introduces indirect comparisons each time $\mathbf{C}$ is raised to a higher power. From the matrix $\mathbf{D}$, the threshold vector is derived in different ways:

- `'ls'` is the least-squares method (Choppin, 1968, 1985), which is the logarithm of the simple arithmetic row-wise mean of $\mathbf{D}$.
- `'evm'` is the eigenvector method (Garner & Engelhard, 2002), which uses the logarithm of the eigenvector corresponding to the principal eigenvalue of $\mathbf{D}$.
- `'cos'` is the cosine maximisation method (Kou & Lin, 2014), which first normalises the columns of $\mathbf{D}$ before taking the logarithms of the row-wise means.

In all these cases, the matrix power n is user-definable.

`'log-lik'` is the iterative maximum likelihood estimator (Bradley & Terry, 1952; Zermelo, 1929), which iteratively maximises the log-likelihood until the change is less than the set tolerance, using `'cos'` for a warm start. `'log-lik'` is the default method because although usually only trivially more accurate, it is highly robust to extremely sparse data, even at small sample sizes, provided the comparison graph is strongly connected (which a non-zero `constant` ensures; see below). `'cos'` and the other non-iterative methods can sometimes produce noticeably worse estimates under such conditions. For well conditioned data, `'cos'` in particular produces practically identical results at a faster speed. `'log-lik'` does not involve raising matrix powers, since it degrades accuracy for `'log-lik'` rather than improving it.

The additive smoothing constant `constant` keeps the comparison matrix free of zero cells (for `'log-lik'`, this is the only useful mechanism to do so since matrix powering degrades performance) and acts as a mild regulariser, particularly for smaller samples: sampling-error zeros or near-zeros can exert an outsized pull on the estimates, which `constant` damps. A non-zero value is recommended; default values were determined through simulation studies.

By default `constant` is `None`, which resolves to a data-adaptive value, $0.1\,N^{0.15}(1-p)^{0.75}\bar{m}^{-1.4}$, where N is the number of persons with at least one response, p is the proportion of missing responses among them and $\bar{m}$ is the mean maximum score across items. It gives less smoothing to designs with more categories or more missing data. With mixed maximum scores the constant is applied per cell of the threshold-pair matrix: cell (i, j) gets $0.1\,N^{0.15}(1-p)^{0.75}(m_i m_j)^{-0.7}$, which reduces to the same formula when all items share one maximum score. The per-cell values are stored as `self.cell_constants`, and `self.constant` records the value given by the formula for a single maximum score at the mean maximum score. `robust` is deprecated and has no effect.

Usage

```
self.calibrate(constant=None, method='log-lik', matrix_power=None, log_lik_tol=0.000001,
               robust=False)
```

Arguments

`constant`	Float or `None`: additive smoothing constant (Elliott & Buttery, 2022b). Default is `constant=None`, which resolves to $0.1\,N^{0.15}(1-p)^{0.75}\bar{m}^{-1.4}$, where N is the number of persons with at least one response, p is the proportion of missing responses among them and $\bar{m}$ is the mean maximum score across items. `constant=0` disables smoothing. The resolved value is stored as `self.constant`.
`robust`	Boolean: deprecated and has no effect; passing `robust=True` raises a `FutureWarning`. It will be removed in a future release. Default is `robust=False`.
`method`	String: method for derivation of vector of central item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are `'ls'` for least squares (Choppin, 1968, 1985), `'evm'` for the eigenvector method(Garner & Engelhard, 2002), `'cos'` for cosine maximisation (Kou & Lin, 2014) or `'log-lik'` for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is `method='log-lik'`.
`matrix_power`	Integer or `None`: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) for central item location estimates is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is `matrix_power=None`, which resolves to `0` (no matrix powering) when `method='log-lik'` or `5` for other methods. Passing `matrix_power=0` with a method other than `'log-lik'` issues a `UserWarning`.
`log_lik_tol`	Float: convergence criterion for log-likelihood method for central item location estimates. Ignored for other methods.

Returns

Returns three attributes:

`self.items`	pandas series: Central item location estimates (mean of uncentred thresholds) with item names as keys and estimates as values.
`self.thresholds`	pandas DataFrame: Centred threshold estimates, one row per item, one column per threshold. Values sum to zero within each item. NaN-padded for items with fewer thresholds than the maximum.
`self.thresholds_uncentred`	pandas DataFrame: Uncentred threshold estimates, one row per item, one column per threshold position. NaN-padded for items with fewer thresholds than the maximum.

Examples

To generate a set of estimates using the default Bradley–Terry log-likelihood method for central item locations, with additive smoothing constant of 0.1:

```
self.calibrate()
```

To generate a set of estimates using the log-likelihood method for central item locations with the pairwise matrix raised to power 7 (overriding the unpowered default) and a tighter convergence criterion:

```
self.calibrate(method='log-lik', matrix_power=7, log_lik_tol=0.00000001)
```

To use the closed-form cosine maximisation method (Kou & Lin, 2014) for central item locations instead of the default:

```
self.calibrate(method='cos')
```

4.3.2 std_errors

Description

Produces standard errors of central item locations, thresholds and category widths. `se_method='sandwich'` computes analytic Huber–White sandwich standard errors (Huber, 1967), with persons as the independent units; `se_method='bootstrap'` draws `no_of_samples` bootstrap resamples of persons. `se_method=None` (default) uses the sandwich, falling back to the bootstrap, with a `UserWarning`, if any item has a score category with no responses.

Usage

```
self.std_errors(interval=None, no_of_samples=500, constant=None, robust=False,
                method='log-lik', matrix_power=None, log_lik_tol=0.000001,
                seed=None, se_method=None)
```

Arguments

`interval`	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval. Defines a central interval of proportion p, e.g. `interval=0.95` defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is `interval=None`.
`no_of_samples`	Integer: Number of bootstrap samples to generate. More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is `no_of_samples=500`.
`constant`	Float or `None`: additive smoothing constant (Elliott & Buttery, 2022b). Default is `constant=None`, which resolves to $0.1\,N^{0.15}(1-p)^{0.75}\bar{m}^{-1.4}$, where N is the number of persons with at least one response, p is the proportion of missing responses among them and $\bar{m}$ is the mean maximum score across items (see Section 4.3.1).
`robust`	Boolean: deprecated and has no effect; passing `robust=True` raises a `FutureWarning`. It will be removed in a future release. Default is `robust=False`.
`method`	String: method for derivation of vector of central item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are `'ls'` for least squares (Choppin, 1968, 1985), `'evm'` for the eigenvector method(Garner & Engelhard, 2002), `'cos'` for cosine maximisation (Kou & Lin, 2014) or `'log-lik'` for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is `method='log-lik'`

Continued on next page...

matrix_power	Integer or None: power to which conditional category response frequency matrix for central item location estimates (Elliott & Buttery, 2022b:991) is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.
seed	Integer or None: seed for the bootstrap resampling random number generator. Pass an integer for fully reproducible standard errors; None (default) draws fresh entropy each call. See Section 1.
se_method	String or None: 'sandwich', 'bootstrap' or None (sandwich where it applies). Default is se_method=None.

Returns

Attributes:

self.se_method	String: 'sandwich' or 'bootstrap', the method used.
self.se_covariance	pandas dataframe or None: sandwich covariance matrix of the uncentred thresholds (sandwich only; None for the bootstrap).
self.central_bootstrap	pandas dataframe: Full central item location bootstrap results, with a row for each bootstrap sample and a column for each central item location estimate.
self.central_se	pandas series: Item names as keys and item standard errors as values.
self.threshold_bootstrap	pandas dataframe: Full bootstrap results, with a row for each bootstrap sample and a column for each threshold estimate. self.threshold_bootstrap contains uncentred threshold estimates (standard errors would be identical with centred thresholds).
self.threshold_se	pandas series: Threshold numbers as keys and threshold standard errors as values.
self.cat_width_bootstrap	pandas dataframe: Full bootstrap results, with a row for each bootstrap sample and a column for each category width estimate.
self.cat_width_se	pandas series: Category numbers as keys and category width standard errors as values.

If an argument is passed to interval, also returns:

`self.item_low`	Lower bound of the specified interval for item estimates.
`self.item_high`	Upper bound of the specified interval for item estimates.
`self.threshold_low`	Lower bound of the specified interval for threshold estimates.
`self.threshold_high`	Upper bound of the specified interval for threshold estimates.
`self.cat_width_low`	Lower bound of the specified interval for category estimates.
`self.cat_width_high`	Upper bound of the specified interval for category estimates.

Example

To generate item standard errors with a 95% interval from 200 samples: `self.std_errors(interval=0.95, no_of_samples=200)`

Modifications to the estimation method are discussed in Section 4.3.1.

4.3.3 calibrate_anchor

Description

Anchors item location estimates onto externally-supplied reference values, supporting item banking – item locations are calibrated as usual, then the item-location scale is shifted by a translation constant (by default via robust modified z-score outlier trimming (Iglewicz & Hoaglin, 1993)) so that a subset of common anchor items line up with externally-supplied reference locations. This is a mean-shift translation only; each item's own centred threshold offsets (`self.thresholds`) are left untouched by the anchoring process.

Optionally, if reference threshold structures `anchor_thresholds` are also supplied for some anchor items (thresholds are not needed for every anchor item), likelihood-ratio tests check whether the reference thresholds are compatible with the estimated thresholds. This can be run per item (`threshold_check='per_item'`, one independent test per supplied item, BH/Bonferroni-corrected for multiple comparisons (Benjamini & Hochberg, 1995)), or as a single combined test across every supplied item at once (`threshold_check='aggregate'`), or both (default).

Usage

```
self.calibrate_anchor(anchors, calibrate=False, selection_method='robust_z',
                      corr_tol=0.95, sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05,
                      no_of_samples=500, adj=None, overwrite_anchors='none',
                      plot=True, plot_kwargs=None, anchor_thresholds=None,
                      threshold_check='both', alpha=0.05, correction='bh',
                      warm_corr=True, tolerance=0.00001, max_iters=100,
                      ext_score_adjustment=0.5, constant=None, robust=False, method='log-lik',
                      matrix_power=None, log_lik_tol=0.000001, seed=None)
```

Arguments

`anchors`	Dictionary or pandas series: reference locations for a subset of items, keyed/indexed by item name.
`calibrate`	Boolean: if `True`, (re-)runs `calibrate` before anchoring. If `False`, `calibrate` is still auto-triggered if it has not already been run. Default is `calibrate=False`.
`selection_method`	String: `'robust_z'`, `'wald'` or `'none'`. `'robust_z'` (default): excludes outlying anchor items via modified z-score procedure (Iglewicz & Hoaglin, 1993); uses `corr_tol`, `sd_ratio_tol`, `min_anchors`. `'wald'`: formal per-item sequential significance test based on z-scores from item standard errors – auto-triggers `std_errors` if not already computed; uses `wald_alpha` and `min_anchors`, not `corr_tol`/`sd_ratio_tol`. `'none'`: plain mean shift over every supplied anchor, no trimming.

Continued on next page...

corr_tol	Float: minimum acceptable Pearson correlation between anchor and observed values on the surviving (non-outlier) item set (selection_method='robust_z' only). Default is corr_tol=0.95.
sd_ratio_tol	Float: the ratio of observed to anchor standard deviations on the surviving item set must fall within $[1/\texttt{sd_ratio_tol}, \texttt{sd_ratio_tol}]$ (selection_method='robust_z' only). Default is sd_ratio_tol=1.1, i.e. the ratio must lie in $[0.909, 1.1]$.
min_anchors	Integer: floor on the number of surviving anchor items – selection stops (with a warning) once this floor is reached, even if the active method's tolerance/significance criterion is not yet satisfied, and is skipped altogether if the initial common item set is at or below this floor (selection_method in 'robust_z'/'wald' only). Default is min_anchors=6.
wald_alpha	Float: significance level for each anchor item's Wald test (selection_method='wald' only). Default is wald_alpha=0.05.
no_of_samples	Integer: bootstrap samples for the auto-triggered std_errors call, only used if item standard errors are not already computed (selection_method='wald' only). Default is no_of_samples=500.
adj	Float or None: if provided, applied as translation constant and selection step skipped. Intended for reuse across bootstrap resamples (Section 4.3.2), where recomputing anchor selection every time would inflate standard errors with anchor-item sampling variance. Default adj=None.
overwrite_anchors	String, one of 'none', 'rejected' or 'all': controls what self.anchor_items holds for the anchor items themselves, as opposed to non-anchor items, which always receive the shifted (calibrated + translation constant) value. 'none': every anchor item keeps exactly its externally-supplied value, unchanged – the usual convention, since an anchor's value is fixed by definition, not re-estimated. 'rejected': retained anchors keep their supplied values, but rejected anchors have their values overwritten with new, freshly-calibrated values; requires selection_method in 'robust_z'/'wald' and no adj override (otherwise falls back to 'none' with a warning). 'all': every anchor item has its value overwritten with a new, freshly-calibrated value like any other item. Default is overwrite_anchors='none'.
plot	Boolean: if True and selection_method in 'robust_z'/'wald' (so a selection table exists), calls plot_anchor_selection (Section 3.5.10). Has no effect when selection_method='none' or adj is supplied. Default plot=True.

Continued on next page...

plot_kwargs	Dictionary or None: extra keyword arguments forwarded to plot_anchor_selection when plot=True (e.g. filename, xmin/xmax, title). Default is plot_kwargs=None.
anchor_thresholds	Dictionary or None: {item_name: array-like} of externally-supplied reference threshold offsets, matching the shape of self.thresholds for that item (that item's own centred category-structure shape, not an absolute/uncentred value). Only items present in this dictionary are checked; need not cover every anchor item. If None, the threshold-structure check is skipped entirely regardless of threshold_check. Default is anchor_thresholds=None.
threshold_check	String: one of 'per_item', 'aggregate', 'both' or 'none'. 'per_item' runs one independent LR test per item in anchor_thresholds (df = that item's max score -1 each), correction-adjusted. 'aggregate' runs a single combined LR test swapping every supplied item's thresholds in simultaneously (df = sum of each item's max score -1), needing no multiple-comparison correction since it is a single test. 'both' runs both tests. 'none' skips the checks. Default is threshold_check='both'
alpha	Float: significance level for all LR tests (compared against the corrected p-value for per-item tests, raw p for the aggregate test). Default is alpha=0.05.
correction	String or None, one of 'bh', 'bonferroni' or None: multiple-comparison correction across the per-item tests only – Benjamini-Hochberg false discovery rate control (Benjamini & Hochberg, 1995) or Bonferroni. None uses raw, uncorrected p-values. Ignored for the aggregate test. Default is correction='bh'.
warm_corr, tolerance, max_iters, ext_score_adjustment	Person-estimation arguments, used only by the threshold_check comparison (locations must be re-estimated under each candidate threshold table before the log-likelihoods are comparable), or if calibrate/std_errors (selection_method='wald') triggered.
constant, method, matrix_power, log_lik_tol	Calibration/bootstrap arguments, used only if calibrate or std_errors (selection_method='wald') triggered – see Section 4.3.1.
seed	Integer or None: seed passed through to the internal std_errors call (only used if item standard errors are not already computed, selection_method='wald' only). None (default) draws fresh entropy each call. See Section 1.

Returns

self.anchor_items	pandas series: item locations shifted onto the anchor scale.
self.anchor_item_names	pandas index: names of the items supplied as anchors.
self.anchor_adj	Float: the translation constant actually applied.
self.anchor_selection	pandas dataframe or None: per-item diagnostics from whichever selection method was used – Anchor, Observed, Deviation, Robust z, Selected ('robust_z'), or Anchor, Observed, SE, Deviation, z, p, Selected ('wald'). None if selection_method='none' or adj supplied.
self.anchor_selected_items, self.anchor_dropped_items	pandas index or None: items retained / excluded as outliers. None if selection_method='none' or adj supplied.
self.anchor_original_corr, self.anchor_original_sd_ratio	Float or None: correlation / SD ratio before trimming.
self.anchor_corr self.anchor_sd_ratio	Float or None: correlation / SD ratio after trimming.
self.anchor_summary	pandas series: one-line summary – anchors supplied/common/selected/dropped, correlation and SD ratio before/after trimming, translation constant applied.
self.anchor_plot	matplotlib figure or None: figure from the auto-triggered plot_anchor_selection call. None if plot=False, selection_method='none', or adj was supplied directly.
self.calibrate_anchor_runs	Dictionary: every call's results above, keyed by the sorted anchor items supplied, so results from an earlier call survive a later call with a different anchor set instead of being overwritten in place.

The threshold-structure check additionally sets:

self.anchor_threshold_test	pandas dataframe or None: per-item LR test table (LL_estimated, LL_given, LR, df, p, p_corrected, Flagged), indexed by item. None if anchor_thresholds not supplied, threshold_check='none', or threshold_check='aggregate' only.
self.anchor_threshold_test_aggregate	pandas series or None: combined LR test across every item in anchor_thresholds at once (LL_estimated, LL_given, LR, df, p, Flagged). None if anchor_thresholds not supplied, threshold_check='none', or threshold_check='per_item' only.

Examples

To anchor item locations onto a bank of reference values stored in a dictionary `my_bank`, with automatic outlier trimming and diagnostic plot:

```
self.calibrate_anchor(my_bank)
```

To additionally check compatibility with reference threshold structures for a subset of items, stored as `my_bank_thresholds`, using only the per-item test with Bonferroni correction:

```
self.calibrate_anchor(my_bank, anchor_thresholds=my_bank_thresholds,
                      threshold_check='per_item', correction='bonferroni')
```

4.3.4 person

Description

Generates a location estimate for a person using the Newton-Raphson method to produce a maximum likelihood estimate, or, by default, Warm's weighted likelihood estimate (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.person(persons, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
            ext_score_adjustment=0.5, missing_as_incorrect=False)
```

Arguments

persons	String or list: person name(s) for the location(s) being estimated. Pass 'all' to estimate for all persons in self.responses.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations. Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
missing_as_incorrect	Boolean: if True, treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is missing_as_incorrect=False.

Returns

pandas series: person location estimate(s), in logits, indexed by person identifier. Returns `NaN` for any person for whom estimation fails.

Examples

To generate a person location estimate for `Person_1` using the default settings and store the result as a variable, `my_person_location`:

```
my_person_location = my_pcm.person('Person_1')
```

To generate an MLE person location estimate without Warm bias correction for `Person_1` based on the first three items and store the result as a variable, `my_person_location`:

```
my_person_location = my_pcm.person('Person_1', ['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

4.3.5 person_estimates

Description

Generates location estimates for all persons using the Newton-Raphson method to produce maximum likelihood estimates, or, by default, Warm's weighted likelihood estimates (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.person_estimates(items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
                      ext_score_adjustment=0.5, missing_as_incorrect=False)
```

Arguments

items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations. Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
missing_as_incorrect	Boolean: if True, treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is missing_as_incorrect=False.

Returns

Attribute self.persons: pandas series with person names as keys and location estimates as values.

Examples

To generate a set of person location estimates with Warm bias correction:

```
self.person_estimates()
```

To generate a set of person location estimates without Warm bias correction, on a subset of the first three items only:

```
my_pcm.person_estimates(items=['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

4.3.6 score_lookup

Description

Generates a location estimate for a given raw score on responses to a given set of items using the Newton-Raphson method to produce a maximum likelihood estimate, or, by default, Warm's weighted likelihood estimate (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.score_lookup(score, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
                  ext_score_adjustment=0.5)
```

Arguments

score	Integer: The raw score for which location is being estimated.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations. Default max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.

Returns

pandas series with raw scores as keys and person location estimates as values.

Examples

To generate a location estimate for a score of 10 on all items, with Warm bias correction, and store the result as a variable, my_score_location:

```
my_score_location = my_pcm.score_lookup(10)
```

To generate a location estimate for a score of 10 on a subset of items saved as a variable `my_items`, without Warm bias correction, and store the result as a variable, `my_score_location`:

```
my_score_location = my_pcm.score_lookup(10, items=my_items, warm_corr=False)
```

4.3.7 score_lookup_table

Description

Generates a lookup table of location estimates corresponding to all available raw scores on a set of items with no missing responses, using the Newton-Raphson method to produce maximum likelihood estimates, or, by default, Warm's weighted likelihood estimates (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.score_lookup_table(items=None, ext_scores=True, warm_corr=True, tolerance=0.00001,
                        max_iters=100, ext_score_adjustment=0.5,
                        filename=None, file_format='csv')
```

Arguments

items	List: List of items names on which to generate estimates. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
ext_scores	Boolean: If True, location estimates for extreme scores (all correct/all incorrect) generated using ext_score_adjustment. Default ext_scores=True.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations. Default max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute `self.score_table`: pandas series with raw scores as keys and corresponding location estimates as values.

Examples

To generate a location lookup table for all items, including extreme scores, with Warm bias correction:

```
my_pcm.score_lookup_table()
```

To generate a location lookup table for a subset of items saved as a variable `my_items)`, without extreme scores and without Warm bias correction:

```
my_pcm.score_lookup_table(items=my_items, ext_scores=False)
```

4.3.8 csem

Description

Calculates conditional standard error of measurement for one or more persons.

Usage

```
self.csem(persons=None, person_locations=None, items=None)
```

Arguments

persons	List or string: person name(s). Overrides person_locations if provided. Default is persons=None, which uses all persons.
person_locations	pandas series, float, list, numpy array, or dictionary, with person names as keys (where applicable) and locations as values. If None, uses self.persons, automatically generating via self.person_estimates() if necessary. A raw location value (or values not indexed by a real person, e.g. a raw score lookup) is treated as hypothetical: since there is no observed response data for it, all items in items are treated as answered. Default is person_locations=None.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.

Returns

pandas series: conditional standard error of measurement in logits for each person (or location), indexed by person name (or by a location label, for hypothetical locations).

Examples

To generate the CSEM for Person_1 on all items and save the result as a variable, my_csem:

```
my_csem = my_pcm.csem('Person_1')
```

To generate the CSEM for a raw score of 3 on a subset of items saved to a variable my_items and save the result as a variable, my_csem:

```
my_csem = my_pcm.csem(3, person_locations=self.score_table, items=my_items)
```

where self.score_table is the output from self.score_lookup_table(items=my_items) (see Section 4.3.7).

4.4 Statistical output

4.4.1 item_stats_df

Description

Produces a table of item statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

```
self.item_stats_df(full=False, zstd=False, point_measure_corr=False, dp=3, se=True,
                   warm_corr=True, tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                   method='log-lik', constant=None, robust=False, matrix_power=None,
                   log_lik_tol=0.000001, no_of_samples=500, interval=None, seed=None,
                   filename=None, file_format='csv')
```

Arguments

full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a summary table with the most commonly reported statistics.
zstd	Boolean: If True, infit and outfit standardised z-scores are reported. Default is zstd=False. Automatically True if full=True.
point_measure_corr	Boolean: If True, point-polyserial correlation (Kornbrot, 2014) between person location and item score is reported, together with the expected value of the point-measure polyserial correlation for an ideal item. Default is point_measure_corr=False. Automatically True if full=True.
dp	Integer: Number of decimal places reported in output.
se	Boolean: If True, computes and reports the SE column (and CI bound columns, if interval is set). If False, skips the bootstrap entirely – useful when only Infit MS/Outfit MS are needed (e.g. repeated simulation runs), since those do not depend on the bootstrap and are unaffected by this flag. Forces interval=None when False. Default is se=True.
warm_corr	Boolean: If True, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 4.3.4). Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is max_iters=100.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure an item location estimate is returned if a person has an extreme score (all responses correct or incorrect). Since there is no finite ML location estimate for extreme scores, adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimation. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 4.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation. Default is constant=None, which resolves to $0.1\,N^{0.15}(1-p)^{0.75}\bar{m}^{-1.4}$, where N is the number of persons with at least one response, p is the proportion of missing responses among them and $\bar{m}$ is the mean maximum score across items (see Section 4.3.1).
robust	Boolean: deprecated and has no effect; passing robust=True raises a FutureWarning. It will be removed in a future release. Default is robust=False.
matrix_power	Integer or None: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of item location estimates. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 4.3.1). Ignored for other methods.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 4.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Unused if se=False. Default is no_of_samples=500.

Continued on next page...

`interval`	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 4.3.2). Defines a central interval of proportion p, e.g. `interval=0.95` defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Ignored if `se=False`. Default is `interval=None`.
`seed`	Integer or `None`: seed passed through to the internal `std_errors`/`fit_statistics` calls (only used if not already computed). `None` (default) draws fresh entropy each call. See Section 1.
`filename`	String: The filename for the saved table, with no suffix for format. If `None`, no file will be saved. Default is `filename=None`.
`file_format`	The format of the file: `csv` or `xlsx`. Default is `file_format=csv`.

Returns

Returned and stored as attribute `self.item_stats`, a pandas dataframe with one row for each item and the following columns:

`Estimate`	Central item location estimate.
`SE`	Bootstrapped standard error of central item location estimate. Only produced if `se=True`.
`2.5%`	Lower bound of empirical bootstrap estimate interval. Column name will change to fit `interval` value: 2.5% represents the lower bound for `interval=0.95` but, for example, `interval=0.9` will produce a column labelled `5%`. Only produced if `se=True` and `interval` is set.
`97.5%`	Upper bound of empirical bootstrap estimate interval. Column name will change to fit `interval` value: 97.5% represents the lower bound for `interval=0.95` but, for example, `interval=0.9` will produce a column labelled `95%`. Only produced if `se=True` and `interval` is set.
`Count`	Count of responses.
`Facility`	Item facility: proportion of correct responses.
`Infit MS`	Infit mean square.
`Infit Z`	Standardised infit z-score. Only produced if `zstd=True`.
`Outfit MS`	Outfit mean square.
`Outfit Z`	Standardised outfit z-score. Only produced if `zstd=True`.
`PM corr`	Point-polyserial correlation. Only produced if `point_measure_corr=True`.
`Exp PM corr`	Expected value of point-measure polyserial correlation. Only produced if `point_measure_corr=True`.
`Disordered`	If `True`, threshold estimates indicate the presence of disordered thresholds within the item (Andrich, 2010).

Examples

To produce a summary `item_stats` table with the most commonly reported statistics:

```
my_pcm.item_stats_df()
```

To produce a full `item_stats` table with all statistics:

```
my_pcm.item_stats_df(full=True)
```

To produce an `item_stats` table with with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
my_pcm.item_stats_df(zstd=True)
```

Other arguments may be used to alter parameters of central item location and/or person location estimation.

4.4.2 threshold_stats_df

Description

Produces a table of threshold statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

```
self.threshold_stats_df(full=False, zstd=False, disc=False, point_measure_corr=False, dp=3,
                        warm_corr=True, tolerance=0.00001, max_iters=100,
                        ext_score_adjustment=0.5, method='log-lik', constant=None,
                        robust=False, no_of_samples=500, interval=None,
                        filename=None, file_format='csv')
```

Arguments

full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a table with the most commonly reported statistics.
zstd	Boolean: If True, infit and outfit standardised z-scores are reported. Default is zstd=False. Automatically True if full=True.
disc	Boolean: If True, item discrimination is reported. The discrimination of the empirical threshold slope, which functions as an SLM item conditioned on responses being in the categories either side, relative to the ideal logistic ogive, with 1 perfect, > 1 showing overfit and < 1 showing underfit; discrimination is similar to the 2PL IRT discrimination parameter (Linacre, 2023), but is a descriptive statistic here rather than an item parameter.
point_measure_corr	Boolean: If True, point-biserial correlation between person location and score $k-1$ versus k is reported, together with its expected value for an ideal item. Default is point_measure_corr=False. Always True if full=True.
dp	Integer: Number of decimal places reported in output.
warm_corr	Boolean: If True, Warm's (1989) bias correction is applied during generation of person location estimates (see Section 4.3.4). Default is warm_corr=True.
tolerance	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations during generation of person estimates (Section 4.3.4). Default max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a person location estimate is returned if a person has an extreme score (all responses correct or incorrect). Adjusts the score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimation.

Continued on next page...

method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 4.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation. Default is constant=None, which resolves to $0.1\,N^{0.15}(1-p)^{0.75}\bar{m}^{-1.4}$, where N is the number of persons with at least one response, p is the proportion of missing responses among them and $\bar{m}$ is the mean maximum score across items (see Section 4.3.1).
robust	Boolean: deprecated and has no effect; passing robust=True raises a FutureWarning. It will be removed in a future release. Default is robust=False.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 4.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is no_of_samples=500.
interval	Float. Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is interval=None.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Two attributes (the centred table is also returned): self.threshold_stats and self.threshold_stats_uncentred, pandas dataframes which differ only in their first column, Estimate. The dataframes have one row for each individual threshold and the following columns:

Estimate	Threshold location estimate. For the threshold_stats table, this contains the estimates of the centred Rasch-Andrich thresholds, while for the threshold_stats_uncentred table, it contains the estimates of the uncentred thresholds.
SE	Bootstrapped standard error of item location estimate.

Continued on next page...

2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 2.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 5%.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 97.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 95%.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if zstd=True.
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if zstd=True.
Discrim	Item discrimination. Only produced if disc=True.
PM corr	Point-biserial correlation. Only produced if point_measure_corr=True.
Exp PM corr	Expected value of point-measure biserial correlation. Only produced if point_measure_corr=True.

Examples

To produce summary threshold_stats tables with the most commonly reported statistics:

```
self.threshold_stats_df()
```

To produce a full my_pcm.threshold_stats table with all statistics:

```
my_pcm.threshold_stats_df(full=True)
```

To produce threshold_stats tables with with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
my_pcm.threshold_stats_df(zstd=True)
```

Other arguments may be used to alter parameters of central item location, threshold and person location estimation.

4.4.3 category_stats_df

Description

Produces a table of category width statistics – threshold_stats_df (Section 4.4.2) reports threshold locations; this reports the difference between consecutive thresholds for each item, $\tau_{k+1} - \tau_k$. Each item's zero-summed threshold vector has only max_score_vector[item] −1 true degrees of freedom, so the per-threshold standard errors in threshold_stats_df are correlated rather than independent, and understate the true amount of error. Widths are identical whether computed from centred or uncentred thresholds, so there is only one category_stats table. No infit/outfit or other fit statistics are reported, since these apply to thresholds rather than categories. Auto-triggers calibrate (Section 4.3.1) and std_errors (Section 4.3.2) directly if not already run.

A category width can be negative – a negative width at category k means thresholds τ_k and τ_{k+1} are disordered for that item (category k is never the most likely response at any person location). Prop disordered makes this a continuous diagnostic rather than a straightforward yes/no: the proportion of bootstrap resamples in which that item's category width was negative, reasonably read as the probability that the true category is disordered.

Usage

```
self.category_stats_df(dp=3, constant=None, robust=False, method='log-lik', matrix_power=None,
                       log_lik_tol=0.000001, no_of_samples=500, interval=None, seed=None,
                       filename=None, file_format='csv')
```

Arguments

dp	Integer: Number of decimal places reported in output. Default is dp=3.
constant, method, matrix_power, log_lik_tol	Calibration keyword arguments, used only if calibrate (Section 4.3.1) has not already been run. See Section 4.3.1 for details of each.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates, used only if std_errors (Section 4.3.2) has not already been run. Default is no_of_samples=500.
interval	Float. Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is interval=None.
seed	Integer or None: seed passed through to std_errors, used only if not already run. None (default) draws fresh entropy each call. See Section 1.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute `self.category_stats`, a pandas dataframe with a (item, category) multiindex – one fewer row per item than `threshold_stats`, since a width is defined between each pair of consecutive thresholds – and the following columns:

`Estimate`	Category width (can be negative).
`SE`	Bootstrapped standard error of the width, differenced *within* each bootstrap resample before taking the standard deviation, so it correctly reflects the covariance between adjacent thresholds rather than naively combining their individual standard errors.
`2.5%` / `97.5%`	Bounds of the empirical bootstrap estimate interval, if `interval` is available (see `threshold_stats_df`, Section 4.4.2, for the column-naming convention).
`Disordered`	Boolean: `True` if `Estimate` is negative.
`Prop disordered`	Proportion of bootstrap resamples in which this category's width was negative – reasonably read as the probability that the true category is disordered. Useful for interpreting `Disordered` in both directions: a `True` that is only weakly supported (`Prop disordered` close to 0.5) versus robust, or a `False` that is nonetheless uncertain versus clear-cut.

Examples

To produce the `my_pcm.category_stats` table, alongside the threshold-level table:

```
my_pcm.threshold_stats_df()
my_pcm.category_stats_df()
```

To check which categories, if any, are disordered:

```
my_pcm.category_stats_df()
my_pcm.category_stats[my_pcm.category_stats['Disordered']]
```

4.4.4 person_stats_df

Description

Produces a table of person statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

```
self.person_stats_df(full=False, rsem=False, dp=3, se=True, warm_corr=True, tolerance=0.00001,
                     max_iters=100, ext_score_adjustment=0.5, method='log-lik', constant=None,
                     robust=False, filename=None, file_format='csv')
```

Arguments

full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a summary table with the most commonly reported statistics.
rsem	Boolean: If True, realistic standard error of measurement (RSEM), which takes into account item fit (Wright, 1996), is reported alongside the conditional standard error of measurement (CSEM). Default is rsem=False. Automatically True if full=True.
dp	Integer: Number of decimal places reported in output.
se	Boolean: passed through to the internal fit_statistics call (only used if not already computed). If False, skips the bootstrap entirely – this table's own columns (CSEM, RSEM, Infit/Outfit) do not depend on it, so se=False is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is se=True.
warm_corr	Boolean: If True, Warm's (1989) bias correction is applied during generation of person estimates (see Section 4.3.4). Default warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is max_iters=100.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 4.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 4.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation. Default is constant=None, which resolves to $0.1\,N^{0.15}(1-p)^{0.75}\bar{m}^{-1.4}$, where N is the number of persons with at least one response, p is the proportion of missing responses among them and $\bar{m}$ is the mean maximum score across items (see Section 4.3.1).
robust	Boolean: deprecated and has no effect; passing robust=True raises a FutureWarning. It will be removed in a future release. Default is robust=False.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 4.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is no_of_samples=500.
interval	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 4.3.2). Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is interval=None.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.person_stats, a pandas dataframe with one row for each person and the following columns:

`Estimate`	Item location estimate.
`CSEM`	Conditional standard error of measurement for person location estimate.
`RSEM`	Realistic standard error of measurement for person location estimate. Only produced if `rsem=True`
`Score`	Number of correct responses.
`Max score`	Maximum available score (number of items attempted).
`p`	Proportion of correct repsonses.
`Infit MS`	Infit mean square.
`Infit Z`	Standardised infit z-score.
`Outfit MS`	Outfit mean square.
`Outfit Z`	Standardised outfit z-score.

Examples

To produce a summary `person_stats` table with the most commonly reported statistics:

```
my_pcm.person_stats_df()
```

To produce a full `self.person_stats` table with all statistics:

```
my_pcm.person_stats_df(full=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

4.4.5 test_stats_df

Description

Produces a table of test-level statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX:

`Separation ratio` (Wright, 1996; Wright & Masters, 1982) is the SD of person locations reported as a ratio of SE units. For persons, $G_p = \sigma_p / \sqrt{\sum_n SE_n^2}$, where σ_p is the variance of the person estimates and SE_n is the RSEM (see Section 6.4.6) for person n. The formula is symmetrical for items, substituting the SE of estimation for RSEM.

`Strata` is the number of statistically distinct levels of either person or item locations as strata with centers three measurement errors apart (Wright & Masters, 1982:106). For persons, $H_p = (4G_p + 1)/3$, with symmetrical results for items.

`Reliability` is Rasch-specific reliability statistic (Wright, 1996), derived from PSI. reliability is similar to Cronbach's Alpha (Cronbach, 1951), and may be interpreted the same way: the proportion of variance of the estimates which stems from variation in location or location rather than estimation error. For persons, $R_p = G_p^2/(1 + G_p^2)$, with symmetrical results for items.

`Cronbach alpha` (Cronbach, 1951) (optional) is the most common classical test theory reliability statistic, computed directly from raw response data rather than derived from Rasch estimates.

Usage

```
self.test_stats_df(dp=3, warm_corr=True, tolerance=0.00001, max_iters=100,
                   ext_score_adjustment=0.5, method='log-lik', constant=None, robust=False, alpha=False,
                   seed=None,
                   filename=None, file_format='csv')
```

Arguments

`dp`	Integer: Number of decimal places reported in output.
`warm_corr`	Boolean: If `True`, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 4.3.4). Default is `warm_corr=True`.
`tolerance`	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is `tolerance=0.00001`.
`max_iters`	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is `max_iters=100`.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 4.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 4.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation. Default is constant=None, which resolves to $0.1\,N^{0.15}(1-p)^{0.75}\bar{m}^{-1.4}$, where N is the number of persons with at least one response, p is the proportion of missing responses among them and $\bar{m}$ is the mean maximum score across items (see Section 4.3.1).
robust	Boolean: deprecated and has no effect; passing robust=True raises a FutureWarning. It will be removed in a future release. Default is robust=False.
alpha	Boolean: if True, adds a Cronbach alpha row to the Persons column of the returned table (the Items and Thresholds columns are left blank, since Cronbach's alpha (Cronbach, 1951) is a person-side reliability statistic). Computed on complete cases; if the data contain missing responses, a warning is raised and alpha is computed after listwise deletion, which may underestimate the true value. Default is alpha=False.
seed	Integer or None: seed passed through to the internal fit_statistics call (only used if not already computed). None (default) draws fresh entropy each call. See Section 1.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.test_stats, a pandas dataframe with three columns, Items, Thresholds and Persons and rows for a range of descriptive statistics describing the distributions of

the estimates and different statistics related to reliability – these statistics describe the suitability of the data for estimating and differentiating the parameters, rather than properties of the parameters themselves. The statistics are:

`Mean`	Mean of estimates.
`SD`	Standard deviation of estimates.
`Separation ratio`	Separation ratio
`Strata`	Number of strata
`Reliability`	Reliability
`Cronbach alpha`	Cronbach's alpha

Example

To produce a `my_pcm.test_stats` table:

```
my_pcm.test_stats_df()
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

4.4.6 model_selection

Description

Compares PCM against RSM – RSM is the constrained (nested) model, sharing one threshold structure across all items; PCM is the unconstrained model, giving each item its own threshold structure – via a likelihood ratio test, AIC, or BIC, to help decide which model is the more appropriate, parsimonious fit for a given dataset. Requires uniform maximum scores across items (a precondition for RSM itself to apply at all). Internally constructs and calibrates an RSM object from the same response data for comparison; auto-triggers calibrate (Section 4.3.1) and person (Section 4.3.4) on self directly if not already run.

Usage

```
self.model_selection(test='AIC', aic_sig_test=True, alpha=0.05, min_effect=0,
                     sampling='dynamic', warm_corr=True, tolerance=0.00001, max_iters=100,
                     ext_score_adjustment=0.5, constant=None, robust=False, method='log-lik',
                     log_lik_tol=0.000001, seed=None)
```

Arguments

test	String: 'LR', 'AIC', or 'BIC'. Default is test='AIC'.
aic_sig_test	Boolean: When test='AIC', if True, applies a significance test with RSM as the null hypothesis. The p-value is the relative likelihood of RSM vs PCM, $p = e^{-\Delta/2}$ where $\Delta = \text{AIC}_{\text{RSM}} - \text{AIC}_{\text{PCM}}$. PCM is preferred only if $p <$ alpha; otherwise RSM is retained as the more parsimonious default. Default is aic_sig_test=True.
alpha	Float: significance level for test='LR' or test='AIC' with aic_sig_test=True. Default is alpha=0.05.
min_effect	Float: minimum effect size, in logits, required to prefer PCM over RSM. Optional; 0.43 and 0.64 mirror the ETS DIF categories. Default is min_effect=0 (no minimum).
sampling	None, 'dynamic', or Integer: Subsamples T non-extreme persons before computing log-likelihoods: at very large N, the LR/AIC/BIC comparison becomes biased toward the more complex (PCM) model, since log-likelihood differences scale with N; capping the effective N mitigates this. An integer defines T.'dynamic' uses $T = \min(20(I-1)(m-1), 1500)$, where I is the number of items and m the shared maximum score; subsampling is skipped whenever the actual non-extreme sample size is $\leq T$. None applies no subsampling. Default is sampling='dynamic'.
warm_corr, tolerance, max_iters, ext_score_adjustment	Person location estimation keyword arguments, used only if person (Section 4.3.4) has not already been run on self. Always used for the internally-constructed RSM comparison object. See Section 4.3.4 for details of each.

Continued on next page...

constant, method, log_lik_tol	Calibration keyword arguments, used only if calibrate (Section 4.3.1) has not already been run on self. Always used for the internally-constructed RSM comparison object. See Section 4.3.1 for details of each.
seed	Integer or None: seed for the person-subsampling random number generator, only used when sampling triggers. Pass an integer for a reproducible comparison; None (default) draws fresh entropy. See Section 1.

Returns

Sets whichever of the following attributes correspond to test (all named from the RSM-vs-PCM comparison regardless of which class this method is called from, since the comparison itself is symmetric):

self.model_comparison_rsm_pcm_lr, _df, _p, _lr_preferred, _lr_summary	Likelihood ratio statistic, degrees of freedom, p-value, the preferred model as a string (RSM is the null; PCM is preferred if $p <$ alpha), and a one-line pandas Series summary (including a Preferred row). Only set if test='LR'.
self.model_comparison_rsm_pcm_aic, _aic_preferred, _aic_summary	AIC values for both models, the preferred model as a string, and a summary Series. Only set if test='AIC'.
self.model_comparison_rsm_pcm_aic_p	The AIC relative-likelihood p-value described above. Only set if test='AIC' and aic_sig_test=True.
self.model_comparison_rsm_pcm_bic, _bic_preferred, _bic_summary	BIC values for both models, the preferred model as a string, and a summary Series. Only set if test='BIC'.

Examples

To compare my_pcm against the equivalent RSM using the default AIC significance test:

```
my_pcm.model_selection()
my_pcm.model_comparison_rsm_pcm_aic_summary
```

To instead run a likelihood ratio test, with a fixed subsampling size of 800 persons to guard against large-N bias:

```
my_pcm.model_selection(test='LR', sampling=800)
```

4.4.7 global_fit

Description

Wald test of invariance of the uncentred thresholds between two groups of persons, each calibrated separately, using sandwich covariances. `method='andersen-lr'` is not yet available.

Usage

```
self.global_fit(method='wald', split='score', covariate=None, welch=True, alpha=0.05, seed=None,
                    split_prop=0.5, se_correction='kauermann-carroll')
```

Arguments

`method`	String: `'wald'` or `'andersen-lr'` (raises `NotImplementedError`). Default is `method='wald'`.
`split`	String: `'score'` (median mean score), `'person_location'`, `'random'` or `'exogenous'`. Default is `split='score'`.
`covariate`	String or `None`: column of `self.exogenous` for `split='exogenous'`. Default is `covariate=None`.
`split_prop`	Float: proportion of persons in each outer group for `'score'` or `'person_location'` splits (0.5 = median split). Default is `split_prop=0.5`.
`welch`	Boolean: refer W/df to an F distribution with denominator degrees of freedom from the smaller group; otherwise W to χ^2. Default is `welch=True`.
`alpha`	Float: significance level. Default is `alpha=0.05`.
`seed`	Integer or `None`: seed for `split='random'`. Default is `seed=None`.
`se_correction`	String or `None`: small-sample correction of the sandwich covariances, `'kauermann-carroll'` (Kauermann & Carroll, 2001) or `None`. Default is `se_correction='kauermann-carroll'`.

Returns

Attributes:

`self.global_fit_summary`	pandas series: statistic, degrees of freedom, p-value and decision.
`self.global_fit_table`	pandas dataframe or `None`: per-parameter difference, SE and z (Wald only).
`self.global_fit_excluded`	List: parameters not compared (not estimable in both groups).
`self.global_fit_groups`	Dictionary: {`group name: calibrated model`}.

Examples

To test invariance between low and high scorers (the default median score split):

```
self.global_fit()
```

To test invariance between two groups defined by an exogenous variable `'Gender'`, stored in `self.exogenous`:

```
self.global_fit(split='exogenous', covariate='Gender')
```

4.4.8 res_corr_analysis

Description

Analysis of correlations of standardised residuals to test for violations of local item interdependence and unidimensionality requirements.

Usage

```
self.res_corr_analysis(warm_corr=True, tolerance=0.00001, max_iters=100,
                       ext_score_adjustment=0.5, constant=None, robust=False, method='log-lik',
                       matrix_power=None, log_lik_tol=0.000001, se=True)
```

Arguments

warm_corr	Boolean: If True, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 4.3.4). Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 4.3.4). Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 4.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for generation of item location estimates. Default is constant=None, which resolves to $0.1\,N^{0.15}(1-p)^{0.75}\bar{m}^{-1.4}$, where N is the number of persons with at least one response, p is the proportion of missing responses among them and $\bar{m}$ is the mean maximum score across items (see Section 4.3.1).
robust	Boolean: deprecated and has no effect; passing robust=True raises a FutureWarning. It will be removed in a future release. Default is robust=False.

Continued on next page...

method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) for generation of central item location estimates (see Section 4.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
matrix_power	Integer or None: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of central item location estimates. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 4.3.1). Ignored for other methods.
se	Boolean: passed through to the internal fit_statistics call (only used if not already computed). If False, skips the bootstrap entirely – this analysis's own output (residual correlations, PCA) does not depend on it, so se=False is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is se=True.

Returns

For tests of violation of the requirement for local item independence (Andrich & Kreiner, 2010; Marais, 2012):

self.residual_correlations	A pandas dataframe of pairwise correlations between item standardised residuals.

For tests of violation of the requirement for unidimensionality based on principal component analysis of the standardised residual correlations (Pallant & Tennant, 2007; Smith, 2002):

self.eigenvectors	The eigenvectors of the standardised residual correlations matrix.
self.eigenvalues	The eigenvalues corresponding to the eigenvectors.
self.variance_explained	The variance explained by each principal component.

Continued on next page...

`self.loadings`	The loading of each item onto each of the principal components, for the the first of which large loadings ('large'typically interpreted as > 0.4 or < -0.4) may be interpreted as representing the presence of significant dimensionality, in analogy to factor analysis.

Example

To produce a residual correlation analysis:

```
my_pcm.res_corr_analysis()
```

Arguments may be used to alter parameters of item location and/or person location estimation.

4.4.9 save_stats

Description

Exports item_stats, threshold_stats (both centred and uncentred), person_stats and test_stats to file in a single call. Auto-triggers item_stats_df, threshold_stats_df, person_stats_df and test_stats_df (Sections 4.4.1, 4.4.2, 4.4.4, 4.4.5) if any have not been run with their default arguments plus whichever of dp/warm_corr/etc. below are passed, so call those methods directly first (e.g. with full=True) if non-default table columns are wanted in the saved output. Does not include category_stats (Section 4.4.3) – save that table separately with self.category_stats.to_csv(...) if needed.

Usage

```
self.save_stats(filename, format='csv', dp=3, warm_corr=True, tolerance=0.00001,
                max_iters=100, ext_score_adjustment=0.5, method='log-lik', constant=None,
                robust=False, no_of_samples=500, interval=None)
```

Arguments

filename	String: output filename or path. For format='csv', used as a prefix (any trailing .csv is stripped first) for five files, _item_stats.csv, _threshold_stats_uncentred.csv, _threshold_stats_centred.csv, _person_stats.csv and _test_stats.csv. For format='xlsx', used as a single workbook filename (.xlsx appended automatically if not already present), with the five tables on separate sheets.
format	String: 'csv' or 'xlsx'. Default is format='csv'.
dp, warm_corr, tolerance, max_iters, ext_score_adjustment, method, constant, no_of_samples, interval	Passed through to item_stats_df/threshold_stats_df/person_stats_df/test_stats_df (Sections 4.4.1, 4.4.2, 4.4.4, 4.4.5), used only for whichever of those tables has not already been computed. See those sections for details of each.

Returns

None. Writes the five tables to file as described above.

Example

To save item, threshold, person and test statistics as five CSV files prefixed my_pcm_results:

```
my_pcm.save_stats('my_pcm_results')
```

To save all tables to separate sheets of a single Excel workbook:

```
my_pcm.save_stats('my_pcm_results', format='xlsx')
```

4.4.10 save_residuals

Description

Exports the residual correlation analysis (self.eigenvectors, self.eigenvalues, self.variance_explained and self.loadings) to file. Auto-triggers res_corr_analysis (Section 4.4.8) if not already run.

Usage

```
self.save_residuals(filename, format='csv', single=True, dp=3, warm_corr=True,
                    tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                    method='log-lik', constant=None, robust=False)
```

Arguments

filename	String: output filename or path (extension appended automatically if not already present).
format	String: 'csv' or 'xlsx'. Default is format='csv'.
single	Boolean: if True, writes all four tables (eigenvectors, eigenvalues, variance explained, loadings) into a single file – sequentially, separated by blank lines, for format='csv'; at successive row offsets on one sheet ('Item residual analysis'), for format='xlsx'. If False, writes each table to its own file (csv) or sheet (xlsx). Default is single=True.
dp, warm_corr, tolerance, max_iters, ext_score_adjustment, method, constant, robust	dp controls rounding of the saved values; the rest are passed through to res_corr_analysis (Section 4.4.8), used only if it has not already been run. See that section for details of each.

Returns

None. Writes the residual correlation analysis tables to file as described above.

Example

To save the residual correlation analysis as a single CSV file:

```
my_pcm.save_residuals('my_pcm_residuals')
```

To save each table to its own sheet of a single Excel workbook:

```
my_pcm.save_residuals('my_pcm_residuals', format='xlsx', single=False)
```

4.4.11 category_counts_df

Description

Produces a table of counts of scores in each category, plus total and missing responses. Response categories run from 0 to the highest maximum score among the included items; items whose own maximum score is lower show blank cells (not 0) for the categories above it, to avoid implying those categories exist for that item.

Usage

```
self.category_counts_df(persons=None, items=None, counts_name=None,
                        filename=None, file_format='csv')
```

Arguments

persons	List or String: Person or subset of persons over which to count responses. Default persons=None, which returns counts for all persons.
items	List or String: Item or subset of items over which to count responses. Default items=None, which returns counts for all items.
counts_name	String or Int: Name under which to store counts on a subset of persons or items so multiple sets of counts can be made available. Useful if e.g. user wants to compare counts between different subgroups of persons according to a criterion such as an exogenous variable. If passed, stores counts within a namespace, self.counts. Default counts_name=None, when counts are returned as a simple attribute, self.category_counts_table.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

If counts_name=None:

Returned and stored as attribute self.category_counts_table, a pandas dataframe of category counts with one row per item and one column per response category (0 to the highest included max score, blank above an item's own max score), plus total and missing responses per item, and grand total row.

If counts_name=my_counts_name:

Returned and stored as attribute self.counts.my_counts_name stored in the namespace self.counts, a pandas dataframe of category counts with one row per item and one column per response category (0 to the highest included max score, blank above an item's own max score), plus total and missing responses per item and grand total row.

Example

To produce a dataframe of category counts for all persons and items:

```
my_pcm.category_counts_df()
```

To produce a dataframe of category counts for a subset of persons saved as a list `my_persons`, with a matching attribute name within the namespace `self.counts`:

```
my_pcm.category_counts_df(persons=my_persons, counts_name=my_persons)
```

4.4.12 dif_test

Description

Differential Item Functioning (DIF) test by an exogenous person covariate (e.g. gender, first language) (Holland & Wainer, 1993). Splits non-extreme persons into groups by the named column in `self.exogenous`, designates one group as the reference (by default the largest), and compares each other ("focal") group against it individually – a reference-group design, not all-pairwise, so covariates with more than two levels are supported directly. Two independent components are tested:

1. **Item-location DIF.** Both groups are calibrated independently (each self-centred), then purified onto a common item scale via either a per-item Wald significance test (`selection_method='wald'`, default) or the robust modified z-score outlier-trimming procedure used by `calibrate_anchor` (Section 4.3.3; Iglewicz & Hoaglin, 1993) – the reference group's central locations supply the "anchor" values, the focal group's supply the "observed" values – so that genuine DIF items cannot contaminate the common scale used to test for DIF in the first place. Item-level DIF is then tested via a per-item Wald test (`test='wald'`, default), a per-item likelihood-ratio test (`test='lr'`), or both (`test='both'`), using bootstrap standard errors from `std_errors` (Section 4.3.2) for the Wald test. An omnibus LR test (Andersen-style: reference vs. focal, every common item jointly) runs by default (`omnibus=True`). Results in `self.dif_table` and `self.dif_omnibus_table`.

2. **Threshold-structure DIF** (differential category functioning, or DCF). PCM's per-item centred threshold offsets (`self.thresholds`) describe each item's own category structure relative to its own central location, not its location, so they are not translated by the item-scale purification above. Tested via each item's category widths (consecutive threshold differences) rather than threshold locations – a change to one category width imputes changes in all threshold locations, potentially causing false positives elsewhere; widths isolate the effect to the category that actually changed. Each item is Wald-tested category by category, reference versus focal, using a bootstrap standard error of the width itself (differenced *within* each bootstrap resample before taking the standard deviation, so it already reflects the covariance between adjacent thresholds). Each item's own category-width tests form their own multiple-comparison correction pool, independent of other items and of the item-location correction pool above. Not affected by `test`/`omnibus` – those only govern component (1). Results in `self.threshold_dif_table`.

`test='lr'/'both'` is mildly liberal; `test='wald'` (the default) is preferred. An item or threshold cell is flagged only if it clears both a (corrected) p-value threshold and an absolute logit-difference threshold, the latter an ETS-style convention, so that a statistically significant but practically negligible difference is not flagged. `dif_test` also supports ETS-style A/B/C severity-category classification (Zwick et al., 1997) (`category`, `category_thresholds`, `category_alpha`) for the item-location component only – the 0.43/0.64 logit defaults are calibrated for item-location DIF specifically, so this does not extend to `self.threshold_dif_table` – as described in Table 124:

Table 124: ETS-style DIF category classification (item-location component only). Sign: + indicates that an item is harder for the focal group; - indicates that an item is easier for the focal group. The 0.43/0.64 logit thresholds are the Rasch-logit equivalents of the classic ETS Mantel-Haenszel delta-scale B/C boundaries (Tristan, 2006); the significance level (default `category_alpha=0.05`) is independent of `alpha`, which governs the separate `Flagged`/`Flagged_LR` columns.

Category	**Criteria**	**Interpretation**
A	|Difference| < 0.43, or not statistically significant	Negligible DIF
B+ / B-	Both |Difference| ≥ 0.43 *and* $p(\text{DIF} = 0) <$ `category_alpha` (two-sided Wald/Welch test), but not C	Slight to moderate DIF
C+ / C-	Both |Difference| ≥ 0.64 *and* $p(\|\text{Difference}\| \leq 0.43) <$ `category_alpha` (one-sided test that difference significantly exceeds B boundary)	Moderate to large DIF

Optionally, standard errors may be adjusted by setting `size_adjust=True` to avoid over-sensitivity for large samples by adjusting the sample size: at very large N, SE shrinks enough that trivial differences become "significant". Item-location SE is rescaled by $\sqrt{n_{\text{actual}}/\texttt{reference_n}}$ unconditionally (Tristan, 2006); threshold (differential category functioning, or DCF) SE is rescaled when the threshold's CPAT information weight exceeds a cap (`dcf_cap`). Each group's SE is rescaled independently. Applied to the Wald/Welch test, category tests, and (if `welch=True`) Welch-Satterthwaite df, which then also uses the adjusted n rather than each group's actual N, for internal consistency. Default is `size_adjust=False`. `dcf_cap` with `size_adjust=True` controls SEs for DCF by capping each item-threshold's harmonic-mean information weight, for `size_adjust=True`'s item-threshold (DCF) component; does not affect `self.dif_table`. Per item-threshold cell, independently for each group: effective weight = min(harmonic_weight, `dcf_cap`); adjusted SE = actual SE $\times \max(1, \sqrt{\text{harmonic_weight}/\texttt{dcf_cap}})$, where the harmonic weight is a cell's row-summed harmonic mean of the raw pairwise comparison matrix and its transpose. Below the cap, bootstrap SE is used. Default is `dcf_cap=15000`.

The omnibus test can take one of two scopes. With `omnibus_scope='item'`, it tests uniform DIF only: each item's category structure is held at the pooled model's values and only a single location shift per item is estimated for each group, with $df = k - 1$, where k is the number of common items. With `omnibus_scope='full'`, it tests uniform DIF and differential category functioning together, with each item's thresholds also free in each group, and $df = (k-1) + \sum_i (m_i - 1)$, where m_i is the maximum score of item i. The result is a single joint test; `self.dif_table` and `self.threshold_dif_table` give the item-by-item results.

Usage

```
self.dif_test(covariate, reference=None, selection_method='wald', corr_tol=0.95,
              sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05, test='wald',
              omnibus=True, omnibus_scope='item', welch=False, size_adjust=False,
              reference_n=100, dcf_cap=15000, correction='bh', alpha=0.05, logit_threshold=0.43,
              category=False, category_thresholds=(0.43, 0.64), category_alpha=0.05,
              no_of_samples=500, plot=False, plot_kwargs=None, warm_corr=True,
              tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5, constant=None, robust=False,
```

method='log-lik', matrix_power=None, log_lik_tol=0.000001, seed=None)

Arguments

covariate	String: column name in self.exogenous to group persons by.
reference	String or None: value of covariate to use as the reference group. If None, the largest group is used. Default is reference=None.
selection_method	String, one of 'wald', 'robust_z' or 'none': how the common item-location scale is purified before testing (never the threshold-structure component) – see calibrate_anchor (Section 4.3.3) for what each option does. Default is selection_method='wald'.
corr_tol	Float: minimum correlation between the two groups' item locations on the retained items ('robust_z' only). Default is corr_tol=0.95.
sd_ratio_tol	Float: the ratio of the two groups' item-location standard deviations on the retained items must lie in [1/sd_ratio_tol, sd_ratio_tol] ('robust_z' only). Default is sd_ratio_tol=1.1.
min_anchors	Integer: minimum number of items retained by purification ('wald' and 'robust_z'). Default is min_anchors=6.
wald_alpha	Float: significance level for each item's purification test ('wald' only). Default is wald_alpha=0.05.
test	String, one of 'wald', 'lr' or 'both': per-item item-location DIF test(s) to compute – affects only the item-location component, never self.threshold_dif_table. 'wald' (default) matches the original behaviour exactly, at no added cost. 'lr'/'both' additionally compute a per-item likelihood-ratio test (extra cost: two person location re-estimations per item per focal group). The existing Flagged column always reflects the Wald test; a separate Flagged_LR column is added when test is 'lr' or 'both'. Default is test='wald'.
omnibus	Boolean: if True (default), also runs an Andersen-style omnibus LR test (reference vs. focal, every common item jointly) per focal group for the item-location component, stored in self.dif_omnibus_table. Cheap relative to the per-item LR test – one extra combined-group model fit per focal group, not per item. See omnibus_scope below for how PCM's per-item threshold freedom is accounted for.
omnibus_scope	String, 'item' or 'full': scope of the omnibus test (see Description): 'item' tests uniform DIF only; 'full' tests uniform DIF and differential category functioning together. Default is omnibus_scope='item'.

Continued on next page...

`welch`	Boolean: if `True`, every Wald test here (item-location *and* threshold-structure) becomes a Welch's t-test: statistic unchanged (`Difference / SE`), but p-value computed against a t-distribution with Welch-Satterthwaite degrees of freedom instead of a normal distribution. Adds a `df` column to both `self.dif_table` and `self.threshold_dif_table`. Does not affect the LR test or omnibus test (already exact chi-squared tests). Default is `welch=False`.
`size_adjust`	Boolean: if `True`, applies the Tristan (2006)rescaling to each group's SE.
`reference_n`	Integer: reference sample size for `size_adjust=True`'s item-location component only (`self.dif_table` – does not affect `self.threshold_dif_table`, which uses `dcf_cap` instead (see below). 100 (the default) is Tristán's own recommendation; around 60 is suggested there for closer alignment with the ETS Category B boundary specifically. Unused unless `size_adjust=True`.
`dcf_cap`	Float: cap on each item-threshold's harmonic-mean information weight. Default is `dcf_cap=15000`.
`correction`	String or `None`, one of `'bh'`, `'bonferroni'` or `None`: multiple-comparison correction applied across items (within each focal-group comparison) and, separately, across every item × category cell (within each focal-group comparison) – Benjamini-Hochberg false discovery rate control (Benjamini & Hochberg, 1995) or Bonferroni. Default is `correction='bh'`.
`alpha`	Float: p-value threshold for flagging in both components (Wald, per-item LR, and omnibus alike), compared against the corrected p-value if `correction` is not `None`. Default is `alpha=0.05`.
`logit_threshold`	Float: absolute logit-difference threshold for flagging in both components (ETS-style convention). Default is `logit_threshold=0.43`.
`category`	Boolean: if `True`, adds an ETS-style `Category` column to `self.dif_table` (item-location component only – see Table 124 above). Default is `category=False`.
`category_thresholds`	Tuple of two floats: (B boundary, C boundary) in logits, per Zwick et al. (1997). Default is `category_thresholds=(0.43, 0.64)`.
`category_alpha`	Float: significance level for both category tests, independent of `alpha`. Default is `category_alpha=0.05`.
`no_of_samples`	Integer: bootstrap resamples for each group's `std_errors` call (Section 4.3.2). Default is `no_of_samples=500`.

Continued on next page...

plot	Boolean: if True, calls plot_anchor_selection (Section 3.5.10) for each focal group that has a selection table (selection_method in 'wald'/'robust_z' only) and stores the resulting figures in self.dif_plots. Default is plot=False.
plot_kwargs	Dictionary or None: extra keyword arguments forwarded to plot_anchor_selection for every focal group when plot=True. Default is plot_kwargs=None.
warm_corr, tolerance, max_iters, ext_score_adjustment	Person-estimation arguments, passed through to the group models – see Section 4.3.4. Only used if test is 'lr'/'both' or omnibus=True.
constant, method, matrix_power, log_lik_tol	Calibration arguments passed to the group models – see Section 4.3.1.
seed	Integer or None: seed passed through to each group's internal std_errors call. None (default) draws fresh entropy each call. See Section 1.

Returns

self.dif_reference	The reference group value.
self.dif_covariate	The covariate column name used.
self.dif_reference_model	PCM object: fitted model for the reference group.
self.dif_focal_models	Dictionary: {focal_value: PCM}, fitted models for each focal group.
self.dif_tc	Dictionary: {focal_value: float}, the item-scale translation constant applied to each focal group.
self.dif_anchor_selection	Dictionary: {focal_value: dataframe or None}, per-item selection diagnostics for each focal group (None if selection_method='none').
self.dif_plots	Dictionary: {focal_value: figure or None}, the plot_anchor_selection figure for each focal group, if plot=True and a selection table exists for that group.
self.dif_group_sizes	Dictionary: {group_value: int}, non-extreme, non-missing-covariate N used for each group.

The two tested components additionally set:

self.dif_table	pandas dataframe: one row per (item, focal group) pair, indexed by item. Columns: Group, Reference/Focal/Focal (purified) (central location estimates), Difference, SE, z, p, p (corrected), Selected and Flagged. If welch=True, also df. If test is 'lr' or 'both', also LR, p_LR, p_LR (corrected), Flagged_LR. If category=True, also Category.
self.dif_omnibus_table	pandas dataframe or None: one row per focal group – LR, df, p, Flagged. Item-location component only if omnibus_scope='item' (default); item location and threshold/category-structure jointly if omnibus_scope='full'. None if omnibus=False.
self.threshold_dif_table	pandas dataframe: differential category functioning, tested via category widths rather than threshold locations. One row per (item, category, focal group), indexed by (Item, Category) where category k is the width between threshold k and k+1. Columns: Group, Reference/Focal (category widths), Difference, SE (bootstrap SE of the width itself), z, p, p (corrected), Flagged – and df if welch=True – corrected independently of self.dif_table, per item. Items whose maximum score differs between the reference and a focal group, or whose maximum score is 1 (no category width to test), are excluded from this table for that group, the former with a warning.

Examples

To test for DIF by an exogenous variable 'L1' (which may have any number of levels), having supplied exogenous to the constructor, using the largest level as the reference group by default:

```
self.dif_test('L1')
```

To inspect the threshold-structure DIF results for a specific item 'Item_3' after running the test:

```
self.threshold_dif_table.loc['Item_3']
```

To run the joint omnibus test of uniform DIF and differential category functioning together, rather than uniform DIF alone:

```
self.dif_test('L1', omnibus_scope='full')
```

4.4.13 recovery_stats

Description

Compares recovered parameter estimates of an object created from a simulation against the generating values (self.generating, Section 4.1.1). Issues a UserWarning and returns None unless the object was built from a simulation. The generating model may be SLM_Sim, PCM_Sim or RSM_Sim. With RSM_Sim, every item's max score must equal the generating max_score, otherwise the method returns None. Each item's thresholds are compared against the generating thresholds. With SLM_Sim, every max score must be 1; thresholds and category widths are skipped with a warning.

Person estimates are compared against*pseudo-estimates* (Elliott & Buttery, 2022b) to isolate systemic error from stochastic response noise: the person estimate from self.responses using the generating item and threshold locations. The persons error column is (recovered estimate − pseudo-estimate), and the persons generating column holds the pseudo-estimate. The error combines parameter estimation error and model misspecification.

Usage

```
self.recovery_stats(anchor=False, alpha=0.05, correction='bh', divergence_tol=0.1,
                        plot=True, **plot_kwargs)
```

Arguments

anchor	Boolean: if True, compares self.anchor_items (from calibrate_anchor, Section 4.3.3) against self.generating.items, instead of self.items. Raises ValueError if calibrate_anchor has not been run. Default anchor=False.
alpha	Float: significance threshold for the per-parameter z-test (Flagged/Flagged_corrected columns, below). Default alpha=0.05.
correction	String or None, one of 'bh', 'bonferroni' or None: multiple-comparison correction applied within each parameter type's own family of tests (matches dif_test's convention, Section 4.4.12). Default correction='bh'.
divergence_tol	Float: threshold on the absolute difference between the OLS and Huber regression slopes (below) for the Divergence_flag summary row. Default divergence_tol=0.1.
plot	Boolean: if True, also produces a recovery plot (scatter, identity line, OLS and Huber fit lines) for each parameter type. Default plot=True.
**plot_kwargs	Forwarded to the internal plotting routine: filename, file_format, dpi, plot_style, font, title_font_size, axis_font_size, labelsize, figsize (same vocabulary as wright_map). A shared filename is suffixed per parameter type.

Returns

`self.recovery.items.table`, `self.recovery.thresholds.table`, `self.recovery.category_widths.table`, `self.recovery.persons.table`	pandas dataframes, one per parameter type (`items`, `thresholds`, `category_widths` and `persons`), each with columns `generating`, `estimated`, `SE`, `error` (estimated − generating), `abs_error`, `z`, `p`, `p_corrected`, `Flagged` and `Flagged_corrected`. `thresholds`/`category_widths` are indexed by (threshold position, item) – `self.thresholds_uncentred.unstack().dropna()`'s own convention. For `persons`, `generating` holds the pseudo-estimate, and `SE` is `self.csem_vector` (the analytic conditional SEM), unaffected by `anchor`. `self.recovery.thresholds`/`self.recovery.category_widths` are `None` for `SLM_Sim`-generated data.
`self.recovery.items.summary`, `self.recovery.thresholds.summary`, `self.recovery.category_widths.summary`, `self.recovery.persons.summary`	pandas dataframes, one column (`Overall`) with rows `RMSE`, `MAE`, `bias` (mean signed error), `r` (Pearson correlation), `SD_ratio` (estimated SD over generating SD), `OLS_slope`, `OLS_intercept`, `Huber_slope`, `Huber_intercept`, `Divergence_flag` (`True` if the OLS and Huber slopes differ by more than `divergence_tol`) and `Prop_flagged` (proportion of parameters with `Flagged=True`). `r`, the two slopes and `SD_ratio` are set to `NaN` when fewer than 3 parameters are compared or the generating values have standard deviation below 0.05.

Examples

To compare recovered item, threshold, category-width and person estimates against the generating simulation, plotting all four:

```
self.recovery_stats()
```

To compare anchored item estimates instead (having already run `calibrate_anchor`), without plotting:

```
self.recovery_stats(anchor=True, plot=False)
```

4.5 Plotting functionality

4.5.1 Shared plotting arguments

All the plotting methods described in this section share a set of arguments which may be used to customise the appearance of the plot or save the plot to file automatically. These arguments are:

`title`	String: Title for the plot, to appear in the image. Default is `title=None`.
`xmin`	Float: Minimum displayed point on x-axis, in logits. Default is `xmin=-5`.
`xmax`	Float: Maximum displayed point on x-axis, in logits. Default is `xmax=5`.
`plot_style`	String: Plot style to use. Available styles are `'white'`, `'dark'`, `'black'`, `'parchment'`, `'minimal'`, `'print'`, `'solarized-light'`, `'solarized-dark'`, `'slate'`, `'blueprint'`, `'newsprint'` and `'chalkboard'` – twelve Seaborn (Waskom, 2021)-based colour schemes. Default is `plot_style='white'`.
`palette`	String: Controls colours of lines and overplotted class intervals dots. Options are `'dark blue'`, `'light blue'`, `'dark red'`, `'light red'`, `'dark green'`, `'light green'`, `'dark grey'`, `'light grey'`, `'dark multi'`, `'light multi'` and `'colorblind multi'`. `'colorblind multi'` is the Okabe & Ito (2008) colour-vision-deficiency-safe qualitative palette. Default is `palette='colorblind multi'`. Not used for `std_residual_plot`.
`black`	Boolean: If `True`, the plot will be rendered in black and white. Default is `black=False`.
`marker_border`	Boolean: If `True`, observed data point markers are drawn with a black edge for contrast. Only used by `icc`, `crcs`, `threshold_ccs` and `tcc` (the methods with an `obs` argument). Default is `marker_border=True`.
`line_border`	Boolean: If `True`, curve lines are drawn with a contrasting outline stroke (white on dark `plot_style` backgrounds, black otherwise). Only used by `icc`, `crcs`, `threshold_ccs` and `tcc`. Default is `line_border=False`.
`font`	String: The font to use in the plot. Default is `font='Times New Roman'`.
`title_font_size`	Float: The size of the title font in points. Default is `title_font_size=15`.
`axis_font_size`	Float: The size of the axis label font in points. Default is `axis_font_size=12`.
`labelsize`	Float: The size of the axis tick label font in points. Default is `labelsize=12`.
`filename`	String: The filename for the saved plot, with no suffix for format. If `None`, no file will be saved. Default is `filename=None`.
`file_format`	The format of the file: `png`, `jpg` or `svg`. Default is `file_format=png`.
`dpi`	The resolution of the plot in dpi (dots per inch) – higher resolution plots are better quality but have larger file sizes. Default is `dpi=300`.

4.5.2 icc

Description

Plots the item characteristic curves (or item response function) for an item: person location on the x-axis against expected score on the y-axis. Options to plot observed responses, item threshold line and lines showing locations corresponding to specified expected scores, and to highlight a specified response category.

Usage

```
self.icc(item, obs=False, no_of_classes=5, thresh_lines=False, central_location=False,
         score_lines=None, score_labels=False, cat_highlight=None)
```

Arguments

item	String: The name of the item(s) to plot.
obs	Boolean: If True, mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is obs=False.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
thresh_lines	Boolean: If True, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is thresh_lines=False.
central_location	Boolean: If True, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is central_location=False.
score_lines	List of floats between 0 and the item's maximum score: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score.
score_labels	Boolean: If True, scores and locations corresponding to arguments passed to score_lines will be labelled on the plot. Default is score_labels=False.
cat_highlight	Integer: Passing a score between 0 and the item's maximum score will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is cat_highlight=None (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns
matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples
To plot a basic item characteristic curve for `Item_1` and store the output as a variable `my_icc_plot` and save it to file as `my_icc_plot.png`:

```
my_icc_plot = my_pcm.icc('Item_1', filename=my_icc_plot)
```

To plot an item characteristic curve for `Item_1` with observed responses for 8 response classes and store the output as a variable `my_icc_plot`:

```
my_icc_plot = self.icc('Item_1', obs=True, no_of_classes=8)
```

To plot an item characteristic curve for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = my_pcm.icc('Item_1', thresh_lines=True, cat_highlight=0)
```

To plot an item characteristic curve for `Item_1` with lines showing the locations corresponding to expected scores of 0.7 and 1.6, with the expected score and corresponding location labelled, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = my_pcm.icc('Item_1', score_lines=[0.7, 1.6], score_labels=True)
```

4.5.3 crcs

Description

Plots category response curves for an item: person location on the x-axis against the probability of obtaining a score in each category, from 0 to the item's maximum score, on the y-axis. Options to plot observed proportions and item threshold line, and to highlight a specified response category.

Usage

```
self.crcs(item, obs=None, no_of_classes=5, thresh_lines=False, central_location=False,
          cat_highlight=None)
```

Arguments

item	String: The name of the item to plot.
obs	List: List of integers between 0 and self.max_score. For each value, mean observed proportions in each ordered response category scoring in that category are plotted against the mean location of the corresponding response class.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
thresh_lines	Boolean: If True, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is thresh_lines=False.
central_location	Boolean: If True, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is central_location=False.
cat_highlight	Integer: Passing a score between 0 and self.max_score will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is cat_highlight=None (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot basic category response curves for `Item_1` and store the output as a variable `my_crcs_plot` and save it to file as `my_crcs_plot.png`:

```
my_crcs_plot = my_pcm.crcs('Item_1', filename=my_crcs_plot)
```

To plot category response curves for `Item_1` with observed response proportions for category 0 for 8 response classes and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = my_pcm.crcs('Item_1', obs=[0], no_of_classes=8)
```

To plot category response curves for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = my_pcm.crcs('Item_1', thresh_lines=True, cat_highlight=0)
```

4.5.4 threshold_ccs

Description

Plots the threshold characteristic curves (or threshold response functions)for an item. For threshold τ_k, $k \in \{1, ..., \texttt{self.max_score}\}$, the threshold characteristic curve is the probability of obtaining a score of k rather than $k-1$, conditional on the score being either $k-1$ rather than k, for a given person location. Each threshold characteristic curve functions as a dichotomous item characteristic curve under the SLM (see Sections 3.2.2 and 3.5.2).

For threshold τ_k, `threshold_ccs` plots person location on the x-axis against the probability of obtaining a score of k on the y-axis. Options to plot threshold lines and central item locations, and to highlight a specified response category.

Usage

```
self.threshold_ccs(item, obs=None, no_of_classes=5, thresh_lines=False, central_location=False,
                   cat_highlight=None)
```

Arguments

`item`	String: The name of the item to plot.
`obs`	List: List of integers corresponding to thresholds τ_1 to τ_m, where $m =$`self.max_score`, or `'all'` or `'none'`. If `obs=[k]`, mean proportions of persons obtaining a score of k rather than $k-1$, conditional on the score being either $k-1$ or k, for each of the ordered response categories, will be plotted against the mean location of the corresponding response class. Multiple thresholds may be passed. Default is `obs=None`.
`no_of_classes`	Integer: The number of observed response categories. Default is `no_of_classes=5`.
`thresh_lines`	Boolean: If `True`, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is `thresh_lines=False`.
`central_location`	Boolean: If `True`, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is `central_location=False`.
`cat_highlight`	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is `cat_highlight=None` (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns
matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples
To plot basic threshold characteristics curves for `Item_1`, store the output as variable `my_threshold_ccs_plot`, and save it to file as `my_threshold_ccs_plot.png`:

```
my_threshold_ccs_plot = my_pcm.threshold_ccs(item='Item_1', filename='my_threshold_ccs_plot')
```

To plot threshold characteristics curves for `Item_1` with threshold lines and category 1 highlighted, and store the output as a variable `my_threshold_ccs_plot`:

```
my_threshold_ccs_plot = my_pcm.threshold_ccs(item='Item_1', thresh_lines=True, cat_highlight=1)
```

To plot threshold characteristics curves for `Item_1` with observed responses plotted for thresholds 2 and 4 and central item location line, and store the output as a variable `my_threshold_ccs_plot`:

```
my_threshold_ccs_plot = my_pcm.threshold_ccs(item='Item_1', obs=[2, 4], central_location=True)
```

4.5.5 iic

Description

Plots the item information curve for an item: person location on the x-axis against Fisher information on the y-axis. Options to plot item threshold line and lines showing Fisher information corresponding to specified locations, and to highlight a specified response category.

Usage

```
self.iic(item, ymax=None, thresh_lines=False, central_location=False, point_info_lines=None,
         point_info_labels=False, cat_highlight=None)
```

Arguments

item	String: The name of the item(s) to plot.
ymax	Float: The maximum value to show on the y-axis. If None, will infer, plotting a maximum of 1.1 times the maximum item information. Default is ymax=None
thresh_lines	Boolean: If True, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is thresh_lines=False.
central_location	Boolean: If True, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is central_location=False.
point_info_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the Fisher information corresponding to the location. Default is point_info_lines=None.
point_info_labels	Boolean: If True, locations and Fisher information corresponding to arguments passed to point_info_lines will be labelled on the plot. Default is point_info_labels=False.
cat_highlight	Integer: Passing a score between 0 and self.max_score will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is cat_highlight=None (no category highlighted).
ymax	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic item information curve for `Item_1` and store the output as a variable `my_iic_plot` and save it to file as `my_iic_plot.png`:

```
my_iic_plot = my_pcm.iic('Item_1', filename='my_iic_plot')
```

To plot an item information curve for `Item_1` with threshold lines, central item location line and category 1 highlighted, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = my_pcm.iic('Item_1', thresh_lines=True, central_location=True, cat_highlight=1)
```

To plot an item information curve for `Item_1` with lines showing the Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding Fisher information labelled, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = my_pcm.iic('Item_1', point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

4.5.6 tcc

Description

Plots the test characteristic curve (or test response function) for a set of items: person location on the x-axis against expected score on the y-axis. Options to plot observed responses and lines showing locations corresponding to specified expected scores.

Usage

```
self.tcc(items=None, obs=False, no_of_classes=5, score_lines=None, score_labels=False)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
obs	Boolean: If True, mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is obs=False.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
score_lines	List of floats between 0 and self.max_score: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score. Default is score_lines=None.
score_labels	Boolean: If True, scores and locations corresponding to arguments passed to score_lines will be labelled on the plot. Default is score_labels=False.

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic test characteristic curve for all items and store the output as a variable my_tcc_plot and save it to file as my_tcc_plot.png:

```
my_tcc_plot = my_pcm.tcc(filename=my_tcc_plot)
```

To plot a test characteristic curve for Item_1 for a subset of items stored as a list **my_item_list**, with observed responses for 8 response classes and store the output as a variable my_tcc_plot:

```
my_tcc_plot = my_pcm.tcc(obs=True, no_of_classes=8)
```

To plot a test characteristic curve for `Item_1` for all items with lines showing the locations corresponding to expected scores of 13 and 20, with the expected score and corresponding location labelled, and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = my_pcm.tcc(score_lines=[13, 20], score_labels=True)
```

4.5.7 test_info

Description

Plots the test information curve: person location on the x-axis against total Fisher information on the y-axis. Option to plot lines showing Fisher information corresponding to specified locations.

Usage

```
self.test_info(items=None, ymax=None, point_info_lines=None, point_info_labels=False)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
point_info_lines	List of floats: Each float is a location for which a vertical line is plotted from the x-axis to its intersection with the item information curve; from there a horizontal line is plotted to the y-axis to show the total Fisher information corresponding to the location. Default point_info_lines=None.
point_info_labels	Boolean: If True, locations and total Fisher information corresponding to arguments passed to point_info_lines will be labelled on the plot. Default is point_info_labels=False.
ymax	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic test information curve and store the output as a variable my_test_info_plot and save it to file as my_test_info_plot.png:

```
my_test_info_plot = my_pcm.test_info(filename='my_test_info_plot')
```

To plot a test information curve for a subset of items stored as a list **my_item_list** and store the output as a variable my_test_info_plot:

```
my_test_info_plot = my_pcm.test_info(items=my_item_list)
```

To plot a test information curve with lines showing the total Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding total Fisher information labelled, and store the output as a variable my_test_info_plot:

```
my_test_info_plot = my_pcm.test_info(point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

4.5.8 test_csem

Description

Plots the test conditional standard error of measurement (CSEM) curve: person location on the x-axis against CSEM (in logits) on the y-axis. Option to plot lines showing CSEM corresponding to specified locations.

Usage

```
self.test_csem(items=None, point_csem_lines=None, point_csem_labels=False, ymax=5)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
point_csem_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the CSEM curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the CSEM corresponding to the location. Default is point_csem_lines=None.
point_csem_labels	Boolean: If True, locations and CSEM corresponding to arguments passed to point_csem_lines will be labelled on the plot. Default is point_csem_labels=False.
ymax	The maximum point displayed on the y-axis, in logits.

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic CSEM curve and store the output as a variable my_test_csem_plot and save it to file as my_test_csem_plot.png:

```
my_test_csem_plot = my_pcm.test_csem(filename='my_test_csem_plot')
```

To plot a CSEM curve for a subset of items stored as a list **my_item_list** and store the output as a variable my_test_csem_plot:

```
my_test_csem_plot = my_pcm.test_csem(items=my_item_list)
```

To plot a CSEM curve with lines showing the CSEM corresponding to locations of -0.3 and 0.7, with the location and corresponding CSEM labelled, and store the output as a variable my_test_csem_plot:

```
my_test_csem_plot = my_pcm.test_csem(point_csem_lines=[-0.3, 0.7], point_csem_labels=True)
```

4.5.9 std_residuals_plot

Description

Plots histogram of standardised residuals, with optional overplotting of standard Normal distribution.

Usage

```
self.std_residuals_plot(items=None, bin_width=0.5, normal=False, dpi=300)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
bin_width	Float: The width of the histogram bins along the x-axis. Default is bin_width=0.5.
dpi	Integer: resolution of the saved plot in dpi (dots per inch). Default is dpi=300.
normal	Boolean: If True, plots a standard normal distribution over the standardised residual histogram for comparison. Default is normal=False.

Additional arguments to customise the appearance of the plot are detailed in Section 4.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot and display a basic standardised residuals histogram and save it to file as my_std_residuals_plot.png:

```
my_pcm.std_residuals_plot(filename='my_std_residuals_plot')
```

To plot and display a standardised residuals histogram with bin width 1, with standard normal curve:

```
self.std_residuals_plot(bin_width=1, normal=True)
```

To plot and display a standardised residuals histogram on a subset of items stored as a list in a variable my_item_list:

```
my_pcm.std_residuals_plot(items=my_item_list)
```

4.5.10 wright_map

Description

Plots a Wright map (person-item map): back-to-back person and item distributions on a shared axis. Options for histograms or kernel density estimate (KDE) curves; items may also be presented by name in histogram format, or – for polytomous models – as a Winsteps-style divided category strip (one row per item, spanning its Rasch-Andrich threshold range). wright_map does not use the shared plotting arguments described in Section 4.5.1.

Usage

```
self.wright_map(person_names=None, item_names=None, item_level='items',
                threshold_type='rasch-andrich',
                orientation='vertical', map_type='hist', item_labels=False,
                item_strip=False, item_distribution=False, strip_thickness=2.0,
                strip_alpha=0.3, sort='location', palette='colorblind multi',
                neutral_extremes=False, item_row_height=None, narrow_font=False,
                edge_padding=0.5, distribution_markers=False, group_by=None,
                group_colors=None, stack=True, blend=False, prop=False, person_lim=None,
                item_lim=None, person_scaling=None, no_of_bins=20, pad=False,
                plot_range=None, kde_points=500, bw_method='scott', line_width=1,
                figsize=None, title=None, plot_style='white', edge_color='black',
                person_color='skyblue', item_color='salmon',
                person_line_color='black', item_line_color='black',
                marker_color=None, alpha=0.6, black=False,
                font='Times New Roman', title_font_size=15, axis_font_size=12,
                labelsize=12, item_label_size=8, tick_interval=1, filename=None,
                file_format='png', dpi=300)
```

Arguments

person_names	String, list or None: Person subset to include. Default is person_names=None (all persons).
item_names	String, list or None: Item subset to include. Default is item_names=None (all items).
item_level	String: 'items' plots one point per item at its central location. 'thresholds' plots one point per Rasch-Andrich threshold instead. Ignored when item_strip=True. Default is item_level='items'.

Continued on next page...

threshold_type	String, one of 'rasch-andrich' (default), 'thurstonian' or 'half-point': which threshold convention to plot when item_level='thresholds'. 'rasch-andrich' plots the estimated Rasch-Andrich thresholds, which can be disordered); 'thurstonian' plots the median threshold ($P(X \geq k) = 0.5$); 'half-point' plots the location where $E[X] = k - 0.5$. 'thurstonian' and 'half-point' are always ordered. Also governs item_strip=True category-boundary segments. Ignored when item_level='items'.
orientation	String: 'vertical' places location on the x-axis, with persons above the baseline; 'horizontal' places location on the y-axis, with persons to the left of the baseline. Default is orientation='vertical'.
map_type	String: 'hist' plots back-to-back histograms; 'kde' plots kernel density estimate curves instead. If item_labels=True, only governs the person side; the item side becomes a label panel rather than a distribution. Default is map_type='hist'.
item_labels	Boolean: If True, item names are listed instead of a bare histogram, grouped into no_of_bins histogram-style location bins and ordered by location. Default is item_labels=False.
item_strip	Boolean: If True, draws each item as a strip divided into coloured category segments showing the most probable category for a given person location, with a boundary line at each threshold. Forces item_labels=True. Default is item_strip=False.
item_distribution	Boolean: If True, plots a histogram/KDE plot as well as item labels. Forces item_labels=True. Default is item_distribution=False.
strip_thickness	Float: Controls the height (vertical orientation) or width (horizontal orientation) of category strips as a multiplier of the base thickness when item_strip=True. Default is strip_thickness=2.0.
strip_alpha	Float: Fill transparency for strips when item_strip=True. Default is strip_alpha=0.3.
sort	String: Ordering of item strip rows. 'location' sorts by the lowest threshold; 'order' orders according to self.items. Only used when item_strip=True. Default is sort='location'.
palette	String or None: Named colour palette for category strips (see Section 4.5.1). None uses a flat item_color fill with no distinction between categories. Only used when item_strip=True. Default is palette='colorblind multi'.

Continued on next page...

neutral_extremes	Boolean: The strip's open (unbounded) top and bottom categories always draw out to the location axis limits – there is no real boundary to stop short at. If True, they fade from a neutral cream colour toward the real threshold instead of using a flat fill in the category palette, de-emphasising the open-ended extremes rather than drawing the eye to an arbitrary width. Only used when item_strip=True. Default is neutral_extremes=False.
item_row_height	Float or None: Physical height, in inches, allocated per stacked item-label row. If None, derived automatically. Only used when item_labels=True. Default is item_row_height=None.
narrow_font	Boolean: If True, item labels (and, with item_strip=True, row labels beside category strips) are set in 'Arial Narrow' instead of font to save space. Falls back to Matplotlib's font substitution if 'Arial Narrow' is not installed. Default is narrow_font=False.
edge_padding	Float: Extra padding (in inches) added around item-label margin. The same value produces matching-looking margins in both orientations. Only used when item_labels=True. Default is edge_padding=0.5.
distribution_markers	Boolean: If True, overlays mean and plus/minus 1 and 2 standard deviation reference markers, labelled μ, $\mu \pm \sigma$, $\mu \pm 2\sigma$ for persons and items. Default is distribution_markers=False.
group_by	pandas Series or None: Grouping of persons (e.g. a DIF group) used to split the person distribution into sub-distributions. Default group_by=None (persons as a single series).
group_colors	Dict or None: Controls colours for group_by. If None, colours are drawn from the 'tab10' colour map. Default is group_colors=None.
stack	Boolean: When group_by is set, for map_type='hist', True cumulatively stacks histogram bars while False draws separate columns per group: dodged, or alpha-blended if blend=True. For map_type='kde', True cumulatively stacks group KDE plots; False overlays group KDE plots. Default is stack=True.
blend	Boolean: When group_by is set, map_type='hist' and stack=False, if True, blends histogram bars using colour instead of dodging them (showing them side-by-side). Default is blend=False.
prop	Boolean: If True, plots distributions as proportions/density rather than counts. Default is prop=False.
person_lim	Float or None: Controls axis limit for person distribution. If None, derived automatically from the distribution. Default is person_lim=None.

Continued on next page...

item_lim	Float or None: Controls axis limit for item distribution. If None, derived automatically from the distribution. Default is item_lim=None.
person_scaling	Float or None: Adjusts the scaling of the person distribution for clarity, particularly when the number of persons is very large compared to items. A scaling of 10 means that the person scale is 10 times the item scale, and so on. Default is person_scaling=None, which chooses the largest of 1, 2, 5, 10, 20, 50, ... that keeps the rescaled item distribution within half the height of the person distribution, and never below 1.
no_of_bins	Integer: Number of histogram or label bins. Default is no_of_bins=20.
pad	Boolean: If True, adds 5% padding to both ends of the location axis limits. If False, the location axis is bound to the plotted range. Default is pad=False.
plot_range	Tuple of two floats, or None: (lo, hi) limits for the location axis. If None, derived automatically. Default is plot_range=None.
kde_points	Integer: Number of points at which each KDE curve is evaluated when map_type='kde'. Default is kde_points=500.
bw_method	String, scalar or callable: Bandwidth selection method passed to scipy.stats.gaussian_kde when map_type='kde'. Default is bw_method='scott'.
line_width	Float: Line width of KDE curves when map_type='kde'. Default is line_width=1.
figsize	Tuple of two floats, or None: Base figure size in inches for the person distribution. If None, defaults to (8, 6) for orientation='vertical' or (4, 8) for orientation='horizontal'. Default is figsize=None.
title	String or None: Plot title. Default is title=None (no title).
plot_style	String: Plot background style, as per plot_style in Section 4.5.1. Default is plot_style='white'.
edge_color	String: Edge colour of histogram bars when map_type='hist' and group_by is None. Default is edge_color='black'.
person_color	String: Fill colour for person distribution; ignored when group_by is set, overridden by a grey shade when black=True. Default is person_color='skyblue'.
item_color	String: Fill colour for item distribution when item_labels=False; overridden by a grey shade when black=True. Default is item_color='salmon'.

Continued on next page...

person_line_color	String: Line colour for person KDE plot when map_type='kde' and group_by is None. Default is person_line_color='black'.
item_line_color	String: Colour for item labels (item_labels=True) or item KDE plot line (item_labels=False, map_type='kde'). Default is item_line_color='black'.
marker_color	String, tuple or None: Colour of tick marks and labels drawn when distribution_markers=True. A single colour applies to both the person and item marks; a (person, item) pair sets them separately. Person marks use it only when group_by=None; grouped marks use the assigned group colours. Default is marker_color=None, which gives each set of marks a darker shade of its own distribution's colour (person_color or item_color).
alpha	Float: Fill transparency for distributions. Default is alpha=0.6.
black	Boolean: If True, renders person_color/item_color as grey shades instead. Has no effect on group_by colours. Default is black=False.
font	String: Plot font. Default font='Times New Roman'.
title_font_size	Float: Title font size in points. Default title_font_size=15.
axis_font_size	Float: Axis label font size in points. Default axis_font_size=12.
labelsize	Float: Axis tick label font size in points. Default labelsize=12.
item_label_size	Integer: Item name font size in points. Default item_label_size=8.
tick_interval	Float: Spacing between Location axis ticks. Default tick_interval=1.
filename	String or None: Filename for saved plot, with no suffix for format. If None, no file is saved. Default filename=None.
file_format	String: Format of saved file, 'png', 'jpg' or 'svg'. Default file_format='png'.
dpi	Integer: Resolution of saved plot in dpi (dots per inch). Default dpi=300.

Returns

matplotlib figure. Displays and closes the figure; if a filename argument is passed, also saves the image to file.

Examples

To plot a basic Wright map, with items and persons as back-to-back histograms:

```
self.wright_map()
```

To plot a Wright map with each item drawn as a divided category strip, plus mean/±1SD/±2SD reference marks for persons, and save it to file as my_wright_map.png:

```
self.wright_map(item_strip=True, distribution_markers=True, title='PCM Wright Map',
```

```
                filename='my_wright_map')
```

To plot a horizontally-oriented Wright map, split by an exogenous grouping variable stored in a pandas Series `my_group` (indexed like `self.persons`), with each group's histogram bars alpha-blended rather than stacked:

```
self.wright_map(orientation='horizontal', group_by=my_group, stack=False,
                blend=True)
```

4.5.11 plot_anchor_selection

Description

Scatter plot of anchor item locations (x-axis) against observed values for the same items (y-axis) to visualise the outlier-trimming diagnostics produced by `calibrate_anchor` (Section 4.3.3) or `dif_test` (Section 4.4.12). Selected and dropped items are plotted in different colours, with a solid $y = x$ identity line for reference and a dotted OLS best-fit line through the selected points only. `plot_anchor_selection` does not use the shared plotting arguments described in Section 4.5.1; its own, smaller set of arguments is described in full below.

Usage

```
self.plot_anchor_selection(result, xmin=-4, xmax=4, ymin=-3, ymax=3, figsize=(8, 6),
                           font='Times New Roman', title_font_size=15, axis_font_size=12,
                           labelsize=12, title='', dpi=300, filename=None,
                           file_format='png')
```

Arguments

`result`	dict or pandas dataframe: Either the full dictionary returned internally by the robust anchor-selection routine, or just its `'table'`, i.e. one of `self.anchor_selection` (from `calibrate_anchor`) or `self.dif_anchor_selection` (from `dif_test`), each a dataframe with `'Anchor'`, `'Observed'` and boolean `'Selected'` columns.
`xmin`, `xmax`, `ymin`, `ymax`	Floats: axis limits. Defaults are `xmin=-4`, `xmax=4`, `ymin=-3`, `ymax=3`.
`figsize`	Tuple of two floats: figure size in inches (width, height). Default is `figsize=(8, 6)`.
`font`	String: the font to use in the plot. Default is `font='Times New Roman'`.
`title_font_size`, `axis_font_size`, `labelsize`	Integers: font sizes in points for the title, axis labels and axis tick labels respectively. Defaults are `title_font_size=15`, `axis_font_size=12`, `labelsize=12`.
`title`	String: plot title. Default is `title=''` (no title).
`filename`	String or `None`: if provided, saves the plot to this path (with the suffix determined by `file_format`). Default is `filename=None` (not saved).
`file_format`	String: file format for saved plots, e.g. `'png'`, `'pdf'`, `'svg'`. Default is `file_format='png'`.
`dpi`	Integer: output resolution in dpi (dots per inch) when saving to file. Default is `dpi=300`.

Returns

matplotlib figure object. If a `filename` argument is passed, also saves the image to file. `calibrate_anchor` and `dif_test` both call this method automatically when their own `plot` argument is `True` (see Sections 4.3.3 and 4.4.12), so it need not usually be called directly; it is documented separately here since the resulting figure objects (`self.anchor_plot`, `self.dif_plots`) can also be regenerated with different display arguments by calling it directly on the stored selection table.

Example

To regenerate the anchor-selection plot for an existing calibration's diagnostics, stored in `self.anchor_selection`, with wider axis limits and a custom title:

```
self.plot_anchor_selection(self.anchor_selection, xmin=-6, xmax=6, ymin=-6, ymax=6,
                           title='Anchor calibration')
```

5 class RSM

5.1 Preliminaries

5.1.1 RSM

Description

Creates an object of the class RSM from a pandas dataframe of polytomously scored data of items which share the same maximum score and Rasch-Andrich threshold structure for analysis. No analysis can be run until an object is created.

Usage

```
RSM(responses, max_score=None, extreme_persons=True, no_of_classes=5,
    validate=True, exogenous=None)
```

Arguments

responses	pandas dataframe with items as columns (item names as column names) and persons as index (person names as row names).
max_score	Integer: The maximum possible score, shared across all items. If no score is passed, max_score will be inferred from the data, although passing an argument is recommended. Default is max_score=None.
extreme_persons	Boolean: if False, all persons with extreme scores (all responses correct or all responses incorrect) are removed from the response dataframe. Default is extreme_persons=True.
no_of_classes	Integer: the number of classes of persons grouped by location for overplotting observed responses on theoretical curves. Default is no_of_classes=5
validate	Boolean: if True, automatically calls check_data_connectivity (Section 5.1.2) on creation and issues a UserWarning if the item response network is disconnected, or if any item has a structurally unresolvable zero in the pairwise calibration matrix, or the directed comparison network is connected but not strongly connected (which prevents a finite method='log-lik' solution on an unpowered matrix). Default is validate=True.
exogenous	pandas dataframe or None: optional person-level covariates for Differential Item Functioning (DIF) analysis (Section 5.4.12). See the equivalent argument on the SLM constructor (Section 3.1.1) for full details – behaviour is identical. Default is exogenous=None.

Returns

Object of class RSM. Analyses are run using methods defined on the RSM object, with results stored as attributes of the RSM object.

Several attributes of object RSM are automatically generated on its creation:

self.responses	pandas dataframe: Dataframe of valid responses.
self.invalid_responses	pandas dataframe: Dataframe of invalid responses (persons with no responses to any items, i.e. all missing data).
self.extreme_persons	pandas dataframe: Dataframe of persons with extreme scores (all-correct or all-incorrect). When extreme_persons=True (the default), extreme scorers are retained in self.responses and this attribute is empty; when extreme_persons=False, they are removed and stored here.
self.max_score	Integer: The maximum possible score, shared across all items.
self.no_of_items	Integer: Number of items.
self.item_names self.item_ids	List: List of item names. self.item_ids is an alias for users who find *_ids more natural than *_names.
self.no_of_persons	Integer: Number of persons.
self.person_names self.person_ids	List: List of person names. self.person_ids is an alias for users who find *_ids more natural than *_names.
self.generating	Namespace object (_SimParams): Only present when the model is instantiated by passing a simulation object (e.g. RSM_Sim) as the responses argument. Contains all attributes of the simulation object (e.g. self.generating.persons, self.generating.items, self.generating.thresholds), enabling direct comparison of generating and estimated parameters.
self.connectivity_status	Dictionary: result of check_data_connectivity (Section 5.1.2), present only when validate=True. Always contains at least 'connected' (Boolean), 'components_count' (Integer) and 'strongly_connected' (Boolean).
self.exogenous	pandas dataframe or None: person-level covariates passed via the exogenous argument, re-indexed onto self.person_names. None if not supplied.
self.no_exogenous_persons	pandas index: Persons present in the response data with no matching exogenous record.
self.exogenous_only_persons	pandas index: Persons present in the exogenous data but not in the response data.

Example

To create an object from a dataframe my_rsm_dataframe, with a maximum score of 5 and 10 observed classes:

```
my_rsm = RSM(my_rsm_dataframe, max_score=5, no_of_classes=10)
```

To create an object from a dataframe my_rsm_dataframe, additionally carrying exogenous variables for DIF analysis loaded into my_exogenous_df:

```
my_rsm = RSM(my_rsm_dataframe, max_score=5, exogenous=my_exogenous_df)
```

5.1.2 check_data_connectivity

Description

Validates whether the item response network is fully connected, in the same sense as SLM's check_data_connectivity (Section 3.1.2): two items are connected if at least one person's response differentiates between them. For RSM, the underlying directed matrix is one row per item, with entry (i, j) counting persons who scored exactly one point higher on item i than item j – matching the item-location stage of calibrate (Section 5.3.1) exactly, since items share a single threshold structure and so need no category-level breakdown.

Also verifies *strong* connectivity of the directed win-graph – the network in which an edge runs from item i to item j whenever at least one person scored higher on i than on j. The Bradley–Terry log-likelihood estimator (calibrate(method='log-lik')) has a finite, unique solution only when this directed graph is strongly connected (**Ford1957**). When the network is connected but not strongly connected – for example with near-Guttman data or a very short test – a UserWarning names the strongly connected components and notes that method='log-lik' on its default unpowered matrix will not converge for the affected items. The other methods ('cos', 'ls', 'evm'), and 'log-lik' with matrix_power ≥ 1, are unaffected.

Called automatically on object creation when validate=True (the default; see Section 5.1.1), storing its result as self.connectivity_status.

Usage

```
self.check_data_connectivity()
```

Returns

Dictionary: 'connected', 'components_count', 'directionally_isolated_items', 'strongly_connected' always present; if disconnected, 'isolated_items'/'all_sub_groups' also present.

Example

To manually re-check connectivity on object my_rsm and inspect the number of connected components:

```
my_rsm.connectivity_status = my_rsm.check_data_connectivity()
my_rsm.connectivity_status['components_count']
```

5.1.3 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of `self.responses` with new name.

Example
To rename an item in object `my_rsm` from `Item_1` to `my_new_item_name`:

```
my_rsm.rename_item('Item_1', 'my_new_item_name')
```

5.1.4 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of `self.responses` with new names.

Example
To rename all items in object `my_rsm` with item names in a list stored as a variable `my_new_item_names`:

```
my_rsm.rename_items_all(my_new_item_names)
```

5.1.5 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the index of self.responses with new name. Also replaces person name in self.exogenous or self.no_exogenous_persons as appropriate if self.exogenous exists.

Example
To rename a person in object my_rsm from Person_1 to my_new_person_name:

```
my_rsm.rename_person('Person_1', 'my_new_person_name')
```

5.1.6 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the index of self.responses with new names. Also replaces all person names in self.exogenous and self.no_exogenous_persons if self.exogenous exists.

Example
To rename all persons in object my_rsm with person names in a list stored as a variable my_new_person_names:

```
my_rsm.rename_persons_all(my_new_person_names)
```

5.2 Core functions

5.2.1 cat_prob

Description

Category probability function which calculates the probability $P(X_{ni} = k)$ of scoring k, with $k \in \{0,\ m\}$, where m is the maximum score, from person location, central item location and Rasch-Andrich thresholds. For a person n with location β_n attempting an item i with central item location δ_i and Rasch-Andrich thresholds $\{\tau_0, ...,\ \tau_m\}$, the probability of obtaining a score of k is given by:

$$P(X_{ni} = k) = \frac{e^{k(\beta_n - \delta_i) - \sum_{t=0}^{k} \tau_t}}{\sum_{k=0}^{m} e^{k(\beta_n - \delta_i) - \sum_{t=0}^{k} \tau_t}} \tag{15}$$

In this formulation, an item is defined by a central item location, δ_i and a set of centred Rasch-Andrich thresholds $\{\tau_k\}$, $k \in \{1, ..., m\}$ which sum to zero, plus a dummy/sentinel threshold $\tau_t = 0$ for the convenience of Equation 15. An alternative formulation would be to define the item solely by m uncentred thresholds, $\{\tau'_{ik}\}$, $k \in \{1, ..., m\}$, where $\tau'_{ik} = \delta_i + \tau_k$, in analogy with the partial credit model formulation described in Section 4.2.2.

Usage

```
self.cat_prob(person_location, item_location, category, thresholds)
```

Arguments

person_location	Float: Person location
item_location	Float: Item location
category	Integer: Response category k, with $k \in \{0,\ 1\}$.
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Returns

Float: probability of obtaining score k.

Example

To obtain the probability of a person of location 0.5 scoring 0 on an item with central location 0 and a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_cat_prob`:

```
my_cat_prob = self.cat_prob(0.5, 0, 0, my_thresholds)
```

5.2.2 exp_score

Description

Expected score function which calculates the expected score $E(X_{ni})$ from person location, central item location and set of Rasch-Andrich thresholds. The expected score is given by:

$$E(X_{ni}) = \sum_{k=0}^{m} kP(X_{ni} = k) \tag{16}$$

where m is the maximum score and $P(X_{ni} = k)$ is as described in Section 5.2.1.

Usage

```
self.exp_score(person_location, item_location, thresholds)
```

Arguments

person_location	Float: Person location
item_location	Float: Item location
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Returns

Float: expected score.

Example

To obtain the expected score for a person of location 0.5 attempting an item of location 0 with a set of Rasch-Andrich thresholds stored as a variable my_thresholds, and store the result as a variable my_exp_score:

```
my_exp_score = self.exp_score(0.5, 0, my_thresholds)
```

5.2.3 variance

Description

Variance function which calculates the variance of the score $V(X_{ni})$ from person location, central item location and set of Rasch-Andrich thresholds. The variance is given by:

$$V(X_{ni}) = \sum_{k=0}^{m} P(X_{ni} = k)(k - E(X_{ni}))^2 \tag{17}$$

where m is the maximum score and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 5.2.1 and 5.2.2 respectively.

The variance is also both the Fisher information for the response and the first partial differential of the expected score function with respect to person location.

Usage

```
self.variance(person_location, item_location, thresholds)
```

Arguments

person_location	Float: Person location
item_location	Float: Item location
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Returns

Float: variance (also the Fisher information provided by the response).

Example

To obtain the variance for a person of location 0.5 attempting an item with central location 0 and a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_variance`:

```
my_variance = self.variance(0.5, 0, my_thresholds)
```

5.2.4 kurtosis

Description

Kurtosis function which calculates the kurtosis of the score $\kappa(X_{ni})$ from person location, central item location and Rasch-Andrich thresholds. The kurtosis is given by:

$$\kappa(X_{ni}) = \sum_{k=0}^{m} P(X_{ni} = k)(k - E(X_{ni}))^4 \tag{18}$$

where m is the maximum score and $P(X_{ni} = k)$ and $E(X_{ni})$ are as described in Sections 5.2.1 and 5.2.2 respectively.

Usage

```
self.kurtosis(person_location, item_location, thresholds)
```

Arguments

person_location	Float: Person location
item_location	Float: Item location
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Returns

Float: kurtosis

Example

To obtain the kurtosis for a person of location 0.5 attempting an item of location 0 with a set of Rasch-Andrich thresholds stored as a variable `my_thresholds`, and store the result as a variable `my_kurtosis`:

```
my_kurtosis = self.kurtosis(0.5, 0, my_thresholds)
```

5.3 Parameter estimation

5.3.1 `calibrate`

Description

Produces central item location estimates and a single shared set of Rasch–Andrich threshold estimates. There are three approaches to estimation available, selectable through the `estimation` argument: `'cp-full-rcl'`, `'cpat-rcl'` and `'pair-cpat'`.

CP-Full-RCL

The default, `estimation='cp-full-rcl'`, is regularised composite likelihood (RCL). For two items, every pair of response patterns (a, b) and $(a-1, b+1)$ with the same total is a binomial comparison whose log-odds depend only on the two items' locations and two thresholds. Both fit all their patterns jointly, as one composite likelihood, by Newton-Raphson, with additive smoothing constants on the item comparisons and CPAT threshold patterns ((k, k) against $(k-1, k+1)$ on two items). A category width with fewer than 20 threshold patterns, or with all its patterns on one side, is pulled toward the mean of the other widths by a ridge of $0.5\,(1 - T/20)$ (0.5 if one-sided), where T is its pattern count. The widths next to an empty middle category are left out of the joint fit and estimated separately by conditional maximum likelihood from their threshold patterns (`threshold_method='cpat-cml'`), given the item locations from the joint fit.

CPAT-RCL

`estimation='cpat-rcl'` is the same composite likelihood restricted to CPAT's patterns (RCL-CPAT). CPAT conditions on response patterns with equal totals: odd totals give the item comparisons (persons scoring one point higher on one item than on another), even totals the threshold patterns ((k, k) against $(k-1, k+1)$ on two items).

PAIR-CPAT

`estimation='pair-cpat'` is the original non-iterative CPAT method (Elliott & Buttery, 2022b), described in the rest of this section. With `estimation='pair-cpat'`, item location estimates use PAIR (Choppin, 1968, 1985), eigenvector method (Garner & Engelhard, 2002) or related conditional pairwise methods. These methods are all based on the production of a conditional category response frequency matrix $\mathbf{C}$ of pairwise comparison counts between all possible pairs of items – persons who scored one category higher on item i than on item j – pooled across all possible adjacent pairs.

Three methods are entirely non-iterative and are derived from the positive pairwise reciprocal matrix $\mathbf{D} : \mathbf{D}_{ij} = \mathbf{C}_{ji}^n / \mathbf{C}_{ij}^n$, the ratios of the counts of the $\mathbf{C}_{ij}$ after raising $\mathbf{C}$ to a power n, which introduces indirect comparisons each time $\mathbf{C}$ is raised to a higher power. From the matrix $\mathbf{D}$, the final vector is derived in different ways:

- `'ls'` is the least-squares method (Choppin, 1968, 1985), which is the logarithm of the simple arithmetic row-wise mean of $\mathbf{D}$.
- `'evm'` is the eigenvector method (Garner & Engelhard, 2002), which uses the logarithm of the eigenvector corresponding to the principal eigenvalue of $\mathbf{D}$.
- `'cos'` is the cosine maximisation method (Kou & Lin, 2014), which first normalises the columns of $\mathbf{D}$ before taking the logarithms of the row-wise means.

In all these cases, the matrix power n is user-definable.

'log-lik' is the iterative maximum likelihood estimator (Bradley & Terry, 1952; Zermelo, 1929), which iteratively maximises the log-likelihood until the change is less than the set tolerance, using 'cos' for a warm start. 'log-lik' is the default method because although usually only trivially more accurate, it is highly robust to extremely sparse data, even at small sample sizes, provided the comparison graph is strongly connected (which a non-zero constant ensures; see below). 'cos' and the other non-iterative methods can sometimes produce noticeably worse estimates under such conditions. For well conditioned data, 'cos' in particular produces practically identical results at a faster speed. 'log-lik' does not involve raising matrix powers, since it degrades accuracy for 'log-lik' rather than improving it.

The additive smoothing constant constant is a form of regularisation, and also ensures connectivity (no zeros in the matrix): zeros or very low counts in comparisons resulting from sampling error can have an outsized influence on resulting estimates, leading to bias; the constant mitigates this effect for small sample sizes with diminishing impact on larger sample sizes.

Rasch-Andrich thresholds are estimated using the conditional pairwise adjacent threshold (CPAT) method (Elliott & Buttery, 2022b), which constructs a chain of category widths – distances between adjacent thresholds – based on a conditional pairwise contrast between items that uses the already calculated item locations; the category widths are then chained together to determine the set of threshold locations, which are then zero-summed. This stage uses constant but not method or matrix_power. Since the calculations for each category use fewer persons than item comparisons, the estimates are more sensitive to the value of constant: constant=0 will sometimes cause the method to break down where zero counts are present, so keeping every cell count non-zero matters more (Elliott & Buttery, 2022b). A non-zero value for constant is therefore strongly recommended.

Items and thresholds take separate smoothing constants. By default constant is None (resolving to $\min(1, 0.548\, I^{-0.2 \log_{10} Nm})$, where N is the number of persons with at least one response, I is the number of items and m is the maximum score, for the item-location stage) and threshold_constant is None, resolving to 0.03 for the threshold stage, which needs less additive smoothing. If threshold_constant is left None and constant is set to a float, that float is used for both stages, so constant=0.1 reproduces earlier versions exactly. Passing a float to threshold_constant sets the threshold constant independently of the item constant. With estimation='cp-full-rcl' or 'cpat-rcl', the item constant (per item pair) is spread over the m boundaries: constant $/m$ on each observed (item pair, boundary) cell.

Usage

```
self.calibrate(constant=None, threshold_constant=None, method=None, matrix_power=None,
               log_lik_tol=None, threshold_method=None, no_info='mean', estimation=None)
```

Arguments

constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for the item comparisons, per item pair (with estimation='cp-full-rcl' or 'cpat-rcl', spread over the m boundaries: constant $/m$ on each observed (item pair, boundary) cell). Default constant=None resolves to a simulation-tuned value. When threshold_constant is None it also sets the Stage 2 threshold constant: to 0.03 if constant is None, otherwise to the same explicit value.
threshold_constant	Float or None: additive smoothing constant for CPAT threshold distance estimates. Default None resolves to 0.03 when constant is None; otherwise it takes the constant value. See the Description for the supporting simulation.
method	String or None: estimation='pair-cpat' only. Method for derivation of vector of central item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default method=None resolves to 'log-lik'.
matrix_power	Integer or None: estimation='pair-cpat' only. Power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) for central item location estimates is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' or 5 for other methods. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float or None: estimation='pair-cpat' only. Convergence criterion for log-likelihood method for central item location estimates. Ignored for other methods. Default log_lik_tol=None resolves to 0.000001.
threshold_method	String or None: estimation='pair-cpat' only. Estimator for the threshold stage, 'cpat-cml' or 'cpat-info-weighted' (Elliott & Buttery, 2022b). Default threshold_method=None resolves to 'cpat-cml'.
no_info	String: 'mean' or 'raise', for a category width with no information. Default is no_info='mean'.
estimation	String or None: 'cp-full-rcl', 'cpat-rcl' or 'pair-cpat' (see Description). Default estimation=None resolves to the method of the object's last calibration, or 'cp-full-rcl' for the first one. method, matrix_power, log_lik_tol and threshold_method apply to 'pair-cpat' only; other values passed with 'cp-full-rcl' or 'cpat-rcl' are ignored, with a UserWarning. 'pair-cpat' is faster on very large data.

Returns

Returns eight attributes:

`self.items`	pandas series: Item location estimates with the item names as keys and estimates as values.
`self.thresholds`	pandas series: Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, an array of m estimates, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel $\tau_0 = 0$ has been removed; the series now has length m, not $m + 1$.
`self.cat_widths`	numpy array: Array of $m - 1$ category width estimates, where m is the maximum score.
`self.constant`	Float: the Stage 1 (item-location) smoothing constant actually used, after resolving `None`.
`self.threshold_constant`	Float: the Stage 2 (CPAT threshold) smoothing constant actually used.
`self.threshold_method`	String or `None`: the threshold estimator used (`None` with `estimation='cp-full-rcl'` or `'cpat-rcl'`).
`self.estimation`	String: the estimation method used.
`self.convergence`	Dictionary or `None`: with `estimation='cp-full-rcl'` or `'cpat-rcl'`, the number of Newton–Raphson iterations (`'iterations'`) and whether the fit converged (`'converged'`); a `UserWarning` is issued if it did not. `None` with `estimation='pair-cpat'`.

Examples

To generate a set of estimates using the default RCL method, with the default data-adaptive item smoothing constant and a threshold smoothing constant of 0.03:

```
self.calibrate()
```

To generate a set of estimates using the two-stage PAIR + CPAT method used before v1.4.0, with the default Bradley–Terry log-likelihood method for central item locations:

```
self.calibrate(estimation='pair-cpat')
```

To reproduce v1.3.1 and earlier exactly, with a single shared smoothing constant of 0.1 for both stages:

```
self.calibrate(estimation='pair-cpat', constant=0.1)
```

To use a fixed smoothing constant of your own for both stages:

```
self.calibrate(constant=0.2)
```

To keep the default item-stage constant but set the threshold stage yourself:

```
self.calibrate(threshold_constant=0.05)
```

To generate a set of estimates using the log-likelihood method for central item locations with the pairwise matrix raised to power 7 (overriding the unpowered default) and a tighter convergence criterion:

```
self.calibrate(estimation='pair-cpat', method='log-lik', matrix_power=7,
               log_lik_tol=0.00000001)
```

To use the closed-form cosine maximisation method (Kou & Lin, 2014) for central item locations instead of the default:

```
self.calibrate(estimation='pair-cpat', method='cos')
```

5.3.2 std_errors

Description

Produces standard errors of central item locations, thresholds and category widths. `se_method='sandwich'` computes analytic Huber–White sandwich standard errors (Huber, 1967), with persons as the independent units; `se_method='bootstrap'` draws `no_of_samples` bootstrap resamples of persons. `se_method=None` (default) uses the sandwich, falling back to the bootstrap, with a `UserWarning`, if a score category is unused; with `estimation='cp-full-rcl'` or `'cpat-rcl'`, also if the fit did not converge, some estimates are not identified by the data, or the design is too large; with `estimation='pair-cpat'`, also if a category width has fewer than 20 threshold patterns or `threshold_method='cpat-info-weighted'`. An explicit `se_method='sandwich'` gives NaN for estimates the data do not identify.

Usage

```
self.std_errors(interval=None, no_of_samples=500, constant=None, threshold_constant=None,
                method='log-lik', matrix_power=None, log_lik_tol=0.000001, seed=None,
                se_method=None)
```

Arguments

`interval`	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval. Defines a central interval of proportion p, e.g. `interval=0.95` defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is `interval=None`.
`no_of_samples`	Integer: Number of bootstrap samples to generate. More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is `no_of_samples=500`.
`constant`	Float: additive smoothing constant (Elliott & Buttery, 2022b). Default is `constant=None`, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} Nm})$, where N is the number of persons with at least one response, I is the number of items and m is the maximum score.
`threshold_constant`	Float or `None`: additive smoothing constant for the Stage 2 CPAT threshold distances (see Section 5.3.1). `None` (default) resolves to `0.03`.
`method`	String: method for derivation of vector of central item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are `'ls'` for least squares (Choppin, 1968, 1985), `'evm'` for the eigenvector method(Garner & Engelhard, 2002), `'cos'` for cosine maximisation (Kou & Lin, 2014) or `'log-lik'` for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is `method='log-lik'`

Continued on next page...

matrix_power	Integer or None: power to which conditional category response frequency matrix for central item location estimates (Elliott & Buttery, 2022b:991) is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.
seed	Integer or None: seed for the bootstrap resampling random number generator. Pass an integer for fully reproducible standard errors; None (default) draws fresh entropy each call. See Section 1.
se_method	String or None: 'sandwich', 'bootstrap' or None (see Description).

Returns

Attributes:

self.se_method	String: 'sandwich' or 'bootstrap', the method used.
self.se_covariance	Dictionary or None: sandwich covariance matrices, {'items': pandas dataframe, 'cat_widths': pandas dataframe} (sandwich only; None for the bootstrap).
self.item_bootstrap	pandas dataframe: Full bootstrap results, with a row for each bootstrap sample and a column for each item estimate (None with the sandwich).
self.item_se	pandas series: Item names as keys and item standard errors as values.
self.threshold_bootstrap	pandas dataframe: Full bootstrap results, with a row for each bootstrap sample and a column for each threshold estimate.
self.threshold_se	pandas series: Threshold numbers as keys and threshold standard errors as values.
self.cat_width_bootstrap	pandas dataframe: Full bootstrap results, with a row for each bootstrap sample and a column for each category width estimate.
self.cat_width_se	pandas series: Category numbers as keys and category width standard errors as values.

If an argument is passed to `interval`, also returns:

`self.item_low`	Lower bound of the specified interval for item estimates.
`self.item_high`	Upper bound of the specified interval for item estimates.
`self.threshold_low`	Lower bound of the specified interval for threshold estimates.
`self.threshold_high`	Upper bound of the specified interval for threshold estimates.
`self.cat_width_low`	Lower bound of the specified interval for category estimates.
`self.cat_width_high`	Upper bound of the specified interval for category estimates.

Example

To generate item standard errors with a 95% interval from 200 samples:

```
self.std_errors(interval=0.95, no_of_samples=200)
```

Modifications to the estimation method are discussed in Section 5.3.1.

5.3.3 calibrate_anchor

Description

Anchors item location estimates onto externally-supplied reference values, supporting item banking: after running a standard calibration (Section 5.3.1), the whole item-location scale is shifted by a translation constant so that a subset of common ("anchor") items line up with externally-supplied reference locations, e.g. from a bank of previously-calibrated items. This is a simple mean-shift.

By default (`selection_method='robust_z'`), the translation constant is computed via a robust, iterative outlier-trimming procedure based on the modified z-score (Iglewicz & Hoaglin, 1993): anchor items whose estimated values have drifted relative to the other anchor items when compared to the externally-supplied reference values are excluded before the shift is computed, so that a handful of stale or otherwise misbehaving anchor items cannot distort the shift applied to the rest of the scale. `selection_method='wald'` is an alternative that uses a formal significance test per anchor item instead of a descriptive-statistics trim, and requires bootstrap item standard errors (auto-triggers `std_errors`, Section 5.3.2, if not already computed). `selection_method='none'` takes a plain mean shift over all supplied anchors, with no trimming. Diagnostics from either selection procedure can be visualised with `plot_anchor_selection` (Section 3.5.10).

The shared threshold vector `self.thresholds` is left untouched by anchoring: thresholds describe the relative category structure common to all items, not an item's location on the location scale, so an item-bank shift does not apply to them. Accordingly, there is no `anchor_thresholds` attribute with a modified version of `self.thresholds`.

Optionally, if an external threshold vector is available to go with the anchor items, passing it via `anchor_thresholds` runs two checks of whether it is compatible with the response data:

1. An omnibus likelihood-ratio test: compares the model fit (person locations re-estimated in each case) using the response data's estimated thresholds against the given `anchor_thresholds`, holding item locations fixed at `self.anchor_items`. A significantly worse fit under `anchor_thresholds` warns that the rating-scale structures may not actually be shared, undermining the validity of a translation-only equating assumption.

2. A per-category test, based on the response data's category widths (consecutive threshold differences), with bootstrap standard errors from `cat_width_se` (auto-triggers `std_errors`, Section 5.3.2, if not already computed). Estimated category widths are compared against `anchor_thresholds` widths via a Wald z-test, one per category. This localised test identifies which categories have different widths from the anchor values, in the same way `dif_test`'s category-width (DCF) test does across groups (Section 5.4.12). The test operates on category widths rather than threshold locations since threshold locations, and their standard errors, are mutually interdependent due to the sum-zero constraint.

Usage

```
self.calibrate_anchor(anchors, calibrate=False, selection_method='robust_z',
                      corr_tol=0.95, sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05,
                      no_of_samples=500, adj=None, overwrite_anchors='none',
                      plot=True, plot_kwargs=None, anchor_thresholds=None,
                      alpha=0.05, correction='bh', warm_corr=True,
                      tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                      constant=None, threshold_constant=None, method='log-lik',
                      matrix_power=None, log_lik_tol=0.000001, seed=None,
                      estimation=None, threshold_method=None)
```

Arguments

`anchors`	Dictionary or pandas series: reference locations for a subset of items, keyed/indexed by item name.
`calibrate`	Boolean: if `True`, (re-)runs `calibrate` before anchoring. If `False`, `calibrate` is still auto-triggered if it has not already been run. Default is `calibrate=False`.
`selection_method`	String, one of `'robust_z'`, `'wald'` or `'none'`. `'robust_z'` (default): excludes outlying anchor items via the modified z-score procedure (Iglewicz & Hoaglin, 1993); uses `corr_tol`, `sd_ratio_tol`, `min_anchors`. `'wald'`: a formal per-item sequential significance test based on z-scores computed from item standard errors – needs bootstrap standard errors, so auto-triggers `std_errors` if not already computed; uses `wald_alpha` and `min_anchors`, not `corr_tol`/`sd_ratio_tol`. `'none'`: plain mean shift over every supplied anchor, no trimming.
`corr_tol`	Float: minimum acceptable Pearson correlation between anchor and observed values on the trimmed item set (`selection_method='robust_z'` only). Default is `corr_tol=0.95`.
`sd_ratio_tol`	Float: the ratio of observed to anchor standard deviations on the trimmed item set must fall within $[1/\texttt{sd_ratio_tol}, \texttt{sd_ratio_tol}]$ (`robust_z` only). Default is `sd_ratio_tol=1.1`, i.e. the ratio must lie in $[0.909, 1.1]$.
`min_anchors`	Integer: minimum number of anchor items after trimming – selection stops (with a warning) once this floor is reached, even if tolerance/significance criterion is not yet satisfied, and is skipped altogether if the initial common item set is at or below this floor (`selection_method` in `'robust_z'`/`'wald'` only). Default is `min_anchors=6`.
`wald_alpha`	Float: significance level for each anchor item's Wald test (`selection_method='wald'` only). Default is `wald_alpha=0.05`.

Continued on next page...

no_of_samples	Integer: bootstrap samples for the auto-triggered std_errors call, only used if item standard errors are not already computed (selection_method='wald' only). Default is no_of_samples=500.
adj	Float or None: if provided, applied as translation constant and selection step skipped. Intended for reuse across bootstrap resamples (Section 5.3.2), where recomputing anchor selection every time would inflate standard errors with anchor-item sampling variance. Default adj=None.
overwrite_anchors	String, one of 'none', 'rejected' or 'all': controls what self.anchor_items holds for the anchor items. 'none': every anchor item keeps exactly its externally-supplied value, unchanged – the usual convention, since an anchor's value is fixed by definition, not re-estimated. 'rejected': retained anchors keep supplied values, rejected anchors have their values overwritten with new estimates; requires selection_method in 'robust_z'/'wald' and no adj override (otherwise falls back to 'none' with a warning). 'all': every anchor item has its value overwritten with a new, freshly-calibrated value like any other item. Default is overwrite_anchors='none'.
plot	Boolean: if True and selection_method in 'robust_z'/'wald', calls plot_anchor_selection (Section 3.5.10). Has no effect when selection_method='none' or adj is supplied. Default plot=True.
plot_kwargs	Dictionary or None: extra keyword arguments forwarded to plot_anchor_selection when plot=True (e.g. filename, x_min/x_max, graph_title). Default is plot_kwargs=None.
anchor_thresholds	Array-like or None: externally-supplied reference threshold vector, length max_score. If supplied, runs both checks described above and warns if either flags a difference. If None, both checks are skipped entirely (self.anchor_threshold_test and self.anchor_category_width_test are set to None). Default is anchor_thresholds=None.
alpha	Float: significance level for both the omnibus likelihood-ratio test (estimated thresholds, df = max_score $-$ 1, versus anchor_thresholds, fixed, $df = 0$, chi-squared with df = max_score $-$ 1) and the per-category Wald z-tests (compared against the corrected p-value if correction is set, else raw p-value). Default is alpha=0.05.
correction	String, one of 'bh', 'bonferroni' or None: multiple-comparison correction across the per-category width tests only. Not applied to the omnibus test, which is a single test. Default is correction='bh'.
warm_corr, tolerance, max_iters, ext_score_adjustment	Person-estimation arguments, used only by the omnibus anchor_thresholds comparison.

Continued on next page...

constant, threshold_constant, estimation, threshold_method, method, matrix_power, log_lik_tol	Calibration arguments, used only if calibrate is triggered – see Section 5.3.1. method, matrix_power, log_lik_tol and threshold_method apply to estimation='pair-cpat' only.
seed	Integer or None: seed passed through to the internal std_errors call (only used if item standard errors are not already computed, selection_method='wald' only). None (default) draws fresh entropy each call. See Section 1.

Returns

self.anchor_items	pandas series: item locations shifted onto the anchor scale.
self.anchor_item_names	pandas index: names of the items supplied as anchors.
self.anchor_adj	Float: the translation constant actually applied.
self.anchor_selection	pandas dataframe or None: per-item diagnostics from whichever selection method was used – Anchor, Observed, Deviation, Robust z, Selected ('robust_z'), or Anchor, Observed, SE, Deviation, z, p, Selected ('wald'). None if selection_method='none' or adj supplied.
self.anchor_selected_items, self.anchor_dropped_items	pandas index or None: items retained / excluded as outliers. None if selection_method='none' or adj supplied.
self.anchor_original_corr, self.anchor_original_sd_ratio	Float or None: correlation / SD ratio before trimming.
self.anchor_corr self.anchor_sd_ratio	Float or None: correlation / SD ratio after trimming.
self.anchor_summary	pandas series: one-line summary – anchors supplied/common/selected/dropped, correlation and SD ratio before/after trimming, translation constant applied.
self.anchor_plot	matplotlib figure or None: figure from the auto-triggered plot_anchor_selection call. None if plot=False, selection_method='none', or adj was supplied directly.
self.calibrate_anchor_runs	Dictionary: every call's results above, keyed by the sorted anchor items supplied, so results from an earlier call survive a later call with a different anchor set instead of being overwritten in place.

Optional threshold-structure checks additionally return:

`self.anchor_threshold_test`	pandas series or `None`: omnibus test – `LL_estimated`, `LL_given`, `LR`, `df`, `p` and a boolean `Flagged`. `None` if `anchor_thresholds` not supplied.
`self.anchor_category_width_test`	pandas dataframe or `None`: per-category test, indexed by category – `Estimate` (response data's estimated category width), `Given` (`anchor_thresholds`' corresponding category width), `Difference`, `SE` (from `cat_width_se`), `z`, `p`, `p_corrected` (if `correction` is set), boolean `Flagged`. `None` if `anchor_thresholds` not supplied.

Examples

To anchor item locations onto a bank of reference values stored in a dictionary `my_bank`, with automatic outlier trimming and diagnostic plot:

```
self.calibrate_anchor(my_bank)
```

To additionally check compatibility with a reference threshold vector stored as `my_bank_thresholds`:

```
self.calibrate_anchor(my_bank, anchor_thresholds=my_bank_thresholds)
```

5.3.4 person

Description

Generates a location estimate for a person using the Newton-Raphson method to produce a maximum likelihood estimate, or, by default, Warm's weighted likelihood estimate (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.person(person, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
            ext_score_adjustment=0.5)
```

Arguments

person	String: The person name for the location being estimated.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations. Default max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimation. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.

Returns

Float: person location estimate.

Examples

To generate a person location estimate for Person_1 using the default settings and store the result as a variable, my_person_location:

`my_person_location = my_person_location = self.person('Person_1')`

To generate an MLE person location estimate without Warm bias correction for `Person_1` based on the first three items and store the result as a variable, `my_person_location`:

```
my_person_location = self.person('Person_1', ['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

5.3.5 person_estimates

Description

Generates location estimates for all persons using the Newton-Raphson method to produce maximum likelihood estimates, or, by default, Warm's weighted likelihood estimates (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.person_estimates(items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
                      ext_score_adjustment=0.5)
```

Arguments

items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations. Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimation. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.

Returns

Attribute self.persons: pandas series with person names as keys and location estimates as values.

Examples

To generate a set of person location estimates with Warm bias correction:

```
self.person_estimates()
```

To generate a set of person location estimates without Warm bias correction, on a subset of the first three items only:

```
self.person_estimates(items=['Item_1', 'Item_2', 'Item_3'], warm_corr=False)
```

5.3.6 score_lookup

Description

Generates a location estimate for a given raw score on responses to a given set of items using the Newton-Raphson method to produce a maximum likelihood estimate, or, by default, Warm's weighted likelihood estimate (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.score_lookup(score, items=None, warm_corr=True, tolerance=0.00001, max_iters=100,
           ext_score_adjustment=0.5)
```

Arguments

score	Integer: The raw score for which location is being estimated.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations. Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the score is extreme (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.

Returns

pandas series with raw scores as keys and person location estimates as values.

Examples

To generate a location estimate for a score of 10 on all items, with Warm bias correction, and store the result

as a variable, `my_score_location`:

```
my_score_location = self.score_lookup(10)
```

To generate a location estimate for a score of 10 on a subset of items saved as a variable `my_items`, without Warm bias correction, and store the result as a variable, `my_score_location`: `my_score_location = self.score_lookup(10, items=my_items, warm_corr=False)`

5.3.7 score_lookup_table

Description

Generates a lookup table of location estimates corresponding to all available raw scores on a set of items with no missing responses, using the Newton-Raphson method to produce maximum likelihood estimates, or, by default, Warm's weighted likelihood estimates (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.score_lookup_table(items=None, ext_scores=True, warm_corr=True, tolerance=0.00001,
                  max_iters=100, ext_score_adjustment=0.5,
                  filename=None, file_format='csv')
```

Arguments

items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
ext_scores	Boolean: If True, estimates for extreme scores (all correct/all incorrect) generated using the ext_score_adjustment argument. Default ext_scores=True.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence criterion for Newton-Raphson iterations. Default tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations. Default max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned if the score is extreme (all responses correct or incorrect). Adjusts the score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimation.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.score_table: pandas series with raw scores as keys and location estimates as values.

Examples

To generate a location lookup table for all items, including extreme scores, with Warm bias correction:

```
self.score_lookup_table()
```

To generate a location lookup table for a subset of items saved as a variable `my_items)`, without extreme scores and without Warm bias correction:

```
self.score_lookup_table(items=my_items, ext_scores=False)
```

5.3.8 csem

Description

Calculates conditional standard error of measurement for a person.

Usage

```
self.csem(person, person_locations=None, items=None)
```

Arguments

person	Person name.
person_locations	pandas series (or dictionary) with person names as keys and locations as values. If None, uses self.persons, automatically generating if necessary. Default is self.persons=None.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.

Returns

Float: conditional standard error of measurement for location estimate.

Examples

To generate the CSEM for Person_1 on all items and save the result as a variable, my_csem:

```
my_csem = self.csem('Person_1')
```

To generate the CSEM for a raw score of 3 on a subset of items saved to a variable my_items and save the result as a variable, my_csem:

```
my_csem = self.csem(3, person_locations=self.score_table, items=my_items)
```

where self.score_table is the output from self.score_lookup_table(items=my_items) (see Section 5.3.7).

5.4 Statistical output

5.4.1 item_stats_df

Description

Produces a table of item statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

```
self.item_stats_df(full=False, zstd=False, point_measure_corr=False, dp=3, se=True,
                   warm_corr=True, tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                   method='log-lik', constant=None, matrix_power=None, log_lik_tol=0.000001,
                   no_of_samples=500, interval=None, seed=None,
                   filename=None, file_format='csv')
```

Arguments

full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a summary table with the most commonly reported statistics.
zstd	Boolean: If True, infit and outfit standardised z-scores are reported alongside mean square statistics. Default is zstd=False. Automatically True if full=True.
point_measure_corr	Boolean: If True, point-polyserial correlation (Kornbrot, 2014) between person location and item score is reported, together with the expected value of the point-measure polyserial correlation for an ideal item. Default is point_measure_corr=False. Automatically True if full=True.
dp	Integer: Number of decimal places reported in output.
se	Boolean: If True, computes and reports the SE column (and CI bound columns, if interval is set). If False, skips the bootstrap entirely – useful when only Infit MS/Outfit MS are needed (e.g. repeated simulation runs), since those do not depend on the bootstrap and are unaffected by this flag. Forces interval=None when False. Default is se=True.
warm_corr	Boolean: If True, Warm's (1989) bias correction applied during generation of person estimates (see Section 5.3.4). Default warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is max_iters=100.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person estimates (see Section 5.3.4) if a person has an extreme score (all responses correct or incorrect). Adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimation.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 5.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation (see section 5.3.1). Default is constant=None, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} Nm})$, where N is the number of persons with at least one response, I is the number of items and m is the maximum score. Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: additive smoothing constant for the Stage 2 CPAT threshold distances (see Section 5.3.1). None (default) resolves to 0.03.
matrix_power	Integer or None: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of item location estimates. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 5.3.1). Ignored for other methods.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 5.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Unused if se=False. Default is no_of_samples=500.
interval	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 5.3.2). Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Ignored if se=False. Default is interval=None.

Continued on next page...

`seed`	Integer or `None`: seed passed through to the internal `std_errors`/`fit_statistics` calls (only used if not already computed). `None` (default) draws fresh entropy each call. See Section 1.
`filename`	String: The filename for the saved table, with no suffix for format. If `None`, no file will be saved. Default is `filename=None`.
`file_format`	The format of the file: `csv` or `xlsx`. Default is `file_format=csv`.

Returns

Returned and stored as attribute `self.item_stats`, a pandas dataframe with one row for each item and the following columns:

`Estimate`	Central item location estimate.
`SE`	Bootstrapped standard error of central item location estimate. Only produced if `se=True`.
`2.5%`	Lower bound of empirical bootstrap estimate interval. Column name will change to fit `interval` value: 2.5% represents the lower bound for `interval=0.95` but, for example, `interval=0.9` will produce a column labelled `5%`. Only produced if `se=True` and `interval` is set.
`97.5%`	Upper bound of empirical bootstrap estimate interval. Column name will change to fit `interval` value: 97.5% represents the lower bound for `interval=0.95` but, for example, `interval=0.9` will produce a column labelled `95%`. Only produced if `se=True` and `interval` is set.
`Count`	Count of responses.
`Facility`	Item facility: proportion of correct responses.
`Infit MS`	Infit mean square.
`Infit Z`	Standardised infit z-score. Only produced if `zstd=True`.
`Outfit MS`	Outfit mean square.
`Outfit Z`	Standardised outfit z-score. Only produced if `zstd=True`.
`PM corr`	Point-polyserial correlation. Only produced if `point_measure_corr=True`.
`Exp PM corr`	Expected value of point-measure polyserial correlation. Only produced if `point_measure_corr=True`.
`Disordered`	If `True`, threshold estimates indicate the presence of disordered thresholds within the item (Andrich, 2010).

Examples

To produce a summary `item_stats` table with the most commonly reported statistics:

```
my_rsm.item_stats_df()
```

To produce a full `item_stats` table with all statistics:

```
my_rsm.item_stats_df(full=True)
```

To produce an `item_stats` table with with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
my_rsm.item_stats_df(zstd=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

5.4.2 threshold_stats_df

Description

Produces a table of threshold statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

```
self.threshold_stats_df(full=False, zstd=False, disc=False, point_measure_corr=False, dp=3,
                        warm_corr=True, tolerance=0.00001, max_iters=100,
                        ext_score_adjustment=0.5, method='log-lik', constant=None,
                        no_of_samples=500, interval=None,
                        filename=None, file_format='csv')
```

Arguments

full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a summary table with the most commonly reported statistics.
zstd	Boolean: If True, infit and outfit standardised z-scores are reported alongside mean square statistics. Default is zstd=False. Automatically True if full=True.
disc	Boolean: If True, item discrimination is reported. The discrimination of the empirical threshold slope, which functions as an SLM item conditioned on responses being in the categories either side, relative to the ideal logistic ogive, with 1 perfect, > 1 showing overfit and < 1 showing underfit; discrimination is similar to the 2PL IRT discrimination parameter (Linacre, 2023), but is a descriptive statistic here rather than an item parameter.
point_measure_corr	Boolean: If True, point-biserial correlation (Kornbrot, 2014) between person location and item score is reported, together with the expected value of the point-measure biserial correlation for an ideal item. Default is point_measure_corr=False. Automatically True if full=True.
dp	Integer: Number of decimal places reported in output.
warm_corr	Boolean: If True, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 5.3.4). Default is warm_corr=True.
tolerance	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default max_iters=100.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 5.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 5.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 5.3.1). Default is constant=None, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} Nm})$, where N is the number of persons with at least one response, I is the number of items and m is the maximum score. Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: additive smoothing constant for the Stage 2 CPAT threshold distances (see Section 5.3.1). None (default) resolves to 0.03.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 5.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is no_of_samples=500.
interval	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 5.3.2). Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is interval=None.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.threshold_stats, a pandas dataframe with one row for each threshold and the following columns:

Estimate	Rasch-Andrich threshold estimate.
SE	Bootstrapped standard error of threshold estimate.
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 2.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 5%.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 97.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 95%.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if zstd=True.
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if zstd=True.
Discrim	Item discrimination. Only produced if disc=True.
PM corr	Point-biserial correlation. Only produced if point_measure_corr=True.
Exp PM corr	Expected value of point-measure biserial correlation. Only produced if point_measure_corr=True.

Examples

To produce a summary threshold_stats table with the most commonly reported statistics:

```
my_rsm.threshold_stats_df()
```

To produce a full threshold_stats table with all statistics:

```
my_rsm.threshold_stats_df(full=True)
```

To produce an threshold_stats table with with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
my_rsm.threshold_stats_df(zstd=True)
```

Other arguments may be used to alter parameters of central item location, threshold and person location estimation.

5.4.3 category_stats_df

Description

Produces a table of category *width* statistics – threshold_stats_df (Section 5.4.2) reports raw threshold locations; this reports the difference between consecutive thresholds for each item, $\tau_{k+1} - \tau_k$. The shared, zero-summed threshold vector has only max_score−1 true degrees of freedom, so the per-threshold standard errors in threshold_stats_df are correlated rather than independent, and understate the true amount of error. No infit/outfit or other fit statistics are reported, since these apply to thresholds rather than categories. Auto-triggers calibrate (Section 5.3.1) and std_errors (Section 5.3.2) directly if not already run.

A category width can be negative – a negative width at category k means thresholds τ_k and τ_{k+1} are disordered for that item (category k is never the most likely response at any person location). Prop disordered makes this a continuous diagnostic rather than a straightforward yes/no: the proportion of bootstrap resamples in which that item's category width was negative, reasonably read as the probability that the true category is disordered.

Usage

```
self.category_stats_df(dp=3, constant=None, threshold_constant=None, method='log-lik', matrix_
power=None,
                       log_lik_tol=0.000001, no_of_samples=500, interval=None, seed=None,
                       filename=None, file_format='csv')
```

Arguments

dp	Integer: Number of decimal places reported in output. Default is dp=3.
constant, method, matrix_power, log_lik_tol	Calibration keyword arguments, used only if calibrate (Section 5.3.1) has not already been run. See Section 5.3.1 for details of each. Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates, used only if std_errors (Section 5.3.2) has not already been run. Default is no_of_samples=500.
interval	Float or None: Empirical interval to define quantiles of estimates from bootstrap samples (see Section 5.3.2). Only included in the output if std_errors is triggered by this call, or was already run with an interval – not retroactively added to an existing standard error run made without one. Default is interval=None.
seed	Integer or None: seed passed through to std_errors, used only if not already run. None (default) draws fresh entropy each call. See Section 1.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute `self.category_stats`, a pandas dataframe with one row per category, indexed `Category 1` to `Category max_score−1` (one fewer row than `threshold_stats`, since a width is defined between each pair of consecutive thresholds) – with the following columns:

`Estimate`	Category width (can be negative).
`SE`	Bootstrapped standard error of the width, differenced *within* each bootstrap resample before taking the standard deviation, so it correctly reflects the covariance between adjacent thresholds rather than naively combining their individual standard errors.
`2.5%` / `97.5%`	Bounds of the empirical bootstrap estimate interval, if `interval` is available (see `threshold_stats_df`, Section 5.4.2, for the column-naming convention).
`Disordered`	Boolean: `True` if `Estimate` is negative.
`Prop disordered`	Proportion of bootstrap resamples in which this category's width was negative – reasonably read as the probability that the true category is disordered. Useful for interpreting `Disordered` in both directions: a `True` that is only weakly supported (`Prop disordered` close to 0.5) versus robust, or a `False` that is nonetheless uncertain versus clear-cut.

Example

To produce the `my_rsm.category_stats` table, alongside the threshold-level table:

```
my_rsm.threshold_stats_df()
my_rsm.category_stats_df()
```

To check which categories, if any, are disordered:

```
my_rsm.category_stats_df()
my_rsm.category_stats[my_rsm.category_stats['Disordered']]
```

5.4.4 person_stats_df

Description

Produces a table of person statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

```
self.person_stats_df(full=False, rsem=False, dp=3, se=True, warm_corr=True, tolerance=0.00001,
                     max_iters=100, ext_score_adjustment=0.5, method='log-lik', constant=None,
                     filename=None, file_format='csv')
```

Arguments

full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a summary table with the most commonly reported statistics.
rsem	Boolean: If True, realistic standard error of measurement (RSEM), which takes into account item fit (Wright, 1996), is reported alongside the conditional standard error of measurement (CSEM). Default is rsem=False. Automatically True if full=True.
dp	Integer: Number of decimal places reported in output.
se	Boolean: passed through to the internal fit_statistics call (only used if not already computed). If False, skips the bootstrap entirely – this table's columns (CSEM, RSEM, Infit/Outfit) do not depend on it, so se=False is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is se=True.
warm_corr	Boolean: If True, Warm's (1989) bias correction is applied during generation of person estimates (see Section 5.3.4). Default warm_corr=True.
tolerance	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default max_iters=100.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 5.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 5.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 5.3.1). Default is constant=None, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} Nm})$, where N is the number of persons with at least one response, I is the number of items and m is the maximum score. Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: additive smoothing constant for the Stage 2 CPAT threshold distances (see Section 5.3.1). None (default) resolves to 0.03.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 5.3.2). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is no_of_samples=500.
interval	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 5.3.2). Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is interval=None.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.person_stats, a pandas dataframe with one row for each person and the following columns:

Estimate	Item location estimate.
CSEM	Conditional standard error of measurement for person location estimate.
RSEM	Realistic standard error of measurement for person location estimate. Only produced if rsem=True
Score	Number of correct responses.
Max score	Maximum available score (number of items attempted).
p	Proportion of correct repsonses.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score.
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score.

Examples

To produce a summary person_stats table with the most commonly reported statistics:

```
my_rsm.person_stats_df()
```

To produce a full person_stats table with all statistics:

```
my_rsm.person_stats_df(full=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

5.4.5 test_stats_df

Description

Produces a table of test-level statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX:

`Separation ratio` (Wright, 1996; Wright & Masters, 1982) is the SD of person locations reported as a ratio of SE units. For persons, $G_p = \sigma_p / \sqrt{\sum_n SE_n^2}$, where σ_p is the variance of the person estimates and SE_n is the RSEM (see Section 6.4.6) for person n. The formula is symmetrical for items, substituting the SE of estimation for RSEM.

`Strata` is the number of statistically distinct levels of either person or item locations as strata with centers three measurement errors apart (Wright & Masters, 1982:106). For persons, $H_p = (4G_p + 1)/3$, with symmetrical results for items.

`Reliability` is Rasch-specific reliability statistic (Wright, 1996), derived from PSI. reliability is similar to Cronbach's Alpha (Cronbach, 1951), and may be interpreted the same way: the proportion of variance of the estimates which stems from variation in location or location rather than estimation error. For persons, $R_p = G_p^2/(1 + G_p^2)$, with symmetrical results for items.

Usage

```
self.test_stats_df(dp=3, warm_corr=True, tolerance=0.00001, max_iters=100,
                   ext_score_adjustment=0.5, method='log-lik', constant=None, alpha=False,
                   seed=None, filename=None, file_format='csv')
```

Arguments

`dp`	Integer: Number of decimal places reported in output.
`warm_corr`	Boolean: If `True`, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 5.3.4). Default is `warm_corr=True`.
`tolerance`	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is `tolerance=0.00001`.
`max_iters`	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is `max_iters=100`.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 5.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 5.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation (see section 5.3.1). Default is constant=None, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} Nm})$, where N is the number of persons with at least one response, I is the number of items and m is the maximum score. Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: additive smoothing constant for the Stage 2 CPAT threshold distances (see Section 5.3.1). None (default) resolves to 0.03.
alpha	Boolean: if True, adds a Cronbach alpha row to the Persons column of the returned table (the Items column is left blank, since Cronbach's alpha (Cronbach, 1951) is a person-side reliability statistic). Computed on complete cases; if the data contain missing responses, a warning is raised and alpha is computed after listwise deletion, which may underestimate the true value. Default is alpha=False.
seed	Integer or None: seed passed through to the internal fit_statistics call (only used if not already computed). None (default) draws fresh entropy each call. See Section 1.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.test_stats, a pandas dataframe with two columns, Items and Persons.

Mean	Mean of estimates.
SD	Standard deviation of estimates.
Separation ratio	Separation ratio
Strata	Number of strata
Reliability	Reliability

Example

To produce a `self.test_stats` table:

```
my_rsm.test_stats_df()
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

5.4.6 model_selection

Description

Compares RSM against PCM – RSM is the constrained (nested) model, sharing one threshold structure across all items; PCM is the unconstrained model, giving each item its own threshold structure – via a likelihood ratio test, AIC, or BIC, to help decide which model is the more appropriate, parsimonious fit for a given dataset. The comparison is symmetric: both RSM.model_selection and PCM.model_selection set the same self.model_comparison_rsm_pcm_* attribute names regardless of which class the method is called from. Internally constructs and calibrates a PCM object from the same response data for comparison; auto-triggers calibrate (Section 5.3.1) and person (Section 5.3.4) on self directly if not already run.

Usage

```
self.model_selection(test='AIC', aic_sig_test=True, alpha=0.05, min_effect=0,
                     sampling='dynamic', warm_corr=True, tolerance=0.00001, max_iters=100,
                     ext_score_adjustment=0.5, constant=None, threshold_constant=None,
                     method='log-lik', log_lik_tol=0.000001, seed=None)
```

Arguments

test	String: 'LR', 'AIC', or 'BIC'. Default is test='AIC'.
aic_sig_test	Boolean: When test='AIC', if True, applies a significance test with RSM as the null hypothesis. The p-value is the relative likelihood of RSM vs PCM, $p = e^{-\Delta/2}$ where $\Delta = \text{AIC}_{\text{RSM}} - \text{AIC}_{\text{PCM}}$. PCM is preferred only if $p <$ alpha; otherwise RSM is retained as the more parsimonious default. Default is aic_sig_test=True.
alpha	Float: significance level for test='LR' or test='AIC' with aic_sig_test=True. Default is alpha=0.05.
min_effect	Float: minimum effect size, in logits, required to prefer PCM over RSM. Optional; 0.43 and 0.64 mirror the ETS DIF categories. Default is min_effect=0 (no minimum).
sampling	None, 'dynamic', or Integer: Subsamples T non-extreme persons before computing log-likelihoods: at very large N, the LR/AIC/BIC comparison becomes biased toward the more complex (PCM) model, since log-likelihood differences scale with N; capping the effective N mitigates this. An integer defines T. 'dynamic' uses $T = \min(20(I-1)(m-1), 1500)$, where I is the number of items and m the shared maximum score; subsampling is skipped whenever the actual non-extreme sample size is $\leq T$. None applies no subsampling. Default is sampling='dynamic'.
warm_corr, tolerance, max_iters,	Person location estimation keyword arguments, used only if person (Section 5.3.4) has not already been run on self. Always used for the internally-

Continued on next page...

ext_score_adjustment	constructed PCM comparison object. See Section 5.3.4 for details of each.
constant, method, log_lik_tol	Calibration keyword arguments, used only if calibrate (Section 5.3.1) has not already been run on self. Always used for the internally-constructed PCM comparison object. See Section 5.3.1 for details of each.
seed	Integer or None: seed for the person-subsampling random number generator, only used when sampling triggers. Pass an integer for a reproducible comparison; None (default) draws fresh entropy. See Section 1.

Returns

Sets whichever of the following attributes correspond to test (all named from the RSM-vs-PCM comparison regardless of which class this method is called from, since the comparison itself is symmetric):

self.model_comparison_rsm_pcm_lr, _df, _p, _lr_preferred, _lr_summary	Likelihood ratio statistic, degrees of freedom, p-value, the preferred model as a string (RSM is the null; PCM is preferred if $p <$ alpha), and a one-line pandas Series summary (including a Preferred row). Only set if test='LR'.
self.model_comparison_rsm_pcm_aic, _aic_preferred, _aic_summary	AIC values for both models, the preferred model as a string, and a summary Series. Only set if test='AIC'.
self.model_comparison_rsm_pcm_aic_p	The AIC relative-likelihood p-value described above. Only set if test='AIC' and aic_sig_test=True.
self.model_comparison_rsm_pcm_bic, _bic_preferred, _bic_summary	BIC values for both models, the preferred model as a string, and a summary Series. Only set if test='BIC'.

Example

To compare my_rsm against the equivalent PCM using the default AIC significance test:

```
my_rsm.model_selection()
my_rsm.model_comparison_rsm_pcm_aic_summary
```

To run a likelihood ratio test, with a fixed subsampling size of 800 persons to guard against large-N bias:

```
my_rsm.model_selection(test='LR', sampling=800)
```

5.4.7 global_fit

Description

Wald test of invariance of the item locations and category widths between two groups of persons, each calibrated separately, using sandwich covariances. `method='andersen-lr'` is not yet available.

Usage

```
self.global_fit(method='wald', split='score', covariate=None, welch=True, alpha=0.05,
                    seed=None, split_prop=0.5, se_correction='kauermann-carroll')
```

Arguments

`method`	String: `'wald'` or `'andersen-lr'` (raises `NotImplementedError`). Default is `method='wald'`.
`split`	String: `'score'` (median mean score), `'person_location'`, `'random'` or `'exogenous'`. Default is `split='score'`.
`covariate`	String or `None`: column of `self.exogenous` for `split='exogenous'`. Default is `covariate=None`.
`split_prop`	Float: proportion of persons in each outer group for `'score'` or `'person_location'` splits (0.5 = median split). Default is `split_prop=0.5`.
`welch`	Boolean: refer W/df to an F distribution with denominator degrees of freedom from the smaller group; otherwise W to χ^2. Default is `welch=True`.
`alpha`	Float: significance level. Default is `alpha=0.05`.
`seed`	Integer or `None`: seed for `split='random'`. Default is `seed=None`.
`se_correction`	String or `None`: small-sample correction of the sandwich covariances, `'kauermann-carroll'` (Kauermann & Carroll, 2001) or `None`. Default is `se_correction='kauermann-carroll'`.

Returns

Attributes:

`self.global_fit_summary`	pandas series: statistic, degrees of freedom, p-value and decision.
`self.global_fit_table`	pandas dataframe or `None`: per-parameter difference, SE and z (Wald only).
`self.global_fit_excluded`	List: parameters not compared (not estimable in both groups).
`self.global_fit_groups`	Dictionary: {`group name: calibrated model`}.

Examples

To test invariance between low and high scorers (the default median score split):

```
self.global_fit()
```

To test invariance between two groups defined by an exogenous variable `'Gender'`, stored in `self.exogenous`:

```
self.global_fit(split='exogenous', covariate='Gender')
```

5.4.8 res_corr_analysis

Description

Analysis of correlations of standardised residuals to test for violations of local item interdependence and unidimensionality requirements.

Usage

```
self.res_corr_analysis(warm_corr=True, tolerance=0.00001, max_iters=100,
                       ext_score_adjustment=0.5, constant=None, threshold_constant=None, method='log-lik'
                       matrix_power=None, log_lik_tol=0.000001, se=True)
```

Arguments

warm_corr	Boolean: If True, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 5.3.4). Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 5.3.4). Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 5.3.4) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
constant	Float: additive smoothing constant (Elliott & Buttery, 2022b) for generation of item location estimates (see Section 5.3.1). Default is constant=None, which resolves to $\min(1, 0.548\, I^{-0.2 \log_{10} Nm})$, where N is the number of persons with at least one response, I is the number of items and m is the maximum score. Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: additive smoothing constant for the Stage 2 CPAT threshold distances (see Section 5.3.1). None (default) resolves to 0.03.

Continued on next page...

method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) for generation of central item location estimates (see Section 5.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). Default is method='log-lik'
matrix_power	Integer or None: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of central item location estimates. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 5.3.1). Ignored for other methods.
se	Boolean: passed through to the internal fit_statistics call (only used if not already computed). If False, skips the bootstrap entirely – this analysis's output (residual correlations, PCA) does not depend on it, so se=False is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is se=True.

Returns

For tests of violation of the requirement for local item independence (Andrich & Kreiner, 2010; Marais, 2012):

self.residual_correlations	A pandas dataframe of pairwise correlations between item standardised residuals.

For tests of violation of the requirement for unidimensionality based on principal component analysis of the standardised residual correlations (Pallant & Tennant, 2007; Smith, 2002):

self.eigenvectors	The eigenvectors of the standardised residual correlations matrix.
self.eigenvalues	The eigenvalues corresponding to the eigenvectors.
self.variance_explained	The variance explained by each principal component.

Continued on next page...

`self.loadings`	The loading of each item onto each of the principal components, for the the first of which large loadings (‘large'typically interpreted as > 0.4 or < -0.4) may be interpreted as representing the presence of significant dimensionality, in analogy to factor analysis.

Example

To produce a residual correlation analysis:

```
my_rsm.res_corr_analysis()
```

Arguments may be used to alter parameters of item location and/or person location estimation.

5.4.9 save_stats

Description

Exports item_stats, threshold_stats (both centred and uncentred), person_stats and test_stats to file in a single call. Auto-triggers item_stats_df, threshold_stats_df, person_stats_df and test_stats_df (Sections 5.4.1, 5.4.2, 5.4.4, 5.4.5) if any have not been run with their default arguments plus whichever of dp/warm_corr/etc. below are passed, so call those methods directly first (e.g. with full=True) if non-default table columns are wanted in the saved output. Does not include category_stats (Section 5.4.3) – save that table separately with self.category_stats.to_csv(...) if needed.

Usage

```
self.save_stats(filename, format='csv', dp=3, warm_corr=True, tolerance=0.00001,
                max_iters=100, ext_score_adjustment=0.5, method='log-lik', constant=None,
                no_of_samples=500, interval=None)
```

Arguments

filename	String: output filename or path. For format='csv', used as a prefix (any trailing .csv is stripped first) for five files, _item_stats.csv, _threshold_stats_uncentred.csv, _threshold_stats_centred.csv, _person_stats.csv and _test_stats.csv. For format='xlsx', used as a single workbook filename (.xlsx appended automatically if not already present), with the five tables on separate sheets.
format	String: 'csv' or 'xlsx'. Default is format='csv'.
dp, warm_corr, tolerance, max_iters, ext_score_adjustment, method, constant, no_of_samples, interval	Passed through to item_stats_df/threshold_stats_df/person_stats_df/test_stats_df (Sections 5.4.1, 5.4.2, 5.4.4, 5.4.5), used only for whichever of those tables has not already been computed. See those sections for details of each. Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).

Returns

None. Writes the five tables to file as described above.

Example

To save item, threshold, person and test statistics as five CSV files prefixed my_rsm_results:

```
my_rsm.save_stats('my_rsm_results')
```

To save all tables to separate sheets of a single Excel workbook:

```
my_rsm.save_stats('my_rsm_results', format='xlsx')
```

5.4.10 save_residuals

Description

Exports the residual correlation analysis (self.eigenvectors, self.eigenvalues, self.variance_explained and self.loadings) to file. Auto-triggers res_corr_analysis (Section 5.4.8) if not already run.

Usage

```
self.save_residuals(filename, format='csv', single=True, dp=3, warm_corr=True,
                    tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                    method='log-lik', constant=None)
```

Arguments

filename	String: output filename or path (extension appended automatically if not already present).
format	String: 'csv' or 'xlsx'. Default is format='csv'.
single	Boolean: if True, writes all four tables (eigenvectors, eigenvalues, variance explained, loadings) into a single file – sequentially, separated by blank lines, for format='csv'; at successive row offsets on one sheet ('Item residual analysis'), for format='xlsx'. If False, writes each table to its own file (csv) or sheet (xlsx). Default is single=True.
dp, warm_corr, tolerance, max_iters, ext_score_adjustment, method, constant, threshold_constant	dp controls rounding of the saved values; the rest are passed through to res_corr_analysis (Section 5.4.8), used only if it has not already been run. See that section for details of each. Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).

Returns

None. Writes the residual correlation analysis tables to file as described above.

Example

To save the residual correlation analysis as a single CSV file:

```
my_rsm.save_residuals('my_rsm_residuals')
```

To save each table to its own sheet of a single Excel workbook:

```
my_rsm.save_residuals('my_rsm_residuals', format='xlsx', single=False)
```

5.4.11 category_counts_df

Description

Produces a table of counts of scores in each category, plus total and missing responses. All items share the same max_score in RSM, so there are no blank cells.

Usage

```
self.category_counts_df(persons=None, items=None, counts_name=None,
                        filename=None, file_format='csv')
```

Arguments

persons	List or String: Person or subset of persons over which to count responses. Default persons=None, which returns counts for all persons.
items	List or String: Item or subset of items over which to count responses. Default items=None, which returns counts for all items.
counts_name	String or Int: Name under which to store counts on a subset of persons or items so multiple sets of counts can be made available. Useful if e.g. user wants to compare counts between different subgroups of persons according to a criterion such as an exogenous variable. If passed, stores counts within a namespace, self.counts. Default counts_name=None, when counts are returned as a simple attribute, self.category_counts_table.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

If counts_name=None:

Returned and stored as attribute self.category_counts_table, a pandas dataframe of category counts with one row per item and one column per response category (0 to max_score), plus total and missing responses per item, and grand total row.

If counts_name=my_counts_name:

Returned and stored as attribute self.counts.my_counts_name stored in the namespace self.counts, a pandas dataframe of category counts with one row per item and one column per response category (0 to max_score), plus total and missing responses per item and grand total row.

Example

To produce a dataframe of category counts for all persons and items:

```
my_rsm.category_counts_df()
```

To produce a dataframe of category counts for a subset of persons saved as a list `my_persons`, with a matching attribute name within the namespace `self.counts`:

```
my_rsm.category_counts_df(persons=my_persons, counts_name=my_persons)
```

5.4.12 dif_test

Description

Differential Item Functioning (DIF) test by an exogenous person covariate (e.g. gender, first language) (Holland & Wainer, 1993). Splits non-extreme persons into groups by the named column in `self.exogenous`, designates one group as the reference (by default the largest), and compares each other ("focal") group against it individually – a reference-group design, not all-pairwise, so covariates with more than two levels are supported directly. Two independent components are tested:

1. **Item-location DIF.** Both groups are calibrated independently (each self-centred), then purified onto a common item scale via either a per-item Wald significance test (`selection_method='wald'`, default) or the robust modified z-score outlier-trimming procedure used by `calibrate_anchor` (Section 5.3.3; Iglewicz & Hoaglin, 1993) – the reference group's item locations supply the "anchor" values, the focal group's supply the "observed" values – so that genuine DIF items cannot contaminate the common scale used to test for DIF. Item-level DIF is tested via a per-item Wald test (`test='wald'`, default), a per-item likelihood-ratio test (`test='lr'`), or both (`test='both'`), using bootstrap standard errors from `std_errors` (Section 5.3.2) for the Wald test. An omnibus LR test (Andersen-style: reference vs. focal, every common item jointly) runs by default (`omnibus=True`). Results in `self.dif_table` and `self.dif_omnibus_table`.

2. **Threshold-structure DIF** (differential category functioning, or DCF). The threshold structure is independent of item locations, so is not translated by the item-scale purification. Tested via category widths (consecutive threshold differences, `thresholds[k+1] - thresholds[k]`) rather than raw threshold locations, since threshold locations are not independent: changing one threshold affects all other threshold locations, while widths isolate a change to the category affected. Each `max_score − 1` category width is Wald-tested directly, reference versus focal, using a bootstrap standard error. Not affected by `test`/`omnibus` – those only govern component (1). Results in `self.threshold_dif_table`.

`test='lr'` is mildly liberal; `test='wald'` (the default) is preferred. An item or category-width cell is flagged only if it clears both a (corrected) p-value threshold and an absolute logit-difference threshold, so that a statistically significant but practically negligible difference is not flagged. `dif_test` also supports ETS-style A/B/C severity-category classification (Zwick et al., 1997) (`category`, `category_thresholds`, `category_alpha`) for the item-location component only – the 0.43/0.64 logit defaults are calibrated for item-location DIF specifically, so this does not extend to `self.threshold_dif_table`, as described in Table 184:

Table 184: ETS-style DIF category classification (item-location component only). Sign: + indicates that an item is harder for the focal group; - indicates that an item is easier for the focal group. The 0.43/0.64 logit thresholds are the Rasch-logit equivalents of the classic ETS Mantel-Haenszel delta-scale B/C boundaries (Tristan, 2006); the significance level (default `category_alpha=0.05`) is independent of `alpha`, which governs the separate `Flagged`/`Flagged_LR` columns.

Category	Criteria	Interpretation
A	|Difference| < 0.43, or not statistically significant	Negligible DIF
B+ / B-	Both |Difference| ≥ 0.43 *and* $p(\text{DIF} = 0) <$ `category_alpha` (two-sided Wald/Welch test), but not C	Slight to moderate DIF
C+ / C-	Both |Difference| ≥ 0.64 *and* $p(\|\text{Difference}\| \leq 0.43) <$ `category_alpha` (one-sided test that difference significantly exceeds B boundary)	Moderate to large DIF

Optionally, standard errors may be adjusted by setting `size_adjust=True` to avoid over-sensitivity for large samples by adjusting the sample size: at very large N, SE shrinks enough that trivial differences become "significant". Item-location SE is rescaled by $\sqrt{n_{\text{actual}}/\texttt{reference_n}}$ unconditionally (Tristan, 2006); threshold (differential category functioning, or DCF) SE is rescaled when the threshold's CPAT information weight exceeds a cap (`dcf_cap`). Each group's SE is rescaled independently. Applied to the Wald/Welch test, category tests, and (if `welch=True`) Welch-Satterthwaite df, which then also uses the adjusted n rather than each group's actual N, for internal consistency. Default is `size_adjust=False`. `dcf_cap` with `size_adjust=True` controls SEs for DCF by capping each threshold's CPAT information weight threshold per threshold, independently for each group: effective weight = min(total_weight, `dcf_cap`); adjusted SE = actual SE $\times \max(1, \sqrt{\text{total_weight}/\texttt{dcf_cap}})$, where total_weight is the threshold's CPAT harmonic-mean information weight, summed across all item pairs (Elliott & Buttery, 2022b). Below the cap, bootstrap SE is used. Default is `dcf_cap=80000`.

Usage

```
self.dif_test(covariate, reference=None, selection_method='wald', corr_tol=0.95,
              sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05, test='wald',
              omnibus=True, welch=False, size_adjust=False, reference_n=100, dcf_cap=80000,
              correction='bh', alpha=0.05, logit_threshold=0.43, category=False,
              category_thresholds=(0.43, 0.64), category_alpha=0.05, no_of_samples=500,
              plot=False, plot_kwargs=None, warm_corr=True, tolerance=0.00001,
              max_iters=100, ext_score_adjustment=0.5, constant=None, threshold_constant=None,
              method='log-lik', matrix_power=None, log_lik_tol=0.000001, seed=None)
```

Arguments

`covariate`	String: column name in `self.exogenous` to group persons by.

Continued on next page...

reference	String or None: value of covariate to use as the reference group. If None, the largest group is used. Default is reference=None.
selection_method	String, one of 'wald', 'robust_z' or 'none': how the common item-location scale is purified before testing (never the threshold-structure component) – see calibrate_anchor (Section 5.3.3) for what each option does. Default is selection_method='wald'.
corr_tol	Float: minimum correlation between the two groups' item locations on the retained items ('robust_z' only). Default is corr_tol=0.95.
sd_ratio_tol	Float: the ratio of the two groups' item-location standard deviations on the retained items must lie in [1/sd_ratio_tol, sd_ratio_tol] ('robust_z' only). Default is sd_ratio_tol=1.1.
min_anchors	Integer: minimum number of items retained by purification ('wald' and 'robust_z'). Default is min_anchors=6.
wald_alpha	Float: significance level for each item's purification test ('wald' only). Default is wald_alpha=0.05.
test	String, one of 'wald', 'lr' or 'both': per-item item-location DIF test(s) to compute – affects only the item-location component, never self.threshold_dif_table. 'wald' (default) matches the original behaviour exactly, at no added cost. 'lr'/'both' additionally compute a per-item likelihood-ratio test (extra cost: two person location re-estimations per item per focal group). The existing Flagged column always reflects the Wald test; a separate Flagged_LR column is added when test is 'lr' or 'both'. Default is test='wald'.
omnibus	Boolean: if True (default), also runs an Andersen-style omnibus LR test (reference vs. focal, every common item jointly) per focal group for the item-location component, stored in self.dif_omnibus_table.
welch	Boolean: if True, every Wald test here (item-location *and* threshold-structure) becomes a Welch's t-test: statistic unchanged (Difference / SE), but p-value computed against a t-distribution with Welch-Satterthwaite degrees of freedom instead of a normal distribution. Adds a df column to both self.dif_table and self.threshold_dif_table. Does not affect the LR test or omnibus test (already exact chi-squared tests). Default is welch=False.
size_adjust	Boolean: if True, applies the Tristan (2006)rescaling to each group's SE.

Continued on next page...

reference_n	Integer: reference sample size for size_adjust=True's item-location component only (self.dif_table) – does not affect self.threshold_dif_table, which uses dcf_cap instead (see below). 100 (the default) is Tristán's recommendation; around 60 is suggested there for closer alignment with the ETS Category B boundary specifically. Unused unless size_adjust=True.
dcf_cap	Float: cap on each threshold's CPAT information weight. Default is dcf_cap=80000.
correction	String or None, one of 'bh', 'bonferroni' or None: multiple-comparison correction applied across items (within each focal-group comparison) and, separately, across every category-width cell (within each focal-group comparison, one pool for the whole test since the threshold vector is shared rather than per item) – Benjamini-Hochberg false discovery rate control (Benjamini & Hochberg, 1995) or Bonferroni. Not applied to the omnibus test. Default is correction='bh'.
alpha	Float: p-value threshold for flagging in both components (Wald, per-item LR, and omnibus alike), compared against the corrected p-value if correction is not None. Default is alpha=0.05.
logit_threshold	Float: absolute logit-difference threshold for flagging in both components (ETS-style convention). Default is logit_threshold=0.43.
category	Boolean: if True, adds an ETS-style Category column to self.dif_table (item-location component only – see Table 184 above). Default is category=False.
category_thresholds	Tuple of two floats: (B boundary, C boundary) in logits, per Zwick et al. (1997). Default is category_thresholds=(0.43, 0.64).
category_alpha	Float: significance level for both category tests, independent of alpha. Default is category_alpha=0.05.
no_of_samples	Integer: bootstrap resamples for each group's std_errors call (Section 5.3.2). Default is no_of_samples=500.
plot	Boolean: if True, calls plot_anchor_selection (Section 3.5.10) for each focal group that has a selection table (selection_method in 'wald'/'robust_z' only) and stores the resulting figures in self.dif_plots. Default is plot=False.
plot_kwargs	Dictionary or None: extra keyword arguments forwarded to plot_anchor_selection for every focal group when plot=True. Default is plot_kwargs=None.

Continued on next page...

warm_corr, tolerance, max_iters, ext_score_adjustment	Person-estimation arguments, passed through to the group models – see Section 5.3.4. Only used if test is 'lr'/'both' or omnibus=True.
constant, method, matrix_power, log_lik_tol	Calibration arguments passed to the group models – see Section 5.3.1.
seed	Integer or None: seed passed through to each group's internal std_errors call. None (default) draws fresh entropy each call. See Section 1.

Returns

`self.dif_reference`	The reference group value.
`self.dif_covariate`	The covariate column name used.
`self.dif_reference_model`	`RSM` object: fitted model for the reference group.
`self.dif_focal_models`	Dictionary: {`focal_value: RSM`}, fitted models for each focal group.
`self.dif_tc`	Dictionary: {`focal_value: float`}, the item-scale translation constant applied to each focal group.
`self.dif_anchor_selection`	Dictionary: {`focal_value: dataframe or None`}, per-item selection diagnostics for each focal group (`None` if `selection_method='none'`).
`self.dif_plots`	Dictionary: {`focal_value: figure or None`}, the `plot_anchor_selection` figure for each focal group, if `plot=True` and a selection table exists for that group.
`self.dif_group_sizes`	Dictionary: {`group_value: int`}, non-extreme, non-missing-covariate N used for each group.

The two tested components additionally set:

`self.dif_table`	pandas dataframe: one row per (item, focal group) pair, indexed by item. Columns: `Group`, `Reference`/`Focal`/`Focal (purified)` (item location estimates), `Difference`, `SE`, `z`, `p`, `p (corrected)`, `Selected` and `Flagged`. If `welch=True`, also `df`. If `test` is `'lr'` or `'both'`, also `LR`, `p_LR`, `p_LR (corrected)`, `Flagged_LR`. If `category=True`, also `Category`.
`self.dif_omnibus_table`	pandas dataframe or `None`: one row per focal group – `LR`, `df`, `p`, `Flagged` – the item-location component only. `None` if `omnibus=False`.
`self.threshold_dif_table`	pandas dataframe: differential category functioning, tested via category widths rather than threshold locations. One row per (category, focal group) pair, where category k is the width between threshold k and $k+1$ ($k = 1, \ldots,$ `max_score` -1). Columns: `Group`, `Category`, `Reference`/`Focal` (category widths), `Difference`, `SE` (bootstrap SE of the width itself), `z`, `p`, `p (corrected)` and `Flagged` – and `df` if `welch=True` – corrected independently of `self.dif_table`.

Examples

To test for DIF by an exogenous variable `'L1'` (which may have any number of levels), having supplied `exogenous` to the constructor, using the largest level as the reference group by default:

```
self.dif_test('L1')
```

To inspect the shared threshold-structure (category-width) DIF results after running the test:

```
self.threshold_dif_table
```

To additionally compute the per-item likelihood-ratio test and ETS-style DIF categories:

```
self.dif_test('L1', test='both', category=True)
```

5.4.13 recovery_stats

Description

Compares recovered parameter estimates of an object created from a simulation against the generating values, (self.generating, Section 5.1.1). Issues a UserWarning and returns None unless the object was built from a simulation. Generating model may be RSM_Sim, PCM_Sim or SLM_Sim. With PCM_Sim, all item max scores must equal self.max_score; thresholds and category widths are compared against the mean of the generating per-item centred thresholds. With SLM_Sim, self.max_score must be 1, thresholds and category widths skipped with a warning.

Person estimates are compared against *pseudo-estimates* (Elliott & Buttery, 2022b) to isolate systemic error from stochastic response noise: the person estimate from self.responses using the generating item and threshold locations. The persons error column is (recovered estimate − pseudo-estimate), and the persons generating column holds the pseudo-estimate. The error combines parameter estimation error and model misspecification.

Usage

```
self.recovery_stats(anchor=False, alpha=0.05, correction='bh', divergence_tol=0.1,
                        plot=True, **plot_kwargs)
```

Arguments

anchor	Boolean: if True, compares self.anchor_items (from calibrate_anchor, Section 5.3.3) against self.generating.items, instead of self.items. Raises ValueError if calibrate_anchor has not been run. Default anchor=False.
alpha	Float: significance threshold for the per-parameter z-test (Flagged/Flagged_corrected columns, below). Default alpha=0.05.
correction	String or None, one of 'bh', 'bonferroni' or None: multiple-comparison correction applied within each parameter type's family of tests (matches dif_test's convention, Section 5.4.12). Default correction='bh'.
divergence_tol	Float: threshold on the absolute difference between the OLS and Huber regression slopes (below) for the Divergence_flag summary row. Default divergence_tol=0.1.
plot	Boolean: if True, also produces a recovery plot (scatter, identity line, OLS and Huber fit lines) for each parameter type. Default plot=True.
**plot_kwargs	Forwarded to the internal plotting routine: filename, file_format, dpi, plot_style, font, title_font_size, axis_font_size, labelsize, figsize (same vocabulary as wright_map). A shared filename is suffixed per parameter type.

Returns

`self.recovery.items.table`, `self.recovery.thresholds.table`, `self.recovery.category_widths.table`, `self.recovery.persons.table`	pandas dataframes, one per parameter type (`items`, `thresholds`, `category_widths` and `persons`), each with columns `generating`, `estimated`, `SE`, `error` (estimated − generating), `abs_error`, `z`, `p`, `p_corrected`, `Flagged` and `Flagged_corrected`. For `persons`, `generating` holds the pseudo-estimate, and `SE` is `self.csem_vector` (the analytic conditional SEM), unaffected by `anchor`. `self.recovery.thresholds`/`self.recovery.category_widths` are `None` for `SLM_Sim`-generated data.
`self.recovery.items.summary`, `self.recovery.thresholds.summary`, `self.recovery.category_widths.summary`, `self.recovery.persons.summary`	pandas dataframes, one column (`Overall`) with rows `RMSE`, `MAE`, `bias` (mean signed error), `r` (Pearson correlation), `SD_ratio` (estimated SD over generating SD), `OLS_slope`, `OLS_intercept`, `Huber_slope`, `Huber_intercept`, `Divergence_flag` (`True` if the OLS and Huber slopes differ by more than `divergence_tol`) and `Prop_flagged` (proportion of parameters with `Flagged=True`). `r`, the two slopes and `SD_ratio` are set to `NaN` when fewer than 3 parameters are compared or the generating values have standard deviation below 0.05.

Examples

To compare recovered item, threshold, category-width and person estimates against the generating simulation, plotting all four:

```
self.recovery_stats()
```

To compare anchored item estimates instead (having already run `calibrate_anchor`), without plotting:

```
self.recovery_stats(anchor=True, plot=False)
```

5.5 Plotting functionality

5.5.1 Shared plotting arguments

All the plotting methods described in this section share a set of arguments which may be used to customise the appearance of the plot or save the plot to file automatically. These arguments are:

`title`	String: Title for the plot, to appear in the image. Default is `title=None`.
`xmin`	Float: Minimum displayed point on x-axis, in logits. Default is `xmin=-5`.
`xmax`	Float: Maximum displayed point on x-axis, in logits. Default is `xmax=5`.
`plot_style`	String: Plot style to use. Available styles are `'white'`, `'dark'`, `'black'`, `'parchment'`, `'minimal'`, `'print'`, `'solarized-light'`, `'solarized-dark'`, `'slate'`, `'blueprint'`, `'newsprint'` and `'chalkboard'` – twelve Seaborn (Waskom, 2021)-based colour schemes. Default is `plot_style='white'`.
`palette`	String: Controls colours of lines and overplotted class intervals dots. Options are `'dark blue'`, `'light blue'`, `'dark red'`, `'light red'`, `'dark green'`, `'light green'`, `'dark grey'`, `'light grey'`, `'dark multi'`, `'light multi'` and `'colorblind multi'`. `'colorblind multi'` is the Okabe & Ito (2008) colour-vision-deficiency-safe qualitative palette. Default is `palette='colorblind multi'`. Not used for `std_residual_plot`.
`black`	Boolean: If `True`, the plot will be rendered in black and white. Default is `black=False`.
`marker_border`	Boolean: If `True`, observed data point markers are drawn with a black edge for contrast. Only used by `icc`, `crcs`, `threshold_ccs` and `tcc` (the methods with an `obs` argument). Default is `marker_border=True`.
`line_border`	Boolean: If `True`, curve lines are drawn with a contrasting outline stroke (white on dark `plot_style` backgrounds, black otherwise) to improve visibility. Only used by `icc`, `crcs`, `threshold_ccs` and `tcc`. Default is `line_border=False`.
`font`	String: The font to use in the plot. Default is `font='Times New Roman'`.
`title_font_size`	Float: The size of the title font in points. Default is `title_font_size=15`.
`axis_font_size`	Float: Size of axis label font in points. Default `axis_font_size=12`.
`labelsize`	Float: The size of the axis tick label font in points. Default is `labelsize=12`.
`filename`	String: The filename for the saved plot, with no suffix for format. If `None`, no file will be saved. Default is `filename=None`.
`file_format`	The format of the file: `png`, `jpg` or `svg`. Default is `file_format=png`.
`dpi`	The resolution of the plot in dpi (dots per inch) – higher resolution plots are better quality but have larger file sizes. Default is `dpi=300`.

5.5.2 icc

Description

Plots the item characteristic curves (or item response function) for an item: person location on the x-axis against expected score on the y-axis. Options to plot observed responses, item threshold line and lines showing locations corresponding to specified expected scores, and to highlight a specified response category.

Usage

```
self.icc(item, obs=False, no_of_classes=5, thresh_lines=False, central_location=False,
         score_lines=None, score_labels=False, cat_highlight=None)
```

Arguments

item	String: The name of the item(s) to plot.
obs	Boolean: If True, mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is obs=False.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
thresh_lines	Boolean: If True, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is thresh_lines=False.
central_location	Boolean: If True, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is central_location=False.
score_lines	List of floats between 0 and self.max_score: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score.
score_labels	Boolean: If True, scores and locations corresponding to arguments passed to score_lines will be labelled on the plot. Default is score_labels=False.
cat_highlight	Integer: Passing a score between 0 and self.max_score will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is cat_highlight=None (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic item characteristic curve for `Item_1` and store the output as a variable `my_icc_plot` and save it to file as `my_icc_plot.png`:

```
my_icc_plot = self.icc('Item_1', filename=my_icc_plot)
```

To plot an item characteristic curve for `Item_1` with observed responses for 8 response classes and store the output as a variable `my_icc_plot`:

```
my_icc_plot = my_rsm.icc('Item_1', obs=True, no_of_classes=8)
```

To plot an item characteristic curve for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = my_rsm.icc('Item_1', thresh_lines=True, cat_highlight=0)
```

To plot an item characteristic curve for `Item_1` with lines showing the locations corresponding to expected scores of 0.7 and 1.6, with the expected score and corresponding location labelled, and store the output as a variable `my_icc_plot`:

```
my_icc_plot = my_rsm.icc('Item_1', score_lines=[0.7, 1.6], score_labels=True)
```

5.5.3 crcs

Description

Plots category response curves for an item: person location on the x-axis against expected the probability of obtaining a score in each category (0 or 1) on the y-axis. Options to plot observed proportions and item threshold line, and to highlight a specified response category.

Usage

```
self.crcs(item=None, obs=None, no_of_classes=5, thresh_lines=False, central_location=False,
          cat_highlight=None)
```

Arguments

item	String: The name of the item to plot. If item=None, the shared set of Rasch-Andrich thresholds with mean zero will be plotted, and any observed proportions overplotted using obs will use data from all items. Default is item=None.
obs	List: List of integers between 0 and self.max_score. For each value, mean observed proportions in each ordered response category scoring in that category are plotted against the mean location of the corresponding response class.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
thresh_lines	Boolean: If True, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is thresh_lines=False.
central_location	Boolean: If True, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is central_location=False.
cat_highlight	Integer: Passing a score between 0 and self.max_score will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is cat_highlight=None (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot basic category response curves and store the output as a variable `my_crcs_plot` and save it to file as `my_crcs_plot.png`:

```
my_crcs_plot = my_rsm.crcs(filename=my_crcs_plot)
```

To plot category response curves for `Item_1` with observed response proportions for category 0 for 8 response classes and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = my_rsm.crcs(item='Item_1', obs=[0], no_of_classes=8)
```

To plot category response curves for `Item_1` with a threshold line and highlighted zero category, and store the output as a variable `my_crcs_plot`:

```
my_crcs_plot = my_rsm.crcs(item='Item_1', thresh_lines=True, cat_highlight=0)
```

5.5.4 threshold_ccs

Description

Plots the threshold characteristic curves (or threshold response functions)for an item. For threshold τ_k, $k \in \{1, ..., \texttt{self.max_score}\}$, the threshold characteristic curve is the probability of obtaining a score of k rather than $k-1$, conditional on the score being either $k-1$ rather than k, for a given person location. Each threshold characteristic curve functions as a dichotomous item characteristic curve under the SLM, conditional on scores being in the adjacent categories either side of the threshold (see Sections 3.2.2 and 3.5.2).

For threshold τ_k, `threshold_ccs` plots person location on the x-axis against the probability of obtaining a score of k on the y-axis. Options to plot threshold lines and central item locations, and to highlight a specified response category.

Usage

```
self.threshold_ccs(item=None, obs=None, no_of_classes=5, thresh_lines=False,
                   central_location=False, cat_highlight=None,)
```

Arguments

`item`	String: The name of the item to plot. If `item=None`, the shared set of Rasch-Andrich thresholds with mean zero will be plotted, and any observed proportions overplotted using `obs` will use data from all items. Default is `item=None`.
`obs`	List: List of integers corresponding to thresholds τ_1 to τ_m, where $m =$`self.max_score`, or `'all'` or `'none'`. If `obs=[k]`, mean proportions of persons obtaining a score of k rather than $k-1$, conditional on the score being either $k-1$ or k, for each of the ordered response categories, will be plotted against the mean location of the corresponding response class. Multiple thresholds may be passed. Default is `obs=None`.
`no_of_classes`	Integer: The number of observed response categories. Default is `no_of_classes=5`.
`thresh_lines`	Boolean: If `True`, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is `thresh_lines=False`.
`central_location`	Boolean: If `True`, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is `central_location=False`.
`cat_highlight`	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is `cat_highlight=None` (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns
matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples
To plot basic threshold characteristics curves, store the output as variable `my_threshold_ccs_plot`, and save it to file as `my_threshold_ccs_plot.png`:

```
my_threshold_ccs_plot = my_rsm.threshold_ccs(filename='my_iic_plot')
```

To plot threshold characteristics curves for `Item_1` with threshold lines and category 1 highlighted, and store the output as a variable `my_threshold_ccs_plot`:

```
my_threshold_ccs_plot = my_rsm.threshold_ccs(item='Item_1', thresh_lines=True,
                                             cat_highlight=1)
```

To plot threshold characteristics curves for `Item_1` with observed responses plotted for thresholds 2 and 4 and central item location line, and store the output as a variable `my_threshold_ccs_plot`:

```
my_threshold_ccs_plot = my_rsm.threshold_ccs(item='Item_1', obs=[2, 4], central_location=True)
```

5.5.5 iic

Description

Plots the item information curve for an item: person location on the x-axis against Fisher information on the y-axis. Options to plot item threshold line and lines showing Fisher information corresponding to specified locations, and to highlight a specified response category.

Usage

```
self.iic(item, ymax=None, thresh_lines=False, central_location=False, point_info_lines=None,
         point_info_labels=False, cat_highlight=None)
```

Arguments

item	String: The name of the item(s) to plot.
ymax	Float: The maximum value to show on the y-axis. If None, will infer, plotting a maximum of 1.1 times the maximum item information. Default is ymax=None
thresh_lines	Boolean: If True, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is thresh_lines=False.
central_location	Boolean: If True, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 5.2.1). Default is central_location=False.
point_info_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the Fisher information corresponding to the location. Default is point_info_lines=None.
point_info_labels	Boolean: If True, locations and Fisher information corresponding to arguments passed to point_info_lines will be labelled on the plot. Default is point_info_labels=False.
cat_highlight	Integer: Passing a score between 0 and self.max_score will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is cat_highlight=None (no category highlighted).
ymax	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples

To plot a basic item information curve for `Item_1` and store the output as a variable `my_iic_plot` and save it to file as `my_iic_plot.png`:

```
my_iic_plot = my_rsm.iic('Item_1', filename='my_iic_plot')
```

To plot an item information curve for `Item_1` with threshold lines, central item location line and category 1 highlighted, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = my_rsm.iic('Item_1', thresh_lines=True, central_location=True, cat_highlight=1)
```

To plot an item information curve for `Item_1` with lines showing the Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding Fisher information labelled, and store the output as a variable `my_iic_plot`:

```
my_iic_plot = my_rsm.icc('Item_1', point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

5.5.6 tcc

Description

Plots the test characteristic curve (or test response function) for a set of items: person location on the x-axis against expected score on the y-axis. Options to plot observed responses and lines showing locations corresponding to specified expected scores.

Usage

```
self.tcc(items=None, obs=False, no_of_classes=5, score_lines=None, score_labels=False)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
obs	Boolean: If True, mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is obs=False.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
score_lines	List of floats between 0 and self.max_score: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score. Default is score_lines=None.
score_labels	Boolean: If True, scores and locations corresponding to arguments passed to score_lines will be labelled on the plot. Default is score_labels=False.

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic test characteristic curve for all items and store the output as a variable my_tcc_plot and save it to file as my_tcc_plot.png:

```
my_tcc_plot = my_rsm.tcc(filename=my_tcc_plot)
```

To plot a test characteristic curve for Item_1 for a subset of items stored as a list my_item_list, with observed responses for 8 response classes and store the output as a variable my_tcc_plot:

```
my_tcc_plot = my_rsm.tcc(obs=True, no_of_classes=8)
```

To plot a test characteristic curve for `Item_1` for all items with lines showing the locations corresponding to expected scores of 13 and 20, with the expected score and corresponding location labelled, and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = my_rsm.tcc(score_lines=[13, 20], score_labels=True)
```

5.5.7 test_info

Description

Plots the test information curve: person location on the x-axis against total Fisher information on the y-axis. Option to plot lines showing Fisher information corresponding to specified locations.

Usage

```
self.test_info(items=None, ymax=None, point_info_lines=None, point_info_labels=False)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
point_info_lines	List of floats: Each float represents a location, with a vertical line plotted from the x-axis to where it intersects the item information curve, then from there a horizontal line to the y-axis to show the total Fisher information corresponding to the location. Default is point_info_lines=None.
point_info_labels	Boolean: If True, locations and total Fisher information corresponding to arguments passed to point_info_lines will be labelled on the plot. Default is point_info_labels=False.
ymax	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic test information curve and store the output as a variable my_test_info_plot and save it to file as my_test_info_plot.png:

```
my_test_info_plot = my_rsm.test_info(filename='my_test_info_plot')
```

To plot a test information curve for a subset of items stored as a list **my_item_list** and store the output as a variable my_test_info_plot:

```
my_test_info_plot = my_rsm.test_info(items=my_item_list)
```

To plot a test information curve with lines showing the total Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding total Fisher information labelled, and store the output as a variable my_test_info_plot:

```
my_test_info_plot = my_rsm.test_info(point_info_lines=[-0.3, 0.7], point_info_labels=True)
```

5.5.8 test_csem

Description

Plots the test conditional standard error of measurement (CSEM) curve: person location on the x-axis against CSEM (in logits) on the y-axis. Option to plot lines showing CSEM corresponding to specified locations.

Usage

```
self.test_csem(items=None, point_csem_lines=None, point_csem_labels=False, ymax=5)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
point_csem_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the CSEM curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the CSEM corresponding to the location. Default is point_csem_lines=None.
point_csem_labels	Boolean: If True, locations and CSEM corresponding to arguments passed to point_csem_lines will be labelled on the plot. Default is point_csem_labels=False.
ymax	The maximum point displayed on the y-axis, in logits.

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic CSEM curve and store the output as a variable my_test_csem_plot and save it to file as my_test_csem_plot.png:

```
my_test_csem_plot = my_rsm.test_csem(filename='my_test_csem_plot')
```

To plot a CSEM curve for a subset of items stored as a list **my_item_list** and store the output as a variable my_test_csem_plot:

```
my_test_csem_plot = my_rsm.test_csem(items=my_item_list)
```

To plot a CSEM curve with lines showing the CSEM corresponding to locations of -0.3 and 0.7, with the location and corresponding CSEM labelled, and store the output as a variable my_test_csem_plot:

```
my_test_csem_plot = my_rsm.test_csem(point_csem_lines=[-0.3, 0.7], point_csem_labels=True)
```

5.5.9 std_residuals_plot

Description

Plots histogram of standardised residuals, with optional overplotting of standard Normal distribution.

Usage

```
self.std_residuals_plot(items=None, bin_width=0.5, normal=False, dpi=300)
```

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
bin_width	Float: The width of the histogram bins along the x-axis. Default is bin_width=0.5.
dpi	Integer: resolution of the saved plot in dpi (dots per inch). Default is dpi=300.
normal	Boolean: If True, plots a standard normal distribution over the standardised residual histogram for comparison. Default is normal=False.

Additional arguments to customise the appearance of the plot are detailed in Section 5.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot and display a basic standardised residuals histogram and save it to file as my_std_residuals_plot.png:

```
my_rsm.std_residuals_plot(filename='my_std_residuals_plot')
```

To plot and display a standardised residuals histogram with bin width 1, with standard normal curve:

```
self.std_residuals_plot(bin_width=1, normal=True)
```

To plot and display a standardised residuals histogram on a subset of items stored as a list in a variable my_item_list:

```
my_rsm.std_residuals_plot(items=my_item_list)
```

5.5.10 wright_map

Description

Plots a Wright map (person-item map): back-to-back person and item distributions on a shared axis. Options for histograms or kernel density estimate (KDE) curves; items may also be presented by name in histogram format, or – for polytomous models – as a Winsteps-style divided category strip (one row per item, spanning its Rasch-Andrich threshold range). wright_map does not use the shared plotting arguments described in Section 5.5.1.

Usage

```
self.wright_map(person_names=None, item_names=None, item_level='items',
                threshold_type='rasch-andrich',
                orientation='vertical', map_type='hist', item_labels=False,
                item_strip=False, item_distribution=False, strip_thickness=2.0,
                strip_alpha=0.3, sort='location', palette='colorblind multi',
                neutral_extremes=False, item_row_height=None, narrow_font=False,
                edge_padding=0.5, distribution_markers=False, group_by=None,
                group_colors=None, stack=True, blend=False, prop=False, person_lim=None,
                item_lim=None, person_scaling=None, no_of_bins=20, pad=False,
                plot_range=None, kde_points=500, bw_method='scott', line_width=1,
                figsize=None, title=None, plot_style='white', edge_color='black',
                person_color='skyblue', item_color='salmon',
                person_line_color='black', item_line_color='black',
                marker_color=None, alpha=0.6, black=False,
                font='Times New Roman', title_font_size=15, axis_font_size=12,
                labelsize=12, item_label_size=8, tick_interval=1, filename=None,
                file_format='png', dpi=300)
```

Arguments

person_names	String, list or None: Person subset to include. Default is person_names=None (all persons).
item_names	String, list or None: Item subset to include. Default is item_names=None (all items).
item_level	String: 'items' plots one point per item at its central location. 'thresholds' plots one point per Rasch-Andrich threshold instead. Ignored when item_strip=True. Default is item_level='items'.

Continued on next page...

threshold_type	String, one of 'rasch-andrich' (default), 'thurstonian' or 'half-point': which threshold convention to plot when item_level='thresholds'. 'rasch-andrich' plots the estimated Rasch-Andrich thresholds, which can be disordered); 'thurstonian' plots the median threshold ($P(X \geq k) = 0.5$); 'half-point' plots the location where $E[X] = k - 0.5$. 'thurstonian' and 'half-point' are always ordered. Also governs item_strip=True category-boundary segments. Ignored when item_level='items'.
orientation	String: 'vertical' places location on the x-axis, with persons above the baseline; 'horizontal' places location on the y-axis, with persons to the left of the baseline. Default is orientation='vertical'.
map_type	String: 'hist' plots back-to-back histograms; 'kde' plots kernel density estimate curves instead. If item_labels=True, only governs the person side; the item side becomes a label panel rather than a distribution. Default is map_type='hist'.
item_labels	Boolean: If True, item names are listed instead of a bare histogram, grouped into no_of_bins histogram-style location bins and ordered by location. Default is item_labels=False.
item_strip	Boolean: If True, draws each item as a strip divided into coloured category segments showing the most probable category for a given person location, with a boundary line at each threshold. Forces item_labels=True. Default is item_strip=False.
item_distribution	Boolean: If True, plots a histogram/KDE plot as well as item labels. Forces item_labels=True. Default is item_distribution=False.
strip_thickness	Float: Controls the height (vertical orientation) or width (horizontal orientation) of category strips as a multiplier of the base thickness when item_strip=True. Default is strip_thickness=2.0.
strip_alpha	Float: Fill transparency for strips when item_strip=True. Default is strip_alpha=0.3.
sort	String: Ordering of item strip rows. 'location' sorts by the lowest threshold; 'order' orders according to self.items. Only used when item_strip=True. Default is sort='location'.
palette	String or None: Named colour palette for category strips (see Section 5.5.1). None uses a flat item_color fill with no distinction between categories. Only used when item_strip=True. Default is palette='colorblind multi'.

Continued on next page...

neutral_extremes	Boolean: The strip's open (unbounded) top and bottom categories always draw out to the location axis limits – there is no real boundary to stop short at. If True, they fade from a neutral cream colour toward the real threshold instead of using a flat fill in the category palette, de-emphasising the open-ended extremes rather than drawing the eye to an arbitrary width. Only used when item_strip=True. Default is neutral_extremes=False.
item_row_height	Float or None: Physical height, in inches, allocated per stacked item-label row. If None, derived automatically. Only used when item_labels=True. Default is item_row_height=None.
narrow_font	Boolean: If True, item labels (and, with item_strip=True, row labels beside category strips) are set in 'Arial Narrow' instead of font to save space. Falls back to Matplotlib's font substitution if 'Arial Narrow' is not installed. Default is narrow_font=False.
edge_padding	Float: Extra padding (in inches) added around item-label margin. The same value produces matching-looking margins in both orientations. Only used when item_labels=True. Default is edge_padding=0.5.
distribution_markers	Boolean: If True, overlays mean and plus/minus 1 and 2 standard deviation reference markers, labelled μ, $\mu \pm \sigma$, $\mu \pm 2\sigma$ for persons and items. Default is distribution_markers=False.
group_by	pandas Series or None: Grouping of persons (e.g. a DIF group) used to split the person distribution into sub-distributions. Default group_by=None (persons as a single series).
group_colors	Dict or None: Controls colours for group_by. If None, colours are drawn from the 'tab10' colour map. Default is group_colors=None.
stack	Boolean: When group_by is set, for map_type='hist', True cumulatively stacks histogram bars while False draws separate columns per group: dodged, or alpha-blended if blend=True. For map_type='kde', True cumulatively stacks group KDE plots; False overlays group KDE plots. Default is stack=True.
blend	Boolean: When group_by is set, map_type='hist' and stack=False, if True, blends histogram bars using colour instead of dodging them (showing them side-by-side). Default is blend=False.
prop	Boolean: If True, plots distributions as proportions/density rather than counts. Default is prop=False.
person_lim	Float or None: Controls axis limit for person distribution. If None, derived automatically from the distribution. Default is person_lim=None.

Continued on next page...

item_lim	Float or None: Controls axis limit for item distribution. If None, derived automatically from the distribution. Default is item_lim=None.
person_scaling	Float or None: Adjusts the scaling of the person distribution for clarity, particularly when the number of persons is very large compared to items. A scaling of 10 means that the person scale is 10 times the item scale, and so on. Default is person_scaling=None, which chooses the largest of 1, 2, 5, 10, 20, 50, ... that keeps the rescaled item distribution within half the height of the person distribution, and never below 1.
no_of_bins	Integer: Number of histogram or label bins. Default is no_of_bins=20.
pad	Boolean: If True, adds 5% padding to both ends of the location axis limits. If False, the location axis is bound to the plotted range. Default is pad=False.
plot_range	Tuple of two floats, or None: (lo, hi) limits for the location axis. If None, derived automatically. Default is plot_range=None.
kde_points	Integer: Number of points at which each KDE curve is evaluated when map_type='kde'. Default is kde_points=500.
bw_method	String, scalar or callable: Bandwidth selection method passed to scipy.stats.gaussian_kde when map_type='kde'. Default is bw_method='scott'.
line_width	Float: Line width of KDE curves when map_type='kde'. Default is line_width=1.
figsize	Tuple of two floats, or None: Base figure size in inches for the person distribution. If None, defaults to (8, 6) for orientation='vertical' or (4, 8) for orientation='horizontal'. Default is figsize=None.
title	String or None: Plot title. Default is title=None (no title).
plot_style	String: Plot background style, as per plot_style in Section 5.5.1. Default is plot_style='white'.
edge_color	String: Edge colour of histogram bars when map_type='hist' and group_by is None. Default is edge_color='black'.
person_color	String: Fill colour for person distribution; ignored when group_by is set, overridden by a grey shade when black=True. Default is person_color='skyblue'.
item_color	String: Fill colour for item distribution when item_labels=False; overridden by a grey shade when black=True. Default is item_color='salmon'.

Continued on next page...

person_line_color	String: Line colour for person KDE plot when map_type='kde' and group_by is None. Default is person_line_color='black'.
item_line_color	String: Colour for item labels (item_labels=True) or item KDE plot line (item_labels=False, map_type='kde'). Default is item_line_color='black'.
marker_color	String, tuple or None: Colour of tick marks and labels drawn when distribution_markers=True. A single colour applies to both the person and item marks; a (person, item) pair sets them separately. Person marks use it only when group_by=None; grouped marks use the assigned group colours. Default is marker_color=None, which gives each set of marks a darker shade of its own distribution's colour (person_color or item_color).
alpha	Float: Fill transparency for distributions. Default is alpha=0.6.
black	Boolean: If True, renders person_color/item_color as grey shades instead. Has no effect on group_by colours. Default is black=False.
font	String: Plot font. Default font='Times New Roman'.
title_font_size	Float: Title font size in points. Default title_font_size=15.
axis_font_size	Float: Axis label font size in points. Default axis_font_size=12.
labelsize	Float: Axis tick label font size in points. Default labelsize=12.
item_label_size	Integer: Item name font size in points. Default item_label_size=8.
tick_interval	Float: Spacing between Location axis ticks. Default tick_interval=1.
filename	String or None: Filename for saved plot, with no suffix for format. If None, no file is saved. Default filename=None.
file_format	String: Format of saved file, 'png', 'jpg' or 'svg'. Default file_format='png'.
dpi	Integer: Resolution of saved plot in dpi (dots per inch). Default dpi=300.

Returns

matplotlib figure. Displays and closes the figure; if a filename argument is passed, also saves the image to file.

Examples

To plot a basic Wright map, with items and persons as back-to-back histograms:

```
self.wright_map()
```

To plot a Wright map with each item drawn as a divided category strip, plus mean/±1SD/±2SD reference marks for persons, and save it to file as my_wright_map.png:

```
self.wright_map(item_strip=True, distribution_markers=True, title='RSM Wright Map',
```

```
                filename='my_wright_map')
```

To plot a horizontally-oriented Wright map, split by an exogenous grouping variable stored in a pandas Series `my_group` (indexed like `self.persons`), with each group's histogram bars alpha-blended rather than stacked:

```
self.wright_map(orientation='horizontal', group_by=my_group, stack=False,
                blend=True)
```

5.5.11 plot_anchor_selection

Description

Scatter plot of anchor item locations (x-axis) against observed values for the same items (y-axis) to visualise the outlier-trimming diagnostics produced by `calibrate_anchor` (Section 5.3.3) or `dif_test` (Section 5.4.12). Selected and dropped items are plotted in different colours, with a solid $y = x$ identity line for reference and a dotted OLS best-fit line through the selected points only. `plot_anchor_selection` does not use the shared plotting arguments described in Section 5.5.1; its smaller set of arguments is described in full below.

Usage

```
self.plot_anchor_selection(result, xmin=-4, xmax=4, ymin=-3, ymax=3, figsize=(8, 6),
                           font='Times New Roman', title_font_size=15, axis_font_size=12,
                           labelsize=12, title='', dpi=300, filename=None,
                           file_format='png')
```

Arguments

`result`	dict or pandas dataframe: Either the full dictionary returned internally by the robust anchor-selection routine, or just its `'table'`, i.e. one of `self.anchor_selection` (from `calibrate_anchor`) or `self.dif_anchor_selection` (from `dif_test`), each a dataframe with `'Anchor'`, `'Observed'` and boolean `'Selected'` columns.
`xmin`, `xmax`, `ymin`, `ymax`	Floats: axis limits. Defaults are `xmin=-4`, `xmax=4`, `ymin=-3`, `ymax=3`.
`figsize`	Tuple of two floats: figure size in inches (width, height). Default is `figsize=(8, 6)`.
`font`	String: the font to use in the plot. Default is `font='Times New Roman'`.
`title_font_size`, `axis_font_size`, `labelsize`	Integers: font sizes in points for the title, axis labels and axis tick labels respectively. Defaults are `title_font_size=15`, `axis_font_size=12`, `labelsize=12`.
`title`	String: plot title. Default is `title=''` (no title).
`filename`	String or `None`: if provided, saves the plot to this path (with the suffix determined by `file_format`). Default is `filename=None` (not saved).
`file_format`	String: file format for saved plots, e.g. `'png'`, `'pdf'`, `'svg'`. Default is `file_format='png'`.
`dpi`	Integer: output resolution in dpi (dots per inch) when saving to file. Default is `dpi=300`.

Returns

matplotlib figure object. If a `filename` argument is passed, also saves the image to file. `calibrate_anchor` and `dif_test` both call this method automatically when their `plot` argument is `True` (see Sections 5.3.3 and 5.4.12), so it need not usually be called directly; it is documented separately here since the resulting figure objects (`self.anchor_plot`, `self.dif_plots`) can also be regenerated with different display arguments by calling it directly on the stored selection table.

Example

To regenerate the anchor-selection plot for an existing calibration's diagnostics, stored in `self.anchor_selection`, with wider axis limits and a custom title:

```
self.plot_anchor_selection(self.anchor_selection, xmin=-6, xmax=6, ymin=-6, ymax=6,
                           title='Anchor calibration')
```

6 class MFRM

6.1 Preliminaries

6.1.1 MFRM

Description

Creates an object of the class MFRM from a pandas multiindex dataframe of polytomously scored data of items, rated by multiple facet elements (e.g. raters) which share the same maximum score for analysis. No analysis can be run until an object is created.

Usage

```
MFRM(responses, max_score=0, extreme_persons=True, no_of_classes=5, facet=None,
     facet_plural=None, validate=True, exogenous=None, drop_facet_elements=None,
     min_persons=3)
```

Arguments

responses	pandas multiindex dataframe with items as columns (item names as column names) and facet elements and persons as the two levels of the multiindex (facet element names as index level 0 names and person names as index level 1 names).
max_score	Integer: The maximum possible score, shared across all items. max_score=0 (the default) infers the maximum from the data, although passing an argument explicitly is recommended, to avoid issues when the true maximum is never observed.
extreme_persons	Boolean: if False, all persons with extreme scores (all responses correct or all responses incorrect across all facet elements) are removed from the response dataframe. Default is extreme_persons=True.
no_of_classes	Integer: the number of classes of persons grouped by location for overplotting observed responses on theoretical curves. Default is no_of_classes=5
facet	String: singular name for the second (non-item, non-person) facet of the data – e.g. 'judge' or 'delivery_mode' if the facet elements in your design are not literally raters. Since *RaschPy*'s arguments cannot be renamed per instance, method arguments and the shared, generic self.facet_* attributes always use facet_element/facet_elements regardless of this setting; only the convenience *attribute* aliases (below) pick up the custom name. If None, messages refer to 'facet element(s)', with '(e.g. rater)' at first mention, and the attribute aliases use 'rater' (e.g. self.rater_names). Default is facet=None.

Continued on next page...

facet_plural	String or None: plural form of facet, used for the pluralised attribute aliases (e.g. self.no_of_judges). If None (default), facet + 's' is used (so the default alias 'rater' pluralises to 'raters' automatically) – only needed for an irregular plural (e.g. facet='judge' with an irregular plural you want to override). Default is facet_plural=None.
validate	Boolean: if True, automatically calls check_data_connectivity (Section 6.1.2) on creation and issues a UserWarning if the item response network is disconnected, or if any item has a structurally unresolvable zero in the pairwise item-location matrix, or the directed comparison network is connected but not strongly connected (which prevents a finite method='log-lik' solution on an unpowered matrix). Default is validate=True.
exogenous	pandas dataframe or None: optional person-level covariates (e.g. gender, first language or age) for Differential Item Functioning (DIF, Section 6.4.13) and Differential Rater Functioning (DRF, Section 6.4.14) analysis, indexed by person identifier. Values are kept as raw category labels. Persons in the response data without a matching exogenous record (and vice versa) are allowed, and are reported via a UserWarning and the attributes self.no_exogenous_persons and self.exogenous_only_persons, rather than raising an error. Default is exogenous=None.
drop_facet_elements	None, String or List: facet elements to remove: 'thin' (fewer than min_persons persons), 'extreme' (only the lowest or only the highest category) or a list of names. Facet elements with no responses are always removed. Removed facet elements are listed in self.dropped_facet_elements. Default is drop_facet_elements=None.
min_persons	Integer: facet elements that rated at least one but fewer than this many persons are reported as thin. Default is min_persons=3.

Returns

Object of class MFRM. Analyses are run using methods defined on the MFRM object, with results stored as attributes of the MFRM object.

Several attributes of object MFRM are automatically generated on its creation:

self.responses	pandas muliindex dataframe: Dataframe of valid responses.
self.invalid_responses	pandas multiindex dataframe: Dataframe of invalid responses (persons with no responses to any items, i.e. all missing data).

Continued on next page...

self.extreme_persons	pandas multiindex dataframe: Dataframe of persons with extreme scores (all-correct or all-incorrect across all facet elements). When extreme_persons=True (the default), extreme scorers are retained in self.responses and this attribute is empty; when extreme_persons=False, they are removed and stored here.
self.max_score	Integer: The maximum possible score, shared across all items.
self.no_of_items	Integer: Number of items.
self.item_names self.item_ids	List: List of item names. self.item_ids is an alias for users who find *_ids more natural than *_names.
self.no_of_persons	Integer: Number of persons.
self.person_names self.person_ids	List: List of unique person names. self.person_ids is an alias for users who find *_ids more natural than *_names.
self.no_of_facet_elements	Integer: Number of facet elements (raters, by default). Aliased as self.no_of_raters and, if a custom facet was passed to the constructor, as self.no_of_{facet_plural} (e.g. self.no_of_judges).
self.facet_names	List: List of facet element names. Aliased as self.rater_names/self.rater_ids and, if a custom facet was passed, as self.{facet}_names/self.{facet}_ids and self.{facet_plural} (e.g. self.judge_names, self.judges).
self.no_of_classes	Integer: Number of response classes, defined by the argument no_of_classes.
self.generating	Namespace object (_SimParams): Only present when the model is instantiated by passing a simulation object (e.g. MFRM_Sim_Global) as the responses argument. Contains all attributes of the simulation object (e.g. self.generating.persons, self.generating.items, self.generating.facet_effects), enabling direct comparison of generating and estimated parameters.
self.facet_element_person_counts	pandas series: number of persons each facet element rated, before any removal. Aliased as self.rater_person_counts and, if a custom facet was passed, as self.{facet}_person_counts.
self.empty_facet_elements	List: facet elements with no responses (always removed). Aliased as self.empty_raters and, if a custom facet was passed, as self.empty_{facet_plural}.
self.thin_facet_elements	List: facet elements that rated at least one but fewer than min_persons persons, whether or not they were removed. Aliased as self.thin_raters and, if a custom facet was passed, as self.thin_{facet_plural}.

Continued on next page...

self.extreme_facet_elements	List: facet elements that used only the lowest or only the highest score category. Aliased as self.extreme_raters.
self.dropped_facet_elements	List: facet elements removed: those with no responses, plus any removed via drop_facet_elements. Aliased as self.dropped_raters and, if a custom facet was passed, as self.dropped_{facet_plural}.
self.exogenous	pandas dataframe or None: person-level covariates passed via the exogenous argument, re-indexed onto self.person_names. None if not supplied.
self.no_exogenous_persons	pandas index: persons present in the response data with no matching exogenous record.
self.exogenous_only_persons	pandas index: persons present in the exogenous data but not in the response data.
self.connectivity_status	Dictionary: result of check_data_connectivity (Section 6.1.2), present only when validate=True. Always contains at least 'connected' (Boolean), 'components_count' (Integer) and 'strongly_connected' (Boolean).

Example

To create an object from a dataframe my_mfrm_dataframe, with a maximum score of 5 and 10 observed classes:

```
my_mfrm = MFRM(my_mfrm_dataframe, max_score=5, no_of_classes=10)
```

To create an object where the second facet represents judges rather than raters, so that self.judge_names/self.judges/self.no_of_judges are available alongside the generic self.facet_* attributes:

```
my_mfrm = MFRM(my_mfrm_dataframe, max_score=5, facet='judge')
```

To create an object from the same dataframe, removing any facet elements that rated fewer than 3 persons:

```
my_mfrm = MFRM(my_mfrm_dataframe, max_score=5, drop_facet_elements='thin')
```

6.1.2 check_data_connectivity

Description

Validates whether the item response network is fully connected, in the same sense as SLM's check_data_connectivity (Section 3.1.2): two items are connected if at least one person's response differentiates between them. For MFRM, the underlying directed matrix is one row per item, with entry (i, j) counting persons who scored exactly one point higher on item i than item j, summed across facet elements (e.g. raters) – matching item_locations's own matrix exactly (the item-location estimation step used by calibrate, Section 6.3.1).

Because each comparison requires the *same* facet element to have scored both items for a given person, two items rated only by disjoint sets of facet elements will register as disconnected here even if every person was rated by someone on both items – this correctly reflects that item_locations itself cannot extract a comparison across facet elements, only within a facet element's own scores for a person. This is a stronger requirement than mere co-occurrence, and worth checking directly if a rating design is heavily spiralled (each facet element seeing only a subset of items).

Also verifies *strong* connectivity of the directed win-graph – the network in which an edge runs from item i to item j whenever at least one person scored higher on i than on j. The Bradley–Terry log-likelihood estimator (calibrate(method='log-lik')) has a finite, unique solution only when this directed graph is strongly connected (**Ford1957**). When the network is connected but not strongly connected – for example with near-Guttman data or a very short test – a UserWarning names the strongly connected components and notes that method='log-lik' on its default unpowered matrix will not converge for the affected items. The other methods ('cos', 'ls', 'evm'), and 'log-lik' with matrix_power ≥ 1, are unaffected.

Called automatically on object creation when validate=True (the default; see Section 6.1.1), storing its result as self.connectivity_status.

Usage

```
self.check_data_connectivity()
```

Returns

Dictionary: 'connected', 'components_count', 'directionally_isolated_items', 'strongly_connected' always present; if disconnected, 'isolated_items'/'all_sub_groups' also present.

Example

To manually re-check connectivity on object my_mfrm, e.g. after restricting to a subset of facet elements, and inspect which items ended up isolated:

```
my_mfrm.connectivity_status = my_mfrm.check_data_connectivity()
my_mfrm.connectivity_status.get('isolated_items', [])
```

6.1.3 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of `self.responses` with new name.

Example
To rename an item in object `my_mfrm` from `Item_1` to `my_new_item_name`:

```
my_mfrm.rename_item('Item_1', 'my_new_item_name')
```

6.1.4 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of `self.responses` with new names.

Example
To rename all items in object `my_mfrm` with item names in a list stored as a variable `my_new_item_names`:

```
my_mfrm.rename_items_all(my_new_item_names)
```

6.1.5 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the second level of the multiindex of self.responses with new name.

Example
To rename a person in object my_mfrm from Person_1 to my_new_person_name:

```
my_mfrm.rename_person('Person_1', 'my_new_person_name')
```

6.1.6 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the second level of the multiindex of self.responses with new names.

Example
To rename all persons in object my_mfrm with person names in a list stored as a variable my_new_person_names:

```
my_mfrm.rename_persons_all(my_new_person_names)
```

6.1.7 rename_facet_element

Description
Method to rename a single facet element (rater, by default).

Usage
```
self.rename_facet_element(old, new)
```

Arguments

old	String: the old name for the facet element
new	String: the new name for the facet element

Returns
Replaces specified facet element name in the first level of the multiindex of self.responses with new name.

Example
To rename a facet element in object my_mfrm from Rater_1 to my_new_rater_name:

```
my_mfrm.rename_facet_element('Rater_1', 'my_new_rater_name')
```

6.1.8 rename_facet_elements_all

Description
Method to rename all facet elements (raters, by default).

Usage
```
self.rename_facet_elements_all(new_names)
```

Arguments

new_names	List of new facet element names as strings, in the same order as self.facet_names

Returns
Replaces all facet element names in the first level of the multiindex of self.responses with new names.

Example
To rename all facet elements in object my_mfrm with facet element names in a list stored as a variable my_new_rater_names:

```
my_mfrm.rename_facet_elements_all(my_new_rater_names)
```

6.1.9 drop_facet_elements

Description

Removes facet elements from the object, clearing all calibrations and results. Aliased as `self.drop_raters` (or `self.drop_{facet_plural}` for a custom `facet`).

Usage

```
self.drop_facet_elements(which)
```

Arguments

`which`	String or List: `'thin'` (fewer than `min_persons` persons), `'extreme'` (only the lowest or only the highest category) or a list of names.

Returns

`self.dropped_facet_elements` (aliased as `self.dropped_raters`): List of all facet elements removed.

Example

To remove facet elements that rated fewer than `min_persons` persons from object `my_mfrm`:

```
my_mfrm.drop_raters('thin')
```

To remove `Rater_3` from object `my_mfrm`:

```
my_mfrm.drop_facet_elements(['Rater_3'])
```

6.2 Core functions

6.2.1 cat_prob

Description

Category probability function which extends the RSM formulation described in 5.2.1 to include an additional facet, taken here to represent a rater (the default facet alias) for convenience, without loss of generality. Calculates the probability $P(X_{nir} = k)$ of person n scoring k, on item i with $k \in \{0, m\}$, where m is the maximum score, with facet element (e.g. rater) r, from person location, central item location, Rasch-Andrich thresholds and facet element representation. The precise formulation depends on the facet element representation, which may be:

- `global`: a single global scalar representation which assumes uniform facet element behaviour across items and thresholds.
- `items`: an extended vector representation which applies a different facet element effect for each item but assumes uniform facet element behaviour across thresholds.
- `thresholds`: an extended vector representation which applies a different facet element effect for each threshold but assumes uniform facet element behaviour across items.
- `matrix`: an extended matrix representation which applies a different facet element effect for every possible item/threshold combination.
- `bivector`: an extended decomposed matrix representation in which the facet element effect for facet element r, item i and threshold k is $\lambda_{rik} = \lambda_{ri.} + \lambda_{r.k}$, the sum of a per-(facet element, item) effect and a per-(facet element, threshold) effect (Elliott & Buttery, 2022a, 2025).
- `pseudo_halo`/`item_stretch`: a restricted form of `items`, characterised by two parameters – a global shift λ_r and a multiplicative item stretch factor ω_r, by which each item location δ_i is scaled.
- `centrality`/`threshold_stretch`: a restricted form of `thresholds`, the centrality/extremity model (Jin & Wang, 2018), characterised by two parameters – a global shift λ_r and a multiplicative threshold stretch factor ω_r, by which each threshold location τ_k is scaled.
- `bistretch`: a restricted form of `bivector`, characterised by three parameters – a global shift λ_r, a multiplicative item stretch factor ω_{rI}, by which each item location δ_i is scaled, and a multiplicative threshold stretch factor ω_{rT}, by which each threshold location τ_k is scaled.

See Elliott and Buttery (2022a) and Elliott (2025) for a detailed discussion of extended facet element representations.

Formulae

`global`

For a person n with location β_n attempting an item i with central item location δ_i, Rasch-Andrich thresholds $\{\tau_0, ..., \tau_m\}$ and facet element r with facet element effect λ_r, the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i - \lambda_r) - \sum_{t=0}^{k} \tau_t}}{\sum_{k=0}^{m} e^{k(\beta_n - \delta_i - \lambda_r) - \sum_{t=0}^{k} \tau_t}} \tag{19}$$

`items`

For a person n with location β_n attempting an item i with central item location δ_i, Rasch-Andrich thresholds $\{\tau_0, ..., \tau_m\}$ and facet element r with facet element effect $\{\lambda_{ri}\}$ for item i, the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i - \lambda_{ri}) - \sum_{t=0}^{k} \tau_t}}{\sum_{k=0}^{m} e^{k(\beta_n - \delta_i - \lambda_{ri}) - \sum_{t=0}^{k} \tau_t}} \tag{20}$$

`thresholds`

For a person n with location β_n attempting an item i with central item location δ_i, Rasch-Andrich thresholds $\{\tau_0, ..., \tau_m\}$ and facet element r with facet element effect vector $\{\lambda_{rj}\}$, $j \in \{0, ..., m\}$, the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i) - \sum_{t=0}^{k} (\tau_t + \lambda_{rt})}}{\sum_{k=0}^{m} e^{k(\beta_n - \delta_i) - \sum_{t=0}^{k} (\tau_t + \lambda_{rt})}} \tag{21}$$

`matrix`

For a person n with location β_n attempting an item i with central item location δ_i, Rasch-Andrich thresholds $\{\tau_0, ..., \tau_m\}$ and facet element r with facet element effect vector $\{\lambda_{rij}\}$, $j \in \{0, ..., m\}$ for item i, the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i) - \sum_{t=0}^{k} (\tau_t + \lambda_{rit})}}{\sum_{k=0}^{m} e^{k(\beta_n - \delta_i) - \sum_{t=0}^{k} (\tau_t + \lambda_{rit})}} \tag{22}$$

`bivector`

For a person n with location β_n attempting an item i with central item location δ_i, Rasch-Andrich thresholds $\{\tau_0, ..., \tau_m\}$ and facet element r with facet element effect vector $\{\lambda_{rij}\}$, $j \in \{0, ..., m\}$ for item i, the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i - \lambda_{ri.}) - \sum_{t=0}^{k} (\tau_t + \lambda_{r.t})}}{\sum_{k=0}^{m} e^{k(\beta_n - \delta_i - \lambda_{ri.}) - \sum_{t=0}^{k} (\tau_t + \lambda_{r.t})}} \tag{23}$$

`pseudo_halo/item_stretch`

For a person n with location β_n attempting an item i with central item location δ_i, Rasch-Andrich thresholds $\{\tau_0, ..., \tau_m\}$ and facet element r with location shift λ_r and item-difficulty-stretch ω_r, the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \omega_r \delta_i - \lambda_r) - \sum_{t=0}^{k} \tau_t}}{\sum_{k=0}^{m} e^{k(\beta_n - \omega_r \delta_i - \lambda_r) - \sum_{t=0}^{k} \tau_t}} \tag{24}$$

`centrality/threshold_stretch`

For a person n with location β_n attempting an item i with central item location δ_i, Rasch-Andrich thresholds $\{\tau_0, ..., \tau_m\}$ and facet element r with location shift λ_r and threshold-stretch ω_r, the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \delta_i - \lambda_r) - \sum_{t=0}^{k} \omega_r \tau_t}}{\sum_{k=0}^{m} e^{k(\beta_n - \delta_i - \lambda_r) - \sum_{t=0}^{k} \omega_r \tau_t}} \tag{25}$$

`bistretch`

For a person n with location β_n attempting an item i with central item location δ_i, Rasch-Andrich thresholds $\{\tau_0, ..., \tau_m\}$ and facet element r with location shift λ_r, item-difficulty-stretch ω_{rI} and threshold-stretch ω_{rT}, the probability of obtaining a score of k is given by:

$$P(X_{nir} = k) = \frac{e^{k(\beta_n - \omega_{rI}\delta_i - \lambda_r) - \sum_{t=0}^{k} \omega_{rT}\tau_t}}{\sum_{k=0}^{m} e^{k(\beta_n - \omega_{rI}\delta_i - \lambda_r) - \sum_{t=0}^{k} \omega_{rT}\tau_t}} \tag{26}$$

Usage

The global, items, thresholds and matrix facet element representations share the same argument format:

```
self.cat_prob_global(person_location, item, item_locations, facet_element, facet_effects,
                     category, thresholds)

self.cat_prob_items(person_location, item, item_locations, facet_element, facet_effects,
                    category, thresholds)

self.cat_prob_thresholds(person_location, item, item_locations, facet_element, facet_effects,
                         category, thresholds)

self.cat_prob_matrix(person_location, item, item_locations, facet_element, facet_effects,
                     category, thresholds)
```

The bivector facet element representation takes the decomposed item and threshold facet effects as two separate arguments instead of one combined `facet_effects` argument:

```
self.cat_prob_bivector(person_location, item, item_locations, facet_element,
                       facet_item_effects, facet_threshold_effects, category, thresholds)
```

`centrality`, `pseudo_halo` and `bistretch` instead take the facet element's own raw parameters directly – a location shift λ_r plus one or two stretch parameters – rather than a reconstructed `thresholds`/`items`/`matrix`-shaped vector, since each is a closed-form restriction with genuinely fewer free parameters than its parent representation:

```
self.cat_prob_centrality(person_location, item, item_locations, facet_element,
                         lambda_r, omega_r, category, thresholds)

self.cat_prob_pseudo_halo(person_location, item, item_locations, facet_element,
                          lambda_r, omega_r, category, thresholds)

self.cat_prob_bistretch(person_location, item, item_locations, facet_element, lambda_r,
                        omega_items_r, omega_thresholds_r, category, thresholds)
```

`facet_item_effects`/`facet_threshold_effects` may also be passed under the dynamic aliases defined by `self.facet`, e.g. `judge_item_effects` if `self.facet = 'judge'`, default `rater_item_effects`/`rater_threshold_effects`.

Arguments

person_location	Float: Person location
item	Name of item.
item_locations	pandas series: Item names as keys and central item locations as values.
facet_element	Name of facet_element.
facet_effects	pandas series or dictionary: Facet element names as keys and the appropriate facet element representations for global, items, thresholds or matrix as values. Not used for bivector (see facet_item_effects/facet_threshold_effects below) or for centrality/pseudo_halo/bistretch (see lambda_r/omega_r below). Details of the formats for facet effects for the different representations are given below.
facet_item_effects	pandas DataFrame or dictionary: bivector only. Facet element names as row index (or dictionary keys) and item names as columns (or, for a dictionary, item names as keys of a per-facet-element pandas series) – e.g. self.facet_effects_bivector_items.
facet_threshold_effects	pandas DataFrame or dictionary: bivector only. Facet element names as row index (or dictionary keys) and threshold positions as columns (or array values), zero-summed per facet element – e.g. self.facet_effects_bivector_thresholds.
lambda_r	Float: centrality/pseudo_halo/bistretch only. The facet element's own location shift – e.g. self.lambda_centrality.loc[facet_element].
omega_r	Float: centrality/pseudo_halo only. The facet element's own stretch parameter – threshold-stretch for centrality (e.g. self.omega_centrality.loc[facet_element]), item-difficulty-stretch for pseudo_halo (e.g. self.omega_pseudo_halo.loc[facet_element]).
omega_items_r	Float: bistretch only. The facet element's own item-difficulty-stretch – e.g. self.omega_items_bistretch.loc[facet_element].
omega_thresholds_r	Float: bistretch only. The facet element's own threshold-stretch – e.g. self.omega_thresholds_bistretch.loc[facet_element].
category	Integer: Response category k, with $k \in \{0, 1\}$.
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Facet element representations for `facet_effects` argument

Toy examples of each facet element representation format, with two facet elements and two items with three categories for `global`, `items`, `thresholds`, `matrix` and `bivector`:

Facet element representation `global`: pandas series with floats (facet effects) for values.

```
facet_effects = pd.Series({'Rater_1':  -0.5,
                           'Rater_2':   0.5}
```

Facet element representation `items`: Dictionary with pandas series for values. The pandas series have item names for keys and floats (item facet effects) for values.

```
facet_effects = {'Rater_1':  pd.Series({'Item_1':  -1, 'Item_2':  0}),
                 'Rater_2':  pd.Series({'Item_1':   0, 'Item_2':  1})}
```

Facet element representation `thresholds`: Dictionaries with numpy arrays for values. The numpy arrays contain the facet effects by threshold, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1':  np.array([0,  -1, -0.5,    0]),
                 'Rater_2':  np.array([0, 1.5,  0.5, -0.5])}
```

Facet element representation `matrix`: Nested dictionary. The outer dictionary has item names for values, which are also the keys for the inner dictionary. Values for the inner dictionary are numpy arrays of facet effects by threshold for the item key, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1':  {'Item_1':  np.array([0, -1.5, -1, 0.5]),
                              'Item_2':  np.array([0,    1,  0,  -1])},
                 'Rater_2':  {'Item_1':  np.array([0, -0.5,  0, 0.5]),
                              'Item_2':  np.array([0,    2,  1,   0])}}
```

Facet element representation `bivector` uses two separate arguments instead of `facet_effects`: `facet_item_effects` (e.g. `self.facet_effects_bivector_items`) and `facet_threshold_effects` (e.g. `self.facet_effects_bivector_thresholds`), both generated by `calibrate_bivector`.

```
facet_item_effects = {'Rater_1':  pd.Series({'Item_1':  -1, 'Item_2':  0}),
                      'Rater_2':  pd.Series({'Item_1':   0, 'Item_2':  1})}
facet_threshold_effects = {'Rater_1':  np.array([0,  -1, -0.5,    0]),
                           'Rater_2':  np.array([0, 1.5,  0.5, -0.5])}
```

`centrality`, `pseudo_halo` and `bistretch` take their own λ_r/ω_r parameters directly instead (see **Usage** above and the worked example below).

Returns

Float: probability of obtaining score k.

Example

To obtain the probability of a person of location 0.5 scoring 1 on 'Item_1' from central item locations set my_item_diffs with a set of Rasch-Andrich thresholds my_thresholds, rated by facet element 'Rater_1' from facet effects set my_facet_effects, using the matrix facet element representation, and store the result as a variable my_cat_prob:

```
my_cat_prob = self.cat_prob_matrix(0.5, 'Item_1', my_item_diffs, 'Rater_1', my_facet_effects,
                                   1, my_thresholds)
```

The same format is followed for the global, items and thresholds facet element representations, with no changes apart from the method name.

For the bivector facet element representation, from item facet effects set my_facet_item_effects and from threshold facet effects set my_facet_threshold_effects:

```
my_cat_prob = self.cat_prob_bivector(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                     my_facet_item_effects, my_facet_threshold_effects, 1,
                                     my_thresholds)
```

For centrality, pseudo_halo and bistretch, given a facet element's global shift set my_lambda_r (or alias my_global_pseudo_halo_r, my_global_centrality_r or my_global_bistretch_r) and stretch parameter(s) my_omega_r, or my_omega_items_r/my_omega_thresholds_r for bistretch (or aliases my_stretch_pseudo_halo_r, my_stretch_centrality_r or my_stretch_items_bistretch_r/my_stretch_thresholds_bistretch_r) – e.g. from self.lambda_centrality.loc['Rater_1']/self.omega_centrality.loc['Rater_1'] after calling calibrate(model='centrality'):

```
my_cat_prob = self.cat_prob_centrality(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                       my_lambda_r, my_omega_r, 1, my_thresholds)
```

cat_prob_pseudo_halo follows the identical pattern; cat_prob_bistretch takes my_omega_items_r and my_omega_thresholds_r in place of the single my_omega_r.

6.2.2 exp_score

Description

Expected score function which calculates the expected score $E(X_{nir})$, rated by facet element (e.g. rater) r from person location, central item location, Rasch-Andrich thresholds and facet effect representation. The expected score is given by:

$$E(X_{nir}) = \sum_{k=0}^{m} kP(X_{nir} = k) \tag{27}$$

where m is the maximum score and $P(X_{nir} = k)$ is the relevant category probability equation for the facet element representation, described in Section 6.2.1.

Usage

The global, items, thresholds and matrix facet element representations share the same argument format:

```
self.exp_score_global(person_location, item, item_locations, facet_element, facet_effects,
                      thresholds)

self.exp_score_items(person_location, item, item_locations, facet_element, facet_effects,
                      thresholds)

self.exp_score_thresholds(person_location, item, item_locations, facet_element, facet_effects,
                          thresholds)

self.exp_score_matrix(person_location, item, item_locations, facet_element, facet_effects,
                      thresholds)
```

The bivector facet element representation takes the decomposed item and threshold facet effects as two separate arguments instead of one combined `facet_effects` argument:

```
self.exp_score_bivector(person_location, item, item_locations, facet_element,
                        facet_item_effects, facet_threshold_effects, thresholds)
```

`centrality`, `pseudo_halo` and `bistretch` instead take the facet element's own raw parameters directly – see `cat_prob` (Section 6.2.1) for the full rationale:

```
self.exp_score_centrality(person_location, item, item_locations, facet_element,
                          lambda_r, omega_r, thresholds)

self.exp_score_pseudo_halo(person_location, item, item_locations, facet_element,
                           lambda_r, omega_r, thresholds)

self.exp_score_bistretch(person_location, item, item_locations, facet_element, lambda_r,
                         omega_items_r, omega_thresholds_r, thresholds)
```

`facet_item_effects`/`facet_threshold_effects` may also be passed under the dynamic aliases defined by `self.facet`, e.g. `judge_item_effects` if `self.facet = 'judge'`, default `rater_item_effects`/`rater_threshold_effects`.

Arguments

person_location	Float: Person location
item	Name of item.
item_locations	pandas series: Item names as keys and central item locations as values.
facet_element	Name of facet_element.
facet_effects	pandas series or dictionary: Facet element names as keys and the appropriate facet element representations for global, items, thresholds or matrix as values. Not used for bivector (see facet_item_effects/facet_threshold_effects below) or for centrality/pseudo_halo/bistretch (see lambda_r/omega_r below). Details of the formats for facet effects for the different representations are given below.
facet_item_effects	pandas DataFrame or dictionary: bivector only. Facet element names as row index (or dictionary keys) and item names as columns (or, for a dictionary, item names as keys of a per-facet-element pandas series) – e.g. self.facet_effects_bivector_items.
facet_threshold_effects	pandas DataFrame or dictionary: bivector only. Facet element names as row index (or dictionary keys) and threshold positions as columns (or array values), zero-summed per facet element – e.g. self.facet_effects_bivector_thresholds.
lambda_r	Float: centrality/pseudo_halo/bistretch only. The facet element's own location shift.
omega_r	Float: centrality/pseudo_halo only. The facet element's own stretch parameter.
omega_items_r	Float: bistretch only. The facet element's own item-difficulty-stretch.
omega_thresholds_r	Float: bistretch only. The facet element's own threshold-stretch.
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Facet element representations for facet_effects argument

Toy examples of each facet element representation format, with two facet elements and two items with three categories for global, items, thresholds, matrix and bivector:

Facet element representation global: pandas series with floats (facet effects) for values.

```
facet_effects = pd.Series({'Rater_1':  -0.5,
                           'Rater_2':   0.5}
```

Facet element representation `items`: Dictionary with pandas series for values. The pandas series have item names for keys and floats (item facet effects) for values.

```
facet_effects = {'Rater_1':  pd.Series({'Item_1':  -1, 'Item_2':  0}),
                 'Rater_2':  pd.Series({'Item_1':   0, 'Item_2':  1})}
```

Facet element representation `thresholds`: Dictionaries with numpy arrays for values. The numpy arrays contain the facet effects by threshold, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1':  np.array([0,  -1, -0.5,    0]),
                 'Rater_2':  np.array([0, 1.5,  0.5, -0.5])}
```

Facet element representation `matrix`: Nested dictionary. Outer dictionary has item names for values, which are also keys for inner dictionary. Values for inner dictionary are numpy arrays of facet effects by threshold for the item key, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1':  {'Item_1':  np.array([0, -1.5, -1, 0.5]),
                              'Item_2':  np.array([0,    1,  0,  -1])},
                 'Rater_2':  {'Item_1':  np.array([0, -0.5,  0, 0.5]),
                              'Item_2':  np.array([0,    2,  1,   0])}}
```

Facet element representation `bivector` uses two separate arguments instead of `facet_effects`: `facet_item_effects` (e.g. `self.facet_effects_bivector_items`) and `facet_threshold_effects` (e.g. `self.facet_effects_bivector_thresholds`), both generated by `calibrate_bivector`.

```
facet_item_effects = {'Rater_1':  pd.Series({'Item_1':  -1, 'Item_2':  0}),
                      'Rater_2':  pd.Series({'Item_1':   0, 'Item_2':  1})}
facet_threshold_effects = {'Rater_1':  np.array([0,  -1, -0.5,    0]),
                           'Rater_2':  np.array([0, 1.5,  0.5, -0.5])}
```

`centrality`, `pseudo_halo` and `bistretch` take their own λ_r/ω_r parameters directly instead (see **Usage** above and the worked example below).

Returns

Float: expected score.

Example

To obtain the expected score for a person of location 0.5 on 'Item_1' from central item locations set my_item_diffs with a set of Rasch-Andrich thresholds my_thresholds, rated by facet element 'Rater_1' from facet effects set my_facet_effects, using the matrix facet element representation, and store the result as a variable `my_exp_score`:

```
my_exp_score = self.exp_score_matrix(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                     my_facet_effects, my_thresholds)
```

The same format is followed for the `global`, `items` and `thresholds` facet element representations, with no changes apart from the method name.

For the bivector facet element representation, from item facet effects set my_facet_item_effects and threshold facet effects set my_facet_threshold_effects:

```
my_exp_score = self.exp_score_bivector(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                       my_facet_item_effects, my_facet_threshold_effects,
                                       my_thresholds)
```

For centrality, pseudo_halo and bistretch, from that facet element's own location shift set my_lambda_r and stretch parameter(s) my_omega_r (or my_omega_items_r/my_omega_thresholds_r for bistretch):

```
my_exp_score = self.exp_score_centrality(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                         my_lambda_r, my_omega_r, my_thresholds)
```

6.2.3 variance

Description

Variance function which calculates the variance of the score $V(X_{ni})$ from person location, item location and Rasch-Andrich thresholds. The variance is given by:

$$V(X_{nir}) = \sum_{k=0}^{m} P(X_{nir} = k)(k - E(X_{nir}))^2 \tag{28}$$

where m is the maximum score and $P(X_{nir} = k)$ and $E(X_{nir})$ as described in Sections 6.2.1 and 6.2.2.

The variance is also both the Fisher information for the response and the first partial differential of the expected score function with respect to person location.

Usage

The global, items, thresholds and matrix facet element (e.g. rater) representations share the same argument format:

```
self.variance_global(person_location, item, item_locations, facet_element, facet_effects,
                     thresholds)

self.variance_items(person_location, item, item_locations, facet_element, facet_effects,
                    thresholds)

self.variance_thresholds(person_location, item, item_locations, facet_element, facet_effects,
                         thresholds)

self.variance_matrix(person_location, item, item_locations, facet_element, facet_effects,
                     thresholds)
```

The bivector facet element representation takes the decomposed item and threshold facet effects as two separate arguments instead of one combined `facet_effects` argument:

```
self.variance_bivector(person_location, item, item_locations, facet_element,
                       facet_item_effects, facet_threshold_effects, thresholds)
```

`centrality`, `pseudo_halo` and `bistretch` instead take the facet element's own raw parameters directly – see `cat_prob` (Section 6.2.1) for the full rationale:

```
self.variance_centrality(person_location, item, item_locations, facet_element,
                         lambda_r, omega_r, thresholds)

self.variance_pseudo_halo(person_location, item, item_locations, facet_element,
                          lambda_r, omega_r, thresholds)

self.variance_bistretch(person_location, item, item_locations, facet_element, lambda_r,
                        omega_items_r, omega_thresholds_r, thresholds)
```

`facet_item_effects`/`facet_threshold_effects` may also be passed under the dynamic aliases defined by `self.facet`, e.g. `judge_item_effects` if `self.facet = 'judge'`, default `rater_item_effects`/`rater_`

threshold_effects.

Arguments

person_location	Float: Person location
item	Name of item.
item_locations	pandas series: Item names as keys and central item locations as values.
facet_element	Name of facet_element.
facet_effects	pandas series or dictionary: Facet element names as keys and the appropriate facet element representations for global, items, thresholds or matrix as values. Not used for bivector (see facet_item_effects/facet_threshold_effects below) or for centrality/pseudo_halo/bistretch (see lambda_r/omega_r below). Details of the formats for facet effects for the different representations are given below.
facet_item_effects	pandas DataFrame or dictionary: bivector only. Facet element names as row index (or dictionary keys) and item names as columns (or, for a dictionary, item names as keys of a per-facet-element pandas series) – e.g. self.facet_effects_bivector_items.
facet_threshold_effects	pandas DataFrame or dictionary: bivector only. Facet element names as row index (or dictionary keys) and threshold positions as columns (or array values), zero-summed per facet element – e.g. self.facet_effects_bivector_thresholds.
lambda_r	Float: centrality/pseudo_halo/bistretch only. The facet element's own location shift.
omega_r	Float: centrality/pseudo_halo only. The facet element's own stretch parameter.
omega_items_r	Float: bistretch only. The facet element's own item-difficulty-stretch.
omega_thresholds_r	Float: bistretch only. The facet element's own threshold-stretch.
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Facet element representations for facet_effects argument

Toy examples of each facet element representation format, with two facet elements and two items with three categories for global, items, thresholds, matrix and bivector:

Facet element representation global: pandas series with floats (facet effects) for values.

```
facet_effects = pd.Series({'Rater_1':  -0.5,
                           'Rater_2':   0.5}
```

Facet element representation items: Dictionary with pandas series for values. The pandas series have item names for keys and floats (item facet effects) for values.

```
facet_effects = {'Rater_1':  pd.Series({'Item_1':  -1, 'Item_2':  0}),
                 'Rater_2':  pd.Series({'Item_1':   0, 'Item_2':  1})}
```

Facet element representation thresholds: Dictionaries with numpy arrays for values. The numpy arrays contain the facet effects by threshold, with the integer of the array index corresponding to each threshold (0 to self.max_score).

```
facet_effects = {'Rater_1':  np.array([0,  -1, -0.5,    0]),
                 'Rater_2':  np.array([0, 1.5,  0.5, -0.5])}
```

Facet element representation matrix: Nested dictionary. The outer dictionary has item names for values, which are also the keys for the inner dictionary. Values for the inner dictionary are numpy arrays of facet effects by threshold for the item key, with the integer of the array index corresponding to each threshold (0 to self.max_score).

```
facet_effects = {'Rater_1':  {'Item_1':  np.array([0, -1.5, -1, 0.5]),
                              'Item_2':  np.array([0,    1,  0,  -1])},
                 'Rater_2':  {'Item_1':  np.array([0, -0.5,  0, 0.5]),
                              'Item_2':  np.array([0,    2,  1,   0])}}
```

Facet element representation bivector uses two separate arguments instead of facet_effects: facet_item_effects (e.g. self.facet_effects_bivector_items) and facet_threshold_effects (e.g. self.facet_effects_bivector_thresholds), both generated by calibrate_bivector.

```
facet_item_effects = {'Rater_1':  pd.Series({'Item_1':  -1, 'Item_2':  0}),
                      'Rater_2':  pd.Series({'Item_1':   0, 'Item_2':  1})}
facet_threshold_effects = {'Rater_1':  np.array([0,  -1, -0.5,    0]),
                           'Rater_2':  np.array([0, 1.5,  0.5, -0.5])}
```

centrality, pseudo_halo and bistretch take their own raw λ_r/ω_r parameters directly instead (see **Usage** above and the worked example below).

Returns

Float: variance

Example

To obtain the variance for a person of location 0.5 on 'Item_1' from central item locations set my_item_diffs with a set of Rasch-Andrich thresholds my_thresholds, rated by facet element 'Rater_1' from facet effects set my_facet_effects, using the matrix facet element representation, and store the result as a variable my_variance:

```
my_variance = self.variance_matrix(0.5, 'Item_1', my_item_diffs, 'Rater_1', my_facet_effects,
                                   my_thresholds)
```

The same format is followed for the global, items and thresholds facet element representations, with no changes apart from the method name.

For the bivector facet element representation, from item facet effects set my_facet_item_effects and threshold facet effects set my_facet_threshold_effects:

```
my_variance = self.variance_bivector(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                     my_facet_item_effects, my_facet_threshold_effects,
                                     my_thresholds)
```

For centrality, pseudo_halo and bistretch, from that facet element's own location shift set my_lambda_r and stretch parameter(s) my_omega_r (or my_omega_items_r/my_omega_thresholds_r for bistretch):

```
my_variance = self.variance_centrality(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                       my_lambda_r, my_omega_r, my_thresholds)
```

6.2.4 kurtosis

Description

Kurtosis function which calculates the kurtosis of the score $\kappa(X_{ni})$ from person location, central item location and Rasch-Andrich thresholds. The kurtosis is given by:

$$\kappa(X_{nir}) = \sum_{k=0}^{m} P(X_{nir} = k)(k - E(X_{nir}))^4 \tag{29}$$

where m is the maximum score and $P(X_{nir} = k)$ and $E(X_{nir})$ are as described in Sections 6.2.1 and 6.2.2 respectively.

Usage

The global, items, thresholds and matrix facet element (e.g. rater) representations share the same argument format:

```
self.kurtosis_global(person_location, item, item_locations, facet_element, facet_effects,
                     thresholds)

self.kurtosis_items(person_location, item, item_locations, facet_element, facet_effects,
                    thresholds)

self.kurtosis_thresholds(person_location, item, item_locations,
                         facet_element, facet_effects, thresholds)

self.kurtosis_matrix(person_location, item, item_locations, facet_element, facet_effects,
                     thresholds)
```

The bivector facet element representation takes the decomposed item and threshold facet effects as two separate arguments instead of one combined `facet_effects` argument:

```
self.kurtosis_bivector(person_location, item, item_locations, facet_element,
                       facet_item_effects, facet_threshold_effects, thresholds)
```

`centrality`, `pseudo_halo` and `bistretch` instead take the facet element's own raw parameters directly – see `cat_prob` (Section 6.2.1) for the full rationale:

```
self.kurtosis_centrality(person_location, item, item_locations, facet_element,
                         lambda_r, omega_r, thresholds)

self.kurtosis_pseudo_halo(person_location, item, item_locations, facet_element,
                          lambda_r, omega_r, thresholds)

self.kurtosis_bistretch(person_location, item, item_locations, facet_element, lambda_r,
                        omega_items_r, omega_thresholds_r, thresholds)
```

`facet_item_effects`/`facet_threshold_effects` may also be passed under the dynamic aliases defined by `self.facet`, e.g. `judge_item_effects` if `self.facet = 'judge'`, default `rater_item_effects`/`rater_threshold_effects`.

Arguments

person_location	Float: Person location
item	Name of item.
item_locations	pandas series: Item names as keys and central item locations as values.
facet_element	Name of facet_element.
facet_effects	pandas series or dictionary: Facet element names as keys and the appropriate facet element representations for global, items, thresholds or matrix as values. Not used for bivector (see facet_item_effects/facet_threshold_effects below) or for centrality/pseudo_halo/bistretch (see lambda_r/omega_r below). Details of the formats for facet effects for the different representations are given below.
facet_item_effects	pandas DataFrame or dictionary: bivector only. Facet element names as row index (or dictionary keys) and item names as columns (or, for a dictionary, item names as keys of a per-facet-element pandas series) – e.g. self.facet_effects_bivector_items.
facet_threshold_effects	pandas DataFrame or dictionary: bivector only. Facet element names as row index (or dictionary keys) and threshold positions as columns (or array values), zero-summed per facet element – e.g. self.facet_effects_bivector_thresholds.
lambda_r	Float: centrality/pseudo_halo/bistretch only. The facet element's own location shift.
omega_r	Float: centrality/pseudo_halo only. The facet element's own stretch parameter.
omega_items_r	Float: bistretch only. The facet element's own item-difficulty-stretch.
omega_thresholds_r	Float: bistretch only. The facet element's own threshold-stretch.
thresholds	List or numpy array: Set of m Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel value $\tau_0 = 0$ has been removed; vectors should now have length m, not $m + 1$.

Facet element representations for facet_effects argument

Toy examples of each facet element representation format, with two facet elements and two items with three categories for global, items, thresholds, matrix and bivector:

Facet element representation global: pandas series with floats (facet effects) for values.

```
facet_effects = pd.Series({'Rater_1':  -0.5,
                           'Rater_2':   0.5}
```

Facet element representation `items`: Dictionary with pandas series for values. The pandas series have item names for keys and floats (item facet effects) for values.

```
facet_effects = {'Rater_1':  pd.Series({'Item_1':  -1, 'Item_2':  0}),
                 'Rater_2':  pd.Series({'Item_1':   0, 'Item_2':  1})}
```

Facet element representation `thresholds`: Dictionaries with numpy arrays for values. The numpy arrays contain the facet effects by threshold, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1':  np.array([0,  -1, -0.5,    0]),
                 'Rater_2':  np.array([0, 1.5,  0.5, -0.5])}
```

Facet element representation `matrix`: Nested dictionary. The outer dictionary has item names for values, which are also the keys for the inner dictionary. Values for the inner dictionary are numpy arrays of facet effects by threshold for the item key, with the integer of the array index corresponding to each threshold (0 to `self.max_score`).

```
facet_effects = {'Rater_1':  {'Item_1':  np.array([0, -1.5, -1, 0.5]),
                              'Item_2':  np.array([0,    1,  0,  -1])},
                 'Rater_2':  {'Item_1':  np.array([0, -0.5,  0, 0.5]),
                              'Item_2':  np.array([0,    2,  1,   0])}}
```

Facet element representation `bivector` uses two separate arguments instead of `facet_effects`: `facet_item_effects` (e.g. `self.facet_effects_bivector_items`) and `facet_threshold_effects` (e.g. `self.facet_effects_bivector_thresholds`), both generated by `calibrate_bivector`.

```
facet_item_effects = {'Rater_1':  pd.Series({'Item_1':  -1, 'Item_2':  0}),
                      'Rater_2':  pd.Series({'Item_1':   0, 'Item_2':  1})}
facet_threshold_effects = {'Rater_1':  np.array([0,  -1, -0.5,    0]),
                           'Rater_2':  np.array([0, 1.5,  0.5, -0.5])}
```

`centrality`, `pseudo_halo` and `bistretch` take their own λ_r/ω_r parameters directly instead (see **Usage** above and the worked example below).

Returns

Float: kurtosis

Example

To obtain the kurtosis for a person of location 0.5 on 'Item_1' from central item locations set my_item_diffs with a set of Rasch-Andrich thresholds my_thresholds, rated by facet element 'Rater_1' from facet effects set my_facet_effects, using the matrix facet element representation, and store the result as a variable `my_kurtosis`:

```
my_kurtosis = self.kurtosis_matrix(0.5, 'Item_1', my_item_diffs, 'Rater_1', my_facet_effects,
                                   my_thresholds)
```

The same format is followed for the `global`, `items` and `thresholds` facet element representations, with no changes apart from the method name.

For the bivector facet element representation, from item facet effects set my_facet_item_effects and threshold facet effects set my_facet_threshold_effects:

```
my_kurtosis = self.kurtosis_bivector(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                     my_facet_item_effects, my_facet_threshold_effects,
                                     my_thresholds)
```

For centrality, pseudo_halo and bistretch, from that facet element's own location shift set my_lambda_r and stretch parameter(s) my_omega_r (or my_omega_items_r/my_omega_thresholds_r for bistretch):

```
my_kurtosis = self.kurtosis_centrality(0.5, 'Item_1', my_item_diffs, 'Rater_1',
                                       my_lambda_r, my_omega_r, my_thresholds)
```

6.3 Parameter estimation

6.3.1 `calibrate`

Description

Produces central item location estimates, a single shared set of Rasch–Andrich threshold estimates, and facet element (e.g. rater) location estimates. There are three approaches to estimation available, selectable through the `estimation` argument: `'cp-full-rcl'`, `'cpat-rcl'` and `'pair-cpat'`.

CP-Full-RCL

The default, `estimation='cp-full-rcl'`, fits item, threshold and facet (rater henceforth for simplicity) parameters jointly by iterative regularised composite likelihood (RCL). It uses every pair of response patterns (k, l) and $(k-1, l+1)$ with the same total: two responses by the same rater to two items, or by two raters to the same item (Elliott & Buttery, 2022b). Each representation has its own item locations and thresholds (`self.items_{model}`, `self.thresholds_{model}`). Facet parameters carry ridge penalties (`ridge`, `interaction_ridge`, `stretch_ridge`) and the counts of each response pattern carry additive smoothing (`constant`). Category widths with fewer than 20 threshold patterns are shrunk toward the mean of the other widths; the widths next to an empty middle category are left out of the joint fit and estimated separately by conditional maximum likelihood from their own threshold patterns (`'cpat-cml'`), given the item locations from the joint fit. `centrality`, `pseudo_halo` and `bistretch` are fitted directly in their shift and stretch parameters.

CPAT-RCL

`estimation='cpat-rcl'` restricts `estimation='cp-full-rcl'` to the CPAT patterns (RCL-CPAT), i.e. (k, k) and $(k-1, k+1)$, and represents an iterative form of CPAT.

PAIR-CPAT

`estimation='pair-cpat'` is the original non-iterative CPAT method (Elliott & Buttery, 2022a, 2022b), described in the rest of this section: item locations, thresholds and facet element locations in three stages, with item and threshold locations shared by every representation (at least until anchored). In this case, item and facet (rater) estimates use PAIR (Choppin, 1968, 1985), eigenvector method (Garner & Engelhard, 2002) or related conditional pairwise methods, based on a conditional category response frequency matrix $\mathbf{C}$ of pairwise comparison counts between all possible pairs of items where persons scored one category higher on item i than on item j by the same facet element, pooled across all facet elements and all adjacent category pairs.

Three methods are entirely non-iterative and are derived from the positive pairwise reciprocal matrix $\mathbf{D} : \mathbf{D}_{ij} = \mathbf{C}_{ji}^n / \mathbf{C}_{ij}^n$, the ratios of the counts of the $\mathbf{C}_{ij}$ after raising $\mathbf{C}$ to a power n, which introduces indirect comparisons each time $\mathbf{C}$ is raised to a higher power. From the matrix $\mathbf{D}$, the final vector is derived in different ways:

- `'ls'` is the least-squares method (Choppin, 1968, 1985), which is the logarithm of the simple arithmetic row-wise mean of $\mathbf{D}$.
- `'evm'` is the eigenvector method (Garner & Engelhard, 2002), which uses the logarithm of the eigenvector corresponding to the principal eigenvalue of $\mathbf{D}$.

- `'cos'` is the cosine maximisation method (Kou & Lin, 2014), which first normalises the columns of **D** before taking the logarithms of the row-wise means.

In all these cases, the matrix power n is user-definable.

`'log-lik'` is the iterative maximum likelihood estimator (Bradley & Terry, 1952; Zermelo, 1929), which iteratively maximises the log-likelihood until the change is less than the set tolerance, using `'cos'` for a warm start. `'log-lik'` is the default method because although usually only trivially more accurate, it is highly robust to extremely sparse data, even at small sample sizes, provided the comparison graph is strongly connected (which a non-zero `constant` ensures; see below). For well conditioned data, `'cos'` in particular produces practically identical results at a faster speed; on the thin comparison sub-matrices that arise for the more granular extended MFRMs (by item/threshold or bivector/matrix), the added robustness from matrix powering (which does not help with `'log-lik'`) makes `'cos'` generally more accurate than `'log-lik'`. When `method=None` (default), each representation is estimated using the method determined best by a simulation study: `'log-lik'` for `global` and the three stretch representations (`centrality`, `pseudo_halo`, `bistretch`), and `'cos'` for `items`, `thresholds`, `matrix` and `bivector`.

Rasch-Andrich thresholds are estimated using the conditional pairwise adjacent threshold (CPAT) method (Elliott & Buttery, 2022b), which constructs a chain of category widths – distances between adjacent thresholds – based on an information-weighted conditional pairwise contrast between items that uses the already calculated item locations; the category widths are then chained together to determine the set of threshold locations, which are then zero-summed. This stage uses `constant` but not `method` or `matrix_power`. Since the calculations for each category use fewer persons than item comparisons, the estimates are more sensitive to the value of `constant`: `constant=0` will sometimes cause the method to break down where zero counts are present, so keeping every cell count non-zero matters more (Elliott & Buttery, 2022b) and a non-zero value for `constant` is strongly recommended.

As noted above, facet locations are estimated using the same machinery as item locations. For the `global` representation, in a symmetrical fashion from the relevant subset of data for each facet parameter relevant to the granularity of the model (Elliott & Buttery, 2022a). With `estimation='pair-cpat'`, `bivector` by default takes the marginal mean vectors of the `matrix` representation, one vector by item and one by threshold (which is zero-summed); `matrix_marginals=False` estimates them directly instead. `matrix_power` and `constant` perform the same functions as for item location estimation. With `estimation='pair-cpat'`, `centrality`, `pseudo_halo` and `bistretch` models are estimated in a two-step process: first, the relevant vector representation is estimated, then the additive shift parameter λ_r and the multiplicative stretch parameter(s) ω_r are derived using robust regression. Two robust regression methods are available:

- `regression='huber'` (default): `sklearn.linear_model.HuberRegressor`, which uses a squared error function for small differences and an absolute error function for large differences, so is more robust to outliers.
- `regression='theil-sen'`: the median of pairwise slopes (slope = median over every pair of points of $(\text{dev}_i - \text{dev}_j)/(\text{ref}_i - \text{ref}_j)$; intercept = median of $\text{dev}_k - \text{slope} \cdot \text{ref}_k$).

Additive smoothing

The additive smoothing constant `constant` is a form of regularisation, and also ensures connectivity (no zeros in the matrix): zeros or very low counts in comparisons resulting from sampling error can have

an outsized influence on resulting estimates, leading to bias; the constant mitigates this effect for small sample sizes with diminishing impact on larger sample sizes. `constant=None` (the default) resolves to a separate simulation-tuned value for each of the three estimation stages (item, threshold and facet – facet varies by representation); `threshold_constant` overrides the threshold-location stage alone. An explicit `constant` value is used for every stage and reproduces the pre-1.3.0 behaviour when set to `0.1`. With `estimation='cp-full-rcl'` or `'cpat-rcl'`, `constant=None` uses a dynamic constant based on the number of items I, the number of (facet element, person) combinations with at least one response n and the maximum score m – $\min(1, 0.548\, I^{-0.2 \log_{10} nm})$ – for items, 0.1 for facets, and 0.03 for thresholds. An explicit value replaces all three. All default additive smoothing constants and ridge penalties were determined through simulation studies.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.calibrate_global(constant=None, threshold_constant=None, method=None,
                      matrix_power=None, log_lik_tol=None, robust=False,
                      estimation=None, ridge=None)

self.calibrate_items(constant=None, threshold_constant=None, method=None,
                      matrix_power=None, log_lik_tol=None, robust=False,
                      estimation=None, ridge=None)

self.calibrate_thresholds(constant=None, threshold_constant=None, method=None,
                      matrix_power=None, log_lik_tol=None, robust=False,
                      estimation=None, ridge=None)

self.calibrate_matrix(constant=None, threshold_constant=None, method=None,
                        matrix_power=None, log_lik_tol=None, robust=False,
                        estimation=None, ridge=None, interaction_ridge=None)

self.calibrate_bivector(constant=None, threshold_constant=None, method=None,
                        matrix_power=None, log_lik_tol=None,
                        matrix_marginals=None, robust=False,
                        estimation=None, ridge=None)

self.calibrate_centrality(constant=None, threshold_constant=None, method=None,
                          matrix_power=None, log_lik_tol=None,
                          regression='huber', robust=False,
                          estimation=None, stretch_ridge=None)

self.calibrate_pseudo_halo(constant=None, threshold_constant=None, method=None,
                          matrix_power=None, log_lik_tol=None,
                          regression='huber', robust=False,
                          estimation=None, stretch_ridge=None)

self.calibrate_bistretch(constant=None, threshold_constant=None, method=None,
                         matrix_power=None, log_lik_tol=None,
                         matrix_marginals=None, regression='huber', robust=False,
```

estimation=None, stretch_ridge=None)

Arguments

constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b). None (default) resolves to a separate simulation-tuned value for each of the three estimation stages – item locations, threshold locations, and the facet element stage, the last of which also depends on the chosen representation. An explicit float is used unchanged for all three stages and takes precedence over robust and threshold_constant; constant=0.1 reproduces the pre-1.3.0 output. Default additive smoothing constants were determined through simulation studies. The resolved values are stored as self.constant, self.threshold_constant and self.facet_constant. With estimation='cp-full-rcl' or 'cpat-rcl', see Description.
threshold_constant	Float or None: additive smoothing constant for the threshold-location (CPAT) stage alone. None (default) follows constant – the tuned threshold-stage default when constant is None, otherwise the explicit constant value. A float overrides that stage only. With estimation='cp-full-rcl' or 'cpat-rcl', the constant, divided by R (the number of facet elements), is added to each observed count of a threshold pattern ((k, k) against $(k-1, k+1)$) for each facet element, pair of items and category k; None gives 0.03 (or constant, if given).
method	String or None: estimation='pair-cpat' only. Method for derivation of central item location and facet effect estimates from pairwise reciprocal matrices (Elliott & Buttery, 2022b). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). None (default) resolves per representation to the method a simulation study found best: 'log-lik' for centrality, pseudo_halo and bistretch, 'cos' for the other five. An explicit value is used for every stage.
matrix_power	Integer or None: estimation='pair-cpat' only. Power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) for central item location estimates is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.

Continued on next page...

log_lik_tol	Float or None: estimation='pair-cpat' only. Convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods. None (default) resolves to 0.000001.
matrix_marginals	estimation='pair-cpat' and bivector and bistretch representations only. Boolean or None: if True, the item and threshold vectors underlying the bivector decomposition are estimated as marginal means of the full matrix representation's own PAIR estimates (Elliott & Buttery, 2022b); if False, each vector is instead estimated directly from its own pooled PAIR matrix (items pooled across thresholds; thresholds pooled across items, corrected for the mean per-facet-element item effect). None (default) resolves to not robust for bivector (marginal means unless robust=True) and to True for bistretch (always – direct marginals bias its stretch parameters towards $\omega = 1$). Ignored for the other six representations.
robust	Boolean, default False: estimation='pair-cpat' only. If True, uses a more conservative constant for rough data, resolved per representation. For matrix, switches from the missing-data-sensitive default to an R-only rule (R = number of facet elements); for bivector, switches to the direct pooled-PAIR marginal estimator with a larger facet constant; for bistretch, lowers the facet element/stretch-stage constant. For the other five representations, no effect currently (accepted for forward compatibility). Ignored with a UserWarning, if an explicit constant is also given.
regression	centrality/pseudo_halo/bistretch representations only. String, one of 'huber' or 'theil-sen': the robust regression method used to recover each facet element's raw λ_r/ω_r (see the note above for the full rationale and the automatic 'huber' $\rightarrow$ 'theil-sen' fallback behaviour). Ignored for the other five representations. With estimation='cp-full-rcl' or 'cpat-rcl', it sets the stretch parameters of the estimation='pair-cpat' fit used as a starting value. Default is regression='huber'.
estimation	String or None: 'cp-full-rcl', 'cpat-rcl' or 'pair-cpat' (see Description). Default estimation=None resolves to the method of the object's last calibration, or 'cp-full-rcl' for the first one. method, matrix_power, log_lik_tol, matrix_marginals and robust apply to 'pair-cpat' only; other values passed with 'cp-full-rcl' or 'cpat-rcl' are ignored, with a UserWarning. The method chosen is carried into the refits made by other methods (bootstrap standard errors, DIF and DRF). 'pair-cpat' is faster on very large designs, but its standard errors are bootstrapped.

Continued on next page...

ridge	Float or None: estimation='cp-full-rcl' or 'cpat-rcl' and global, items, thresholds, bivector and matrix (its bivector parts) only. Ridge penalty ridge $\times \sum$ (deviation)^2 on the facet element deviations, in logits. None (default) resolves to 0.3.
interaction_ridge	Float or None: estimation='cp-full-rcl' or 'cpat-rcl' and matrix only. Penalty on the item $\times$ threshold interactions beyond the bivector parts. None (default) resolves to 3.
stretch_ridge	Float or None: estimation='cp-full-rcl' or 'cpat-rcl' and centrality, pseudo_halo and bistretch only. Penalty on $(\omega - 1)^2$. None (default) resolves to 1.

Returns

Returns the following attributes:

self.items	pandas series: Item location estimates with the item names as keys and estimates as values, from the latest calibration.
self.items_{model}	pandas series: the representation's own item location estimates (with estimation='pair-cpat', the same for every representation).
self.thresholds	pandas series: Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, an array of m estimates, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel $\tau_0 = 0$ has been removed; the series now has length m, not $m + 1$. From the latest calibration.
self.thresholds_{model}	pandas series: the representation's own threshold estimates (with estimation='pair-cpat', the same for every representation).
self.facet_effects	pandas series/DataFrame: Facet element names as rows and the appropriate facet element representations for global, items, thresholds, matrix or bivector as columns. For bivector, two attributes are stored: self.facet_effects_bivector_items (R×I DataFrame of per-(facet element, item) effects) and self.facet_effects_bivector_thresholds (R×K DataFrame of per-(facet element, threshold) effects). The reconstructed full matrix is available as self.facet_effects_bivector.

Continued on next page...

	For `centrality`, `pseudo_halo` and `bistretch`, returns pandas DataFrame with facet element names as rows and parameters as columns – `lambda` plus `omega` (or, for `bistretch`, `omega_items` and `omega_thresholds`). Full reconstructed vector representatin is stored separately, as `self.facet_effects_{model}_full_vector`.
`self.cat_widths`	numpy array: Array of $m-1$ category width estimates, where m is the maximum score.
`self.estimation`	String: the estimation method of the latest calibration (`self.estimation_{model}` per representation).
`self.convergence`	Dictionary or `None`: with `estimation='cp-full-rcl'` or `'cpat-rcl'`, the number of Newton–Raphson iterations (`'iterations'`) and whether the fit converged (`'converged'`), with a `UserWarning` if it did not; `None` with `estimation='pair-cpat'`. Also stored per representation as `self.convergence_{model}`.
`self.ridge`, `self.interaction_ridge`, `self.stretch_ridge`	Float or `None`: the regularisation values used (`None` where they do not apply).

Example `self.facet_effects` outputs

Toy examples of each facet element representation format, with two facet elements and two items with three categories:

Facet element representation `global`: pandas series, facet element names as the index, one facet effect per facet element (no column name, since a series has none).

Table 216: `self.facet_effects_global`

`Rater_1`	`-0.5`
`Rater_2`	`0.5`

Facet element representation `items`: pandas DataFrame, facet element names as the row index and item names as columns.

Table 217: self.facet_effects_items

	Item_1	Item_2
Rater_1	-1	0
Rater_2	0	1

Facet element representation thresholds: pandas DataFrame, facet element names as the row index and threshold positions (1 to self.max_score) as columns.

Table 218: self.facet_effects_thresholds

	1	2	3
Rater_1	-1.0	-0.5	0.0
Rater_2	1.5	0.5	-0.5

Facet element representation matrix: pandas MultiIndex DataFrame, facet element and item names as a two-level row index, threshold positions (1 to self.max_score) as columns.

Table 219: self.facet_effects_matrix

		1	2	3
Rater_1	Item_1	-1.5	-1.0	0.5
	Item_2	1.0	0.0	-1.0
Rater_2	Item_1	-0.5	0.0	0.5
	Item_2	2.0	1.0	0.0

Facet element representation bivector: two DataFrames are produced. self.facet_effects_bivector_items has facet element names as the row index and item names as columns (per-facet-element, per-item facet effects):

Table 220: self.facet_effects_bivector_items

	Item_1	Item_2
Rater_1	-0.5	-0.3
Rater_2	0.5	0.3

self.facet_effects_bivector_thresholds has facet element names as the row index and threshold positions as columns (zero-sum per facet element):

Table 221: self.facet_effects_bivector_thresholds

	1	2	3
Rater_1	-0.2	0.0	0.2
Rater_2	0.3	0.0	-0.3

The reconstructed full matrix self.facet_effects_bivector (item effect plus threshold effect per cell) has the same MultiIndex format as self.facet_effects_matrix above:

Table 222: self.facet_effects_bivector

		1	2	3
Rater_1	Item_1	-0.7	-0.5	-0.3
	Item_2	-0.5	-0.3	-0.1
Rater_2	Item_1	0.8	0.5	0.2
	Item_2	0.6	0.3	0.0

Facet element representation centrality (Jin & Wang, 2018): two parameters per facet element, self.lambda_centrality (location shift) and self.omega_centrality (threshold-stretch; also reachable as self.stretch_centrality), columns of DataFrame as self.facet_effects_centrality:

Table 223: self.lambda_centrality, self.omega_centrality

	lambda	omega
Rater_1	-0.5	1.5
Rater_2	0.5	0.6

self.facet_effects_centrality_full_vector is the reconstruction of this into the same R×K format as self.facet_effects_thresholds, for compatibility with every downstream method that expects that shape:

Table 224: self.facet_effects_centrality_full_vector

	1	2	3
Rater_1	-1.0	-0.5	0.0
Rater_2	0.9	0.5	0.1

Facet element representation pseudo_halo: two free parameters per facet element, self.lambda_pseudo_halo (location shift) and self.omega_pseudo_halo (item-difficulty-stretch), columns of DataFrame as self.facet_effects_pseudo_halo:

Table 225: self.lambda_pseudo_halo, self.omega_pseudo_halo

	lambda	omega
Rater_1	-0.5	1.5
Rater_2	0.5	0.6

self.facet_effects_pseudo_halo_full_vector is the reconstruction of this into the same R×I format as self.facet_effects_items:

Table 226: self.facet_effects_pseudo_halo_full_vector

	Item_1	Item_2
Rater_1	-1.0	0.0
Rater_2	0.9	0.1

Facet element representation bistretch: three free parameters per facet element, self.lambda_bistretch (location shift), self.omega_items_bistretch (item-stretch; also reachable as self.stretch_items_bistretch) and self.omega_thresholds_bistretch (threshold-stretch; also reachable as self.stretch_thresholds_bistretch), each a pandas series with facet element names as the index:

Table 227: self.lambda_bistretch, self.omega_items_bistretch, self.omega_thresholds_bistretch

	lambda	omega_items	omega_thresholds
Rater_1	-0.5	1.5	1.5
Rater_2	0.5	0.6	0.6

For bistretch reuses bivector, the threshold axis zero-summed per facet element (pure shape, no level). So the item component is exactly pseudo_halo's formula, $\lambda_r + (\omega_r - 1)\delta_i$, stored as self.facet_effects_bistretch_items:

Table 228: self.facet_effects_bistretch_items

	Item_1	Item_2
Rater_1	-1.0	0.0
Rater_2	0.9	0.1

The threshold component drops centrality's λ_r term since this is absorbed in the item axis above – leaving just $(\omega_{\text{thresholds},r} - 1)\tau_k$, stored as self.facet_effects_bistretch_thresholds:

Table 229: self.facet_effects_bistretch_thresholds

	1	2	3
Rater_1	-0.5	0.0	0.5
Rater_2	0.4	0.0	-0.4

The reconstructed full matrix self.facet_effects_bistretch_full_vector (item component plus threshold component per cell) has the same MultiIndex format as self.facet_effects_bivector above. self.facet_effects_bistretch itself, by contrast, is the compact three-column (lambda, omega_items, omega_thresholds) DataFrame, one row per facet element, matching self.lambda_bistretch/self.omega_items_bistretch/self.omega_thresholds_bistretch collected into one table:

Table 230: self.facet_effects_bistretch_full_vector

		1	2	3
Rater_1	Item_1	-1.5	-1.0	-0.5

Continued on next page...

(continued)

		1	2	3
	Item_2	-0.5	0.0	0.5
Rater_2	Item_1	1.3	0.9	0.5
	Item_2	0.5	0.1	-0.3

Examples

To generate a set of estimates for the global facet element representation using the default Bradley–Terry log-likelihood method for central item locations and facet effects, with additive smoothing constant of 0.1:

```
self.calibrate_global()
```

To generate a set of estimates for the matrix facet element representation using the log-likelihood method for central item locations and facet effects with the pairwise matrix raised to power 7 (overriding the unpowered default) and a tighter convergence criterion:

```
self.calibrate_matrix(method='log-lik', matrix_power=7, log_lik_tol=0.00000001)
```

To use the closed-form cosine maximisation method (Kou & Lin, 2014) for the matrix representation instead of the default:

```
self.calibrate_matrix(method='cos')
```

To generate a set of bivector estimates using the default Bradley–Terry log-likelihood method:

```
self.calibrate_bivector()
```

To generate a set of centrality estimates (recovered closed-form from an internally-calibrated thresholds fit):

```
self.calibrate_centrality()
```

To generate a set of pseudo_halo estimates, using `'theil-sen'` rather than the default `'huber'` to recover λ_r/ω_r:

```
self.calibrate_pseudo_halo(regression='theil-sen')
```

To generate a set of bivector estimates with `estimation='pair-cpat'`, with the item and threshold vectors estimated directly from pooled PAIR matrices rather than as marginal means of the matrix representation (the default):

```
self.calibrate_bivector(estimation='pair-cpat', matrix_marginals=False)
```

'Mixed' facet element model

A further model, `model='mixed'`, is available on `std_errors` (Section 6.3.3), `item_stats_df`, `threshold_stats_df`, `category_stats_df`, `person_stats_df`, `test_stats_df` and `rater_stats_df` (Section 6.4.4), but has no `calibrate_mixed()` convenience alias and is not a valid `model=` value for `calibrate` itself. Instead it depends on a per-facet-element assignment generated by `per_rater_model_selection` (Section 6.3.11) – each facet element is represented under whichever of the eight representations above was found adequate for that specific facet element, rather than one representation being applied uniformly to every facet element.

Calling any of the methods above with `model='mixed'` lazily runs `per_rater_model_selection` if it has not already been run with the specified anchor facet elements.

6.3.2 calibrate_anchor

Description

Anchors a calibration to a defined set of 'gold standard' anchor facet elements (e.g. raters), who provide a frame of reference and for whom mean facet effect is defined as zero (and, for stretch parameters with `estimation='cp-full-rcl'` or `'cpat-rcl'`, mean stretch as 1); other facet elements' facet effects shifted accordingly. For extended facet element representations, this process has a knock-on effect on the central item location and/or Rasch-Andrich threshold estimates. Full details of the anchoring procedure can be found in Elliott and Buttery (2022a). For `centrality`, `pseudo_halo` and `bistretch` calibrated with `estimation='cp-full-rcl'` or `'cpat-rcl'`, item locations and thresholds are rescaled accordingly.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.calibrate_global_anchor(anchors, calibrate=False, constant=None, threshold_constant=None,
method=None, robust=False,
                             matrix_power=None, log_lik_tol=0.000001, adj=None,
                             estimation=None, ridge=None)

self.calibrate_items_anchor(anchors, calibrate=False, constant=None, threshold_constant=None,
method=None, robust=False,
                            matrix_power=None, log_lik_tol=0.000001, adj=None,
                            estimation=None, ridge=None)

self.calibrate_thresholds_anchor(anchors, calibrate=False, constant=None, threshold_constant=None,
robust=False,
                                 method=None, matrix_power=None, log_lik_tol=0.000001,
                                 adj=None,
                                 estimation=None, ridge=None)

self.calibrate_matrix_anchor(anchors, calibrate=False, constant=None, threshold_constant=None,
method=None, robust=False,
                             matrix_power=None, log_lik_tol=0.000001, adj=None,
                             estimation=None, ridge=None, interaction_ridge=None)

self.calibrate_bivector_anchor(anchors, calibrate=False, constant=None, threshold_constant=None,
method=None, robust=False,
                               matrix_power=None, log_lik_tol=0.000001, adj=None,
                               estimation=None, matrix_marginals=None, ridge=None)

self.calibrate_centrality_anchor(anchors, calibrate=False, constant=None, threshold_constant=None,
robust=False,
                               method=None, matrix_power=None, log_lik_tol=0.000001,
                               adj=None, regression='huber',
                               estimation=None, stretch_ridge=None)
```

```
self.calibrate_pseudo_halo_anchor(anchors, calibrate=False, constant=None, threshold_constant=None,
robust=False,
                                  method=None, matrix_power=None, log_lik_tol=0.000001,
                                  adj=None, regression='huber',
                                  estimation=None, stretch_ridge=None)

self.calibrate_bistretch_anchor(anchors, calibrate=False, constant=None, threshold_constant=None,
robust=False,
                                method=None, matrix_power=None, log_lik_tol=0.000001,
                                adj=None, regression='huber',
                                estimation=None, matrix_marginals=None, stretch_ridge=None)
```

Arguments

`anchors`	List: List of names of 'gold standard' anchor facet elements.
`calibrate`	Boolean: Only needed when a standard unanchored calibration has not been run, so unanchored parameters have not yet been generated. If `True`, a standard unanchored calibration will first be run. Default value is `calibrate=False`.
`estimation`	String or `None`: Only relevant when `calibrate=True`. Estimation method: `'cp-full-rcl'` (regularised composite likelihood over all response-pattern pairs), `'cpat-rcl'` (the same, restricted to CPAT patterns) or `'pair-cpat'`. `None` uses the method of the object's latest calibration, or `'cp-full-rcl'` if none. Default is `estimation=None`.
`constant`	Float or `None`: Only relevant when `calibrate=True`. Additive smoothing constant. `None` uses the data-adaptive default. Default is `constant=None`.
`threshold_constant`	Float or `None`: Only relevant when `calibrate=True`. Additive smoothing constant for the thresholds. `None` follows `constant`. Default is `threshold_constant=None`.
`robust`	Boolean: Only relevant when `calibrate=True` and `estimation='pair-cpat'`. For `'bivector'`, estimates the item and threshold vectors directly rather than as marginal means of the matrix representation. Default is `robust=False`.

Continued on next page...

method	String: Only relevant when both calibrate=True and estimation='pair-cpat'. Method for derivation of central item location and facet effect estimates from pairwise reciprocal matrices (Elliott & Buttery, 2022b). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). None (default) resolves per representation (see Section 6.3.1).
matrix_power	Integer or None: Only relevant when calibrate=True and estimation='pair-cpat'. Power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) for central item location estimates is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: Only relevant when calibrate=True and estimation='pair-cpat'. Convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.
adj	Float, dictionary, or None: if provided, applied directly as the anchor translation/adjustment rather than being computed from anchors. Intended for reuse across bootstrap resamples, where recomputing the anchor adjustment from scratch every time would inflate standard errors with anchor-facet-element sampling variance. Default is adj=None.
regression	centrality/pseudo_halo/bistretch representations only. String, one of 'huber' or 'theil-sen': same meaning as calibrate's own regression argument (Section 6.3.1), applied when re-deriving λ_r/ω_r from the anchored facet effects. Also used for the internal calibrate call when calibrate=True. Ignored for the other five representations. Default is regression='huber'.
matrix_marginals	Boolean or None: Only relevant when calibrate=True, estimation='pair-cpat', and 'bivector' or 'bistretch'. If True, the item and threshold vectors are marginal means of the matrix representation; if False, they are estimated directly. None resolves to True, except for 'bivector' with robust=True. Default is matrix_marginals=None.
ridge	Float or None: Only relevant when calibrate=True, estimation='cp-full-rcl' or 'cpat-rcl', and 'global', 'items', 'thresholds', 'bivector' or 'matrix'. Ridge penalty on the facet element deviations, in logits. None resolves to 0.3. Default is ridge=None.

Continued on next page...

`interaction_ridge`	Float or `None`: Only relevant when `calibrate=True`, `estimation='cp-full-rcl'` or `'cpat-rcl'`, and `'matrix'`. Penalty on the item × threshold interactions beyond the bivector parts. `None` resolves to 3. Default is `interaction_ridge=None`.
`stretch_ridge`	Float or `None`: Only relevant when `calibrate=True`, `estimation='cp-full-rcl'` or `'cpat-rcl'`, and `'centrality'`, `'pseudo_halo'` or `'bistretch'`. Penalty on $(\omega - 1)^2$. `None` resolves to 1. Default is `stretch_ridge=None`.

Returns

Returns four attributes:

`self.anchor_items`	pandas series: Item location estimates with the item names as keys and estimates as values. Depending on the facet element representation, the attribute will be stored as: `self.anchor_items_global`, `self.anchor_items_items`, `self.anchor_items_thresholds`, `self.anchor_items_matrix`, `self.anchor_items_bivector`, `self.anchor_items_centrality`, `self.anchor_items_pseudo_halo` or `self.anchor_items_bistretch`.
`self.anchor_thresholds`	pandas series: Rasch-Andrich threshold estimates $\tau_1, \ldots, \tau_m$, an array of m estimates, where m is the maximum score, which sum to zero. **Note (breaking change from earlier versions):** the leading sentinel $\tau_0 = 0$ has been removed; the series now has length m, not $m + 1$. Depending on the facet element representation, the attribute will be stored as: `self.anchor_thresholds_global`, `self.anchor_thresholds_items`, `self.anchor_thresholds_thresholds`, `self.anchor_thresholds_matrix`, `self.anchor_thresholds_bivector`, `self.anchor_thresholds_centrality`, `self.anchor_thresholds_pseudo_halo` or `self.anchor_thresholds_bistretch`.
`self.anchor_facet_effects`	pandas series, dictionary, or DataFrame: Facet element names as keys and the appropriate facet element representations for `global`, `items`, `thresholds`, `matrix`, `bivector`, `centrality`, `pseudo_halo` or `bistretch` as values. For `bivector` and `bistretch`, two attributes are stored: `self.anchor_facet_effects_bivector_items` and `self.anchor_facet_effects_bivector_thresholds` (or, for `bistretch`, the `_bistretch_items`/`_bistretch_thresholds` equivalents); the reconstructed full matrix is available as `self.anchor_facet_effects_bivector` (or `_bistretch`). Depending on the facet element representation, the attribute will be stored as: `self.anchor_facet_effects_global`, `self.anchor_facet_effects_items`, `self.anchor_facet_effects_thresholds`, `self.anchor_facet_effects_matrix` or

Continued on next page...

	self.anchor_facet_effects_bivector.
self.anchor_cat_widths	numpy array Array of $m-1$ category width estimates, where m is the maximum score, as values. Depending on the facet element representation, the attribute will be stored as: self.anchor_cat_widths_global, self.anchor_cat_widths_items, self.anchor_cat_widths_thresholds, self.anchor_cat_widths_matrix, self.anchor_cat_widths_bivector, self.anchor_cat_widths_centrality, self.anchor_cat_widths_pseudo_halo or self.anchor_cat_widths_bistretch.

Example self.facet_effects **outputs**

Toy examples of each facet element representation format, with two facet elements and two items with three categories:

Facet element representation global: pandas series with floats (facet effects) for values.

```
self.facet_effects_global = pd.Series({'Rater_1':  -0.5,
                                       'Rater_2':   0.5}
```

Facet element representation items: Dictionary with pandas series for values. The pandas series have item names for keys and floats (item facet effects) for values.

```
self.facet_effects_items = {'Rater_1':  pd.Series({'Item_1':  -1, 'Item_2':  0}),
                            'Rater_2':  pd.Series({'Item_1':   0, 'Item_2':  1})}
```

Facet element representation thresholds: Dictionaries with numpy arrays for values. The numpy arrays contain the facet effects by threshold, with the integer of the array index corresponding to each threshold (0 to self.max_score).

```
self.facet_effects_thresholds = {'Rater_1':  np.array([0,  -1, -0.5,    0]),
                                 'Rater_2':  np.array([0, 1.5,  0.5, -0.5])}
```

Facet element representation matrix: Nested dictionary. The outer dictionary has item names for values, which are also the keys for the inner dictionary. Values for the inner dictionary are numpy arrays of facet effects by threshold for the item key, with the integer of the array index corresponding to each threshold (0 to self.max_score).

```
self.facet_effects_matrix = {'Rater_1':  {'Item_1':  np.array([0, -1.5, -1, 0.5]),
                                          'Item_2':  np.array([0,    1,  0,  -1])},
                             'Rater_2':  {'Item_1':  np.array([0, -0.5,  0, 0.5]),
                                          'Item_2':  np.array([0,    2,  1,   0])}}
```

Facet element representation bivector: Two DataFrames as for calibrate_bivector above: self.anchor_facet_effects_bivector_items and self.anchor_facet_effects_bivector_thresholds, with the reconstructed full matrix in self.anchor_facet_effects_bivector.

Examples

To generate a set of anchored estimates for the global facet element representation from an existing unanchored calibration, anchored to facet elements `'Rater_1'` and `'Rater_2'`:

```
self.calibrate_global_anchor(['Rater_1', 'Rater_2'])
```

To generate a set of estimates for the matrix facet element representation anchored to facet elements `'Rater_1'` and `'Rater_2'`, first running an unanchored calibration, using the log-likelihood method for central item locations and facet effects, with matrix raised to power 7 and a convergence stopping criterion of 0.00000001:

```
self.calibrate_matrix_anchor(['Rater_1', 'Rater_2'], calibrate=True,
                        method='log-lik', matrix_power=7, log_lik_tol=0.00000001)
```

6.3.3 std_errors

Description

Produces standard errors of central item locations, thresholds, category widths and facet element effects. `se_method='sandwich'` (representations calibrated with `estimation='cp-full-rcl'` or `'cpat-rcl'`) computes analytic Huber-White Sandwich Estimator standard errors (Huber, 1967); `se_method='bootstrap'` draws `no_of_samples` bootstrap resamples of persons. `se_method=None` (default) uses the sandwich where it applies; if it does not apply (an unused score category, a fit that did not converge, estimates not identified by the data, or a design too large), the bootstrap is used, with a `UserWarning`. An explicit `se_method='sandwich'` gives NaN for estimates the data do not identify. Standard errors of facet elements that used only the lowest or only the highest score category are NaN.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.std_errors_global(anchors=None, interval=None, no_of_samples=500, constant=None,
                       threshold_constant=None, robust=False,
                       method=None, matrix_power=None, log_lik_tol=0.000001,
                       store_bootstrap=False, seed=None, se_method=None)

self.std_errors_items(anchors=None, interval=None, no_of_samples=500, constant=None,
                       threshold_constant=None, robust=False,
                       method=None, matrix_power=None, log_lik_tol=0.000001,
                       store_bootstrap=False, seed=None, se_method=None)

self.std_errors_thresholds(anchors=None, interval=None, no_of_samples=500,
                         constant=None, threshold_constant=None, method=None,
                         robust=False, matrix_power=None,
                         log_lik_tol=0.000001, store_bootstrap=False, seed=None, se_method=None)

self.std_errors_matrix(anchors=None, interval=None, no_of_samples=500, constant=None,
                       threshold_constant=None, robust=False,
                       method=None, matrix_power=None, log_lik_tol=0.000001,
                       store_bootstrap=False, seed=None, se_method=None)

self.std_errors_bivector(anchors=None, interval=None, no_of_samples=500,
                         constant=None, threshold_constant=None, method=None,
                         robust=False, matrix_power=None,
                         log_lik_tol=0.000001, store_bootstrap=False, seed=None, se_method=None)

self.std_errors_centrality(anchors=None, interval=None, no_of_samples=500,
                          constant=None, threshold_constant=None, method=None,
                          robust=False, matrix_power=None,
                          log_lik_tol=0.000001, store_bootstrap=False, seed=None, se_method=None)

self.std_errors_pseudo_halo(anchors=None, interval=None, no_of_samples=500,
                            constant=None, threshold_constant=None, method=None,
```

```
                                robust=False, matrix_power=None,
                                log_lik_tol=0.000001, store_bootstrap=False, seed=None, se_method=None)

self.std_errors_bistretch(anchors=None, interval=None, no_of_samples=500,
                          constant=None, threshold_constant=None, method=None,
                          robust=False, matrix_power=None,
                          log_lik_tol=0.000001, store_bootstrap=False, seed=None, se_method=None)
```

Arguments

`anchors`	List: List of names of 'gold standard' anchor facet elements. Only used if anchored standard errors are being produced. Default is `anchors=None`.
`interval`	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval. Defines a central interval of proportion p, e.g. `interval=0.95` defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is `interval=None`.
`no_of_samples`	Integer: Number of bootstrap samples to generate. More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is `no_of_samples=500`.
`constant`	Float or `None`: additive smoothing constant (Elliott & Buttery, 2022b). `None` (default) resolves to the simulation-tuned per-stage defaults (see Section 6.3.1).
`threshold_constant`	Float or `None`: smoothing constant for the threshold-location (CPAT) stage alone; `None` (default) follows `constant`. See Section 6.3.1.
`robust`	Boolean, default `False`: selects the more conservative per-representation calibration for rough data. See Section 6.3.1.
`method`	String: method for derivation of vector of central item location estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b). Options are `'ls'` for least squares (Choppin, 1968, 1985), `'evm'` for the eigenvector method(Garner & Engelhard, 2002), `'cos'` for cosine maximisation (Kou & Lin, 2014) or `'log-lik'` for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). `None` (default) resolves per representation (see Section 6.3.1).
`matrix_power`	Integer or `None`: power to which conditional category response frequency matrix for central item location estimates (Elliott & Buttery, 2022b:991) is raised. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is `matrix_power=None`, which resolves to `0` (no matrix powering) when `method='log-lik'` and to `5` for every other method. Passing `matrix_power=0` with a method other than `'log-lik'` issues a `UserWarning`.

Continued on next page...

log_lik_tol	Float: convergence stopping criterion for log-likelihood method for central item location estimates. Ignored for other methods.
store_bootstrap	Boolean: if True, stores the fitted (unanchored) bootstrap samples as self._bootstrap_samples_{model} rather than discarding them once the SEs above are computed. Costs memory (no_of_samples fitted MFRM objects) but allows anchor_std_errors (Section 6.3.4) to reuse them – recalibrating each stored sample to a chosen anchor set – instead of refitting the whole bootstrap from scratch for every anchor set tried. Default is store_bootstrap=False.
seed	Integer or None: seed for the bootstrap resampling random number generator (and, for model='mixed', passed through to the lazily-triggered per_rater_model_selection call if it has not already been run). Pass an integer for fully reproducible standard errors; None (default) draws fresh entropy each call. See Section 1.
se_method	String or None: 'sandwich', 'bootstrap' or None (see Description). 'sandwich' raises a ValueError for a representation not calibrated with estimation='cp-full-rcl' or 'cpat-rcl', for mixed, or with store_bootstrap=True. With the sandwich, the _bootstrap attributes are None. Default is se_method=None.

Returns

Creates a set of attributes, which depend on whether an argument is passed to anchors. Where store_bootstrap=True, the underlying bootstrap replicas themselves remain retrievable too, with the full complement of raw per-sample values behind every summary below – see the note at the end of this section.

If anchors=None:

self.item_se	pandas series: Item names as keys and item standard errors as values, for the latest representation. Stored per representation as self.item_se_{model}.
self.threshold_se	pandas series: Threshold numbers as keys and item standard errors as values: self.threshold_se_global, self.threshold_se_items, self.threshold_se_thresholds, self.threshold_se_matrix, self.threshold_se_bivector, self.threshold_se_centrality, self.threshold_se_pseudo_halo or self.threshold_se_bistretch
self.cat_width_bootstrap	dict: Category numbers as keys and arrays of the per-sample bootstrap replicate category width values as values: self.cat_width_bootstrap_global, self.cat_width_bootstrap_items, self.cat_width_bootstrap_thresholds, self.cat_width_bootstrap_matrix,

Continued on next page...

	self.cat_width_bootstrap_bivector, self.cat_width_bootstrap_centrality, self.cat_width_bootstrap_pseudo_halo or self.cat_width_bootstrap_bistretch
self.cat_width_se	dict: Category numbers as keys and item standard errors as values: self.cat_width_se_global, self.cat_width_se_items, self.cat_width_se_thresholds, self.cat_width_se_matrix, self.cat_width_se_bivector, self.cat_width_se_centrality, self.cat_width_se_pseudo_halo or self.cat_width_se_bistretch
self.rater_se	pandas series or DataFrame: Item names as keys and item standard errors as values. For bivector and bistretch, two additional attributes self.rater_se_marginal_items and self.rater_se_marginal_thresholds are also stored. Depending on the facet element representation: self.rater_se_global, self.rater_se_items, self.rater_se_thresholds, self.rater_se_matrix, self.rater_se_bivector, self.rater_se_centrality, self.rater_se_pseudo_halo or self.rater_se_bistretch

If an argument is passed to interval, also returns:

self.item_low	Lower bound of the specified interval for item estimates, for the latest representation; per representation as self.item_low_{model}.
self.item_high	Upper bound of the specified interval for item estimates, for the latest representation; per representation as self.item_high_{model}.
self.threshold_low	Lower bound of the specified interval for threshold estimates: self.threshold_low_global, self.threshold_low_items, self.threshold_low_thresholds, self.threshold_low_matrix, self.threshold_low_bivector, self.threshold_low_centrality, self.threshold_low_pseudo_halo or self.threshold_low_bistretch
self.threshold_high	Upper bound of the specified interval for threshold estimates: self.threshold_high_global, self.threshold_high_items, self.threshold_high_thresholds, self.threshold_high_matrix, self.threshold_high_bivector, self.threshold_high_centrality, self.threshold_high_pseudo_halo or self.threshold_high_bistretch
self.cat_width_low	Lower bound of the specified interval for category estimates: self.cat_width_low_global, self.cat_width_low_items, self.cat_width_low_thresholds, self.cat_width_low_matrix, self.cat_width_low_bivector, self.cat_width_low_centrality,

Continued on next page...

	self.cat_width_low_pseudo_halo or self.cat_width_low_bistretch
self.cat_width_high	Upper bound of the specified interval for category estimates: self.cat_width_high_global, self.cat_width_high_items, self.cat_width_high_thresholds, self.cat_width_high_matrix, self.cat_width_high_bivector, self.cat_width_high_centrality, self.cat_width_high_pseudo_halo or self.cat_width_high_bistretch
self.rater_low	Lower bound of the specified interval for facet effect estimates: self.rater_low_global, self.rater_low_items, self.rater_low_thresholds, self.rater_low_matrix, self.rater_low_bivector, self.rater_low_centrality, self.rater_low_pseudo_halo or self.rater_low_bistretch
self.rater_high	Upper bound of the specified interval for facet effect estimates: self.rater_high_global, self.rater_high_items, self.rater_high_thresholds, self.rater_high_matrix, self.rater_high_bivector, self.rater_high_centrality, self.rater_high_pseudo_halo or self.rater_high_bistretch

If anchors is not None:

self.anchor_item_se	pandas series: Item names as keys and item standard errors as values, for the latest representation. Stored per representation as self.anchor_item_se_{model}.
self.anchor_threshold_se	pandas series: Threshold numbers as keys and item standard errors as values: self.anchor_threshold_se_global, self.anchor_threshold_se_items, self.anchor_threshold_se_thresholds, self.anchor_threshold_se_matrix, self.anchor_threshold_se_bivector, self.anchor_threshold_se_centrality, self.anchor_threshold_se_pseudo_halo or self.anchor_threshold_se_bistretch
self.anchor_cat_width_bootstrap	dict: Category numbers as keys and arrays of the per-sample bootstrap replicate category width values as values: self.anchor_cat_width_bootstrap_global, self.anchor_cat_width_bootstrap_items, self.anchor_cat_width_bootstrap_thresholds, self.anchor_cat_width_bootstrap_matrix, self.anchor_cat_width_bootstrap_bivector, self.anchor_cat_width_bootstrap_centrality, self.anchor_cat_width_bootstrap_pseudo_halo or self.anchor_cat_width_bootstrap_bistretch
self.cat_width_se	dict: Category numbers as keys and item standard errors as values: self.anchor_cat_width_se_global, self.anchor_cat_width_se_items,

Continued on next page...

	self.anchor_cat_width_se_thresholds, self.anchor_cat_width_se_matrix, self.anchor_cat_width_se_bivector, self.anchor_cat_width_se_centrality, self.anchor_cat_width_se_pseudo_halo or self.anchor_cat_width_se_bistretch
self.anchor_ater_se	pandas series: Item names as keys and item standard errors as values: self.anchor_rater_se_global, self.anchor_rater_se_items, self.anchor_rater_se_thresholds, self.anchor_rater_se_matrix, self.anchor_rater_se_bivector, self.anchor_rater_se_centrality, self.anchor_rater_se_pseudo_halo or self.anchor_rater_se_bistretch

If an argument is passed to interval, also returns:

self.anchor_item_low	Lower bound of the specified interval for item estimates, for the latest representation; per representation as self.anchor_item_low_{model}.
self.anchor_item_high	Upper bound of the specified interval for item estimates, for the latest representation; per representation as self.anchor_item_high_{model}.
self.anchor_threshold_low	Lower bound of the specified interval for threshold estimates: self.anchor_threshold_low_global, self.anchor_threshold_low_items, self.anchor_threshold_low_thresholds, self.anchor_threshold_low_matrix, self.anchor_threshold_low_bivector, self.anchor_threshold_low_centrality, self.anchor_threshold_low_pseudo_halo or self.anchor_threshold_low_bistretch
self.anchor_threshold_high	Upper bound of the specified interval for threshold estimates: self.anchor_threshold_high_global, self.anchor_threshold_high_items, self.anchor_threshold_high_thresholds, self.anchor_threshold_high_matrix, self.anchor_threshold_high_bivector, self.anchor_threshold_high_centrality, self.anchor_threshold_high_pseudo_halo or self.anchor_threshold_high_bistretch
self.anchor_cat_width_low	Lower bound of the specified interval for category estimates: self.anchor_cat_width_low_global, self.anchor_cat_width_low_items, self.anchor_cat_width_low_thresholds, self.anchor_cat_width_low_matrix, self.anchor_cat_width_low_bivector, self.anchor_cat_width_low_centrality, self.anchor_cat_width_low_pseudo_halo or self.anchor_cat_width_low_bistretch
self.anchor_cat_	Upper bound of the specified interval for category estimates:

Continued on next page...

width_high	self.anchor_cat_width_high_global, self.anchor_cat_width_high_items, self.anchor_cat_width_high_thresholds, self.anchor_cat_width_high_matrix, self.anchor_cat_width_high_bivector, self.anchor_cat_width_high_centrality, self.anchor_cat_width_high_pseudo_halo or self.anchor_cat_width_high_bistretch
self.anchor_rater_low	Lower bound of the specified interval for facet effect estimates: self.anchor_rater_low_global, self.anchor_rater_low_items, self.anchor_rater_low_thresholds, self.anchor_rater_low_matrix, self.anchor_rater_low_bivector, self.anchor_rater_low_centrality, self.anchor_rater_low_pseudo_halo or self.anchor_rater_low_bistretch
self.anchor_rater_high	Upper bound of the specified interval for facet effect estimates: self.anchor_rater_high_global, self.anchor_rater_high_items, self.anchor_rater_high_thresholds, self.anchor_rater_high_matrix, self.anchor_rater_high_bivector, self.anchor_rater_high_centrality, self.anchor_rater_high_pseudo_halo or self.anchor_rater_high_bistretch

Retrieving the raw bootstrap samples

The tables above report summary statistics (standard errors and optional interval bounds) derived from the bootstrap. If store_bootstrap=True is passed, the no_of_samples bootstrapped samples are stored too, as a list assigned to self._bootstrap_samples_{model}. Each replica is a fully-calibrated MFRM object in its own right, carrying every attribute an ordinary calibrate (or, if anchors was passed, calibrate_anchor) call would produce for that model, giving direct access to the per-sample values behind every summary statistic reported above. For example, [s.items for s in self._bootstrap_samples_global] recovers the full set of bootstrapped item vectors – with no separate item_bootstrap, threshold_bootstrap or rater_bootstrap attribute needed. If store_bootstrap=False (the default), the samples are discarded once the summary statistics above have been computed.

Examples

To generate unanchored item standard errors for the global representation with a 95% interval from 200 samples:

```
self.std_errors_global(interval=0.95, no_of_samples=200)
```

To generate anchored item standard errors for the matrix representation with a 90% interval from 200 samples, anchored to facet elements 'Rater_1' and 'Rater_2':

```
self.anchor_std_errors_matrix(anchors=['Rater_1', 'Rater_2'], no_of_samples=200)
```

Modifications to the estimation method are discussed in Section 6.3.1.

'Mixed' facet element representation

std_errors(model='mixed') produces bootstrap standard errors under the per-facet-element representation assignment from per_rater_model_selection (Section 6.3.11), lazily run with its own default selection

criteria if it has not already been run (or was run with a different anchor set) – see the 'mixed' representation note under `calibrate` (Section 6.3.1). There is no `std_errors_mixed()` convenience alias; call `self.std_errors(model='mixed', ...)` instead.

6.3.4 anchor_std_errors

Description

Produces standard errors from pre-existing bootstraps when std_errors was previously run with store_bootstrap=True, as a means of avoiding the need to re-run bootstrapping when re-anchoring to a different set of anchor facet elements (e.g. raters). Standard errors of facet elements that used only the lowest or only the highest score category are NaN.

Confidence interval width is inherited from the std_errors run for this model if not explicitly provided, so anchored and unanchored confidence intervals stay on a consistent basis; pass interval=0 to explicitly suppress this inheritance. Stores anchor_item_se (and _low/_high if an interval is set), anchor_threshold_se_{model}, anchor_rater_se_{model}, and the corresponding low/high attributes, mirroring std_errors's own naming convention.

When the fast path is taken (stored samples reused), each replica in self._bootstrap_samples_{model} gains the anchor-calibrated attributes too, so it remains the same list to consult for the raw per-sample values behind these summaries – see "Retrieving the raw bootstrap replicas" under std_errors (Section 6.3.3).

Usage

Eight methods are defined, one for each facet element representation; all methods share the same argument format:

```
self.anchor_std_errors_global(anchors=None, interval=None, no_of_samples=500,
                              constant=None, threshold_constant=None, method=None, robust=False,
matrix_power=None,
                              log_lik_tol=0.000001, seed=None)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch).

Arguments

anchors	List: facet elements whose mean facet effect is anchored to zero. If None, falls back to anchor_rater_names_{model} set by calibrate_anchor (Section 6.3.2) – so calibrate_anchor must already have been run for this model if anchors is left as None. Default is anchors=None.
interval	Float or None: as for std_errors (Section above), but inherits the interval used by that model's std_errors run if left as None. Pass interval=0 to explicitly suppress CIs even if std_errors used one. Default is interval=None.
no_of_samples	Integer: number of bootstrap samples. Only used on the slow (full refit) path, when stored samples from a prior store_bootstrap=True run are not available. Default is no_of_samples=500.

Continued on next page...

constant, method, matrix_power, log_lik_tol	Calibration keyword arguments, as for std_errors (Section above). See Section 6.3.1 for details of each.
seed	Integer or None: seed for the bootstrap resampling random number generator. Only used on the slow path. None (default) draws fresh entropy each call. See Section 1.

Returns

None returned directly; sets self.anchor_item_se and self.anchor_item_se_{model} (plus _low/_high if interval is set), self.anchor_threshold_se_{model} and self.anchor_rater_se_{model} (plus their own _low/_high), mirroring std_errors's unanchored attribute names with an anchor_ prefix.

Example

To run the unanchored bootstrap once, storing samples for reuse, then compute anchored SEs for two different candidate anchor sets under the matrix representation without refitting the bootstrap each time:

```
self.std_errors_matrix(store_bootstrap=True)
self.anchor_std_errors_matrix(anchors=['Rater_1', 'Rater_2'])
self.anchor_std_errors_matrix(anchors=['Rater_1', 'Rater_3'])
```

If calibrate_anchor has already been run for a model, anchors can be omitted to reuse its anchor set:

```
self.calibrate_anchor('matrix', ['Rater_1', 'Rater_2'])
self.anchor_std_errors_matrix()
```

6.3.5 person

Description

Generates a location estimate for a person using the Newton-Raphson method to produce a maximum likelihood estimate, or, by default, Warm's weighted likelihood estimate (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

```
self.person(persons, model='global', anchor=False, items=None, facet_elements=None,
            warm_corr=True, tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
            missing_as_incorrect=False)
```

Arguments

persons	String or list: person name(s) for the location(s) being estimated. Pass 'all' to estimate for all persons in self.responses.
model	String, one of 'global', 'items', 'thresholds', 'matrix', 'bivector', 'centrality', 'pseudo_halo' or 'bistretch': the facet element (e.g. rater) representation to use. Default is model='global'.
anchor	Boolean:If True, parameters from an anchored calibration will be used. Default is anchor=False.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
facet_elements	List: List of names of a subset of facet element, based on which to generate the location estimate. Default is facet_elements=None, which generates a location based on the full set of facet elements. Only use when an estimate based on a subset of facet elements or an individual facet element is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations. Default is max_iters=100.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure an ML estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. Ignored when warm_corr=True.
missing_as_incorrect	Boolean: if True, treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is missing_as_incorrect=False.

Returns

pandas series: person location estimate(s), in logits, indexed by person identifier. Returns NaN for any person for whom estimation fails.

Example

To generate an unanchored person location estimate for Person_1 using a global facet element representation using the default settings and store the result as a variable, my_person_location:

```
my_person_location = self.person('Person_1')
```

To generate an anchored MLE person location estimate under the matrix facet element representation without Warm bias correction for Person_1 based on the first three items rated by Rater_1 and store the result as a variable, my_person_location:

```
my_person_location = self.person('Person_1', model='matrix',
                                 items=['Item_1', 'Item_2', 'Item_3'],
                                 facet_elements=['Rater_1'], warm_corr=False)
```

6.3.6 person_estimates

Description

Generates location estimates for all persons using the Newton-Raphson method to produce maximum likelihood estimates, or, by default, Warm's weighted likelihood estimates (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

Eight methods are defined, one for each facet element (e.g. rater) representation; all methods share the same argument format:

```
self.person_estimates_global(anchor=False, items=None, facet_elements=None, warm_corr=True,
                         tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                         missing_as_incorrect=False)

self.person_estimates_items(anchor=False, items=None, facet_elements=None, warm_corr=True,
                        tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                        missing_as_incorrect=False)

self.person_estimates_thresholds(anchor=False, items=None, facet_elements=None, warm_corr=True,
                           tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                           missing_as_incorrect=False)

self.person_estimates_matrix(anchor=False, items=None, facet_elements=None, warm_corr=True,
                         tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                         missing_as_incorrect=False)

self.person_estimates_bivector(anchor=False, items=None, facet_elements=None, warm_corr=True,
                           tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                           missing_as_incorrect=False)


self.person_estimates_centrality(anchor=False, items=None, facet_elements=None, warm_corr=True,
                            tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                            missing_as_incorrect=False)


self.person_estimates_pseudo_halo(anchor=False, items=None, facet_elements=None, warm_corr=True,
                              tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                              missing_as_incorrect=False)


self.person_estimates_bistretch(anchor=False, items=None, facet_elements=None, warm_corr=True,
                            tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                            missing_as_incorrect=False)
```

Arguments

anchor	Boolean:If True, parameters from an anchored calibration will be used. Default is anchor=False.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.
facet_elements	List: List of names of a subset of facet element, based on which to generate the location estimate. Default is facet_elements=None, which generates a location based on the full set of facet elements. Only use when an estimate based on a subset of facet elements or an individual facet element is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations. Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure an ML estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. Ignored when warm_corr=True.
missing_as_incorrect	Boolean: if True, treats missing responses as score 0 rather than excluding them from the likelihood – relevant for educational testing contexts where non-response implies an incorrect answer. Default is missing_as_incorrect=False.

Returns

- If anchor=False, attribute self.persons_global, self.persons_items, self.persons_thresholds, self.persons_matrix, self.persons_bivector, self.persons_centrality, self.persons_pseudo_halo or self.persons_bistretch: pandas series with person names as keys and location estimates as values.
- If anchor=True, attribute self.anchor_persons_global, self.anchor_persons_items, self.anchor_persons_thresholds, self.anchor_persons_matrix, self.anchor_persons_bivector,

`self.anchor_persons_centrality`, `self.anchor_persons_pseudo_halo` or `self.anchor_persons_bistretch`: pandas series with person names as keys and location estimates as values.

Examples

To generate a set of unanchored person location estimates for the global facet element representation from all items and facet elements with Warm bias correction:

```
self.person_estimates_global()
```

To generate a set of anchored person location estimates for the matrix facet element representation without Warm bias correction, on a subset of the first three items only, rated by `Rater_1`:

```
self.person_estimates_matrix(anchor=True, items=['Item_1', 'Item_2', 'Item_3'],
                             facet_elements=['Rater_1'], warm_corr=False)
```

6.3.7 score_lookup

Description

Generates a location estimate for a given raw score on responses to a given set of items using the Newton-Raphson method to produce a maximum likelihood estimate, or, by default, Warm's weighted likelihood estimate (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

Eight methods are defined, one for each facet element (e.g. rater) representation; all methods share the same argument format:

```
self.score_lookup_global(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                         tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_items(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                        tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_thresholds(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                             tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_matrix(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                         tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_bivector(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                           tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_centrality(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                             tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_pseudo_halo(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                              tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)

self.score_lookup_bistretch(score, anchor=False, items=None, facet_elements=None, warm_corr=True,
                            tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5)
```

Arguments

score	Integer: The raw score for which location is being estimated.
anchor	Boolean:If True, parameters from an anchored calibration will be used. Default is anchor=False.
items	List: List of names of a subset of items, based on which to generate the location estimate. Default is items=None, which generates a location based on the full set of items. Only use when an estimate based on a subset of items is required.

Continued on next page...

facet_elements	List: List of names of a subset of facet element, based on which to generate the location estimate. Default is facet_elements=None, which generates a location based on the full set of facet elements. Only use when an estimate based on a subset of facet elements or an individual facet element is required.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations. Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure an ML estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. Ignored when warm_corr=True.

Returns

Float: Location estimate corresponding to raw score.

Examples

To generate an unanchored location estimate under the global facet element representation for a score of 10 on all items rated by all facet elements, with Warm bias correction, and store the result as a variable, my_score_location:

```
my_score_location = self.score_lookup_global(10)
```

To generate an anchored location estimate under the matrix facet element representation for a score of 10 on a subset of items saved as a variable my_items, rated by Rater_1 without Warm bias correction, and store the result as a variable, my_score_location:

```
my_score_location = self.score_lookup_matrix(10, anchor=True, items=my_items,
                                             facet_elements=['Rater_1'], warm_corr=False)
```

6.3.8 score_lookup_table

Description

Generates a lookup table of location estimates corresponding to all available raw scores on a set of items with no missing responses, using the Newton-Raphson method to produce maximum likelihood estimates, or, by default, Warm's weighted likelihood estimates (Warm, 1989), found by solving the weighted likelihood equation directly. Warm estimates are finite for extreme scores.

Usage

Eight methods are defined, one for each facet element (e.g. rater) representation; all methods share the same argument format:

```
self.score_lookup_table_global(anchor=False, attribute=True, items=None,
                               facet_elements=None, ext_scores=True, warm_corr=True,
                               tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                               filename=None, file_format='csv')

self.score_lookup_table_items(anchor=False, attribute=True, items=None,
                              facet_elements=None, ext_scores=True, warm_corr=True,
                              tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                              filename=None, file_format='csv')

self.score_lookup_table_thresholds(anchor=False, attribute=True, items=None,
                                   facet_elements=None, ext_scores=True, warm_corr=True,
                                   tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                   filename=None, file_format='csv')

self.score_lookup_table_matrix(anchor=False, attribute=True, items=None,
                               facet_elements=None, ext_scores=True, warm_corr=True,
                               tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                               filename=None, file_format='csv')

self.score_lookup_table_bivector(anchor=False, attribute=True, items=None,
                                 facet_elements=None, ext_scores=True, warm_corr=True,
                                 tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                 filename=None, file_format='csv')

self.score_lookup_table_centrality(anchor=False, attribute=True, items=None,
                                   facet_elements=None, ext_scores=True, warm_corr=True,
                                   tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                   filename=None, file_format='csv')

self.score_lookup_table_pseudo_halo(anchor=False, attribute=True, items=None,
                                    facet_elements=None, ext_scores=True, warm_corr=True,
                                    tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                    filename=None, file_format='csv')

self.score_lookup_table_bistretch(anchor=False, attribute=True, items=None,
```

```
facet_elements=None, ext_scores=True, warm_corr=True,
tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
filename=None, file_format='csv')
```

Arguments

anchor	Boolean:If True, parameters from an anchored calibration will be used. Default is anchor=False.
attribute	Boolean: if True, stores the result as self.score_table_{model}. If False, the table is only returned, not stored. Default is attribute=True.
items	List: List of subset of items on which to generate the location estimate. Default items=None, which generates a location based on all items. Only use when an estimate based on a subset of items is required.
facet_elements	List: List of names of a subset of facet element, based on which to generate the location estimate. Default is facet_elements=None, which generates a location based on the full set of facet elements. Only use when an estimate based on a subset of facet elements or an individual facet element is required.
ext_scores	Boolean: If True, location estimates for extreme scores (all correct/all incorrect) will be generated using the ext_score_adjustment argument. Default is ext_scores=True.
warm_corr	Boolean: if True, Warm's weighted likelihood estimate (Warm, 1989) is returned, found by solving the weighted likelihood equation directly; if False, the maximum likelihood estimate is returned. Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations. Default is tolerance=0.00001.
max_iters	Integer: maximum Newton-Raphson iterations. Default max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure an ML estimate is returned if the person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. Ignored when warm_corr=True.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

pandas series with raw scores as keys and corresponding location estimates as values. If attribute=True (the default), also stored as self.score_table_{model} (e.g. self.score_table_global).

Examples

To generate an unanchored location lookup table under the global facet element representation for all items

rated by all facet elements, including extreme scores, with Warm bias correction:

```
self.score_lookup_table()
```

To generate an anchored location lookup table under the matrix facet element representation for a subset of items saved as a variable `my_items`, rated by `Rater_1`, with extreme scores and Warm bias correction:

```
self.score_lookup_table(anchor=True, items=my_items, facet_elements=['Rater_1']
```

6.3.9 csem

Description

Calculates the conditional standard error of measurement (CSEM = $1/\sqrt{I}$, where I is the total Fisher information summed across all observed facet-element/item combinations) for one or more persons, from a set of items and facet elements (e.g. raters).

Usage

Eight methods are defined, one for each facet element representation; all methods share the same argument format:

```
self.csem_global(anchor=False, persons=None, person_locations=None, items=None,
                 facet_elements=None)

self.csem_items(anchor=False, persons=None, person_locations=None, items=None,
                facet_elements=None)

self.csem_thresholds(anchor=False, persons=None, person_locations=None, items=None,
                     facet_elements=None)

self.csem_matrix(anchor=False, persons=None, person_locations=None, items=None,
                 facet_elements=None)

self.csem_bivector(anchor=False, persons=None, person_locations=None, items=None,
                   facet_elements=None)

self.csem_centrality(anchor=False, persons=None, person_locations=None, items=None,
                     facet_elements=None)

self.csem_pseudo_halo(anchor=False, persons=None, person_locations=None, items=None,
                      facet_elements=None)

self.csem_bistretch(anchor=False, persons=None, person_locations=None, items=None,
                    facet_elements=None)
```

Arguments

anchor	Boolean:If True, parameters from an anchored calibration will be used. Default is anchor=False.
persons	String, list, or None: subset of persons to compute CSEM for. Overrides person_locations if both are supplied (i.e. person_locations is filtered down to just these persons). Default is persons=None, which uses all persons.

Continued on next page...

person_locations	pandas series, float, list, numpy array, or None: person location estimate(s) to compute CSEM for. If None (default), uses the stored self.persons_{model} (or self.anchor_persons_{model} if anchor=True) location estimates, auto-triggering person_estimates (Section 6.3.6) if not already present. A raw float, list, or numpy array of locations (i.e. not indexed by a real person) is treated as *hypothetical*: since there is no observed response row to consult for such a location, every item in items is treated as answered by every facet element in facet_elements. Default is person_locations=None.
items	List or None: List of names of a subset of items, based on which to compute CSEM. Default is items=None, which uses the full set of items.
facet_elements	List, string, or None: subset of facet elements used to compute CSEM. Default is facet_elements=None, which uses the full set of facet elements.

Returns

pandas series: CSEM values, in logits, indexed by person (or, for hypothetical locations passed directly via person_locations, indexed by a generated label for each value).

Examples

To generate the unanchored CSEM for all persons under the global facet element representation, on all items and facet elements, and save the result as a variable, my_csem:

```
my_csem = self.csem_global()
```

To generate the anchored CSEM for Person_1 under the matrix facet element representation on a subset of items ['Item_1', 'Item_2'], and a subset of facet elements ['Rater_1', 'Rater_2'], and save the result as a variable, my_csem:

```
my_csem = self.csem_matrix(anchor=True, persons='Person_1',
                           items=['Item_1', 'Item_2'],
                           facet_elements=['Rater_1', 'Rater_2'])
```

To generate the CSEM for a hypothetical person location of 0.5 on a subset of items saved as a variable my_items:

```
my_csem = self.csem_global(person_locations=0.5, items=my_items)
```

6.3.10 model_selection

Description

Compares the eight MFRM facet element (e.g. rater) representations (global, items, thresholds, matrix, bivector, centrality, pseudo_halo, bistretch – see Section 6.3.1) via AIC, BIC or a likelihood ratio test, to choose which single representation is the most appropriate, parsimonious fit for every facet element at once. Contrast with per_rater_model_selection (Section 6.3.11), which instead assigns an individual, potentially different, representation to each facet element, which may vary. Calibrates and runs person_estimates (Section 6.3.6) for all active representations if not already done – by default all eight, or only those listed in models if it is set.

Usage

```
self.model_selection(test='AIC', models=None, aic_sig_test=True, alpha=0.05,
                     min_effect=0, sampling='dynamic', warm_corr=True, tolerance=0.00001,
                     max_iters=100, ext_score_adjustment=0.5, constant=None,
                     threshold_constant=None, method=None, robust=False,
                     matrix_power=None, log_lik_tol=0.000001, seed=None)
```

Arguments

test	String: 'AIC', 'BIC', or 'LR'. For 'LR', nested pairwise tests are computed rather than a single ranking, since the eight representations do not form one single nested chain: global vs. each of items, thresholds, centrality, pseudo_halo and bistretch; items vs. bivector and pseudo_halo; thresholds vs. bivector and centrality; bivector vs. matrix and bistretch; centrality vs. bistretch; pseudo_halo vs. bistretch. Degrees of freedom fall directly out of the parameter-count difference between each pair. Default is test='AIC'.
models	List/tuple of strings, or None: restricts the comparison to the models in the list. For test='LR', pairs of models must be nested for comparison, but for AIC/BIC, this is not necessary. None (default) uses all eight representations. Where the design makes one representation the same model as another, the later one in the order global, items, thresholds, bivector, matrix, centrality, pseudo_halo, bistretch is left out, with a warning, and all other comparisons still run: centrality = thresholds when $m = 2$; pseudo_halo = items with 2 items, and also bistretch = bivector when $m = 2$; for dichotomous data, thresholds and centrality = global, bivector and matrix = items, and bistretch = pseudo_halo.

Continued on next page...

aic_sig_test	Boolean: when True (requires test='AIC'), applies a significance test against a baseline representation – whichever representation in the active set has the fewest free parameters, ties broken by lowest AIC. This is global whenever it is in the active set, since it always has the fewest parameters of all eight representations, but otherwise the simplest representation actually available (e.g. centrality/pseudo_halo if models excludes global). The p-value is the relative likelihood of the baseline vs. the best-fitting representation, $p = e^{-\|\Delta\|/2}$ where $\Delta = \mathrm{AIC}_{\mathrm{baseline}} - \mathrm{AIC}_{\mathrm{best}}$. A more complex representation is preferred only if $p <$ alpha; otherwise the baseline is retained as the more parsimonious default. Default is aic_sig_test=True.
alpha	Float: significance level for the AIC relative-likelihood test (used only when test='AIC' and aic_sig_test=True). Default is alpha=0.05.
min_effect	Float: minimum effect size, in logits, required to prefer a more complex representation over a simpler one, measured as the maximum absolute difference between their facet effect surfaces. For test='LR', an additional condition alongside the p-value for each pair; for AIC/BIC, applied as a post-hoc walk-back from the selected representation toward global along the five-model ladder (matrix, bivector, items/thresholds, global). 0.43 and 0.64 correspond to the ETS DIF category boundaries (moderate and large). Default min_effect=0 (no minimum).
sampling	None, 'dynamic', or Integer: subsamples non-extreme persons before computing log-likelihoods – parameters are always estimated on the full data; only the log-likelihood evaluation itself is subsampled. Not primarily about speed: at very large N, the LR/AIC/BIC comparison becomes biased toward the more complex facet element representation, since log-likelihood differences scale with N; capping the effective N mitigates this. 'dynamic' uses $T = \min(20R(I-1)(m-1), 1500)$, where I is the number of items, R the number of facet elements and m the maximum score ($m-1$ is taken as 1 for dichotomous data); an integer fixes T directly; None disables subsampling. Default is sampling='dynamic'.
warm_corr, tolerance, max_iters, ext_score_adjustment	Person location estimation keyword arguments, used only for representations not already estimated. See Section 6.3.5 for details of each.
constant, method, matrix_power, log_lik_tol	Calibration keyword arguments, used only for representations not already calibrated. See Section 6.3.1 for details of each.
seed	Integer or None: seed for the person-subsampling random number generator, only used when sampling triggers. None (default) draws fresh entropy. See Section 1.

Returns

Sets whichever of the following attributes correspond to `test`:

`self.model_comparison_mfrm_aic`, `_aic_baseline`, `_aic_p`, `_aic_preferred`, `_aic_summary`	{`representation: AIC value`} dictionary, the baseline representation tested against (see `aic_sig_test` above) as a string, the AIC relative-likelihood p-value, the preferred representation as a string, and a ranked comparison table (its Δ`AIC` column is relative to the baseline). Only set if `test='AIC'`.
`self.model_comparison_mfrm_bic`, `_bic_preferred`, `_bic_summary`	{`representation: BIC value`} dictionary, the preferred representation, and a ranked comparison table. Only set if `test='BIC'`.
`self.model_comparison_mfrm_lr`, `_lr_summary`	{`pair_label: (LR, df, p)`} dictionary of nested pairwise tests, and a summary table. Only set if `test='LR'`.

Examples

To compare all eight representations using the default AIC significance test:

```
self.model_selection()
self.model_comparison_mfrm_aic_summary
```

To instead run pairwise likelihood ratio tests, with a fixed subsampling size of 800 persons to guard against large-N bias:

```
self.model_selection(test='LR', sampling=800)
```

To restrict the comparison to a single nested pair – testing whether `bistretch`'s combined threshold-and-item stretch is needed over the unrestricted `bivector` representation, without calibrating the other six:

```
self.model_selection(test='LR', models=['bivector', 'bistretch'])
```

6.3.11 per_rater_model_selection

Description

Assigns the simplest adequate facet effect parameterisation to each facet element (e.g. rater) individually, producing a "mixed" model in which different facet elements may be represented under different facet element representations (any of the eight in Section 6.3.1) according to which is actually needed to capture that facet element's own behaviour. This differs from the global model_selection comparison (Section 6.3.10), which picks a single representation for every facet element at once: here, e.g. one facet element exhibiting extreme or complex behaviour does not force every other facet element into a potentially overparameterised representation.

The matrix model (the saturated model) is calibrated; every more restricted representation is then derived from it as marginal means of the matrix facet effects (centrality/pseudo_halo/bistretch as restricted forms of those marginals, via the same recipe as calibrate – see Section 6.3.1). By default (test='AIC'), each facet element's representation is then chosen directly as the minimum-criterion representation among the active set, guarded by a significance test against the simplest active baseline (aic_sig_test) and, if set, the min_effect practical-difference gate — exactly as for the global model_selection comparison (Section 6.3.10), just applied per facet element; test='BIC' works the same way, substituting BIC for AIC. Under test='LR', each facet element is instead tested top-down along a fixed ladder, in two stages.

Each candidate representation is scored by the joint log-likelihood at the matrix model's person estimates, subsampled according to sampling.

In the first stage, matrix v bivector is tested first; a facet element who needs the matrix representation is assigned it directly. A facet element for whom bivector suffices is then tested on two further, simultaneous forks – bivector v items and bivector v thresholds – and provisionally assigned global if both simplifications hold, items or thresholds if only one does, or bivector if neither does.

The second stage runs one extra, targeted stretch test depending on which of those five representations the first stage produced – a simplified design used in place of a full recursive walk over all eight representations' nesting relationships, still reaching every one of the eight possible outcomes:

- matrix winner: no extra test (centrality, pseudo_halo and bistretch all restrict representations matrix has already beaten).
- bivector winner: tested v bistretch.
- thresholds winner: tested v centrality.
- items winner: tested v pseudo_halo.
- global winner: tested v both centrality and pseudo_halo ($df = 1$ each). If exactly one beats global, that one is assigned. If both do, resolved via a further test of bistretch against global; if bistretch itself is not confirmed, falls back to whichever single axis had the stronger evidence. If neither beats global, the assignment stays global.

At every rung, a more complex representation is retained only if it clears both the likelihood-ratio significance test and, if set, the min_effect practical-difference gate.

An anchored variant (called using argument anchors) first calibrates the matrix model in anchor mode

(Section 6.3.2) before running the same ladder, since a different anchoring frame of reference can result in different model selections for facet elements, as their facet effect profiles change. Downstream methods that support `model='mixed'` (Sections 6.4.1, 6.3.3 and 6.4.4) read the assignment produced here.

Usage

```
self.per_rater_model_selection(anchors=None, test='AIC', models=None, aic_sig_test=True,
                               sampling='dynamic', alpha=0.05, min_effect=0, warm_corr=True,
                               tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                               constant=None, threshold_constant=None, method=None,
                               robust=False, matrix_power=None,
                               log_lik_tol=0.000001, seed=None)
```

Arguments

`anchors`	List or `None`: facet element identifiers used to define the anchoring frame of reference (see Section 6.3.2). Different anchor sets can give different, individually valid, per-facet-element assignments, since the zero-sum constraint means one non-uniform facet element can induce apparent non-uniformity in others. All facet elements, including anchors, are still tested. Default `anchors=None` (unanchored).
`test`	String, one of `'AIC'`, `'BIC'` or `'LR'`: model-selection criterion. Default is `test='AIC'`.
`models`	List/tuple of strings, or `None`: restricts which representations each facet element can be assigned (each one of the eight in Section 6.3.1, or their `threshold_stretch`/`item_stretch` aliases). For `test='AIC'`/`'BIC'`, a direct filter – the minimum is taken only over the given representations (any non-empty subset works; see `aic_sig_test` below for how the significance test adapts when `global` is excluded). For `test='LR'`, {`global, items, thresholds, bivector`} must all be included ({`global, items`} for dichotomous data); raises `ValueError` if the LR backbone is not fully included. `None` (default) uses all eight representations. Representations that the design makes the same model as another are left out as for `model_selection()`. For dichotomous data, only `global`, `items` and `pseudo_halo` remain, and the LR ladder tests `global` against `items`, then `items` against `pseudo_halo` (or `global` against `pseudo_halo` when `items` is not selected).
`aic_sig_test`	Boolean: when `test='AIC'`, imposes a significance test rewuiring the more complex model to be more likely with $p > 0.95$, based on the relative-likelihood p-value $p = e^{-\Delta\mathrm{AIC}/2} <$ `alpha`. Adds `p_vs_baseline` and `baseline_model` columns to `per_rater_model_selection_table`. Default is `aic_sig_test=True`.

Continued on next page...

`sampling`	String, integer, or `None`: per-facet-element subsampling used to calculate log-likelihood, to avoid bias towards more complex models. `'dynamic'` uses $T = \min(20(I-1)(m-1), 1500)$ ($T = \min(20(I-1), 1500)$ for dichotomous data); an integer fixes T directly; `None` uses every non-extreme person the facet element has scored. Default is `sampling='dynamic'`.
`seed`	Integer or `None`: seed for the per-facet-element subsampling random number generator (only used when `sampling` triggers). `None` (default) draws fresh entropy each call, matching every other stochastic method in *RaschPy* – see Section 1.
`alpha`	Float: significance level – for `test='LR'`, the threshold for accepting a restriction at a ladder rung; for `test='AIC'` with `aic_sig_test=True`, the threshold for the relative-likelihood p-value. Default is `alpha=0.05`.
`min_effect`	Float: minimum effect size, in logits, required to retain a more complex representation at each ladder rung, measured as the maximum absolute difference between the more complex and simpler representation's facet effect surfaces at that rung. Applied as a post-hoc walk-back from the selected representation toward `global` for AIC/BIC, or as an additional condition alongside the p-value for LR. 0.43 and 0.64 correspond to the ETS DIF category boundaries (moderate and large). Default `min_effect=0` (no minimum).
`warm_corr`, `tolerance`, `max_iters`, `ext_score_adjustment`	Person-estimation arguments for the matrix model – see Section 6.3.6.
`constant`, `method`, `matrix_power`, `log_lik_tol`	Calibration arguments for the matrix model – see Section 6.3.1.

Returns

If `anchors=None`:

`self.rater_models`	pandas series: the selected representation (any of the eight in Section 6.3.1, or whichever subset `models` restricted the assignment to) for each facet element.
`self.facet_effects_mixed`	pandas dataframe: each facet element's facet effects under its own assigned representation, in the common (item, threshold) matrix layout.
`self.per_rater_model_selection_table`	pandas dataframe: full per-facet-element, per-rung test statistics.
`self.per_rater_model_selection_counts`	pandas series: count of facet elements assigned to each representation.

If `anchors` is provided, the equivalent `self.anchor_rater_models`, `self.anchor_facet_effects_mixed`, `self.anchor_per_rater_model_selection_table` and `self.anchor_per_rater_model_selection_counts` are set instead. In both cases:

`self.per_rater_model_selection_runs`	Dictionary: every call's results above, keyed by `tuple(anchors)` (or `None` for an unanchored call), so results from an earlier call survive a later call with a different anchor set. e.g. `self.per_rater_model_selection_runs[tuple(my_anchors)].per_rater_model_selection_table`.

Examples

To assign each facet element the simplest adequate representation using the default AIC criterion:

```
self.per_rater_model_selection()
```

To use a likelihood-ratio ladder with a stricter significance level and a minimum practical effect size of 0.3 logits, anchored to a set of facet elements stored in `my_anchor_raters`:

```
self.per_rater_model_selection(anchors=my_anchor_raters, test='LR', alpha=0.01,
                               min_effect=0.3)
```

To restrict every facet element's assignment to the original five representations – excluding `matrix` and all three stretch representations entirely, so the ladder never considers them:

```
self.per_rater_model_selection(test='LR',
                               models=['global', 'items', 'thresholds', 'bivector'])
```

6.4 Statistical output

6.4.1 item_stats_df

Description

Produces a table of item statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.item_stats_df_global(anchors=None, full=False, ext_scores=True, zstd=False,
                          point_measure_corr=False, dp=3, se=True, warm_corr=True,
                          tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                          method=None, constant=None, threshold_constant=None,
                          robust=False, matrix_power=None,
                          log_lik_tol=0.000001, no_of_samples=500,
                          interval=None, seed=None,
                          filename=None, file_format='csv')

self.item_stats_df_items(anchors=None, full=False, ext_scores=True, zstd=False,
                         point_measure_corr=False, dp=3, se=True, warm_corr=True,
                         tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                         method=None, constant=None, threshold_constant=None,
                         robust=False, matrix_power=None,
                         log_lik_tol=0.000001, no_of_samples=500,
                         interval=None, seed=None,
                         filename=None, file_format='csv')

self.item_stats_df_thresholds(anchors=None, full=False, ext_scores=True, zstd=False,
                              point_measure_corr=False, dp=3, se=True, warm_corr=True,
                              tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                              method=None, constant=None, threshold_constant=None,
                              robust=False, matrix_power=None,
                              log_lik_tol=0.000001, no_of_samples=500,
                              interval=None, seed=None,
                              filename=None, file_format='csv')

self.item_stats_df_matrix(anchors=None, full=False, ext_scores=True, zstd=False,
                          point_measure_corr=False, dp=3, se=True, warm_corr=True,
                          tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                          method=None, constant=None, threshold_constant=None,
                          robust=False, matrix_power=None,
                          log_lik_tol=0.000001, no_of_samples=500,
                          interval=None, seed=None,
                          filename=None, file_format='csv')
```

```
self.item_stats_df_bivector(anchors=None, full=False, ext_scores=True, zstd=False,
                            point_measure_corr=False, dp=3, se=True, warm_corr=True,
                            tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                            method=None, constant=None, threshold_constant=None,
                            robust=False, matrix_power=None,
                            log_lik_tol=0.000001, no_of_samples=500,
                            interval=None, seed=None,
                            filename=None, file_format='csv')

self.item_stats_df_centrality(anchors=None, full=False, ext_scores=True, zstd=False,
                              point_measure_corr=False, dp=3, se=True, warm_corr=True,
                              tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                              method=None, constant=None, threshold_constant=None,
                              robust=False, matrix_power=None,
                              log_lik_tol=0.000001, no_of_samples=500,
                              interval=None, seed=None,
                              filename=None, file_format='csv')

self.item_stats_df_pseudo_halo(anchors=None, full=False, ext_scores=True, zstd=False,
                                 point_measure_corr=False, dp=3, se=True, warm_corr=True,
                                 tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                                 method=None, constant=None, threshold_constant=None,
                                 robust=False, matrix_power=None,
                                 log_lik_tol=0.000001, no_of_samples=500,
                                 interval=None, seed=None,
                                 filename=None, file_format='csv')

self.item_stats_df_bistretch(anchors=None, full=False, ext_scores=True, zstd=False,
                             point_measure_corr=False, dp=3, se=True, warm_corr=True,
                             tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                             method=None, constant=None, threshold_constant=None,
                             robust=False, matrix_power=None,
                             log_lik_tol=0.000001, no_of_samples=500,
                             interval=None, seed=None,
                             filename=None, file_format='csv')
```

Arguments

`anchors`	List: If `None`, unanchored central item location estimates and standard errors will be used. If a list of facet element names is passed, anchored estimates and standard errors, anchored to the facet elements passed, will be used. Other statistics are unaffected by `anchors`. Default is `anchors=None`.

Continued on next page...

full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a summary table with the most commonly reported statistics.
zstd	Boolean: If True, infit and outfit standardised z-scores are reported alongside mean square statistics. Default is zstd=False. Automatically True if full=True.
point_measure_corr	Boolean: If True, point-polyserial correlation (Kornbrot, 2014) between person location and item score is reported, together with the expected value of the point-measure polyserial correlation for an ideal item. Default is point_measure_corr=False. Automatically True if full=True.
dp	Integer: Number of decimal places reported in output.
se	Boolean: If True, computes and reports the SE column (and CI bound columns, if interval is set). If False, skips the bootstrap entirely – useful when only Infit MS/Outfit MS are needed (e.g. repeated simulation runs), since those do not depend on the bootstrap and are unaffected by this flag. Forces interval=None when False. Default is se=True.
warm_corr	Boolean: If True, Warm's (1989) bias correction applied during generation of person estimates (see Section 6.3.5). Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.

Continued on next page...

`method`	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 6.3.1). Options are `'ls'` for least squares (Choppin, 1968, 1985), `'evm'` for the eigenvector method(Garner & Engelhard, 2002), `'cos'` for cosine maximisation (Kou & Lin, 2014) or `'log-lik'` for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). `None` (default) resolves per representation (see Section 6.3.1).
`constant`	Float or `None`: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation. `None` (default) resolves to the simulation-tuned per-stage defaults (see Section 6.3.1). Calibration uses the estimation method of the latest calibration (`'cp-full-rcl'` if none).
`threshold_constant`	Float or `None`: smoothing constant for the threshold-location (CPAT) stage alone; `None` (default) follows `constant`. See Section 6.3.1.
`robust`	Boolean, default `False`: selects the more conservative per-representation calibration for rough data. See Section 6.3.1.
`matrix_power`	Integer or `None`: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of item location estimates. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is `matrix_power=None`, which resolves to `0` (no matrix powering) when `method='log-lik'` and to `5` for every other method. Passing `matrix_power=0` with a method other than `'log-lik'` issues a `UserWarning`.
`log_lik_tol`	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 6.3.1). Ignored for other methods.
`no_of_samples`	Integer: Number of bootstrap samples for standard error estimates (see Section 6.3.3). More samples lead to more accurate standard error estimates, but take longer to compute. Default `no_of_samples=500`.
`interval`	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 6.3.3). Defines a central interval of proportion p, e.g. `interval=0.95` defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is `interval=None`.
`seed`	Integer or `None`: seed passed through to the internal `std_errors`/`fit_statistics` calls (and to the lazily-triggered `per_rater_model_selection` call for `model='mixed'`) if not already computed. `None` (default) draws fresh entropy each call. See Section 1.

Continued on next page...

filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as one of self.item_stats_global, self.item_stats_items, self.item_stats_thresholds, self.item_stats_matrix, self.item_stats_bivector, self.item_stats_centrality, self.item_stats_pseudo_halo or self.item_stats_bistretch, a pandas dataframe with one row for each item and the following columns:

Estimate	Central item location estimate.
SE	Bootstrapped SE of central item estimate. Only produced if se=True.
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 2.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 5%. Only produced if se=True and interval is set.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit interval value as for lower bound above.
Count	Count of responses.
Facility	Item facility: mean score as a proportion of the maximum possible score.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if zstd=True.
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if zstd=True.
PM corr	Point-polyserial correlation. Only produced if point_measure_corr=True.
Exp PM corr	Expected value of point-measure polyserial correlation. Only produced if point_measure_corr=True.
Disordered	If True, threshold estimates indicate the presence of disordered thresholds within the item (Andrich, 2010).

Examples

To produce summary unanchored self.item_stats_global table with most commonly reported statistics:

```
self.item_stats_df_global()
```

To produce a full self.item_stats_items table with all statistics, anchored to facet elements Rater_1 and Rater_2:

```
self.item_stats_df_items(anchors=['Rater_1', 'Rater_2'], full=True)
```

To produce an unanchored `self.item_stats_matrix` table with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
self.item_stats_df_matrix(zstd=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

6.4.2 threshold_stats_df

Description

Produces a table of threshold statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

Eight methods are defined, one for each facet element (e.g. rater) representation; all methods share the same argument format:

```
self.threshold_stats_df_global(anchors=None, full=False, zstd=False, disc=False,
                               point_measure_corr=False, dp=3, se=True,
                               warm_corr=True, tolerance=0.00001, max_iters=100,
                               ext_score_adjustment=0.5, method=None, constant=None,
                               threshold_constant=None, robust=False,
                               matrix_power=None, log_lik_tol=0.000001, no_of_samples=500,
                               interval=None, seed=None,
                               filename=None, file_format='csv')

self.threshold_stats_df_items(anchors=None, full=False, zstd=False, disc=False,
                              point_measure_corr=False, dp=3, se=True,
                              warm_corr=True, tolerance=0.00001, max_iters=100,
                              ext_score_adjustment=0.5, method=None, constant=None,
                              threshold_constant=None, robust=False,
                              matrix_power=None, log_lik_tol=0.000001, no_of_samples=500,
                              interval=None, seed=None,
                              filename=None, file_format='csv')

self.threshold_stats_df_thresholds(anchors=None, full=False, zstd=False, disc=False,
                                   point_measure_corr=False, dp=3, se=True,
                                   warm_corr=True, tolerance=0.00001, max_iters=100,
                                   ext_score_adjustment=0.5, method=None, constant=None,
                                   threshold_constant=None, robust=False,
                                   matrix_power=None, log_lik_tol=0.000001,
                                   no_of_samples=500, interval=None, seed=None,
                                   filename=None, file_format='csv')

self.threshold_stats_df_matrix(anchors=None, full=False, zstd=False, disc=False,
                               point_measure_corr=False, dp=3, se=True,
                               warm_corr=True, tolerance=0.00001, max_iters=100,
                               ext_score_adjustment=0.5, method=None, constant=None,
                               threshold_constant=None, robust=False,
                               matrix_power=None, log_lik_tol=0.000001, no_of_samples=500,
                               interval=None, seed=None,
                               filename=None, file_format='csv')
```

```
self.threshold_stats_df_bivector(anchors=None, full=False, zstd=False, disc=False,
                                 point_measure_corr=False, dp=3, se=True,
                                 warm_corr=True, tolerance=0.00001, max_iters=100,
                                 ext_score_adjustment=0.5, method=None, constant=None,
                                 threshold_constant=None, robust=False,
                                 matrix_power=None, log_lik_tol=0.000001, no_of_samples=500,
                                 interval=None, seed=None,
                                 filename=None, file_format='csv')

self.threshold_stats_df_centrality(anchors=None, full=False, zstd=False, disc=False,
                                   point_measure_corr=False, dp=3, se=True,
                                   warm_corr=True, tolerance=0.00001, max_iters=100,
                                   ext_score_adjustment=0.5, method=None, constant=None,
                                   threshold_constant=None, robust=False,
                                   matrix_power=None, log_lik_tol=0.000001, no_of_samples=500,
                                   interval=None, seed=None,
                                   filename=None, file_format='csv')

self.threshold_stats_df_pseudo_halo(anchors=None, full=False, zstd=False, disc=False,
                                    point_measure_corr=False, dp=3, se=True,
                                    warm_corr=True, tolerance=0.00001, max_iters=100,
                                    ext_score_adjustment=0.5, method=None, constant=None,
                                    threshold_constant=None, robust=False,
                                    matrix_power=None, log_lik_tol=0.000001, no_of_samples=500,
                                    interval=None, seed=None,
                                    filename=None, file_format='csv')

self.threshold_stats_df_bistretch(anchors=None, full=False, zstd=False, disc=False,
                                  point_measure_corr=False, dp=3, se=True,
                                  warm_corr=True, tolerance=0.00001, max_iters=100,
                                  ext_score_adjustment=0.5, method=None, constant=None,
                                  threshold_constant=None, robust=False,
                                  matrix_power=None, log_lik_tol=0.000001, no_of_samples=500,
                                  interval=None, seed=None,
                                  filename=None, file_format='csv')
```

Arguments

anchors	List: If None, unanchored central item location estimates and standard errors will be used. If a list of facet element names is passed, anchored estimates and standard errors, anchored to the facet elements passed, will be used. Other statistics are unaffected by anchors. Default is anchors=None.

Continued on next page...

full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a summary table with the most commonly reported statistics.
zstd	Boolean: If True, infit and outfit standardised z-scores are reported alongside mean square statistics. Default is zstd=False. Automatically True if full=True.
disc	Boolean: If True, item discrimination is reported. The discrimination of the empirical item slope relative to the ideal logistic ogive, with 1 perfect, greater than 1 showing overfit and less than 1 showing underfit; discrimination is similar to the 2PL IRT discrimination parameter (Linacre, 2023), but is a descriptive statistic in the SLM rather than an item parameter.
point_measure_corr	Boolean: If True, point-polyserial correlation (Kornbrot, 2014) between person location and item score is reported, together with the expected value of the point-measure polyserial correlation for an ideal item. Default is point_measure_corr=False. Automatically True if full=True.
dp	Integer: Number of decimal places reported in output.
se	Boolean: If True, computes and reports the SE column (and CI bound columns, if interval is set). If False, skips the bootstrap entirely – useful when only Infit MS/Outfit MS are needed (e.g. repeated simulation runs), since those do not depend on the bootstrap and are unaffected by this flag. Forces interval=None when False. Default is se=True.
warm_corr	Boolean: If True, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 6.3.5). Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.

Continued on next page...

method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 6.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). None (default) resolves per representation (see Section 6.3.1).
constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation. None (default) resolves to the simulation-tuned per-stage defaults (see Section 6.3.1). Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: smoothing constant for the threshold-location (CPAT) stage alone; None (default) follows constant. See Section 6.3.1.
robust	Boolean, default False: selects the more conservative per-representation calibration for rough data. See Section 6.3.1.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 6.3.3). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Unused if se=False. Default is no_of_samples=500.
interval	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 6.3.3). Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Ignored if se=False. Default is interval=None.
seed	Integer or None: seed passed through to the internal std_errors/fit_statistics calls (and to the lazily-triggered per_rater_model_selection call for model='mixed') if not already computed. None (default) draws fresh entropy each call. See Section 1.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.threshold_stats_global, self.threshold_stats_items, self.threshold_stats_thresholds, self.threshold_stats_matrix or self.threshold_stats_bivector, a pandas dataframe with one row for each threshold and the following columns:

Estimate	Rasch-Andrich threshold location estimate.
SE	Bootstrapped standard error of threshold estimate. Only produced if se=True.
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 2.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 5%. Only produced if se=True and interval is set.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 97.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 95%. Only produced if se=True and interval is set.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if zstd=True.
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if zstd=True.
Discrim	Item discrimination. Only produced if disc=True.
PM corr	Point-polyserial correlation. Only produced if point_measure_corr=True.
Exp PM corr	Expected value of point-measure polyserial correlation. Only produced if point_measure_corr=True.

Examples

To produce an unanchored summary self.threshold_stats table under the global representation with the most commonly reported statistics:

```
self.threshold_stats_df_global()
```

To produce a full self.threshold_stats table under the item representation with all statistics, anchored to facet elements Rater_1 and Rater_2:

```
self.threshold_stats_df(anchors=['Rater_1', 'Rater_2'], full=True)
```

To produce an unanchored self.threshold_stats table under the matrix representation with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
self.threshold_stats_df_matrix(zstd=True)
```

Other arguments may be used to alter parameters of central item location, threshold and person location estimation.

6.4.3 category_stats_df

Description

Produces a table of category *width* statistics – threshold_stats_df (Section 6.4.2) reports raw threshold locations; this reports the difference between consecutive thresholds for each item, $\tau_{k+1} - \tau_k$. The shared, zero-summed threshold vector has only max_score−1 true degrees of freedom, so the per-threshold standard errors in threshold_stats_df are correlated rather than independent, and understate the true amount of error. No infit/outfit or other fit statistics are reported, since these apply to thresholds rather than categories. Auto-triggers calibrate (Section 6.3.1) and std_errors (Section 6.3.3) directly if not already run.

A category width can be negative – a negative width at category k means thresholds τ_k and τ_{k+1} are disordered for that item (category k is never the most likely response at any person location). Prop disordered makes this a continuous diagnostic rather than a straightforward yes/no: the proportion of bootstrap resamples in which that item's category width was negative, reasonably read as the probability that the true category is disordered.

Usage

Eight methods are defined, one for each facet element (e.g. rater) representation; all methods share the same argument format:

```
self.category_stats_df_global(anchors=None, dp=3, warm_corr=True, tolerance=0.00001,
                              max_iters=100, ext_score_adjustment=0.5, method=None,
                              constant=None, threshold_constant=None, robust=False,
                              matrix_power=None, log_lik_tol=0.000001,
                              no_of_samples=500, interval=None, seed=None,
                              filename=None, file_format='csv')
```

and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch, or self.category_stats_df(model='global', ...) etc.

Arguments

anchors	List: If None, unanchored thresholds and standard errors are used. If a list of facet element names is passed, anchored thresholds and standard errors, anchored to the facet elements passed, are used. Default is anchors=None.
dp	Integer: Number of decimal places reported in output. Default is dp=3.
warm_corr, tolerance, max_iters, ext_score_adjustment	Person location estimation keyword arguments, used only if not already run. See Section 6.3.5 for details of each.
method, constant, matrix_power, log_lik_tol	Calibration keyword arguments, used only if calibrate (Section 6.3.1) has not already been run. Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none). See Section 6.3.1 for details of each.

Continued on next page...

no_of_samples, interval, seed	Bootstrap standard error keyword arguments, used only if std_errors (Section 6.3.3) has not already been run. See Section 6.3.3 for details of each.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute `self.category_stats_global`, `self.category_stats_items`, `self.category_stats_thresholds`, `self.category_stats_matrix`, `self.category_stats_bivector`, `self.category_stats_centrality`, `self.category_stats_pseudo_halo` or `self.category_stats_bistretch` (as appropriate), a pandas dataframe with one row per category, indexed `Category 1` to `Category max_score`−1 (one fewer row than `threshold_stats`, since a width is defined between each pair of consecutive thresholds), with the following columns:

`Estimate`	Category width (can be negative).
`SE`	Bootstrapped standard error of the width, differenced *within* each bootstrap resample before taking the standard deviation, so it correctly reflects the covariance between adjacent thresholds rather than naively combining their individual standard errors.
`2.5%` / `97.5%`	Bounds of the empirical bootstrap estimate interval, if `interval` is available (see `threshold_stats_df`, Section 6.4.2, for the column-naming convention).
`Disordered`	Boolean: `True` if `Estimate` is negative.
`Prop disordered`	Proportion of bootstrap resamples in which this category's width was negative – reasonably read as the probability that the true category is disordered. Useful for interpreting `Disordered` in both directions: a `True` that is only weakly supported (`Prop disordered` close to 0.5) versus robust, or a `False` that is nonetheless uncertain versus clear-cut.

Example

To produce the `self.category_stats_global` table, alongside the threshold-level table, under the global representation:

```
self.threshold_stats_df_global()
self.category_stats_df_global()
```

To check which categories, if any, are disordered under the matrix representation:

```
self.category_stats_df_matrix()
self.category_stats_matrix[self.category_stats_matrix['Disordered']]
```

6.4.4 rater_stats_df

Description

Produces a table of facet element (e.g. rater) statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.rater_stats_df_global(anchors=None, full=False, zstd=False, dp=3, warm_corr=True,
                           tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                           method=None, constant=None, threshold_constant=None,
                           robust=False, matrix_power=None, log_lik_tol=0.000001,
                           no_of_samples=500, interval=None, seed=None,
                           filename=None, file_format='csv')
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently self.rater_stats_df(model='global', ...) with model set to any of the eight models.

A ninth representation, model='mixed', is also available on the base self.rater_stats_df(model='mixed', ...) call (no rater_stats_df_mixed() alias), reporting each facet element's statistics under whichever representation per_rater_model_selection (Section 6.3.11) assigned it, lazily run with its own default selection criteria if it has not already been run (or was run with a different anchor set).

For centrality, pseudo_halo and bistretch, a single call stores two tables. The primary self.rater_stats_{model} reports the model's own raw shift/stretch parameters (lambda, plus omega – or, for bistretch, omega_items and omega_thresholds) with bootstrap SE and, if interval is set, CI. A table based on the reconstructed vectors from the representations (per item for pseudo_halo, per threshold for centrality, both for bistretch) is stored separately as self.rater_stats_{model}_full_vector.

Passing alias=True relabels the primary table's parameter columns with a human-readable name instead, purely cosmetically – the values themselves are identical either way:

Table 255: rater_stats_df(alias=True) column labels

Model	Raw parameter	alias=True label
centrality	lambda	Global
centrality	omega	Threshold stretch
pseudo_halo	lambda	Global
pseudo_halo	omega	Item stretch
bistretch	lambda	Global
bistretch	omega_items	Item stretch
bistretch	omega_thresholds	Threshold stretch

Arguments

anchors	List: If None, unanchored central item location estimates and standard errors will be used. If a list of facet element names is passed, anchored estimates and standard errors, anchored to the facet elements passed, will be used. Other statistics are unaffected by anchors. Default is anchors=None.
full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a summary table with the most commonly reported statistics.
zstd	Boolean: If True, infit and outfit standardised z-scores are reported alongside mean square statistics. Default is zstd=False. Automatically True if full=True.
marginal	matrix representation only. Boolean: If True, marginal mean facet effect vectors by item and threshold are reported, rather than the full matrix representation. Default is marginal=True.
alias	centrality/pseudo_halo/bistretch only. Boolean: If True, labels the primary table's parameter columns with a human-readable alias instead of the raw parameter name – 'Global' for lambda, 'Item stretch'/'Threshold stretch' for omega/omega_items/omega_thresholds depending on which axis that model's stretch parameter acts on (see Table 255). Purely cosmetic – the values themselves are unchanged, and self.rater_stats_{model}_full_vector is unaffected (it has no lambda/omega columns to alias). Ignored (no effect) for every other model. Default is alias=False.
divergence_test	centrality/pseudo_halo/bistretch only. None, 'wald' or 't': if set, tests for each item (pseudo_halo), threshold (centrality) or both (bistretch) whether the stretch model's value differs from the free model's estimate, with standard errors from a paired bootstrap. 'wald' uses the standard normal distribution; 't' uses a t-distribution with no_of_samples−1 degrees of freedom. Adds 'Flagged items' and/or 'Flagged categories' columns to the table; with full=True, also one Boolean column per item and/or threshold, and a 'Flagged' row in rater_stats_{model}_full_vector. Default is divergence_test=None.
correction	Only relevant when divergence_test is set. None or 'bh': if 'bh', Benjamini-Hochberg FDR correction is applied within each facet element's own family of tests (that facet element's items, or that facet element's thresholds – not pooled across facet elements). Default correction=None compares raw p-values against alpha directly. No-op when divergence_test=None.

Continued on next page...

alpha	Only relevant when divergence_test is set. Float: significance threshold for flagging, applied after correction (if any). Default is alpha=0.05.
plot	Only relevant when divergence_test is set. Boolean: if True, also builds and stores self.fit_plot_{model} (e.g. self.fit_plot_pseudo_halo) – a scatter of the free vector model's estimates against the stretch model's reconstruction, flagged elements in a different colour, in the same style as self.anchor_plot elsewhere in this package's DIF/anchor-selection machinery. Like every other statistic in this family, it also picks up the usual model-alias spelling (e.g. self.fit_plot_item_stretch) automatically – it is a stored attribute, not a separate callable method. Default is plot=False.
dp	Integer: Number of decimal places reported in output.
warm_corr	Boolean: If True, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 6.3.5). Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during item estimation (see section 6.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). None (default) resolves per representation (see Section 6.3.1).

Continued on next page...

constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for item estimation. None (default) resolves to the simulation-tuned per-stage defaults (see Section 6.3.1). Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: smoothing constant for the threshold-location (CPAT) stage alone; None (default) follows constant. See Section 6.3.1.
robust	Boolean, default False: selects the more conservative per-representation calibration for rough data. See Section 6.3.1.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 6.3.3). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is no_of_samples=500.
interval	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 6.3.3). Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is interval=None.
seed	Integer or None: seed passed through to the internal std_errors/fit_statistics calls (and to the lazily-triggered per_rater_model_selection call for model='mixed'), only used if not already computed. None (default) draws fresh entropy each call. See Section 1.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as one of self.rater_stats_global, self.rater_stats_items, self.rater_stats_thresholds, self.rater_stats_matrix, self.rater_stats_bivector, self.rater_stats_centrality, self.rater_stats_pseudo_halo, self.rater_stats_bistretch or self.rater_stats_mixed, a pandas dataframe with one row for each facet element and the following columns. For centrality/pseudo_halo/bistretch, the same call also stores self.rater_stats_{model}_full_vector (see above), with the same column structure applied per item/threshold instead of per parameter:

Estimate	Facet effect estimate (structure depends on model: a scalar for global; a vector by item, threshold, or item×threshold cell for items/thresholds/matrix with marginal=False (also mixed, which is always in this item×threshold cell form – see 'The 'mixed' facet element representation' below); a vector by parameter – lambda, plus omega (or, for bistretch, omega_items and omega_thresholds) – for centrality/pseudo_halo/bistretch).
SE	Bootstrapped standard error of the facet effect estimate.
2.5%	Lower bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 2.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 5%.
97.5%	Upper bound of empirical bootstrap estimate interval. Column name will change to fit interval value: 97.5% represents the lower bound for interval=0.95 but, for example, interval=0.9 will produce a column labelled 95%.
Count	Count of responses made by the facet element.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score. Only produced if zstd=True.
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score. Only produced if zstd=True.

Examples

To produce a summary unanchored self.rater_stats_global table with the most commonly reported statistics:

```
self.rater_stats_df_global()
```

To produce a full self.rater_stats_items table with all statistics, anchored to facet elements Rater_1 and Rater_2:

```
self.rater_stats_df_items(anchors=['Rater_1', 'Rater_2'], full=True)
```

To produce an unanchored self.item_stats_matrix table with the most commonly reported statistics, plus standardised z-scores for infit and outfit:

```
self.item_stats_df_matrix(zstd=True)
```

To produce a full self.rater_stats_pseudo_halo table (lambda/omega with SEs and 95% CIs) and, from the same call, the reconstructed full (facet element × item) location vector as self.rater_stats_pseudo_halo_full_vector:

```
self.rater_stats_df_pseudo_halo(full=True)
```

To produce that same table with human-readable column labels (`'Global'`/`'Item stretch'` instead of `lambda`/`omega` – see Table 255) instead:

```
self.rater_stats_df_pseudo_halo(full=True, alias=True)
```

To run the item-by-item divergence test (Wald, with Benjamini-Hochberg correction), then inspect the per-facet-element flagged-item count and, for one specific item, which facet elements were flagged:

```
self.rater_stats_df_pseudo_halo(full=True, divergence_test='wald',
                                    correction='bh')
print(self.rater_stats_pseudo_halo['Flagged items'])
print(self.rater_stats_pseudo_halo['Item_1'])
print(self.rater_stats_pseudo_halo_full_vector['Item_1'])
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

The 'mixed' facet element representation

`rater_stats_df(model='mixed')` reports facet element statistics under the per-facet-element representation assignment from `per_rater_model_selection` (Section 6.3.11), which is run with default settings if it has not already been run. The table adds a `Model` column giving each facet element's representation. The table always gives statistics for each (item, threshold) combination; `marginal=True` raises a `UserWarning` and returns the full table. There is no `rater_stats_df_mixed()` alias.

6.4.5 check_anchor_homogeneity

Description

Tests whether the facet elements (e.g. raters) selected as an anchor set (Section 6.3.2) are sufficiently homogenous in their behaviours, before calibrate_anchor recentres their group mean to zero. On its own, calibrate_anchor simply subtracts the mean of facet effect element across the anchor set, thereby setting the mean of the anchors for each element to zero; it does not test whether that mean is a stable, shared reference point, or an artefact of averaging over facet elements who do not actually behave alike.

If one or more anchor facet elements have a genuinely different facet location from the rest, the anchored zero becomes unstable and the whole anchored scale inherits that instability. Separate from calibrate_anchor, since the default 'cochran' test requires calculation of additional bootstrap standard errors. Unrelated to per_rater_model_selection (Section 6.3.11) – this test asks whether the selected anchor facet elements agree with each other. Free (unanchored) facet effects are used, so the test is not circularly biased toward finding homogeneity by construction.

Homogeneity is tested by one of two methods:

1. Cochran's (1954) Q heterogeneity test (the standard meta-analysis approach), $Q = \sum_r w_r(\lambda_r - \bar{\lambda})^2$ with weights $w_r = 1/SE_r^2$ from bootstrap standard errors, chi-squared with $df = n_{\text{anchors}} - 1$, evaluated on the supplied anchor set. Which specific facet element or facet elements are responsible are then identified by sequential exclusion: successively drop the facet element with the largest $|z|$ compared to the current remaining group's weighted mean and retest until Q is no longer significant – this isolates true outliers, while a single pass against the full group can cause an outlier to impute artificial significance in the rest. 'robust'
2. Iglewicz & Hoaglin modified (robust) z-score (Iglewicz & Hoaglin, 1993), testing each facet element's component against the anchor group's own median and MAD.

Usage

```
self.check_anchor_homogeneity(model, anchors, rater_homogeneity_test='cochran',
                              per_component='auto', alpha=0.05, correction='bh',
                              robust_z_threshold=3.5, no_of_samples=500, constant=None, threshold_
constant=None, robust=False,
                              method=None, matrix_power=None, log_lik_tol=0.000001,
                              seed=None)
```

Arguments

model	String: one of 'global', 'items', 'thresholds', 'matrix', 'bivector', 'centrality', 'pseudo_halo' or 'bistretch'.
anchors	List: anchor facet element to test. At least two anchor facet elements required to test homogeneity.

Continued on next page...

rater_homogeneity_test	String, 'cochran' for Cochran's (1954) Q heterogeneity test, or 'robust' for robust z. Default is rater_homogeneity_test='cochran'.
per_component	'auto', True or False: if True, as well as testing each anchor's mean facet effect, tests each item, threshold or (item, threshold) cell separately. 'auto' resolves to True for every representation except global. Default is per_component='auto'.
alpha	Float: significance level for every Cochran's Q test run here – the initial omnibus test, each step of the sequential-exclusion procedure, and the per-component tests (rater_homogeneity_test='cochran' only). Default is alpha=0.05.
correction	String or None, one of 'bh', 'bonferroni' or None: multiple-comparison correction across the per-component tests only (Benjamini-Hochberg false discovery rate control (Benjamini & Hochberg, 1995) or Bonferroni) – these are genuinely independent tests, one per component, so correction is appropriate. Not applied to per-facet-element flagging, since covered by sequential exclusion via repeated omnibus tests rather than a family of independent per-facet-element p-values. rater_homogeneity_test='cochran' only. Default is correction='bh'.
robust_z_threshold	Float: flagging threshold for rater_homogeneity_test='robust' (Iglewicz & Hoaglin's own recommended default). Default is robust_z_threshold=3.5.
no_of_samples	Integer: bootstrap samples, only used if unanchored standard errors for this representation are not already stored (rater_homogeneity_test='cochran' only). Default is no_of_samples=500.
constant, method, matrix_power, log_lik_tol	Calibration/bootstrap arguments, used only if calibration or standard errors for this representation are not already computed – see Sections 6.3.1 and 6.3.3.
seed	Integer or None: seed passed through to the internal std_errors call (only used if unanchored standard errors are not already computed, rater_homogeneity_test='cochran' only). None (default) draws fresh entropy each call. See Section 1.

Returns

`self.anchor_homogeneity_test`	pandas series or `None`: `Q`, `df`, `p`, `Flagged` and `N_dropped_for_homogeneity` for the scalar (whole-profile) omnibus test, evaluated on the anchor set as supplied – `N_dropped_for_homogeneity` is how many facet elements the sequential-exclusion procedure needed to remove before Q stopped being significant. `None` if `rater_homogeneity_test='robust'`, since no omnibus statistic exists for robust z.
`self.anchor_homogeneity_per_rater`	pandas dataframe: one row per anchor facet element. For `rater_homogeneity_test='cochran'`: `Facet effect`, `SE`, `z`, `p`, `Flagged` (whether sequential exclusion removed this facet element – `z`/`p` are relative to whichever group they were tested against: their own removal step if dropped, or the final retained group if not). For `rater_homogeneity_test='robust'`: `Facet effect`, `Robust z`, `Flagged`.
`self.anchor_homogeneity_per_component`	pandas dataframe or `None`: one row per component (item name, threshold number, or item-threshold pair) with that component's own omnibus result: `Q`, `df`, `p`, `p_corrected`, `Flagged` (`'cochran'`), or `Flagged` only (`'robust'`, meaning at least one facet element exceeded the threshold on that component). `None` if `per_component` resolves to `False`.

Examples

To test whether a proposed anchor set stored in `my_anchor_raters` agrees with itself under the matrix representation, using the default Cochran's Q approach:

```
self.check_anchor_homogeneity('matrix', my_anchor_raters)
```

To use the purely descriptive robust z-score approach instead, without per-component breakdown:

```
self.check_anchor_homogeneity('matrix', my_anchor_raters,
                              rater_homogeneity_test='robust', per_component=False)
```

6.4.6 person_stats_df

Description

Produces a table of person statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX.

Usage

Eight methods are defined, one for each facet element (e.g. rater) representation; all methods share the same argument format:

```
self.person_stats_df_global(anchors=None, full=False, rsem=False, zstd=False, dp=3,
                            warm_corr=True, tolerance=0.00001, max_iters=100,
                            ext_score_adjustment=0.5, method=None, constant=None, threshold_
constant=None, robust=False,
                            matrix_power=None, log_lik_tol=0.000001, interval=None,
                            no_of_samples=500,
                            filename=None, file_format='csv')
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently self.person_stats_df(model='global', ...) with model set to any of the eight representation names.

Arguments

anchors	List: If None, unanchored central item location estimates will be used to generate person location estimates. If a list of facet element names is passed, anchored estimates and standard errors, anchored to the facet elements passed, will be used. Other statistics are unaffected by anchors. Default is anchors=None.
full	Boolean: If True, a full table with all available statistics is produced. Default is full=False for a summary table with the most commonly reported statistics.
rsem	Boolean: If True, realistic standard error of measurement (RSEM), which takes into account item fit (Wright, 1996), is reported alongside the conditional standard error of measurement (CSEM). Default is rsem=False. Automatically True if full=True.
zstd	Boolean: If True, infit and outfit standardised z-scores are reported alongside mean square statistics. Default is zstd=False.
dp	Integer: Number of decimal places reported in output.

Continued on next page...

warm_corr	Boolean: If True, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 6.3.5). Default is warm_corr=True.
tolerance	Float: convergence criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 6.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). None (default) resolves per representation (see Section 6.3.1).
constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation. None (default) resolves to the simulation-tuned per-stage defaults (see Section 6.3.1). Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: smoothing constant for the threshold-location (CPAT) stage alone; None (default) follows constant. See Section 6.3.1.
robust	Boolean, default False: selects the more conservative per-representation calibration for rough data. See Section 6.3.1.
matrix_power	Integer or None: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for central item location estimation. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.

Continued on next page...

log_lik_tol	Float: convergence stopping criterion for log-likelihood method for central item location estimation. Ignored for other methods.
interval	Float. Empirical interval to define quantiles of estimates from bootstrap samples, as an alternative to a confidence interval (see Section 6.3.3). Defines a central interval of proportion p, e.g. interval=0.95 defines quantiles at 2.5% and 97.5%. More stable with larger numbers of bootstrap samples. Default is interval=None.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 6.3.3). More samples mean more accurate standard error estimates, but take longer to compute. Default is no_of_samples=500.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.person_stats_global, self.person_stats_items, self.person_stats_thresholds, self.person_stats_matrix, self.person_stats_bivector, self.person_stats_centrality, self.person_stats_pseudo_halo or self.person_stats_bistretch, a pandas dataframe with one row for each person and the following columns:

Estimate	Person location estimate.
CSEM	Conditional standard error of measurement for person location estimate.
RSEM	Realistic standard error of measurement for person estimate if rsem=True
Score	Total score summed across all items and facet elements.
Max score	Maximum available score (items attempted × facet elements × max_score).
p	Mean score as a proportion of the maximum possible score.
Infit MS	Infit mean square.
Infit Z	Standardised infit z-score.
Outfit MS	Outfit mean square.
Outfit Z	Standardised outfit z-score.

Examples

To produce an unanchored summary self.person_stats table under the global facet element representation with the most commonly reported statistics:

```
self.person_stats_df_global()
```

To produce a full `self.person_stats` table under the matrix facet element representation with all statistics, anchored to facet elements `Rater_1` and `Rater_2`:

```
self.person_stats_df(anchors=['Rater_1', 'Rater_2'], full=True)
```

Other arguments may be used to alter parameters of item location and/or person location estimation.

6.4.7 test_stats_df

Description

Produces a table of test-level statistics in the form of a pandas dataframe, which may be saved to formats such as csv, xlsx or LaTeX:

`Separation ratio` (Wright, 1996; Wright & Masters, 1982) is the SD of person locations reported as a ratio of SE units. For persons, $G_p = \sigma_p / \sqrt{\sum_n SE_n^2}$, where σ_p is the variance of the person estimates and SE_n is the RSEM (see Section 6.4.6) for person n. The formula is symmetrical for items, substituting the SE of estimation for RSEM.

`Strata` is the number of statistically distinct levels of either person or item locations as strata with centers three measurement errors apart (Wright & Masters, 1982:106). For persons, $H_p = (4G_p + 1)/3$, with symmetrical results for items.

`Reliability` is Rasch-specific reliability statistic (Wright, 1996), derived from PSI. reliability is similar to Cronbach's Alpha (Cronbach, 1951), and may be interpreted the same way: the proportion of variance of the estimates which stems from variation in location or location rather than estimation error. For persons, $R_p = G_p^2/(1 + G_p^2)$, with symmetrical results for items.

Usage

Eight methods are defined, one for each facet element (e.g. rater) representation; all methods share the same argument format:

```
self.test_stats_df_global(dp=3, warm_corr=True, tolerance=0.00001, max_iters=100,
                          ext_score_adjustment=0.5, method=None, constant=None,
                          threshold_constant=None, robust=False,
                          matrix_power=None, log_lik_tol=0.000001, no_of_samples=500,
                          seed=None, filename=None, file_format='csv')
```

and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`, or `self.test_stats_df(model='global', ...)` etc.

Arguments

`dp`	Integer: Number of decimal places reported in output.
`warm_corr`	Boolean: If `True`, Warm's (1989) bias correction is applied during generation of person estimates (see Section 6.3.5). Default `warm_corr=True`.
`tolerance`	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is `tolerance=0.00001`.
`max_iters`	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is `max_iters=100`.

Continued on next page...

ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) during central item location estimation (see section 6.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). None (default) resolves per representation (see Section 6.3.1).
constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for central item location estimation. None (default) resolves to the simulation-tuned per-stage defaults (see Section 6.3.1). Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: smoothing constant for the threshold-location (CPAT) stage alone; None (default) follows constant. See Section 6.3.1.
robust	Boolean, default False: selects the more conservative per-representation calibration for rough data. See Section 6.3.1.
matrix_power	Integer or None: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for central item location estimation. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method for central item location estimation. Ignored for other methods.
no_of_samples	Integer: Number of bootstrap samples to generate for standard error estimates (see Section 6.3.3). More samples lead to more accurate standard error estimates, but take correspondingly longer to compute. Default is no_of_samples=500.

Continued on next page...

seed	Integer or None: seed passed through to the internal std_errors/fit_statistics calls (and to the lazily-triggered per_rater_model_selection call for model='mixed') if not already computed. None (default) draws fresh entropy each call. See Section 1.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

Returned and stored as attribute self.test_stats_global, self.test_stats_items, self.test_stats_thresholds, self.test_stats_matrix, self.test_stats_bivector, self.test_stats_centrality, self.test_stats_pseudo_halo or self.test_stats_bistretch, a pandas dataframe with two columns, Items and Persons.

Mean	Mean of estimates.
SD	Standard deviation of estimates.
Separation ratio	Separation ratio
Strata	Number of strata
Reliability	Reliability

Example

To produce a self.test_stats_global table under the global facet element representation:
self.test_stats_df_global()

Other arguments may be used to alter parameters of item location and/or person location estimation.

6.4.8 item_res_corr_analysis

Description

Analysis of correlations of standardised residuals by item to test for violations of local item interdependence and unidimensionality requirements.

Usage

Eight methods are defined, one for each facet element (e.g. rater) representation; all methods share the same argument format:

```
self.item_res_corr_analysis_global(warm_corr=True, tolerance=0.00001, max_iters=100,
                                   ext_score_adjustment=0.5, constant=None, threshold_constant=None,
method=None, robust=False,
                                   matrix_power=None, log_lik_tol=0.000001)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently self.item_res_corr_analysis(model='global', ...) with model set to any of the eight representation names. These forward any extra keyword arguments (e.g. se=False – see below) to the internal fit_statistics call, only used if the residuals have not already been computed for this model.

Arguments

warm_corr	Boolean: If True, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 6.3.5). Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.

Continued on next page...

constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for generation of item location estimates. None (default) resolves to the simulation-tuned per-stage defaults (see Section 6.3.1). Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: smoothing constant for the threshold-location (CPAT) stage alone; None (default) follows constant. See Section 6.3.1.
robust	Boolean, default False: selects the more conservative per-representation calibration for rough data. See Section 6.3.1.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) for generation of central item location estimates (see Section 6.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). None (default) resolves per representation (see Section 6.3.1).
se	Boolean: not part of the fixed signature above, but forwarded via extra keyword arguments to the internal fit_statistics call. Standardised residuals (and hence this analysis) do not depend on the bootstrap, so se=False is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is se=True.
matrix_power	Integer or None: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of central item location estimates. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 6.3.1). Ignored for other methods.

Returns

For tests of violation of the requirement for local item independence (Andrich & Kreiner, 2010; Marais, 2012):

self.item_residual_correlations_global, self.item_residual_correlations_items, self.item_residual_correlations_thresholds, self.item_residual_correlations_matrix,	A pandas dataframe of pairwise correlations between item standardised residuals.

Continued on next page...

self.item_residual_correlations_bivector,
self.item_residual_correlations_
centrality,
self.item_residual_correlations_pseudo_
halo or
self.item_residual_correlations_bistretch

For tests of violation of the requirement for unidimensionality based on principal component analysis of the standardised residual correlations (Pallant & Tennant, 2007; Smith, 2002):

`self.item_eigenvectors_global`, `self.item_eigenvectors_items`, `self.item_eigenvectors_thresholds`, `self.item_eigenvectors_matrix`, `self.item_eigenvectors_bivector`, `self.item_eigenvectors_centrality`, `self.item_eigenvectors_pseudo_halo` or `self.item_eigenvectors_bistretch`	The eigenvectors of the standardised residual correlations matrix.
`self.item_eigenvalues_global`, `self.item_eigenvalues_items`, `self.item_eigenvalues_thresholds`, `self.item_eigenvalues_matrix`, `self.item_eigenvalues_bivector`, `self.item_eigenvalues_centrality`, `self.item_eigenvalues_pseudo_halo` or `self.item_eigenvalues_bistretch`	The eigenvalues corresponding to the eigenvectors.
`self.item_variance_explained_global` `self.item_variance_explained_items`, `self.item_variance_explained_thresholds`, `self.item_variance_explained_matrix`, `self.item_variance_explained_bivector`, `self.item_variance_explained_centrality`, `self.item_variance_explained_pseudo_halo` or `self.item_variance_explained_bistretch`	The variance explained by each principal component.
`self.item_loadings_global`, `self.item_loadings_items`, `self.item_loadings_thresholds`, `self.item_loadings_matrix`, `self.item_loadings_bivector`, `self.item_loadings_centrality`, `self.item_loadings_pseudo_halo` or `self.item_loadings_bistretch`	The loading of each item onto each of the principal components, for the first of which large loadings ('large'typically interpreted as > 0.4 or < -0.4) may be interpreted as representing the presence of significant dimensionality, in analogy to factor analysis.

Example

To produce an item residual correlation analysis under the global facet element representation:

```
self.item_res_corr_analysis_global()
```

Arguments may be used to alter parameters of item location and/or person location estimation.

6.4.9 rater_res_corr_analysis

Description

Analysis of correlations of standardised residuals by item to test for violations of local item interdependence and unidimensionality requirements.

Usage

Eight methods are defined, one for each facet element (e.g. rater) representation; all methods share the same argument format:

```
self.rater_res_corr_analysis_global(warm_corr=True, tolerance=0.00001, max_iters=100,
                                    ext_score_adjustment=0.5, constant=None, threshold_constant=None,
method=None, robust=False,
                                    matrix_power=None, log_lik_tol=0.000001)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently self.rater_res_corr_analysis(model='global', ...) with model set to any of the eight representation names. These forward any extra keyword arguments (e.g. se=False – see below) to the internal fit_statistics call, only used if the residuals have not already been computed for this model.

Arguments

warm_corr	Boolean: If True, Warm's bias correction (Warm, 1989) is applied during generation of person location estimates (see Section 6.3.5). Default is warm_corr=True.
tolerance	Float: convergence stopping criterion for Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is tolerance=0.00001.
max_iters	Integer: maximum number of Newton-Raphson iterations during generation of person estimates (see Section 6.3.5). Default is max_iters=100.
ext_score_adjustment	Float: Value in range $(0, 1)$ to ensure a estimate is returned during generation of person location estimates (see Section 6.3.5) if a person has an extreme score (all items responded to are correct or incorrect). Since there is no finite ML location estimate for extreme scores, this adjusts the person's score to ext_score_adjustment (if zero) or the maximum possible score minus ext_score_adjustment (if maximum score) before estimating location. This applies to maximum likelihood estimation only: with warm_corr=True, extreme scores have finite Warm estimates and the adjustment only sets the starting value.

Continued on next page...

constant	Float or None: additive smoothing constant (Elliott & Buttery, 2022b) for generation of item location estimates. None (default) resolves to the simulation-tuned per-stage defaults (see Section 6.3.1). Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none).
threshold_constant	Float or None: smoothing constant for the threshold-location (CPAT) stage alone; None (default) follows constant. See Section 6.3.1.
robust	Boolean, default False: selects the more conservative per-representation calibration for rough data. See Section 6.3.1.
method	String: method for derivation of vector of estimates from pairwise reciprocal matrix (Elliott & Buttery, 2022b) for generation of central item location estimates (see Section 6.3.1). Options are 'ls' for least squares (Choppin, 1968, 1985), 'evm' for the eigenvector method(Garner & Engelhard, 2002), 'cos' for cosine maximisation (Kou & Lin, 2014) or 'log-lik' for the Bradley–Terry log-likelihood method (Bradley & Terry, 1952). None (default) resolves per representation (see Section 6.3.1).
matrix_power	Integer or None: power to which conditional category response frequency matrix (Elliott & Buttery, 2022b:991) is raised for generation of central item location estimates. Each additional power increases the number of indirect pairwise comparisons (Choppin, 1985; Elliott & Buttery, 2022b). Default is matrix_power=None, which resolves to 0 (no matrix powering) when method='log-lik' and to 5 for every other method. Passing matrix_power=0 with a method other than 'log-lik' issues a UserWarning.
log_lik_tol	Float: convergence stopping criterion for log-likelihood method for generation of item location estimates (see Section 6.3.1). Ignored for other methods.
se	Boolean: not part of the fixed signature above, but forwarded via extra keyword arguments to the internal fit_statistics call. Standardised residuals (and hence this analysis) do not depend on the bootstrap, so se=False is purely a speed-up (e.g. for repeated simulation runs) with no effect on the output. Default is se=True.

Returns

For tests of violation of the requirement for local facet element independence (Andrich & Kreiner, 2010; Marais, 2012):

self.rater_residual_correlations_global, self.rater_residual_correlations_items, self.rater_residual_correlations_thresholds,	A pandas dataframe of pairwise correlations between item standardised residuals.

Continued on next page...

self.rater_residual_correlations_matrix,
self.rater_residual_correlations_bivector,
self.rater_residual_correlations_
centrality,
self.rater_residual_correlations_pseudo_
halo or
self.rater_residual_correlations_bistretch

For tests of violation of the requirement for unidimensionality based on principal component analysis of the standardised residual correlations (Pallant & Tennant, 2007; Smith, 2002):

`self.rater_eigenvectors_global`, `self.rater_eigenvectors_items`, `self.rater_eigenvectors_thresholds`, `self.rater_eigenvectors_matrix`, `self.rater_eigenvectors_bivector`, `self.rater_eigenvectors_centrality`, `self.rater_eigenvectors_pseudo_halo` or `self.rater_eigenvectors_bistretch`	The eigenvectors of the standardised residual correlations matrix.
`self.rater_eigenvalues_global`, `self.rater_eigenvalues_items`, `self.rater_eigenvalues_thresholds`, `self.rater_eigenvalues_matrix`, `self.rater_eigenvalues_bivector`, `self.rater_eigenvalues_centrality`, `self.rater_eigenvalues_pseudo_halo` or `self.rater_eigenvalues_bistretch`	The eigenvalues corresponding to the eigenvectors.
`self.rater_variance_explained_global` `self.rater_variance_explained_items`, `self.rater_variance_explained_thresholds`, `self.rater_variance_explained_matrix`, `self.rater_variance_explained_bivector`, `self.rater_variance_explained_centrality`, `self.rater_variance_explained_pseudo_halo` or `self.rater_variance_explained_bistretch`	The variance explained by each principal component.
`self.rater_loadings_global`, `self.rater_loadings_items`, `self.rater_loadings_thresholds`, `self.rater_loadings_matrix`, `self.rater_loadings_bivector`, `self.rater_loadings_centrality`, `self.rater_loadings_pseudo_halo` or `self.rater_loadings_bistretch`	The loading of each facet element onto each of the principal components, for the first of which large loadings ('large'typically interpreted as > 0.4 or < -0.4) may be interpreted as representing the presence of significant dimensionality, in analogy to factor analysis.

Example

To produce a facet element residual correlation analysis under the global facet element representation:

```
self.rater_res_corr_analysis_global()
```

Arguments may be used to alter parameters of item location and/or person location estimation.

6.4.10 save_stats

Description

Exports item_stats, threshold_stats, rater_stats, person_stats and test_stats for a given facet element (e.g. rater) representation to file in a single call. Auto-triggers item_stats_df, threshold_stats_df, rater_stats_df, person_stats_df and test_stats_df (Sections 6.4.1, 6.4.2, 6.4.4, 6.4.6 and the section above) for that representation if any have not been run with their default arguments plus whichever of dp/warm_corr/etc. below are passed, so call those methods directly first if non-default table columns are wanted in the saved output. Does not include category_stats (Section 6.4.3) – save that table separately with self.category_stats_{model}.to_csv(...) if needed.

Usage

```
self.save_stats(model='global', filename='', format='csv', alias=False, dp=3,
                warm_corr=True, tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
                method=None, constant=None, threshold_constant=None, robust=False,
                matrix_power=None, log_lik_tol=0.000001,
                no_of_samples=500, interval=None)
```

or equivalently self.save_stats_global(...) (and _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), which fix model and take the remaining arguments unchanged.

Arguments

model	String, one of 'global', 'items', 'thresholds', 'matrix', 'bivector', 'centrality', 'pseudo_halo' or 'bistretch': the facet element representation whose statistics to save. Default is model='global'.
filename	String: output filename or path. For format='csv', used as a prefix (any trailing .csv is stripped first) for five files, _item_stats.csv, _threshold_stats.csv, _rater_stats.csv, _person_stats.csv and _test_stats.csv. For format='xlsx', used as a single workbook filename (.xlsx appended automatically if not already present), with the five tables on separate sheets. For model one of 'centrality'/'pseudo_halo'/'bistretch', a sixth file/sheet is also saved – _rater_stats_full_vector.csv or the Rater statistics (full vector) sheet – holding self.rater_stats_{model}_full_vector (see Section 6.4.4).
format	String: 'csv' or 'xlsx'. Default is format='csv'.

Continued on next page...

alias	centrality/pseudo_halo/bistretch only. Boolean: passed straight through to the internal rater_stats_df call (Section 6.4.4, Table 255), labelling the saved facet element statistics table's parameter columns with a human-readable alias instead of the raw parameter name. Ignored for other models. Only takes effect if rater_stats_{model} has not already been computed – see row below. Default is alias=False.
divergence_test, correction, alpha, plot	centrality/pseudo_halo/bistretch only. Passed to the internal rater_stats_df call (Section 6.4.4). The saved rater_stats_{model}_full_vector includes the 'Flagged' row only if rater_stats_df has been run with full=True. Defaults divergence_test=None, correction=None, alpha=0.05, plot=False.
dp, warm_corr, tolerance, max_iters, ext_score_adjustment, method, constant, matrix_power, log_lik_tol, no_of_samples, interval	Passed through to whichever of item_stats_df/threshold_stats_df/rater_stats_df/person_stats_df/test_stats_df has not already been computed for model. Calibration uses the estimation method of the latest calibration ('cp-full-rcl' if none). See those sections for details of each.

Returns

None. Writes the five tables to file as described above.

Example

To save item, threshold, facet element, person and test statistics under the global representation, as five CSV files prefixed my_mfrm_results:

```
self.save_stats_global('my_mfrm_results')
```

To save the matrix representation's tables to separate sheets of a single Excel workbook:

```
self.save_stats_matrix('my_mfrm_results', format='xlsx')
```

To save the centrality representation's tables with the facet element statistics table's parameter columns aliased ('Global'/'Threshold stretch' rather than lambda/omega):

```
self.save_stats_centrality('my_mfrm_centrality_results', alias=True)
```

6.4.11 save_residuals_items / save_residuals_raters

Description

Export the item residual correlation analysis (item_res_corr_analysis, Section 6.4.8) or facet element (e.g. rater) residual correlation analysis (rater_res_corr_analysis, Section above) to file, for a given facet element representation. Auto-triggers the corresponding analysis if not already run. Two separate method families exist since the two analyses produce independent sets of eigenvectors/eigenvalues/variance explained/loadings tables (Sections 6.4.8 and above), which are not usually wanted in the same output file.

Usage

```
self.save_residuals_items_global(filename, format='csv', single=True, dp=3)
self.save_residuals_raters_global(filename, format='csv', single=True, dp=3)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch in place of _global, for both families). Any extra keyword arguments (e.g. warm_corr, constant) are forwarded to the underlying residual correlation analysis, used only if it has not already been run for that representation.

Arguments

filename	String: output filename or path (extension appended automatically if not already present).
format	String: 'csv' or 'xlsx'. Default is format='csv'.
single	Boolean: if True, writes all four tables (eigenvectors, eigenvalues, variance explained, loadings) into a single file – sequentially, separated by blank lines, for format='csv'; at successive row offsets on one sheet ('Residual analysis'), for format='xlsx'. If False, writes each table to its own file (csv) or sheet (xlsx). Default is single=True.
dp	Integer: Number of decimal places for rounding the saved values. Default is dp=3.

Returns

None. Writes the four tables to file as described above.

Example

To save the item residual correlation analysis under the global representation as a single CSV file:

```
self.save_residuals_items_global('my_mfrm_item_residuals')
```

To save the facet element residual correlation analysis under the matrix representation, each table to its own sheet of a single Excel workbook:

```
self.save_residuals_raters_matrix('my_mfrm_rater_residuals', format='xlsx', single=False)
```

6.4.12 category_counts_df

Description

Produces a table of counts of scores in each category, plus total and missing responses, aggregated across the requested facet elements (e.g. raters), plus by individual facet element. All items share the same max_score in MFRM, so there are no blank cells.

To compare counts between different subgroups of persons according to a criterion such as an exogenous variable, use the argument counts_name; this stores tables within a namespace, self.counts, meaning that a succession of tables, e.g. one per exogenous-variable group, can be kept side by side for comparison rather than overwriting each other.

Usage

```
self.category_counts_df(persons=None, items=None, facet_elements=None, counts_name=None,
                        filename=None, file_format='csv')
```

Arguments

persons	List or String: Person or subset of persons over which to count responses. Default persons=None, which returns counts for all persons.
items	List or String: Item or subset of items over which to count responses. Default items=None, which returns counts for all items.
facet_elements	List or String: Facet element (e.g. facet element) or subset of facet elements over which to count responses – restricts both the aggregated table and the per-facet-element breakdown to this subset. Default facet_elements=None, which uses all facet elements (e.g. self.rater_names).
counts_name	String or Int: Name under which to store counts on a subset of persons, items or facet elements, so multiple sets of counts can be made available. If counts_name='my_counts_name', the aggregated table under self.counts.my_counts_name, the breakdown under self.counts.my_counts_name_facet_elements. If counts_name=None, stores the aggregated table as self.category_counts_table and the per-facet-element breakdown as self.category_counts_facet_elements (also aliased as self.category_counts_raters under the default facet naming). Default is counts_name=None.
filename	String: The filename for the saved table, with no suffix for format. If None, no file will be saved. Default is filename=None.
file_format	The format of the file: csv or xlsx. Default is file_format=csv.

Returns

The aggregated table (`self.category_counts_table` below) is returned, and the tables are stored as follows.

If `counts_name=None`:

`self.category_counts_table`	pandas dataframe: category counts with one row per item and one column per response category, plus `Total` and `Missing` columns, with a `Total` row, aggregated across the facet elements.
`self.category_counts_facet_elements` (also `self.category_counts_raters` under the default facet naming)	pandas multiindex dataframe: the same category counts broken down by facet element within the requested subset, with facet elements as the outer index level and items (plus a `Total` row per facet element) as the inner index level.

If `counts_name=my_counts_name`:

`self.counts.my_counts_name`	The aggregated table described above, stored in the namespace `self.counts`.
`self.counts['my_counts_name_facet_elements']`	The per-facet-element breakdown described above, stored in the same namespace.

Example

To produce the aggregated and per-facet-element dataframes of category counts for all persons, items and facet elements:

```
my_mfrm.category_counts_df()
```

To produce category counts for a subset of persons saved as a list `my_persons`, restricted to facet element `'Rater_1'`, with a matching attribute name within the namespace `self.counts`:

```
my_mfrm.category_counts_df(persons=my_persons, facet_elements='Rater_1',
                                    counts_name=my_persons)
```

6.4.13 dif_test

Description

Differential Item Functioning (DIF) and Differential Category Functioning (DCF) test by an exogenous person covariate, for a specified facet_effect model. Item locations and thresholds are those of the representation given by `model`. Persons (not facet elements) are split into reference and focal groups, each calibrated separately; purification uses items only.

1. **Item-location DIF.** Both groups are calibrated independently (each self-centred), then purified onto a common item scale via either a per-item Wald significance test (`selection_method='wald'`, default) or the robust modified z-score outlier-trimming procedure used by `calibrate_anchor` (Section 5.3.3; Iglewicz & Hoaglin, 1993) – the reference group's item locations supply the "anchor" values, the focal group's supply the "observed" values – so that genuine DIF items cannot contaminate the common scale used to test for DIF. Item-level DIF is tested via a per-item Wald test (`test='wald'`, default), a per-item likelihood-ratio test (`test='lr'`), or both (`test='both'`), using bootstrap standard errors from `std_errors` (Section 5.3.2) for the Wald test. An omnibus LR test (Andersen-style: reference vs. focal, every common item jointly) runs by default (`omnibus=True`). Results in `self.dif_table` and `self.dif_omnibus_table`.
2. **Differential category functioning (DCF)**. Invariance of the threshold structure is tested via category widths (consecutive threshold differences, `thresholds[k+1] - thresholds[k]`) rather than raw threshold locations, since threshold locations are not independent: changing one threshold affects all other threshold locations, while widths isolate a change to the category affected. Each `max_score` − `1` category width is Wald-tested directly, reference versus focal, using a bootstrap standard error. Not affected by `test`/`omnibus` – those only govern component (1). Results in `self.threshold_dif_table`.

An item or category-width cell is flagged only if it clears both a (corrected) p-value threshold and an absolute logit-difference threshold, so that a statistically significant but practically negligible difference is not flagged. `dif_test` also supports ETS-style A/B/C severity-category classification (Zwick et al., 1997) (`category`, `category_thresholds`, `category_alpha`) for the item-location component only – the 0.43/0.64 logit defaults are calibrated for item-location DIF specifically, so this does not extend to `self.threshold_dif_table`, as described in Table 275:

Table 275: ETS-style DIF category classification. Sign: `+` indicates that an item is harder for the focal group; `-` indicates that an item is easier for the focal group.

Category	Criteria	Interpretation
A	|Difference| < 0.43, or not statistically significant	Negligible DIF
B+ / B-	Both |Difference| ≥ 0.43 and $p(\text{DIF} = 0) <$ `category_alpha` (two-sided Wald/Welch test), but not C	Slight to moderate DIF
C+ / C-	Both |Difference| ≥ 0.64 *and* $p(\|\text{Difference}\| \leq 0.43) <$ `category_alpha` (one-sided test that difference significantly exceeds B boundary)	Moderate to large DIF

The omnibus test comes in two forms. `self.dif_omnibus_table` tests items only: facet effects are held at

their full-data values in both groups and items/thresholds estimated separately for each group (df = common items -1). `self.df_omnibus_table` tests items and facet effects together: items, thresholds and facet effects are all estimated separately for each group (same df). `test='lr'/'both'` is mildly liberal; `test='wald'` (the default) is preferred.

Usage

```
self.dif_test(covariate, model='global', reference=None, selection_method='wald',
              corr_tol=0.95, sd_ratio_tol=1.1, min_anchors=6, wald_alpha=0.05,
              test='wald', omnibus=True, welch=False, size_adjust=False, reference_n=100,
              dcf_cap=2000, correction='bh', alpha=0.05, logit_threshold=0.43, category=False,
              category_thresholds=(0.43, 0.64), category_alpha=0.05, no_of_samples=500,
              plot=False, plot_kwargs=None, warm_corr=True, tolerance=0.00001, max_iters=100,
              ext_score_adjustment=0.5, constant=None, threshold_constant=None, method=None,
              matrix_power=None, matrix_marginals=None, regression='huber', robust=False,
              log_lik_tol=0.000001, seed=None)
```

Arguments

`covariate`	String: column of `self.exogenous` by which persons are grouped.
`model`	String, one of `'global'`, `'items'`, `'thresholds'`, `'bivector'`, `'matrix'`, `'centrality'`, `'pseudo_halo'` or `'bistretch'`: the facet element representation each group is calibrated under. Default is `model='global'`.
`reference`	String or `None`: value of `covariate` for the reference group. `None` uses the largest group. Default is `reference=None`.
`selection_method`	String, one of `'wald'`, `'robust_z'` or `'none'`: how the item-location scale is purified before testing. `'wald'`: sequential Wald tests of each item's difference, using `wald_alpha` and `min_anchors`. `'robust_z'`: excludes outlying items by the modified z-score procedure (Iglewicz & Hoaglin, 1993), using `corr_tol`, `sd_ratio_tol` and `min_anchors`. `'none'`: mean shift over all common items. Default is `selection_method='wald'`.
`corr_tol`	Float: minimum correlation between the two groups' item locations on the retained items (`'robust_z'` only). Default is `corr_tol=0.95`.
`sd_ratio_tol`	Float: the ratio of the two groups' item-location standard deviations on the retained items must lie in $[1/\texttt{sd_ratio_tol}, \texttt{sd_ratio_tol}]$ (`'robust_z'` only). Default is `sd_ratio_tol=1.1`.
`min_anchors`	Integer: minimum number of items retained by purification (`'wald'` and `'robust_z'`). Default is `min_anchors=6`.

Continued on next page...

wald_alpha	Float: significance level for each item's purification test ('wald' only). Default is wald_alpha=0.05.
test	String, one of 'wald', 'lr' or 'both': per-item test(s) of item-location DIF. 'lr' and 'both' add a likelihood-ratio test and a Flagged_LR column; Flagged always reflects the Wald test. 'lr' and 'both' are mildly liberal; 'wald' is preferred. Default is test='wald'.
omnibus	Boolean: if True, also computes the two omnibus likelihood-ratio tests for each focal group (self.dif_omnibus_table and self.df_omnibus_table). Default is omnibus=True.
welch	Boolean: if True, the Wald tests (item locations and category widths) use a t-distribution with Welch–Satterthwaite degrees of freedom instead of the standard normal, and a df column is added to both tables. Does not affect the likelihood-ratio tests. Default is welch=False.
size_adjust	Boolean: if True, inflates each group's standard errors at large N. Item-location SEs are multiplied by $\sqrt{n/\texttt{reference_n}}$ (Tristan, 2006), where n is the group size; category-width SEs are multiplied by $\max(1, \sqrt{n/\texttt{dcf_cap}})$. Applies to the Wald tests, the category tests and, if welch=True, the degrees of freedom. Default is size_adjust=False.
reference_n	Integer: reference sample size for the item-location standard errors when size_adjust=True. Default is reference_n=100.
dcf_cap	Float: cap, in persons, for the category-width standard errors when size_adjust=True. Default is dcf_cap=2000.
correction	String or None, one of 'bh' (Benjamini–Hochberg (Benjamini & Hochberg, 1995)), 'bonferroni' or None: multiple-comparison correction, applied separately across items and across category widths within each focal group. Not applied to the omnibus tests. Default is correction='bh'.
alpha	Float: significance level for flagging (corrected p-values if correction is set). Default is alpha=0.05.
logit_threshold	Float: minimum absolute difference, in logits, for flagging. Default is logit_threshold=0.43.
category	Boolean: if True, adds an ETS Category column (Table 275) to self.dif_table. Default is category=False.
category_thresholds	Tuple of two floats: the B and C category boundaries, in logits (Zwick et al., 1997). Default is category_thresholds=(0.43, 0.64).
category_alpha	Float: significance level for the category tests. Default is category_alpha=0.05.

Continued on next page...

no_of_samples	Integer: bootstrap resamples for each group's standard errors. Default is no_of_samples=500.
plot	Boolean: if True, plots the purification for each focal group with plot_anchor_selection ('wald' and 'robust_z' only), stored in self.dif_plots. Default is plot=False.
plot_kwargs	Dictionary or None: keyword arguments passed to plot_anchor_selection. Default is plot_kwargs=None.
warm_corr	Boolean: if True, person locations are Warm's weighted likelihood estimates; otherwise maximum likelihood estimates. Used only when test is 'lr' or 'both', or omnibus=True. Default is warm_corr=True.
tolerance	Float: convergence criterion for person estimation (same conditions as warm_corr). Default is tolerance=0.00001.
max_iters	Integer: maximum Newton–Raphson iterations for person estimation (same conditions as warm_corr). Default is max_iters=100.
ext_score_adjustment	Float in $(0, 1)$: adjustment to extreme scores for maximum likelihood person estimation (same conditions as warm_corr). Default is ext_score_adjustment=0.5.
constant	Float or None: additive smoothing constant for each group's calibration. None uses the data-adaptive default. Default is constant=None.
threshold_constant	Float or None: additive smoothing constant for the thresholds. None follows constant. Default is threshold_constant=None.
method	String or None: pairwise estimation method ('ls', 'evm', 'cos' or 'log-lik'), estimation='pair-cpat' only. None resolves by representation. Default is method=None.
matrix_power	Integer or None: power to which the pairwise matrix is raised, estimation='pair-cpat' only. None resolves to 0 for 'log-lik' and 5 otherwise. Default is matrix_power=None.
matrix_marginals	Boolean or None: 'bivector' and 'bistretch' with estimation='pair-cpat' only: if True, the item and threshold vectors are marginal means of the matrix representation. Not used in the bootstrap. Default is matrix_marginals=None.
regression	String, 'huber' or 'theil-sen': robust regression for the stretch parameters, 'centrality', 'pseudo_halo' and 'bistretch' only. Not used in the bootstrap. Default is regression='huber'.

Continued on next page...

robust	Boolean: estimation='pair-cpat' only; for 'bivector', estimates the item and threshold vectors directly when matrix_marginals=None. Default is robust=False.
log_lik_tol	Float: convergence criterion for the 'log-lik' method. Default is log_lik_tol=0.000001.
seed	Integer or None: seed for each group's bootstrap. None draws fresh entropy (Section 1). Default is seed=None.

Returns

self.dif_model	String: the model used.
self.dif_covariate	String: the covariate used.
self.dif_reference	The reference group value.
self.dif_reference_model	MFRM object: the reference group's fitted model.
self.dif_focal_models	Dictionary: {focal_value: MFRM}, each focal group's fitted model.
self.dif_tc	Dictionary: {focal_value: float}, the item-scale translation constant applied to each focal group.
self.dif_anchor_selection	Dictionary: {focal_value: dataframe or None}, the purification table for each focal group (None if selection_method='none').
self.dif_plots	Dictionary: {focal_value: figure or None}, the plot_anchor_selection figure for each focal group, if plot=True.
self.dif_group_sizes	Dictionary: {group_value: int}, the number of persons (non-extreme, covariate not missing) in each group.
self.dif_table	pandas dataframe: item-location DIF, one row per (item, focal group) pair, indexed by item. Columns: Group, Reference, Focal, Focal (purified), Difference, SE, z, p, p (corrected), Selected, Flagged. Also df if welch=True; LR, p_LR, p_LR (corrected), Flagged_LR if test is 'lr' or 'both'; Category if category=True.
self.dif_omnibus_table	pandas dataframe or None: one row per focal group, with LR, df, p, Flagged: the omnibus test of items, with facet effects held at their pooled values. None if omnibus=False.
self.df_omnibus_table	pandas dataframe or None: as self.dif_omnibus_table, for the joint test of items and facet effects. None if omnibus=False.

Continued on next page...

`self.threshold_dif_table`	pandas dataframe: differential category functioning, tested on category widths. One row per (category, focal group) pair, where category k is the width between thresholds k and $k+1$. Columns: `Group`, `Category`, `Reference`, `Focal`, `Difference`, `SE`, `z`, `p`, `p (corrected)`, `Flagged`, and `df` if `welch=True`.

Examples

To test for DIF and DCF under the `'global'` facet element model, by an exogenous variable `'L1'`:

```
self.dif_test('L1')
```

To test under the `'matrix'` facet element model instead, additionally computing the per-item likelihood-ratio test and ETS-style DIF categories:

```
self.dif_test('L1', model='matrix', test='both', category=True)
```

6.4.14 drf_test

Description

Differential Rater Functioning (DRF) test by an exogenous person covariate: the facet-element-location analogue of DIF. The test depends on the model:

1. `'global'`: one test per common facet element (see below).
2. `'items'/'thresholds'/'bivector'/'matrix'`: a separate test of each facet element's effect on each item (`items`) or threshold (`thresholds`). `'bivector'/'matrix'` test both, in two tables: `drf_table` (items) and `drf_threshold_table` (thresholds). `'matrix'` uses marginal item and threshold projections.
3. `'centrality'/'pseudo_halo'/'bistretch'`: one joint test per common facet element of all its parameters together (λ and ω; for `bistretch`, λ, ω_{items} and $\omega_{\text{thresholds}}$). The statistic is $T = \Delta^\top \Sigma^{-1} \Delta$, where Δ is the vector of focal-minus-reference differences in the facet element's parameters, and Σ is the covariance matrix of those differences estimated from paired bootstraps. T is compared with a χ^2 distribution with 2 degrees of freedom (3 for `bistretch`). Separate z-tests for each parameter are also reported. λ is purified as for `'global'`; ω is not purified.

Only common facet elements are tested: those that rated at least `min_persons_per_group` persons in each group. Others are listed in `self.drf_excluded_raters`. Purification uses the same anchor-selection method as `dif_test`, applied to facet element locations; all common facet elements are tested. For `'global'` there is one translation constant per focal group; for `'items'/'thresholds'/'bivector'/'matrix'` there is one per item or threshold.

The omnibus test is available for `'global'` only, and is run by default. It holds items and thresholds at their pooled values and lets the facet effects vary per group, with *df* = common facet elements −1.

Usage

```
self.drf_test(covariate, model='global', reference=None, selection_method='wald',
              corr_tol=0.95, sd_ratio_tol=1.1, min_anchors=2, wald_alpha=0.05,
              min_persons_per_group=5, omnibus=None, welch=False, correction='bh', alpha=0.05,
              logit_threshold=0.43, no_of_samples=500, plot=False, plot_kwargs=None,
              warm_corr=True, tolerance=0.00001, max_iters=100, ext_score_adjustment=0.5,
              constant=None, threshold_constant=None, method=None, matrix_power=None, robust=False,
              log_lik_tol=0.000001, seed=None)
```

Arguments

`covariate`	String: column of `self.exogenous` by which persons are grouped.
`model`	String, one of `'global'`, `'items'`, `'thresholds'`, `'bivector'`, `'matrix'`, `'centrality'`, `'pseudo_halo'` or `'bistretch'`: the facet element representation each group is calibrated under, which determines the tests (see above). Default is `model='global'`.

Continued on next page...

reference	String or None: value of covariate for the reference group. None uses the largest group. Default is reference=None.
selection_method	String, one of 'wald', 'robust_z' or 'none': how the facet element location scale is purified before testing. 'wald': sequential Wald tests of each common facet element's difference, using wald_alpha and min_anchors. 'robust_z': excludes outlying facet elements by the modified z-score procedure (Iglewicz & Hoaglin, 1993), using corr_tol, sd_ratio_tol and min_anchors. 'none': mean shift over all common facet elements. Default is selection_method='wald'.
corr_tol	Float: minimum correlation between the two groups' facet element locations on the retained facet elements ('robust_z' only). Default is corr_tol=0.95.
sd_ratio_tol	Float: the ratio of the two groups' facet element location standard deviations on the retained facet elements must lie in $[1/\texttt{sd_ratio_tol}, \texttt{sd_ratio_tol}]$ ('robust_z' only). Default is sd_ratio_tol=1.1.
min_anchors	Integer: minimum number of common facet elements retained by purification ('wald' and 'robust_z'). Default is min_anchors=2.
wald_alpha	Float: significance level for each facet element's purification test ('wald' only). Default is wald_alpha=0.05.
min_persons_per_group	Integer: minimum number of persons a facet element must have rated in each group to be a common facet element. Others are excluded and listed in self.drf_excluded_raters. Default is min_persons_per_group=5.
omnibus	Boolean or None: whether to run the omnibus test ('global' only; True with any other model raises a ValueError). None runs it for 'global'. Default is omnibus=None.
welch	Boolean: if True, the Wald tests use a t-distribution with Welch–Satterthwaite degrees of freedom instead of the standard normal, and a df column is added. Does not affect the joint tests or the omnibus test. Default is welch=False.
correction	String or None, one of 'bh' (Benjamini–Hochberg (Benjamini & Hochberg, 1995)), 'bonferroni' or None: multiple-comparison correction across each table's tests. Not applied to the omnibus test. Default is correction='bh'.
alpha	Float: significance level for flagging (corrected p-values if correction is set). Default is alpha=0.05.

Continued on next page...

logit_threshold	Float: minimum absolute difference, in logits, for flagging. Default is logit_threshold=0.43.
no_of_samples	Integer: bootstrap resamples for each group's standard errors. Default is no_of_samples=500.
plot	Boolean: if True, plots the purification for each focal group with plot_anchor_selection ('wald' and 'robust_z' only), stored in self.drf_plots. Default is plot=False.
plot_kwargs	Dictionary or None: keyword arguments passed to plot_anchor_selection. Default is plot_kwargs=None.
warm_corr	Boolean: if True, person locations are Warm's weighted likelihood estimates; otherwise maximum likelihood estimates. Used only for the omnibus test. Default is warm_corr=True.
tolerance	Float: convergence criterion for person estimation (omnibus test only). Default is tolerance=0.00001.
max_iters	Integer: maximum Newton–Raphson iterations for person estimation (omnibus test only). Default is max_iters=100.
ext_score_adjustment	Float in $(0, 1)$: adjustment to extreme scores for maximum likelihood person estimation (omnibus test only). Default is ext_score_adjustment=0.5.
constant	Float or None: additive smoothing constant for each group's calibration. None uses the data-adaptive default. Default is constant=None.
threshold_constant	Float or None: additive smoothing constant for the thresholds. None follows constant. Default is threshold_constant=None.
method	String or None: pairwise estimation method ('ls', 'evm', 'cos' or 'log-lik'), estimation='pair-cpat' only. None resolves by representation. Default is method=None.
matrix_power	Integer or None: power to which the pairwise matrix is raised, estimation='pair-cpat' only. None resolves to 0 for 'log-lik' and 5 otherwise. Default is matrix_power=None.
robust	Boolean: estimation='pair-cpat' only; for 'bivector', estimates the item and threshold vectors directly rather than as marginal means of the matrix representation. Default is robust=False.
log_lik_tol	Float: convergence criterion for the 'log-lik' method. Default is log_lik_tol=0.000001.
seed	Integer or None: seed for each group's bootstrap. None draws fresh entropy (Section 1). Default is seed=None.

Returns

self.drf_model	String: the model used.
self.drf_covariate	String: the covariate used.
self.drf_reference	The reference group value.
self.drf_reference_model	MFRM object: the reference group's fitted model.
self.drf_focal_models	Dictionary: {focal_value: MFRM}, each focal group's fitted model.
self.drf_group_sizes	Dictionary: {group_value: int}, the number of persons (non-extreme, covariate not missing) in each group.
self.drf_excluded_raters	Dictionary: {focal_value: list}, facet elements present in both groups but excluded because they rated fewer than min_persons_per_group persons in at least one.
self.drf_table	pandas dataframe, common facet elements only. 'global': one row per (facet element, focal group), with columns Group, Reference, Focal, Focal (purified) (facet element locations), Difference, SE, z, p, p (corrected), Selected, Flagged, and df if welch=True. 'items', 'thresholds', 'bivector', 'matrix': one row per (facet element, item or threshold, focal group), with the same columns plus an Item or Category column; one correction across the whole table; for 'bivector' and 'matrix', items only. 'centrality', 'pseudo_halo', 'bistretch': one row per (facet element, focal group), with Group; Reference_{param}, Focal_{param} and Difference_{param} for each parameter; T (the joint statistic), df, p, p (corrected), Selected (λ purification) and Flagged (joint test); and z_{param}, p_{param}, p_{param} (corrected) and Flagged_{param} for each parameter, with a separate correction.
self.drf_threshold_table	pandas dataframe or None: 'bivector' and 'matrix' only: the threshold tests, with the same columns as self.drf_table and a Category column, corrected separately. None for other models.
self.drf_omnibus_table	pandas dataframe or None: one row per focal group, with LR, df, p, Flagged. None unless the omnibus test is run.
self.drf_tc	Dictionary: the translation constants applied to each focal group, one per common facet element (each excluding that facet element). 'global', 'centrality', 'pseudo_halo', 'bistretch' (λ only): {focal_value: pandas.Series}. 'items', 'thresholds': {focal_value: {item or threshold: pandas.Series}}. 'bivector', 'matrix': {focal_value: {'items': {item: pandas.Series}, 'thresholds': {threshold: pandas.Series}}}.

Continued on next page...

`self.drf_anchor_selection`	Dictionary: {`focal_value: dataframe or None`}, the purification table for each focal group (`None` if `selection_method='none'`).
`self.drf_plots`	Dictionary: {`focal_value: figure or None`}, the `plot_anchor_selection` figure for each focal group, if `plot=True`.

Examples

To test for DRF under the `'global'` facet element model, by an exogenous variable `'L1'`:

```
self.drf_test('L1')
```

To test under the `'bistretch'` facet element model instead, requiring at least 10 rated persons per group for a facet element to be tested at all:

```
self.drf_test('L1', model='bistretch', min_persons_per_group=10)
```

6.4.15 recovery_stats

Description

Compares recovered parameter estimates of an object created from a simulation against the generating values (self.generating, Section 6.1.1). Issues a UserWarning and returns None unless the object was built from a simulation. model defaults to self.generating.model. A different model may be given, to measure the effect of fitting a simpler representation than the one that generated the data (Elliott & Buttery, 2022a). Items, thresholds and category widths are compared directly for any model.

If model == self.generating.model, facet effects are compared in that representation's own form, with bootstrap standard errors (one set of facet element parameters for global/items/thresholds/matrix, two for bivector/centrality/pseudo_halo, three for bistretch). Otherwise the full (facet element, item, category) location grids are compared, with z/p/Flagged set to NaN/False.

Person estimates are compared against *pseudo-estimates* (Elliott & Buttery, 2022b) to isolate systemic error from stochastic response noise: the person estimate from self.responses using the generating parameters (under self.generating.model). The persons error column is (recovered estimate − pseudo-estimate). It combines parameter estimation error and model misspecification.

groups='rater' gives person recovery by facet element: for each facet element R, the recovered estimate and pseudo-estimate are recomputed from R's responses only (person(facet_elements=[R])). rater_subset restricts the per-facet-element plots, not the summary table. A pandas.Series may also be passed to groups for person-level grouping (as for wright_map's group_by), applying to person recovery only.

Usage

```
self.recovery_stats(model=None, anchor=False, groups=None, rater_subset=None,
                        separate_rater_plots=False, alpha=0.05, correction='bh',
                        divergence_tol=0.1, plot=True, **plot_kwargs)
```

Arguments

model	String or None: which of the eight facet element representations to recover (see Section 6.3.1). None (default) resolves to self.generating.model. If not yet calibrated, it triggers calibrate(model=...).

Continued on next page...

anchor	Boolean: if True, compares against model's anchored attributes (self.anchor_items_{model}, self.anchor_thresholds_{model}, self.anchor_persons_{model}, self.anchor_facet_effects_{model}) instead of the unanchored ones. Raises ValueError, listing every missing attribute, unless the full anchored pipeline (calibrate_anchor, Section 6.3.2; anchor_std_errors, Section 6.3.4; person_estimates(anchor=True), Section 6.3.6) has been run for model. The person standard error, self.csem_vector_{model}, is always the non-anchored one. Default anchor=False.
groups	pandas.Series, 'rater' or None: person-level grouping for person recovery (see above). Default groups=None (no grouping, a single Overall column).
rater_subset	List or None: restricts the by-facet-element direct-comparison plot (when groups='rater') to these facet elements. None (default) uses every facet element in self.facet_names.
separate_rater_plots	Boolean: for bivector's two sets of facet element parameters or bistretch's three, False (default) plots them combined on one set of axes (different colours, separate OLS/Huber fits per colour); True plots them separately, one figure per parameter set. No effect for the representations with one set of facet element parameters (global/items/thresholds/matrix/centrality/pseudo_halo). Default separate_rater_plots=False.
alpha	Float: significance threshold for the per-parameter z-test (Flagged/Flagged_corrected columns, below). Default alpha=0.05.
correction	String or None, one of 'bh', 'bonferroni' or None: multiple-comparison correction applied within each parameter type's own family of tests. Default correction='bh'.
divergence_tol	Float: threshold on the absolute difference between the OLS and Huber regression slopes (below) for the Divergence_flag summary row. Default divergence_tol=0.1.
plot	Boolean: if True, also produces recovery plots (scatter, identity line, OLS and Huber fit lines): one each for items/thresholds/category widths/persons, one for facet element effects (coloured by parameter set, combined or separate per separate_rater_plots), an additional by-facet-element persons comparison and by-facet-element direct-comparison plot when groups='rater', and an anchor-vs-non-anchor plot when anchor=True. Default plot=True.

Continued on next page...

**plot_kwargs	Forwarded to the internal plotting routine: filename, file_format, dpi, plot_style, font, title_font_size, axis_font_size, labelsize, figsize (same vocabulary as wright_map). A shared filename is suffixed per plot.

Returns

self.recovery.items, self.recovery.thresholds, self.recovery.category_widths, self.recovery.persons	Namespaces, each with .table (a per-parameter pandas dataframe: generating, estimated, SE, error, abs_error, z, p, p_corrected, Flagged, Flagged_corrected) and .summary (one column, Overall – or one per group if groups is set – with rows RMSE, MAE, bias, r, SD_ratio, OLS_slope, OLS_intercept, Huber_slope, Huber_intercept, Divergence_flag, Prop_flagged). For persons, generating holds the pseudo-estimate. r, the two slopes and SD_ratio are set to NaN when fewer than 3 parameters are compared or the generating values have standard deviation below 0.05. When groups='rater', self.recovery.persons.summary has one column per facet element in addition to Overall (the pooled comparison).
self.recovery.raters	Namespace. When model == self.generating.model: .table and .summary are None, and .subvectors is a dict keyed by parameter set (e.g. 'global', or 'items'/'thresholds' for bivector), each value itself a namespace with its own .table/.summary in the same shape as items above. When model != self.generating.model: .subvectors is None, and .table/.summary instead hold the (facet element, item, category) location-grid comparison, with SE/z/p/Flagged left NaN/False.

Examples

To compare recovered items/thresholds/category-widths/persons/facet-element-effects against the generating simulation, plotting each:

```
self.recovery_stats()
```

To recover a simpler facet element representation than the generating one (e.g. generating data was bivector):

```
self.recovery_stats(model='global')
```

To break person recovery down by facet element, restricting the direct facet-element-comparison plot to two named facet elements:

```
self.recovery_stats(groups='rater', rater_subset=['Rater_1', 'Rater_2'])
```

To compare anchored estimates instead (having already run the full anchored pipeline for model='global'):

```
self.recovery_stats(model='global', anchor=True)
```

6.5 Plotting functionality

6.5.1 Shared plotting arguments

All the plotting methods described in this section share a set of arguments which may be used to customise the appearance of the plot or save the plot to file automatically. These arguments are:

`title`	String: Title for the plot, to appear in the image. Default is `title=None`.
`xmin`	Float: Minimum displayed point on x-axis, in logits. Default is `xmin=-5`.
`xmax`	Float: Maximum displayed point on x-axis, in logits. Default is `xmax=5`.
`plot_style`	String: Plot style to use. Available styles are `'white'`, `'dark'`, `'black'`, `'parchment'`, `'minimal'`, `'print'`, `'solarized-light'`, `'solarized-dark'`, `'slate'`, `'blueprint'`, `'newsprint'` and `'chalkboard'` – twelve Seaborn (Waskom, 2021)-based colour schemes. Default is `plot_style='white'`.
`palette`	String: Controls colours of lines and overplotted class intervals dots. Options are `'dark blue'`, `'light blue'`, `'dark red'`, `'light red'`, `'dark green'`, `'light green'`, `'dark grey'`, `'light grey'`, `'dark multi'`, `'light multi'` and `'colorblind multi'`. `'colorblind multi'` is the Okabe & Ito (2008) colour-vision-deficiency-safe qualitative palette. Default is `palette='colorblind multi'`. Not used for `std_residual_plot`.
`black`	Boolean: If `True`, the plot will be rendered in black and white. Default is `black=False`.
`marker_border`	Boolean: If `True`, observed data point markers are drawn with a black edge for contrast. Only used by `icc`, `crcs`, `threshold_ccs` and `tcc` (the methods with an `obs` argument). Default is `marker_border=True`.
`line_border`	Boolean: If `True`, curve lines are drawn with a contrasting outline stroke (white on dark `plot_style` backgrounds, black otherwise) to improve visibility. Only used by `icc`, `crcs`, `threshold_ccs` and `tcc`. Default is `line_border=False`.
`font`	String: The font to use in the plot. Default is `font='Times New Roman'`.
`title_font_size`	Float: The size of the title font in points. Default is `title_font_size=15`.
`axis_font_size`	Float: Axis label font size in points. Default `axis_font_size=12`.
`labelsize`	Float: The size of the axis tick label font in points. Default is `labelsize=12`.
`filename`	String: The filename for the saved plot, with no suffix for format. If `None`, no file will be saved. Default is `filename=None`.
`file_format`	The format of the file: `png`, `jpg` or `svg`. Default is `file_format=png`.
`dpi`	The resolution of the plot in dpi (dots per inch) – higher resolution plots are better quality but have larger file sizes. Default is `dpi=300`.

6.5.2 icc

Description

Plots the item characteristic curve (or item response function) for an item: person location on the x-axis against expected score on the y-axis. Options to plot observed responses, item threshold line and lines showing locations corresponding to specified expected scores, and to highlight a specified response category.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.icc_global(item, anchor=False, facet_element=None, obs=None, warm=True,
                no_of_classes=5, thresh_lines=False, central_location=False,
                score_lines=None, score_labels=False, cat_highlight=None)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently `self.icc(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

item	String: The name of the item(s) to plot.
anchor	Boolean: If True, anchored parameters are used; if False, unanchored parameters are used. Default is anchor=False.
facet_element	String: If a facet_element argument is passed, plots the item characteristic curve for the item as rated by the given facet element. If facet_element=None, plots a neutral item characteristic curve for a facet element with zero facet effect. Default is facet_element=None.
obs	Boolean: If True, mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is obs=None.
warm	Boolean: If True, Warm's bias correction (Warm, 1989) is applied when locating any score_lines on the curve. Default is warm=True.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
thresh_lines	Boolean: If True, vertical lines showing the thresholds between each response category will be plotted: uncentred thresholds when a facet_element argument is passed, centred thresholds when facet_element=None. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is thresh_lines=False.

Continued on next page...

central_location	Boolean: If True, a vertical line is plotted showing the threshold corresponding to the central item location, for uncentred thresholds – see Section 6.2.1); if facet_element=None, the line will be at zero facet effect (the mean of the centred thresholds by definition). Default is central_location=False.
score_lines	List of floats between 0 and self.max_score: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score.
score_labels	Boolean: If True, scores and locations corresponding to arguments passed to score_lines will be labelled on the plot. Default is score_labels=False.
cat_highlight	Integer: Passing a score between 0 and self.max_score will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is cat_highlight=None (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns
matplotlib image. If a filename argument is passed, also saves the image to file.

Examples
To plot a basic unanchored ICC for Item_1 under the global facet element representation and store the output as a variable my_icc_plot and save it to file as my_icc_plot.png:

```
my_icc_plot = self.icc_global('Item_1', filename=my_icc_plot)
```

To plot an anchored ICC for Item_1 under the item facet element representation, rated by Rater_1, with observed responses and store the output as a variable my_icc_plot:

```
my_icc_plot = self.icc_items('Item_1', anchor=True, facet_element='Rater_1', obs=True)
```

To plot an ICC for Item_1 under the threshold facet element representation with a threshold line and highlighted zero category, and store the output as a variable my_icc_plot:

```
my_icc_plot = self.icc_thresholds('Item_1', thresh_lines=True, cat_highlight=0)
```

To plot an ICC for Item_1 under the matrix facet element representation with lines showing the locations corresponding to expected scores of 0.7 and 1.6, with the expected score and corresponding location labelled, and store the output as a variable my_icc_plot:

```
my_icc_plot = self.icc_matrix('Item_1', score_lines=[0.7, 1.6], score_labels=True)
```

6.5.3 crcs

Description

Plots category response curves for an item: person location on the x-axis against expected the probability of obtaining a score in each category (0 or 1) on the y-axis. Options to plot observed proportions and item threshold line, and to highlight a specified response category.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.crcs_global(item=None, anchor=False, facet_element=None, obs=None,
                 no_of_classes=5, thresh_lines=False, central_location=False,
                 cat_highlight=None)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently `self.crcs(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

item	String: The name of the item to plot. If item=None, the shared set of Rasch-Andrich thresholds with mean zero will be plotted, and any observed proportions overplotted using obs will use data from all items. Default is item=None.
anchor	Boolean: If True, anchored parameters are used; if False, unanchored parameters are used. Default is anchor=False.
facet_element	String: If a facet_element argument is passed, plots the category response curves as rated by the given facet element, adjusted by facet effect. If facet_element=None, plots a neutral category response curves for a facet element with zero facet effect. Default is facet_element=None.
obs	List: List of integers between 0 and self.max_score. For each value, mean observed proportions in each ordered response category scoring in that category are plotted against the mean location of the corresponding response class.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
thresh_lines	Boolean: If True, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is thresh_lines=False.

Continued on next page...

central_location	Boolean: If True, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 6.2.1). Default is central_location=False.
cat_highlight	Integer: Passing a score between 0 and self.max_score will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is cat_highlight=None (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns
matplotlib image. If a filename argument is passed, also saves the image to file.

Examples
To plot basic unanchored category response curves under the global facet element representation, rated by facet element Rater_1, and store the output as a variable my_crcs_plot and save it to file as my_crcs_plot.png:

```
my_crcs_plot = self.crcs_global(facet_element='Rater_1', filename=my_crcs_plot)
```

To plot anchored category response curves for Item_1 under the threshold facet element representation, rated by facet element Rater_1, with observed response proportions for category 1 for 8 response classes and store the output as a variable my_crcs_plot:

```
my_crcs_plot = self.crcs_thresholds(item='Item_1', anchor=True, facet_element='Rater_1', obs=[1],
                                    no_of_classes=8)
```

To plot unanchored category response curves for Item_1 under the matrix facet element representation, rated by a neutral 'zero facet effect' facet element, with a threshold line and highlighted zero category, and store the output as a variable my_crcs_plot:

```
my_crcs_plot = self.crcs_matrix(item='Item_1', thresh_lines=True, cat_highlight=0)
```

6.5.4 threshold_ccs

Description

Plots the threshold characteristic curves (or threshold response functions)for an item. For threshold τ_k, $k \in \{1, ..., \texttt{self.max_score}\}$, the threshold characteristic curve is the probability of obtaining a score of k rather than $k-1$, conditional on the score being either $k-1$ rather than k, for a given person location. Each threshold characteristic curve functions as a dichotomous item characteristic curve under the SLM (see Sections 3.2.2 and 3.5.2).

For threshold τ_k, `threshold_ccs` plots person location against the probability of obtaining a score of k. Options to plot threshold lines and central item locations, and to highlight a specified response category.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.threshold_ccs_global(item=None, anchor=False, facet_element=None, obs=None,
                          no_of_classes=5, thresh_lines=False,
                          central_location=False, cat_highlight=None)
```

(and equivalently `_items`, `_thresholds`, `_matrix`, `_bivector`, `_centrality`, `_pseudo_halo`, `_bistretch`), or equivalently `self.threshold_ccs(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

`item`	String: The name of the item to plot. If `item=None`, the shared set of Rasch-Andrich thresholds with mean zero plotted, and any observed proportions overplotted using `obs` will use data from all items. Default `item=None`.
`anchor`	Boolean: If `True`, anchored parameters are used; if `False`, unanchored parameters are used. Default is `anchor=False`.
`facet_element`	String: If a `facet_element` argument is passed, plots the category response curves for the item as rated by the given facet element. If `facet_element=None`, plots neutral category response curves for a facet element with zero facet effect. Default is `facet_element=None`.
`obs`	List: List of integers corresponding to thresholds τ_1 to τ_m, where $m =$`self.max_score`, or `'all'` or `'none'`. If `obs=[k]`, mean proportions of persons obtaining a score of k rather than $k-1$, conditional on the score being either $k-1$ or k, for each of the ordered response categories, will be plotted against the mean location of the corresponding response class. Multiple thresholds may be passed. Default is `obs=None`.
`no_of_classes`	Integer: The number of observed response categories. Default is `no_of_classes=5`.

Continued on next page...

thresh_lines	Boolean: If True, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is thresh_lines=False.
central_location	Boolean: If True, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 6.2.1). Default is central_location=False.
cat_highlight	Integer: Passing 0 or 1 will highlight the range of locations for which the selected score is the most probable response (all locations to one side of the item location. Default is cat_highlight=None (no category highlighted).

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns
matplotlib image. If a filename argument is passed, also saves the image to file.

Examples
To plot basic unanchored threshold characteristics curves under the global facet element representation, rated by Rater_1, and store the output as a variable my_threshold_ccs_plot, and save it to file as my_threshold_ccs_plot.png:

```
my_threshold_ccs_plot = self.threshold_ccs_global(facet_element='Rater_1', filename='my_iic_plot')
```

To plot anchored threshold characteristics curves for Item_1 under the item facet element representation, for a neutral 'zero' facet element with threshold lines and category 1 highlighted, and store the output as a variable my_threshold_ccs_plot:

```
my_threshold_ccs_plot = self.threshold_ccs_items(item='Item_1', anchor=True,
                                                 thresh_lines=True, cat_highlight=1)
```

To plot unanchored threshold characteristics curves for Item_1 under the matrix facet element representation, rated by Rater_1, with observed responses plotted for thresholds 2 and 4 and central item location line, and store the output as a variable my_threshold_ccs_plot:

```
my_threshold_ccs_plot = self.threshold_ccs_matrix(item='Item_1', facet_element='Rater_1',
                                                  obs=[2, 4], central_location=True)
```

6.5.5 iic

Description

Plots the item information curve for an item: person location on the x-axis against Fisher information on the y-axis. Options to plot item threshold line and lines showing Fisher information corresponding to specified locations, and to highlight a specified response category.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.iic_global(item, anchor=False, facet_element=None, ymax=None,
                thresh_lines=False, central_location=False, point_info_lines=None,
                point_info_labels=False, cat_highlight=None)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently `self.iic(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

item	String: The name of the item(s) to plot.
anchor	Boolean: If True, anchored parameters are used; if False, unanchored parameters are used. Default is anchor=False.
facet_element	String: If a facet_element argument is passed, plots the item information curve for the item as rated by the given facet element. If facet_element=None, plots a neutral item information curve for a facet element with zero facet effect. Default is facet_element=None.
ymax	Float: The maximum value to show on the y-axis. If None, will infer, plotting a maximum of 1.1 times the maximum item information. Default is ymax=None
thresh_lines	Boolean: If True, vertical lines showing the uncentred thresholds between each response category will be plotted. Threshold τ_k is the person location for which the scores $k-1$ and k are equally probable. Default is thresh_lines=False.
central_location	Boolean: If True, a vertical line showing the threshold corresponding to the central item location (the mean of the uncentred thresholds – see Section 6.2.1). Default is central_location=False.
point_info_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the Fisher information corresponding to the location.

Continued on next page...

point_info_labels	Boolean: If True, locations and Fisher information corresponding to arguments passed to point_info_lines will be labelled on the plot. Default is point_info_labels=False.
cat_highlight	Integer: Passing a score between 0 and self.max_score will highlight the range of locations for which the selected score is the most probable response. If the data has disordered thresholds (Andrich, 2010; Pallant & Tennant, 2007) and response category selected is never the most probable, no area will be highlighted. Default is cat_highlight=None (no category highlighted).
ymax	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns
matplotlib image. If a filename argument is passed, also saves the image to file.

Examples
To plot a basic unanchored item information curve for Item_1 under the global facet element representation, rated by Rater_1, and store the output as a variable my_iic_plot and save it to file as my_iic_plot.png:

```
my_iic_plot = self.iic_global('Item_1', facet_element='Rater_1', filename='my_iic_plot')
```

To plot an anchored item information curve for Item_1 under the item facet element representation, for a neutral 'zero' facet element with threshold lines, central item location line and category 1 highlighted, and store the output as a variable my_iic_plot:

```
my_iic_plot = self.iic_items('Item_1', anchor=True, thresh_lines=True, central_location=True,
                             cat_highlight=1)
```

To plot an unanchored item information curve for Item_1 under the matrix facet element representation, rated by Rater_1, with lines showing the Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding Fisher information labelled, and store the output as a variable my_iic_plot:

```
my_iic_plot = self.icc_matrix('Item_1', point_info_lines=[-0.3, 0.7],
                              point_info_labels=True)
```

6.5.6 tcc

Description

Plots the test characteristic curve (or test response function) for a set of items: person location on the x-axis against expected score on the y-axis. Options to plot observed responses and lines showing locations corresponding to specified expected scores.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.tcc_global(anchor=False, items=None, facet_elements=None, obs=False,
                no_of_classes=5, score_lines=None, score_labels=False)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently self.tcc(model='global', ...) with model set to any of the eight representation names.

Arguments

anchor	Boolean: If True, anchored parameters are used; if False, unanchored parameters are used. Default is anchor=False.
items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
facet_elements	List: The names of the facet elements to be used in the plot. facet_elements='all' uses all facet elements. Default is facet_elements=None, which generates a location based on a single neutral 'zero' facet element. If multiple facet elements are selected, the maximum score will be the sum of the maximum scores across facet elements.
obs	Boolean: If True, mean observed scores for each of the ordered response categories will be plotted against the mean location of the corresponding response class. Default is obs=False.
no_of_classes	Integer: The number of observed response categories. Default is no_of_classes=5.
score_lines	List of floats between 0 and self.max_score: Each float in the list represents an expected score, for which a horizontal line from the y-axis to where it intersects the item characteristic curve will be plotted, and from there a vertical line down to the x-axis will be plotted to show the location corresponding to the expected score. Default is score_lines=None.
score_labels	Boolean: If True, scores and locations corresponding to arguments passed to score_lines will be labelled on the plot. Default is score_labels=False.

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns
matplotlib image. If a `filename` argument is passed, also saves the image to file.

Examples
To plot a basic unanchored test characteristic curve for all items under the global facet element representation for a neutral 'zero' facet element and store the output as a variable `my_tcc_plot`, and save it to file as `my_tcc_plot.png`:

```
my_tcc_plot = self.tcc_global(filename=my_tcc_plot)
```

To plot a test characteristic curve for `Item_1` for a subset of items stored as a list `my_item_list` under the threshold facet element representation for facet element `Rater_1`, with observed responses for 8 response classes and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = self.tcc_thresholds(obs=True, no_of_classes=8)
```

To plot a test characteristic curve for `Item_1` for all items under the matrix facet element representation for all facet elements, with lines showing the locations corresponding to expected scores of 13 and 20, with the expected score and corresponding location labelled, and store the output as a variable `my_tcc_plot`:

```
my_tcc_plot = self.tcc_matrix(facet_elements='all', score_lines=[13, 20], score_labels=True)
```

6.5.7 test_info

Description

Plots the test information curve: person location on the x-axis against total Fisher information on the y-axis. Option to plot lines showing Fisher information corresponding to specified locations.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.test_info_global(anchor=False, items=None, facet_elements=None, ymax=None,
                      point_info_lines=None, point_info_labels=False)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently self.test_info(model='global', ...) with model set to any of the eight representation names.

Arguments

anchor	Boolean: If True, anchored parameters are used; if False, unanchored parameters are used. Default is anchor=False.
items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
facet_elements	List: The names of the facet elements to be used in the plot. facet_elements='all' uses all facet elements. Default is facet_elements=None, which generates a location based on a single neutral 'zero' facet element. If multiple facet elements are selected, the test information will be the sum of the information across facet elements.
point_info_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the item information curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the total Fisher information corresponding to the location. Default is point_info_lines=None.
point_info_labels	Boolean: If True, locations and total Fisher information corresponding to arguments passed to point_info_lines will be labelled on the plot. Default is point_info_labels=False.
ymax	The maximum point displayed on the y-axis, in Fisher information.

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic anchored test information curve under the global facet element representation using all items and a neutral 'zero' facet element, storing the output as a variable `my_test_info_plot` and saving it to file as `my_test_info_plot.png`:

```
my_test_info_plot = self.test_info_global(anchor=True, filename='my_test_info_plot')
```

To plot an unanchored test information curve under the item facet element representation using a subset of items stored as a list **`my_item_list`** and `Rater_1`, storing the output as a variable `my_test_info_plot`:

```
my_test_info_plot = self.test_info_items(items=my_item_list, facet_elements='Rater_1')
```

To plot an anchored test information curve under the matrix facet element representation with lines showing the total Fisher information corresponding to locations of -0.3 and 0.7, with the location and corresponding total Fisher information labelled, and store the output as a variable `my_test_info_plot`:

```
my_test_info_plot = self.test_info_matrix(anchor=True, point_info_lines=[-0.3, 0.7],
                                          point_info_labels=True)
```

6.5.8 test_csem

Description

Plots the test conditional standard error of measurement (CSEM) curve: person location on the x-axis against CSEM (in logits) on the y-axis. Option to plot lines showing CSEM corresponding to specified locations.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.test_csem_global(anchor=False, items=None, facet_elements=None, point_csem_lines=None,
                      point_csem_labels=False, ymax=5)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently self.test_csem(model='global', ...) with model set to any of the eight representation names.

Arguments

anchor	Boolean: If True, anchored parameters are used; if False, unanchored parameters are used. Default is anchor=False.
items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
facet_elements	List: The names of the facet elements to be used in the plot. Default is facet_elements=None, which generates a location based on a single neutral 'zero' facet element. If multiple facet elements are selected, the CSEM will be based on the sum of the information across facet elements.
point_csem_lines	List of floats: Each float in the list represents a location, for which a vertical line from the x-axis to where it intersects the CSEM curve will be plotted, and from there a horizontal line across to the y-axis will be plotted to show the CSEM corresponding to the location. Default is point_csem_lines=None.
point_csem_labels	Boolean: If True, locations and CSEM corresponding to arguments passed to point_csem_lines will be labelled on the plot. Default is point_csem_labels=False.
ymax	The maximum point displayed on the y-axis, in logits.

Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot a basic anchored CSEM curve for all items under the global facet element representation for a neutral 'zero' facet element, storing the output as a variable `my_test_csem_plot` and saving it to file as `my_test_csem_plot.png`:

```
my_test_csem_plot = self.test_csem_global(anchor=True, filename='my_test_csem_plot')
```

To plot an unanchored CSEM curve under the item facet element representation using a subset of items stored as a list `my_item_list` and `Rater_1`, storing the output as a variable `my_test_csem_plot`:

```
my_test_csem_plot = self.test_csem_items(items=my_item_list, facet_elements='Rater_1')
```

To plot an anchored CSEM curve under the matrix facet element representation with lines showing the CSEM corresponding to locations of -0.3 and 0.7, with the location and corresponding CSEM labelled, and store the output as a variable `my_test_csem_plot`:

```
my_test_csem_plot = self.test_csem_matrix(anchor=True, point_info_lines=[-0.3, 0.7],
                                          point_info_labels=True)
```

6.5.9 std_residuals_plot

Description

Plots histogram of standardised residuals, with optional overplotting of standard Normal distribution.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.std_residuals_plot_global(items=None, facet_elements=None, bin_width=0.5, normal=False,
                               dpi=300)
```

(and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch), or equivalently `self.std_residuals_plot(model='global', ...)` with `model` set to any of the eight representation names.

Arguments

items	List: The names of the items to be used in the plot. If None, the full set of items will be used. Default is items=None.
facet_elements	List: The names of the facet elements to be used in the plot.If None, the full set of facet elements will be used. Default is facet_elements=None.
bin_width	Float: Width of the histogram bins along x-axis. Default is bin_width=0.5.
dpi	Integer: resolution of the saved plot in dpi (dots per inch). Default is dpi=300.
normal	Boolean: If True, plots a standard normal distribution over the standardised residual histogram for comparison. Default is normal=False.

Since the anchoring process does not affect residuals, there is no anchor argument for std_residuals_plot. Additional arguments to customise the appearance of the plot are detailed in Section 6.5.1.

Returns

matplotlib image. If a filename argument is passed, also saves the image to file.

Examples

To plot and display a basic standardised residuals histogram under the global facet element representation for all items and facet elements, saving it to file as my_std_residuals_plot.png:

```
self.std_residuals_plot_global(filename='my_std_residuals_plot')
```

To plot and display a standardised residuals histogram under the threshold facet element representation for all items and facet element 'Rater_1', with bin width 1 and a standard normal curve:

```
self.std_residuals_plot_thresholds(facet_element='Rater_1', bin_width=1, normal=True)
```

To plot and display a standardised residuals histogram under the matrix facet element representation on a subset of items stored as a list in a variable `my_item_list`, and all facet elements:

```
self.std_residuals_plot_matrix(items=my_item_list)
```

6.5.10 wright_map

Description

Plots a Wright map (person-item map) extended with a third dimension for the calibration facet (e.g. raters): back-to-back person and item distributions on a shared axis, plus a facet-element location panel alongside (its own panel by default, or overlaid onto the items panel). Persons and items behave identically to RSM's/PCM's own wright_map (Sections 5.5.10/4.5.10), including the divided category-strip item display. The facet is whatever self.facet aliases to ('rater' by default). wright_map does not use the shared plotting arguments described in Section 6.5.1.

Usage

Eight methods are defined, one for each facet representation; all methods share the same argument format:

```
self.wright_map_global(person_names=None, item_names=None,
                       facet_element_names=None, matrix_expansion=False,
                       show_stretch=True, item_level='items',
                       threshold_type='rasch-andrich',
                       orientation='vertical', map_type='hist',
                       item_labels=False, item_strip=False, item_distribution=False,
                       facet_labels=False, overlay_facet=False, strip_thickness=2.0,
                       strip_alpha=0.3, sort='location', palette='colorblind multi',
                       neutral_extremes=False, item_row_height=None,
                       facet_row_height=None, narrow_font=False, edge_padding=0.5,
                       distribution_markers=False, group_by=None, group_colors=None,
                       stack=True, blend=False, prop=False, person_lim=None, item_lim=None,
                       facet_lim=None, person_scaling=None, facet_scaling=1,
                       no_of_bins=20, pad=False, plot_range=None, kde_points=500,
                       bw_method='scott', line_width=1, figsize=None, title=None,
                       plot_style='white', edge_color='black',
                       person_color='skyblue', item_color='salmon',
                       facet_color='mediumseagreen',
                       person_line_color='black', item_line_color='black',
                       facet_line_color='black', marker_color=None,
                       alpha=0.6, black=False, font='Times New Roman',
                       title_font_size=15, axis_font_size=12, labelsize=12,
                       item_label_size=8, facet_label_size=8, tick_interval=1,
                       filename=None, file_format='png', dpi=300)
```

and equivalently _items, _thresholds, _matrix, _bivector, _centrality, _pseudo_halo, _bistretch, or self.wright_map(model, ...) with model etc. Unlike the other plotting methods in this section, model is a required positional argument on the base wright_map call, with no default.

Arguments

`model`	String: One of `'global'`, `'items'`, `'thresholds'`, `'matrix'`, `'bivector'`, `'centrality'`, `'pseudo_halo'` or `'bistretch'`. For the vector/matrix models, each individual element of the facet element representation is plotted, resulting in multiple points for each facet element. For the stretch models, the global shift is plotted in an identical fashion to the `'global'` model, with the stretch parameters plotted in a separate panel since the axis is not shared (unless `expand_model=True` is passed, in which case the stretch model is resolved to its corresponding vectorised representation – see below).
`person_names`	String, list or `None`: Person subset to include. Default is `person_names=None` (all persons).
`item_names`	String, list or `None`: Item subset to include. Default is `item_names=None` (all items).
`facet_element_names`	String, list or `None`: Facet-element subset to include (e.g. specific facet elements). Default is `facet_element_names=None` (all elements of the facet).
`expand_model`	Boolean: Only used when `model` is `'bivector'` or one of the stretch models; silently ignored otherwise. If `True`, plots the full resolved matrix instead of the two vectors for `'bivector'`, or the corresponding resolved vector(s) for stretch models.
`show_stretch`	Boolean: Only used for the stretch models (`centrality`, `pseudo_halo`, `bistretch`); silently ignored otherwise. Adds panel(s) with histograms showing ω (each facet element's stretch factor(s)), centred on 1 rather than 0. Default is `show_stretch=True`.
`item_level`	String: `'items'` plots one point per item at its central location. `'thresholds'` plots one point per Rasch-Andrich threshold instead. Ignored when `item_strip=True`. Default is `item_level='items'`.
`threshold_type`	String, one of `'rasch-andrich'` (default), `'thurstonian'` or `'half-point'`: which threshold convention to plot when `item_level='thresholds'`. `'rasch-andrich'` plots the estimated Rasch-Andrich thresholds, which can be disordered); `'thurstonian'` plots the median threshold ($P(X \geq k) = 0.5$); `'half-point'` plots the location where $E[X] = k - 0.5$. `'thurstonian'` and `'half-point'` are always ordered. Both are computed from the items and thresholds without facet effects. Also governs `item_strip=True` category-boundary segments. Ignored when `item_level='items'`.

Continued on next page...

orientation	String: 'vertical' places location on the x-axis, with persons above the baseline; 'horizontal' places location on the y-axis, with persons to the left of the baseline. Default is orientation='vertical'.
map_type	String: 'hist' plots back-to-back histograms; 'kde' plots kernel density estimate curves instead. If item_labels=True, only governs the person side; the item side becomes a label panel rather than a distribution. Also applies to facets when facet_labels=False. Default is map_type='hist'.
item_labels	Boolean: If True, item names are listed instead of a bare histogram, grouped into no_of_bins histogram-style location bins and ordered by location. Default is item_labels=False.
item_strip	Boolean: If True, draws each item as a strip divided into coloured category segments showing the most probable category for a given person location, with a boundary line at each threshold. Forces item_labels=True. Default is item_strip=False.
item_distribution	Boolean: If True, plots a histogram/KDE plot as well as item labels. Forces item_labels=True. Default is item_distribution=False.
facet_labels	Boolean: If True, facet names are listed instead of a bare histogram, grouped into no_of_bins histogram-style location bins and ordered by location. Forces overlay_facet=False. Default is facet_labels=False.
overlay_facet	Boolean: If True, the facets are overlaid onto the items rather than plotted separately. Forced to False if item_labels=True and/or facet_labels=True. Default is overlay_facet=False.
strip_thickness	Float: Controls the height (vertical orientation) or width (horizontal orientation) of category strips as a multiplier of the base thickness when item_strip=True. Default is strip_thickness=2.0.
strip_alpha	Float: Fill transparency for strips when item_strip=True. Default is strip_alpha=0.3.
sort	String: Ordering of item strip rows. 'location' sorts by the lowest threshold; 'order' orders according to self.items. Only used when item_strip=True. Default is sort='location'.
palette	String or None: Named colour palette for category strips (see Section 6.5.1). None uses a flat item_color fill with no distinction between categories. Only used when item_strip=True. Default is palette='colorblind multi'.

Continued on next page...

neutral_extremes	Boolean: The strip's open (unbounded) top and bottom categories always draw out to the location axis limits – there is no real boundary to stop short at. If True, they fade from a neutral cream colour toward the real threshold instead of using a flat fill in the category palette, de-emphasising the open-ended extremes rather than drawing the eye to an arbitrary width. Only used when item_strip=True. Default is neutral_extremes=False.
item_row_height	Float or None: Physical height, in inches, allocated per stacked item-label row. If None, derived automatically. Only used when item_labels=True. Default is item_row_height=None.
facet_row_height	Float or None: Physical height, in inches, allocated per stacked facet-label row. If None, derived automatically. Only used when facet_labels=True. Default is facet_row_height=None.
narrow_font	Boolean: When item_labels=True and/or facet_labels=True, if True then item labels, facet labels, and (with item_strip=True) row labels beside category strips are set in 'Arial Narrow' instead of font to save space. Falls back to Matplotlib's font substitution if 'Arial Narrow' is not installed. Default is narrow_font=False.
edge_padding	Float: Extra padding (in inches) added around item-label margin. The same value produces matching-looking margins in both orientations. Only used when item_labels=True. Default is edge_padding=0.5.
distribution_markers	Boolean: If True, overlays mean and plus/minus 1 and 2 standard deviation reference markers, labelled μ, $\mu \pm \sigma$, $\mu \pm 2\sigma$ for persons and items. Default is distribution_markers=False.
group_by	pandas Series or None: Grouping of persons (e.g. a DIF group) used to split the person distribution into sub-distributions. Default group_by=None (persons as a single series).
group_colors	Dict or None: Controls colours for group_by. If None, colours are drawn from the 'tab10' colour map. Default is group_colors=None.
stack	Boolean: When group_by is set, for map_type='hist', True cumulatively stacks histogram bars while False draws separate columns per group: dodged, or alpha-blended if blend=True. For map_type='kde', True cumulatively stacks group KDE plots; False overlays group KDE plots. Default is stack=True.
blend	Boolean: When group_by is set, map_type='hist' and stack=False, if True, blends histogram bars using colour instead of dodging them (showing them side-by-side). Default is blend=False.

Continued on next page...

prop	Boolean: If True, plots distributions as proportions/density rather than counts. Default is prop=False.
person_lim	Float or None: Controls axis limit for person distribution. If None, derived automatically from the distribution. Default is person_lim=None.
item_lim	Float or None: Controls axis limit for item distribution. If None, derived automatically from the distribution. Default is item_lim=None.
facet_lim	Float or None: Controls axis limit for facet distribution. If None, derived automatically from the distribution. Default is facet_lim=None.
person_scaling	Float or None: Adjusts the scaling of the person distribution for clarity, particularly when the number of persons is very large compared to items/facet elements. A scaling of 10 means that the person scale is 10 times the item scale, and so on. Default is person_scaling=None, which chooses the largest of 1, 2, 5, 10, 20, 50, ... that keeps the rescaled item (and facet) distribution within half the height of the person distribution, and never below 1.
facet_scaling	Float: Adjusts the scaling of the facet distribution for clarity, particularly when the number of facet elements is very large compared to items elements, for example with the matrix model. A scaling of 10 means that the facet scale is 10 times the item scale, and so on. Default is facet_scaling=1.
no_of_bins	Integer: Number of histogram or label bins. Default is no_of_bins=20.
pad	Boolean: If True, adds 5% padding to both ends of the location axis limits. If False, the location axis is bound to the plotted range. Default is pad=False.
plot_range	Tuple of two floats, or None: (lo, hi) limits for the location axis. If None, derived automatically. Default is plot_range=None.
kde_points	Integer: Number of points at which each KDE curve is evaluated when map_type='kde'. Default is kde_points=500.
bw_method	String, scalar or callable: Bandwidth selection method passed to scipy.stats.gaussian_kde when map_type='kde'. Default is bw_method='scott'.
line_width	Float: Line width of KDE curves when map_type='kde'. Default is line_width=1.
figsize	Tuple of two floats, or None: Base figure size in inches for the person distribution. If None, defaults to (8, 6) for orientation='vertical' or (4, 8) for orientation='horizontal'. Default is figsize=None.

Continued on next page...

title	String or None: Plot title. Default is title=None (no title).
plot_style	String: Plot background style, as per plot_style in Section 6.5.1. Default is plot_style='white'.
edge_color	String: Edge colour of histogram bars when map_type='hist' and group_by is None. Default is edge_color='black'.
person_color	String: Fill colour for person distribution; ignored when group_by is set, overridden by a grey shade when black=True. Default is person_color='skyblue'.
item_color	String: Fill colour for item distribution when item_labels=False; overridden by a grey shade when black=True. Default is item_color='salmon'.
facet_color	String: Fill colour for facet distribution when facet_labels=False; overridden by a grey shade when black=True. Default is facet_color='mediumseagreen'.
person_line_color	String: Line colour for person KDE plot when map_type='kde' and group_by is None. Default is person_line_color='black'.
item_line_color	String: Colour for item labels (item_labels=True) or item KDE plot line (item_labels=False, map_type='kde'). Default is item_line_color='black'.
facet_line_color	String: Colour for facet labels (facet_labels=True) or facet KDE plot line (facet_labels=False, map_type='kde'). Default is facet_line_color='black'.
marker_color	String, tuple or None: Colour of tick marks and labels drawn when distribution_markers=True. A single colour applies to both the person and item marks; a (person, item) pair sets them separately. Person marks use it only when group_by=None; grouped marks use the assigned group colours. Default is marker_color=None, which gives each set of marks a darker shade of its own distribution's colour (person_color or item_color).
alpha	Float: Fill transparency for distributions. Default is alpha=0.6.
black	Boolean: If True, renders person_color/item_color/facet_color as grey shades instead. Has no effect on group_by colours. Default is black=False.
font	String: Plot font. Default font='Times New Roman'.
title_font_size	Float: Title font size in points. Default title_font_size=15.
axis_font_size	Float: Axis label font size in points. Default axis_font_size=12.

Continued on next page...

labelsize	Float: Axis tick label font size in points. Default labelsize=12.
item_label_size	Integer: Item name font size in points. Default item_label_size=8.
facet_label_size	Integer: Facet name font size in points. Default facet_label_size=8.
tick_interval	Float: Spacing between Location axis ticks. Default tick_interval=1.
filename	String or None: Filename for saved plot, with no suffix for format. If None, no file is saved. Default filename=None.
file_format	String: Format of saved file, 'png', 'jpg' or 'svg'. Default file_format='png'.
dpi	Integer: Resolution of saved plot in dpi (dots per inch). Default dpi=300.

Returns
matplotlib figure. Displays and closes the figure; if a filename argument is passed, also saves the image to file.

Examples
To plot a basic Wright map under the global model, with items and persons as back-to-back histograms:

```
self.wright_map_global()
```

To plot a Wright map under the matrix model, with each item shown as a divided category strip, facet elements listed by name, and distribution reference markers for persons, and save to file as my_wright_map.png:

```
self.wright_map_matrix(item_strip=True, facet_labels=True,
                       distribution_markers=True, title='MFRM Wright Map',
                       filename='my_wright_map')
```

To plot a Wright map under the centrality model, with facet elements listed by name and ω (threshold-stretch) panel added:

```
self.wright_map_centrality(facet_labels=True, show_stretch=True)
```

7 class SLM_Sim

7.1 Generating an SLM_Sim simulation

7.1.1 SLM_Sim

Description

Creates an object of the class SLM_Sim. Simulates response data fitting the simple logistic model (SLM) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
SLM_Sim(no_of_items, no_of_persons, item_range=3, person_sd=1.5, offset=0, missing=0,
        manual_persons=None, manual_items=None, manual_person_names=None,
        manual_item_names=None, seed=None,
        missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
item_range	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is item_range=3. Ignored if specified item locations are passed using the manual_items argument.
person_sd	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is person_sd=1.5. Ignored if specified person locations are passed using the manual_persons argument.
offset	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data in the simulation. If missing is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of missing of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly missing. Default is missing=0.

Continued on next page...

manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control the parameters of the simulation. manual_persons=None, random person locations are generated. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified item locations. Allows the user to control the parameters of the simulation. manual_items=None, random item locations are generated. Default is manual_items=None.
manual_person_names	List: Optional list of person names specified by user. If manual_person_names=None, person names are generated automatically. Default is manual_person_names=None.
manual_item_names	List: Optional list of item names specified by user. If manual_item_names=None, item names are generated automatically. Default is manual_item_names=None.
seed	Integer or None: seed for the random number generator. Pass an integer for a fully reproducible simulation; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing. Default is missing_method='bernoulli'.

Returns

Object of class SLM_Sim. Several attributes of object SLM_Sim are automatically generated on its creation:

self.responses	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an SLM object for analysis.
self.no_of_items	Integer: The number of items, specified by argument no_of_items.
self.no_of_persons	Integer: The number of persons, specified by argument no_of_persons.
self.item_range	Float: The generating range of item locations, specified by argument item_range.
self.person_sd	The generating standard deviation of person locations, specified by argument person_sd.
self.offset	The generating offset between the means of person location and item location, specified by argument offset.
self.missing	The generating proportion of missing data, specified by argument missing

Continued on next page...

self.missing_method	The method used to apply missing data, specified by argument missing_method
self.persons	pandas series: Person names as keys and person locations as values.
self.items	pandas series: keys are item names and values are item locations.
self.person_names	List: Person names. If no person names are passed using the manual_person_names argument, default person names are generated in the format Person_1 etc.
self.item_names	List: Item names. If no item names are passed using the manual_item_names argument, default item names are generated in the format Item_1 etc.

Examples

To create an SLM_Sim object called my_slm_sim with randomly generated person locations for 10000 persons and randomly generated item locations for 50 items, specifying an item range of 4 logits, a person SD of 2 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_slm_sim = SLM_Sim(50, 10000, item_range=4, person_sd=2, offset=1, missing=0.3)
```

To create an SLM_Sim object called my_slm_sim with specified person locations saved as a variable named my_person_abils, 50 items with specified item locations saved as a variable named my_item_diffs, 100 persons with specified person names saved as a variable named my_person_names and specified item names saved as a variable named my_item_names, with no missing data:

```
my_slm_sim = SLM_Sim(50, 100, manual_persons=my_person_abils, manual_items=my_item_diffs,
                     manual_person_names=my_person_names, manual_item_names=my_item_names)
```

7.2 Customising an SLM_Sim simulation

7.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_slm from Item_1 to my_new_item_name:

```
my_slm.rename_item('Item_1', 'my_new_item_name')
```

7.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_slm with item names in a list stored as a variable my_new_item_names:

```
my_slm.rename_items_all(my_new_item_names)
```

7.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the index of `self.responses` with new name. Names are also replaced in the generating parameters (e.g. `self.items`, `self.persons`) and the response probabilities.

Example
To rename a person in object `my_slm` from `Person_1` to `my_new_person_name`:

```
my_slm.rename_person('Person_1', 'my_new_person_name')
```

7.2.4 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the index of `self.responses` with new names. Names are also replaced in the generating parameters (e.g. `self.items`, `self.persons`) and the response probabilities.

Example
To rename all persons in object `my_slm` with person names in a list stored as a variable `my_new_person_names`:

```
my_slm.rename_persons_all(my_new_person_names)
```

8 class PCM_Sim

8.1 Generating a PCM_Sim simulation

8.1.1 PCM_Sim

Description

Creates an object of the class PCM_Sim. Simulates response data fitting the partial credit model (PCM) (Masters, 1982) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
PCM_Sim(no_of_items, no_of_persons, max_score_vector, item_range=3, category_base=2,
        person_sd=1.5, max_disorder=0, offset=0, missing=0, manual_persons=None,
        manual_items=None, manual_thresholds=None, manual_person_names=None,
        manual_item_names=None, seed=None,
        missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
max_score_vector	List or 1-dimensional numpy array of integers: Maximum score for each item.
item_range	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is item_range=3. Ignored if specified item locations are passed using the manual_items argument.
category_base	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between max_disorder (see below) and twice category_base minus max_disorder. Default is category_base=2.
person_sd	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is person_sd=1.5. Ignored if specified person locations are passed using the manual_persons argument.
max_disorder	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around category_base. Default is max_disorder=0, with no disordered thresholds.

Continued on next page...

offset	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data in the simulation. If missing is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of missing of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly missing. Default is missing=0.
manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control the parameters of the simulation. manual_persons=None, random person locations are generated. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified item locations. Allows the user to control the parameters of the simulation. manual_items=None, random item locations are generated. Default is manual_items=None.
manual_thresholds	List of lists, or list of 1-dimensional numpy arrays: Optional set of specified Rasch-Andrich thresholds, one list per item (in the same order as the items), each of length max_score_vector's corresponding entry and summing to zero. Allows the user to control the parameters of the simulation. If manual_thresholds=None, random Rasch-Andrich thresholds are generated independently for each item. Default is manual_thresholds=None.
manual_person_names	List: Optional list of person names specified by user. If manual_person_names=None, person names are generated automatically. Default is manual_person_names=None.
manual_item_names	List: Optional list of item names specified by user. If manual_item_names=None, item names are generated automatically. Default is manual_item_names=None.
seed	Integer or None: seed for the random number generator. Pass an integer for a fully reproducible simulation; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing. Default is missing_method='bernoulli'.

Returns

Object of class PCM_Sim. Several attributes of object PCM_Sim are automatically generated on its creation:

self.responses	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an PCM object for analysis.
self.no_of_items	Integer: The number of items, specified by argument no_of_items.
self.no_of_persons	Integer: The number of persons, specified by argument no_of_persons.
self.item_range	Float: The generating range of item locations, specified by argument item_range.
self.max_score_vector	List or 1-dimensional numpy array of integers: List with the maximum possible score for each item.
self.person_sd	The generating standard deviation of person locations, specified by argument person_sd.
self.category_base	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument category_base.
self.max_disorder	The generating maximum permitted threshold disorder, specified by argument max_disorder.
self.offset	The generating offset between the means of person location and item location, specified by argument offset.
self.missing	The generating proportion of missing data, specified by argument missing
self.missing_method	The method used to apply missing data, specified by argument missing_method
self.persons	pandas series: Person names as keys and person locations as values.
self.items	pandas series: keys are item names and values are item locations.
self.thresholds	pandas dataframe: one row per item, one column per threshold index. Since items may have different maximum scores, rows are padded with NaN beyond that item's own max_score_vector entry.
self.person_names	List: Person names. If no person names are passed using the manual_person_names argument, default person names are generated in the format Person_1 etc.
self.item_names	List: Item names. If no item names are passed using the manual_item_names argument, default item names are generated in the format Item_1 etc.

Example

To create a PCM_Sim object called my_pcm_sim with randomly generated person locations for 1000 persons and randomly generated item locations for 4 items, specifying maximum scores of 3, 3, 5 and 5, an item range of 4 logits, a person SD of 2 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_pcm_sim = PCM_Sim(4, 1000, max_score_vector=[3, 3, 5, 5], item_range=4,
                     person_sd=2, offset=1, missing=0.3)
```

To create an `PCM_Sim` object called `my_pcm_sim` with 1,000 persons with specified person locations saved as a variable named `my_person_abils`, 30 items with specified item locations saved as a variable named `my_item_diffs`, specified maximum scores saved as a variable named `my_max_score_vector`,specified person names saved as a variable named `my_person_names` and specified item names saved as a variable named `my_item_names`, with no missing data:

```
my_pcm_sim = PCM_Sim(30, 1000, max_score_vector=my_max_score_vector,
                     manual_persons=my_person_abils, manual_items=my_item_diffs,
                     manual_person_names=my_person_names, manual_item_names=my_item_names)
```

8.2 Customising an PCM_Sim simulation

8.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_slm from Item_1 to my_new_item_name:

```
my_slm.rename_item('Item_1', 'my_new_item_name')
```

8.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_slm with item names in a list stored as a variable my_new_item_names:

```
my_slm.rename_items_all(my_new_item_names)
```

8.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the index of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename a person in object my_slm from Person_1 to my_new_person_name:

```
my_slm.rename_person('Person_1', 'my_new_person_name')
```

8.2.4 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the index of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all persons in object my_slm with person names in a list stored as a variable my_new_person_names:

```
my_slm.rename_persons_all(my_new_person_names)
```

9 class RSM_Sim

9.1 Generating an RSM_Sim simulation

9.1.1 RSM_Sim

Description

Creates an object of the class RSM_Sim. Simulates response data fitting the rating scale model (RSM) (Andrich, 1978) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
RSM_Sim(no_of_items, no_of_persons, max_score, item_range=3, category_base=2, person_sd=1.5,
        max_disorder=0, offset=0, missing=0, manual_persons=None, manual_items=None,
        manual_thresholds=None, manual_person_names=None, manual_item_names=None,
        seed=None,
        missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
item_range	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is item_range=3. Ignored if specified item locations are passed using the manual_items argument.
category_base	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between max_disorder (see below) and twice category_base minus max_disorder. Default is category_base=2.
person_sd	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is person_sd=1.5. Ignored if specified person locations are passed using the manual_persons argument.
max_disorder	Float: If a negative value is passed, disordered Rasch-Andrich thresholds (Andrich, 2010; Pallant & Tennant, 2007) may be generated. A positive value may be passed, which controls the minimum category width and limits the variation in category widths around category_base. Default is max_disorder=0, which does not permit disordered thresholds.

Continued on next page...

offset	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data in the simulation. If missing is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of missing of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly missing. Default is missing=0.
manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control the parameters of the simulation. manual_persons=None, random person locations are generated. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control the parameters of the simulation. manual_items=None, random central item locations are generated. Default is manual_items=None.
manual_thresholds	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control the parameters of the simulation. manual_items=None, random Rasch-Andrich thresholds are generated. Default is manual_thresholds=None.
manual_person_names	List: Optional list of person names specified by user. If manual_person_names=None, person names are generated automatically. Default is manual_person_names=None.
manual_item_names	List: Optional list of item names specified by user. If manual_item_names=None, item names are generated automatically. Default is manual_item_names=None.
seed	Integer or None: seed for the random number generator. Pass an integer for a fully reproducible simulation; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing. Default is missing_method='bernoulli'.

Returns

Object of class RSM_Sim. Several attributes of object RSM_Sim are automatically generated on its creation:

self.responses	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an RSM object for analysis.
self.no_of_items	Integer: The number of items, specified by argument no_of_items.
self.no_of_persons	Integer: The number of persons, specified by argument no_of_persons.
self.item_range	Float: The generating range of item locations, specified by argument item_range.
self.max_score	Integer: The shared maximum possible score on each item, specified by argument max_score.
self.category_base	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument category_base.
self.person_sd	The generating standard deviation of person locations, specified by argument person_sd.
self.offset	The generating offset between the means of person location and item location, specified by argument offset.
self.missing	The generating proportion of missing data, specified by argument missing
self.missing_method	The method used to apply missing data, specified by argument missing_method
self.persons	pandas series: Person names as keys and person locations as values.
self.items	pandas series: keys are item names and values are item locations.
self.thresholds	pandas series: set of self.max_score Rasch-Andrich thresholds $\tau_1, \ldots, \tau_m$, indexed 1 to self.max_score. **Note (breaking change from earlier versions):** the leading sentinel $\tau_0 = 0$ has been removed.
self.person_names	List: Person names. If no person names are passed using the manual_person_names argument, default person names are generated in the format Person_1 etc.
self.item_names	List: Item names. If no item names are passed using the manual_item_names argument, default item names are generated in the format Item_1 etc.

Examples

To create an RSM_Sim object called my_rsm_sim with a maximum score of 5, randomly generated person locations for 5000 persons and randomly generated item locations for 8 items, specifying an item range of 4 logits, a base category width of 1.5 logits, a person SD of 2 logits, a maximum permitted threshold disorder of 0.5 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_rsm_sim = RSM_Sim(8, 5000, item_range=4, category_base=1.5, person_sd=2, max_disorder=0.5,
```

```
offset=1, missing=0.3)
```

To create an RSM_Sim object called my_rsm_sim with a maximum score of 5, specified person locations for 100 persons saved as a variable named my_person_abils, specified item locations for 12 items saved as a variable named my_item_diffs, a specified set of Rasch-Andrich thresholds saved as a variable named my_thresholds, specified person names saved as a variable named my_person_names and specified item names saved as a variable named my_item_names, with no missing data:

```
my_rsm_sim = RSM_Sim(12, 100, manual_persons=my_person_abils, manual_items=my_item_diffs,
                     manual_thresholds=my_thresholds, manual_person_names=my_person_names,
                     manual_item_names=my_item_names)
```

9.2 Customising an RSM_Sim simulation

9.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_rsm from Item_1 to my_new_item_name:

```
my_rsm.rename_item('Item_1', 'my_new_item_name')
```

9.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_rsm with item names in a list stored as a variable my_new_item_names:

```
my_rsm.rename_items_all(my_new_item_names)
```

9.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the index of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename a person in object my_rsm from Person_1 to my_new_person_name:

```
my_rsm.rename_person('Person_1', 'my_new_person_name')
```

9.2.4 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the index of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all persons in object my_rsm with person names in a list stored as a variable my_new_person_names:

```
my_rsm.rename_persons_all(my_new_person_names)
```

10 class MFRM_Sim_Global

10.1 Generating an MFRM_Sim_Global simulation

10.1.1 MFRM_Sim_Global

Description

Creates an object of the class MFRM_Sim_Global. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the global facet element (e.g. rater) representation (Linacre, 1994) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
MFRM_Sim_Global(no_of_items, no_of_persons, max_score,
                no_of_facet_elements, item_range=2,
                facet_range=2, category_base=2, person_sd=1.5, max_disorder=0, offset=0,
                missing=0, shared_missing=True, manual_persons=None, manual_items=None,
                manual_thresholds=None, manual_facet_effects=None, manual_person_names=None,
                manual_item_names=None, manual_facet_names=None, facet=None,
                facet_plural=None, seed=None,
                missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
max_score	Integer: The maximum score on each item.
no_of_facet_elements	Integer or None: The number of facet elements in the simulation.
item_range	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is item_range=2. Ignored if specified item locations are passed using the manual_items argument.
facet_range	Float: The range (maximum - minimum) of facet effects for randomly generated facet effects, which are drawn from a uniform distribution. Default is facet_range=2. Ignored if specified facet effects are passed using the manual_facet_effects argument.
category_base	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between max_disorder (see below) and twice category_base minus max_disorder. Default is category_base=2.

Continued on next page...

person_sd	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is person_sd=1.5. Ignored if specified person locations are passed using the manual_persons argument.
max_disorder	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around category_base. Default is max_disorder=0, with no disordered thresholds.
offset	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data in the simulation. If missing is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of missing of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly missing. Default is missing=0.
shared_missing	Boolean: If True, the same persons are missing across all facet elements (correlated missingness). If False, missingness is independent across facet elements. Default is shared_missing=True.
manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control simulation parameters. manual_persons=None, random locations are generated. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control simulation parameters. If manual_items=None, random locations are generated. Default is manual_items=None.
manual_thresholds	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control simulation parameters. If manual_items=None, random thresholds are generated. Default is manual_thresholds=None.
manual_raters	Dictionary or array-like: alternative to manual_facet_effects, keyed by facet element, in the same structure. Not for model='bivector'. Default is manual_raters=None.

Continued on next page...

manual_facet_effects	List or 1-dimensional numpy array: Optional list of specified facet effects. Allows the user to control simulation parameters. If manual_facet_effects=None, random facet effects are generated. Default is manual_facet_effects=None.
manual_person_names	List: Optional list of person names specified by user. If manual_person_names=None, person names are generated automatically. Default is manual_person_names=None.
manual_item_names	List: Optional list of item names specified by user. If manual_item_names=None, item names are generated automatically. Default is manual_item_names=None.
manual_facet_names	List: Optional list of item names specified by user. If manual_facet_names=None, item names are generated automatically. Default is manual_facet_names=None.
facet	String: singular name for the second (non-item, non-person) facet of the simulated data – e.g. 'judge' if you want the simulated facet elements to be labelled as something other than raters (default generated names follow the format {Facet}_1 etc., e.g. Judge_1). Also aliases self.no_of_{facet_plural} and self.{facet_plural}_effects – note this does *not* alias self.rater_names/self.facet_names, which keep their fixed names regardless. If None, generated names and attribute aliases use 'rater' (Rater_1 etc.) and messages refer to 'facet element(s)', with '(e.g. rater)' at first mention. Default is facet=None.
facet_plural	String or None: plural form of facet, used for the pluralised attribute aliases described above. If None (default), facet + 's' is used. Default is facet_plural=None.
seed	Integer or None: seed for the random number generator. Pass an integer for a fully reproducible simulation; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing, per facet element when shared_missing=False, or in the one shared pattern when shared_missing=True. Default is missing_method='bernoulli'.

Format for `manual_facet_effects` **input**

Toy example of the input for `manual_facet_effects`, with two items and two persons. Pandas series with facet element names as keys and facet effects as values.

```
manual_facet_effects = pd.Series({'Rater_1':  -0.5,
                                  'Rater_2':   0.5})
```

Returns

Object of class `MFRM_Sim_Global`. Several attributes of object `MFRM_Sim_Global` are automatically generated on its creation:

`self.responses`	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an `MFRM` object for analysis.
`self.no_of_items`	Integer: The number of items, specified by argument `no_of_items`.
`self.no_of_persons`	Integer: The number of persons, specified by argument `no_of_persons`.
`self.no_of_raters`	Integer: The number of facet elements, specified by argument `no_of_raters`.
`self.item_range`	Float: The generating range of item locations, specified by argument `item_range`.
`self.facet_range`	Float: The generating range of facet effects, specified by argument `facet_range`.
`self.max_score`	Integer: The shared maximum possible score on each item, specified by argument `max_score`.
`self.category_base`	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument `category_base`.
`self.person_sd`	The generating standard deviation of person locations, specified by argument `person_sd`.
`self.offset`	The generating offset between the means of person location and item location, specified by argument `offset`.
`self.missing`	The generating proportion of missing data, specified by argument `missing`
`self.missing_method`	The method used to apply missing data, specified by argument `missing_method`
`self.persons`	pandas series: Person names as keys and person locations as values.
`self.items`	pandas series: keys are item names and values are item locations.
`self.facet_effects`	pandas series: keys are facet element names and values are facet effects.
`self.thresholds`	pandas series: set of `self.max_score` Rasch-Andrich thresholds $\tau_1, \ldots, \tau_m$, indexed 1 to `self.max_score`. **Note (breaking change from earlier versions):** the leading sentinel $\tau_0 = 0$ has been removed.
`self.person_names`	List: Person names. If no person names are passed using the `manual_person_names` argument, default person names are generated as `Person_1` etc.
`self.item_names`	List: Item names. If no item names are passed using the `manual_item_names` argument, default item names are generated as `Item_1` etc.

Continued on next page...

self.rater_names	List: Facet element names. If no facet element names are passed using the manual_facet_names argument, default facet element names are generated as Rater_1 etc.
self.dropped_facet_elements	List: facet elements removed with drop_facet_elements (empty on creation). Aliased as self.dropped_raters and, if a custom facet was passed, as self.dropped_{facet_plural}.

Example

To create an MFRM_Sim_Global object called my_mfrm_sim with a maximum score of 5, randomly generated person locations for 5,000 persons, randomly generated item locations for 10 items and randomly generated facet effects for 12 facet elements, specifying an item range of 4 logits, a facet element range of 3 logits, a base category width of 1.5 logits, a person SD of 2 logits, a maximum permitted threshold disorder of 0.5 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_mfrm_sim = MFRM_Sim_Global(10, 5000, 12, 5, item_range=4, facet_range=3,
                              category_base=1.5, person_sd=2,
                              max_disorder=0.5, offset=1, missing=0.3)
```

To create an MFRM_Sim_Global object called my_mfrm_sim with a maximum score of 5, specified person locations for 100 persons saved as a variable named my_person_abils, specified item locations for 12 items saved as a variable named my_item_diffs, specified facet effects for 6 facet elements saved as a variable named my_facet_effects, a specified set of Rasch-Andrich thresholds saved as a variable named my_thresholds, specified person names saved as a variable named my_person_names, specified item names saved as a variable named my_item_names and specified facet element names saved as a variable named my_rater_names, with no missing data:

```
my_mfrm_sim = MFRM_Sim_Global(12, 100, 6, 5, manual_persons=my_person_abils,
                              manual_items=my_item_diffs,
                              manual_facet_effects=my_facet_effects,
                              manual_thresholds=my_thresholds,
                              manual_person_names=my_person_names,
                              manual_item_names=my_item_names,
                              manual_facet_names=my_rater_names)
```

10.2 Customising an MFRM_Sim_Global simulation

10.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_mfrm from Item_1 to my_new_item_name:

```
my_mfrm.rename_item('Item_1', 'my_new_item_name')
```

10.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_mfrm with item names in a list stored as a variable my_new_item_names:

```
my_mfrm.rename_items_all(my_new_item_names)
```

10.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the second multiindex level of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename a person in object my_mfrm from Person_1 to my_new_person_name:

```
my_mfrm.rename_person('Person_1', 'my_new_person_name')
```

10.2.4 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the second multiindex level of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all persons in object my_mfrm with person names in a list stored as a variable my_new_person_names:

```
my_mfrm.rename_persons_all(my_new_person_names)
```

10.2.5 rename_rater

Description
Method to rename a single facet element (e.g. rater).

Usage
```
self.rename_rater(old, new)
```

Arguments

old	String: the old name for the facet element
new	String: the new name for the facet element

Returns
Replaces specified facet element name in the first multiindex level of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename a facet element in object my_mfrm from Rater_1 to my_new_rater_name:

```
my_mfrm.rename_rater('Rater_1', 'my_new_rater_name')
```

10.2.6 rename_raters_all

Description Method to rename all facet elements (e.g. raters).

Usage
```
self.rename_raters_all(new_names)
```

Arguments

new_names	List of new facet element names as strings

Returns
Replaces all facet element names in the first multiindex level of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename all facet elements in object my_mfrm with facet element names in a list stored as a variable my_new_rater_names:

```
my_mfrm.rename_raters_all(my_new_rater_names)
```

10.2.7 drop_facet_elements

Description

Method to remove facet elements (e.g. raters) from the simulation: their responses and their generating parameters (self.facet_effects, plus self.item_effects/self.threshold_effects for bivector and the lambda_/omega parameters for the stretch models). The other facet elements' responses, and all item, threshold and person parameters, are unchanged. Aliased as self.drop_raters and, if a custom facet was passed, as self.drop_{facet_plural} (e.g. self.drop_judges).

Usage

```
self.drop_facet_elements(which, min_persons=3)
```

Arguments

which	String or List: 'thin' removes facet elements that rated fewer than min_persons persons (including those with no responses); 'extreme' removes those that used only the lowest or only the highest score category; a list removes the named facet elements (a ValueError is raised if any name is not in the data).
min_persons	Integer: threshold for which='thin'. Default is min_persons=3.

Returns

Removes the specified facet elements from self.responses, self.facet_names and the generating parameters, with a UserWarning listing those removed, and adds them to self.dropped_facet_elements.

Example

To remove Rater_3 from object my_mfrm_sim:

```
my_mfrm_sim.drop_raters(['Rater_3'])
```

11 class MFRM_Sim_Items

11.1 Generating an MFRM_Sim_Items simulation

11.1.1 MFRM_Sim_Items

Description

Creates an object of the class MFRM_Sim_Items. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the vector-by-item facet element (e.g. rater) representation (Elliott & Buttery, 2022a) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
MFRM_Sim_Items(no_of_items, no_of_persons, max_score,
               no_of_facet_elements, item_range=2,
               facet_range=2, category_base=2, person_sd=1.5, max_disorder=0, offset=0,
               missing=0, shared_missing=True, manual_persons=None, manual_items=None,
               manual_thresholds=None, manual_facet_effects=None, manual_person_names=None,
               manual_item_names=None, manual_facet_names=None, facet=None,
               facet_plural=None, seed=None,
               missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
max_score	Integer: The maximum score on each item.
no_of_facet_elements	Integer or None: The number of facet elements in the simulation.
item_range	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is item_range=2. Ignored if specified item locations are passed using the manual_items argument.
facet_range	Float: The range (maximum - minimum) of facet effects for randomly generated facet effects, which are drawn from a uniform distribution. Default is facet_range=2. Ignored if specified facet effects are passed using the manual_facet_effects argument.
category_base	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between max_disorder (see below) and twice category_base minus max_disorder. Default is category_base=2.

Continued on next page...

person_sd	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is person_sd=1.5. Ignored if specified person locations are passed using the manual_persons argument.
max_disorder	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around category_base. Default is max_disorder=0, with no disordered thresholds.
offset	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data in the simulation. If missing is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of missing of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly missing. Default is missing=0.
shared_missing	Boolean: If True, the same persons are missing across all facet elements (correlated missingness). If False, missingness is independent across facet elements. Default is shared_missing=True.
manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control simulation parameters. manual_persons=None, random locations are generated. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control simulation parameters. If manual_items=None, random locations are generated. Default is manual_items=None.
manual_thresholds	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control simulation parameters. If manual_items=None, random thresholds are generated. Default is manual_thresholds=None.
manual_facet_effects	List or 1-dimensional numpy array: Optional list of specified facet effects. Allows the user to control simulation parameters. If manual_facet_effects=None, random facet effects are generated. Default is manual_facet_effects=None.

Continued on next page...

manual_person_names	List: Optional list of person names specified by user. If manual_person_names=None, person names are generated automatically. Default is manual_person_names=None.
manual_item_names	List: Optional list of item names specified by user. If manual_item_names=None, item names are generated automatically. Default is manual_item_names=None.
manual_facet_names	List: Optional list of item names specified by user. If manual_facet_names=None, item names are generated automatically. Default is manual_facet_names=None.
facet	String: singular name for the second (non-item, non-person) facet of the simulated data – e.g. 'judge' if you want the simulated facet elements to be labelled as something other than raters (default generated names follow the format {Facet}_1 etc., e.g. Judge_1). Also aliases self.no_of_{facet_plural} and self.{facet_plural}_effects – note this does *not* alias self.rater_names/self.facet_names, which keep their fixed names regardless. If None, generated names and attribute aliases use 'rater' (Rater_1 etc.) and messages refer to 'facet element(s)', with '(e.g. rater)' at first mention. Default is facet=None.
facet_plural	String or None: plural form of facet, used for the pluralised attribute aliases described above. If None (default), facet + 's' is used. Default is facet_plural=None.
seed	Integer or None: seed for the random number generator. Pass an integer for a fully reproducible simulation; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing, per facet element when shared_missing=False, or in the one shared pattern when shared_missing=True. Default is missing_method='bernoulli'.

Format for manual_facet_effects **input**

Toy example of the nested dictionary input for manual_facet_effects, with two items and two persons. The outer dictionary has item names for values, which are also the keys for the inner dictionary. The inner dictionaries item names for keys and floats (item facet effects) for values.

```
manual_facet_effects = {'Rater_1':  {'Item_1':  -1, 'Item_2':  0},
                        'Rater_2':  {'Item_1':   0, 'Item_2':  1}}
```

Returns

Object of class MFRM_Sim_Items. Several attributes of object MFRM_Sim_Items are automatically generated on

its creation:

self.responses	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an MFRM object for analysis.
self.no_of_items	Integer: The number of items, specified by argument no_of_items.
self.no_of_persons	Integer: The number of persons, specified by argument no_of_persons.
self.no_of_raters	Integer: The number of facet elements, specified by argument no_of_raters.
self.item_range	Float: The generating range of item locations, specified by argument item_range.
self.facet_range	Float: The generating range of facet effects, specified by argument facet_range.
self.max_score	Integer: The shared maximum possible score on each item, specified by argument max_score.
self.category_base	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument category_base.
self.person_sd	The generating standard deviation of person locations, specified by argument person_sd.
self.offset	The generating offset between the means of person location and item location, specified by argument offset.
self.missing	The generating proportion of missing data, specified by argument missing
self.missing_method	The method used to apply missing data, specified by argument missing_method
self.persons	pandas series: Person names as keys and person locations as values.
self.items	pandas series: keys are item names and values are item locations.
self.facet_effects	Nested dictionary: Outer keys are facet element names, inner keys are item names, as in the manual_facet_effects input example given above.
self.thresholds	pandas series: set of self.max_score Rasch-Andrich thresholds $\tau_1, \ldots, \tau_m$, indexed 1 to self.max_score. **Note (breaking change from earlier versions):** the leading sentinel $\tau_0 = 0$ has been removed.
self.person_names	List: Person names. If no person names are passed using the manual_person_names argument, default person names are generated in the format Person_1 etc.
self.item_names	List: Item names. If no item names are passed using the manual_item_names argument, default item names are generated in the format Item_1 etc.

Continued on next page...

`self.rater_names`	List: Facet element names. If no facet element names are passed using the `manual_facet_names` argument, default facet element names are generated in the format `Rater_1` etc.
`self.dropped_facet_elements`	List: facet elements removed with `drop_facet_elements` (empty on creation). Aliased as `self.dropped_raters` and, if a custom `facet` was passed, as `self.dropped_{facet_plural}`.

Examples

To create an MFRM_Sim_Items object called my_mfrm_sim with a maximum score of 5, randomly generated person locations for 5,000 persons, randomly generated item locations for 10 items and randomly generated facet effects for 12 facet elements, specifying an item range of 4 logits, a facet element range of 3 logits, a base category width of 1.5 logits, a person SD of 2 logits, a maximum permitted threshold disorder of 0.5 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_mfrm_sim = MFRM_Sim_Items(10, 5000, 12, 5, item_range=4, facet_range=3,
                             category_base=1.5, person_sd=2, max_disorder=0.5,
                             offset=1, missing=0.3)
```

To create an MFRM_Sim_Items object called my_mfrm_sim with a maximum score of 5, specified person locations for 100 persons saved as a variable named my_person_abils, specified item locations for 12 items saved as a variable named my_item_diffs, specified facet effects for 6 facet elements saved as a variable named my_facet_effects, a specified set of Rasch-Andrich thresholds saved as a variable named my_thresholds, specified person names saved as a variable named my_person_names, specified item names saved as a variable named my_item_names and specified facet element names saved as a variable named my_rater_names, with no missing data:

```
my_mfrm_sim = MFRM_Sim_Items(12, 100, 6, 5, manual_persons=my_person_abils,
                             manual_items=my_item_diffs,
                             manual_facet_effects=my_facet_effects,
                             manual_thresholds=my_thresholds,
                             manual_person_names=my_person_names,
                             manual_item_names=my_item_names,
                             manual_facet_names=my_rater_names)
```

11.2 Customising an MFRM_Sim_Items simulation

11.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_mfrm from Item_1 to my_new_item_name:

```
my_mfrm.rename_item('Item_1', 'my_new_item_name')
```

11.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_mfrm with item names in a list stored as a variable my_new_item_names:

```
my_mfrm.rename_items_all(my_new_item_names)
```

11.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the second multiindex level of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename a person in object my_mfrm from Person_1 to my_new_person_name:

```
my_mfrm.rename_person('Person_1', 'my_new_person_name')
```

11.2.4 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the second multiindex level of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all persons in object my_mfrm with person names in a list stored as a variable my_new_person_names:

```
my_mfrm.rename_persons_all(my_new_person_names)
```

11.2.5 rename_rater

Description
Method to rename a single facet element (e.g. rater).

Usage
```
self.rename_rater(old, new)
```

Arguments

old	String: the old name for the facet element
new	String: the new name for the facet element

Returns
Replaces specified facet element name in the first multiindex level of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename a facet element in object my_mfrm from Rater_1 to my_new_rater_name:

```
my_mfrm.rename_rater('Rater_1', 'my_new_rater_name')
```

11.2.6 rename_raters_all

Description Method to rename all facet elements (e.g. raters).

Usage
```
self.rename_raters_all(new_names)
```

Arguments

new_names	List of new facet element names as strings

Returns
Replaces all facet element names in the first multiindex level of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename all facet elements in object my_mfrm with facet element names in a list stored as a variable my_new_rater_names:

```
my_mfrm.rename_raters_all(my_new_rater_names)
```

11.2.7 drop_facet_elements

Description

Method to remove facet elements (e.g. raters) from the simulation: their responses and their generating parameters (self.facet_effects, plus self.item_effects/self.threshold_effects for bivector and the lambda_/omega parameters for the stretch models). The other facet elements' responses, and all item, threshold and person parameters, are unchanged. Aliased as self.drop_raters and, if a custom facet was passed, as self.drop_{facet_plural} (e.g. self.drop_judges).

Usage

```
self.drop_facet_elements(which, min_persons=3)
```

Arguments

which	String or List: 'thin' removes facet elements that rated fewer than min_persons persons (including those with no responses); 'extreme' removes those that used only the lowest or only the highest score category; a list removes the named facet elements (a ValueError is raised if any name is not in the data).
min_persons	Integer: threshold for which='thin'. Default is min_persons=3.

Returns

Removes the specified facet elements from self.responses, self.facet_names and the generating parameters, with a UserWarning listing those removed, and adds them to self.dropped_facet_elements.

Example

To remove Rater_3 from object my_mfrm_sim:

```
my_mfrm_sim.drop_raters(['Rater_3'])
```

12 class MFRM_Sim_Thresholds

12.1 Generating an MFRM_Sim_Thresholds simulation

12.1.1 MFRM_Sim_Thresholds

Description

Creates an object of the class MFRM_Sim_Thresholds. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the vector-by-threshold facet element (e.g. rater) representation (Elliott & Buttery, 2022a) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
MFRM_Sim_Thresholds(no_of_items, no_of_persons, max_score, no_of_facet_elements, item_range=2,
                    facet_range=2, category_base=2, person_sd=1.5, max_disorder=0,
                    offset=0, missing=0, shared_missing=True, manual_persons=None,
                    manual_items=None, manual_thresholds=None, manual_facet_effects=None,
                    manual_person_names=None, manual_item_names=None,
                    manual_facet_names=None, facet=None, facet_plural=None,
                    seed=None,
                    missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
max_score	Integer: The maximum score on each item.
no_of_facet_elements	Integer or None: The number of facet elements in the simulation.
item_range	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is item_range=2. Ignored if specified item locations are passed using the manual_items argument.
facet_range	Float: The range (maximum - minimum) of facet effects for randomly generated facet effects, which are drawn from a uniform distribution. Default is facet_range=2. Ignored if specified facet effects are passed using the manual_facet_effects argument.
category_base	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between max_disorder (see below) and twice category_base minus max_disorder. Default is category_base=2.

Continued on next page...

person_sd	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is person_sd=1.5. Ignored if specified person locations are passed using the manual_persons argument.
max_disorder	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around category_base. Default is max_disorder=0, with no disordered thresholds.
offset	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data in the simulation. If missing is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of missing of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly missing. Default is missing=0.
shared_missing	Boolean: If True, the same persons are missing across all facet elements (correlated missingness). If False, missingness is independent across facet elements. Default is shared_missing=True.
manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control simulation parameters. manual_persons=None, random locations are generated. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control simulation parameters. If manual_items=None, random locations are generated. Default is manual_items=None.
manual_thresholds	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control simulation parameters. If manual_items=None, random thresholds are generated. Default is manual_thresholds=None.
manual_facet_effects	List or 1-dimensional numpy array: Optional list of specified facet effects. Allows the user to control simulation parameters. If manual_facet_effects=None, random facet effects are generated. Default is manual_facet_effects=None.

Continued on next page...

manual_person_names	List: Optional list of person names specified by user. If manual_person_names=None, person names are generated automatically. Default is manual_person_names=None.
manual_item_names	List: Optional list of item names specified by user. If manual_item_names=None, item names are generated automatically. Default is manual_item_names=None.
manual_facet_names	List: Optional list of item names specified by user. If manual_facet_names=None, item names are generated automatically. Default is manual_facet_names=None.
facet	String: singular name for the second (non-item, non-person) facet of the simulated data – e.g. 'judge' if you want the simulated facet elements to be labelled as something other than raters (default generated names follow the format {Facet}_1 etc., e.g. Judge_1). Also aliases self.no_of_{facet_plural} and self.{facet_plural}_effects – note this does *not* alias self.rater_names/self.facet_names, which keep their fixed names regardless. If None, generated names and attribute aliases use 'rater' (Rater_1 etc.) and messages refer to 'facet element(s)', with '(e.g. rater)' at first mention. Default is facet=None.
facet_plural	String or None: plural form of facet, used for the pluralised attribute aliases described above. If None (default), facet + 's' is used. Default is facet_plural=None.
seed	Integer or None: seed for the random number generator. Pass an integer for a fully reproducible simulation; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing, per facet element when shared_missing=False, or in the one shared pattern when shared_missing=True. Default is missing_method='bernoulli'.

Format for manual_facet_effects **input**

Toy example of the dictionary input for manual_facet_effects, with two items, two persons and four categories. Dictionary with item names as keys and numpy arrays as values, of length self.max_score + 1 of item facet effects for values with the indices of each array corresponding to the threshold, and threshold zero set to zero by convention.

```
manual_facet_effects = {'Rater_1':  np.array([0,  -1, -0.5,    0]),
                        'Rater_2':  np.array([0, 1.5,  0.5, -0.5])}
```

Returns

Object of class MFRM_Sim_Thresholds. Several attributes of object MFRM_Sim_Thresholds are automatically

generated on its creation:

self.responses	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an MFRM object for analysis.
self.no_of_items	Integer: The number of items, specified by argument no_of_items.
self.no_of_persons	Integer: The number of persons, specified by argument no_of_persons.
self.no_of_raters	Integer: The number of facet elements, specified by argument no_of_raters.
self.item_range	Float: The generating range of item locations, specified by argument item_range.
self.facet_range	Float: The generating range of facet effects, specified by argument facet_range.
self.max_score	Integer: The shared maximum possible score on each item, specified by argument max_score.
self.category_base	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument category_base.
self.person_sd	The generating standard deviation of person locations, specified by argument person_sd.
self.offset	The generating offset between the means of person location and item location, specified by argument offset.
self.missing	The generating proportion of missing data, specified by argument missing
self.missing_method	The method used to apply missing data, specified by argument missing_method
self.persons	pandas series: Person names as keys and person locations as values.
self.items	pandas series: keys are item names and values are item locations.
self.facet_effects	Dictionary: Keys are facet element names, values are numpy arrays, as in the manual_facet_effects input example given above.
self.thresholds	pandas series: set of self.max_score Rasch-Andrich thresholds $\tau_1, \ldots, \tau_m$, indexed 1 to self.max_score. **Note (breaking change from earlier versions):** the leading sentinel $\tau_0 = 0$ has been removed.
self.person_names	List: Person names. If no person names are passed using the manual_person_names argument, default person names are generated in the format Person_1 etc.
self.item_names	List: Item names. If no item names are passed using the manual_item_names argument, default item names are generated in the format Item_1 etc.

Continued on next page...

`self.rater_names`	List: Facet element names. If no facet element names are passed using the `manual_facet_names` argument, default facet element names are generated in the format `Rater_1` etc.
`self.dropped_facet_elements`	List: facet elements removed with `drop_facet_elements` (empty on creation). Aliased as `self.dropped_raters` and, if a custom `facet` was passed, as `self.dropped_{facet_plural}`.

Example

To create an MFRM_Sim_Thresholds object called my_mfrm_sim with a maximum score of 5, randomly generated person locations for 5,000 persons, randomly generated item locations for 10 items and randomly generated facet effects for 12 facet elements, specifying an item range of 4 logits, a facet element range of 3 logits, a base category width of 1.5 logits, a person SD of 2 logits, a maximum permitted threshold disorder of 0.5 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_mfrm_sim = MFRM_Sim_Thresholds(10, 5000, 12, 5, item_range=4,
                                  category_base=1.5, person_sd=2,
                                  max_disorder=0.5, offset=1, missing=0.3)
```

To create an MFRM_Sim_Thresholds object called my_mfrm_sim with a maximum score of 5, specified person locations for 100 persons saved as a variable named my_person_abils, specified item locations for 12 items saved as a variable named my_item_diffs, specified facet effects for 6 facet elements saved as a variable named my_facet_effects, a specified set of Rasch-Andrich thresholds saved as a variable named my_thresholds, specified person names saved as a variable named my_person_names, specified item names saved as a variable named my_item_names and specified facet element names saved as a variable named my_rater_names, with no missing data:

```
my_mfrm_sim = MFRM_Sim_Thresholds(12, 100, 6, 5, manual_persons=my_person_abils,
                                  manual_items=my_item_diffs,
                                  manual_facet_effects=my_facet_effects,
                                  manual_thresholds=my_thresholds,
                                  manual_person_names=my_person_names,
                                  manual_item_names=my_item_names,
                                  manual_facet_names=my_rater_names)
```

12.2 Customising an MFRM_Sim_Thresholds simulation

12.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_mfrm from Item_1 to my_new_item_name:

```
my_mfrm.rename_item('Item_1', 'my_new_item_name')
```

12.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_mfrm with item names in a list stored as a variable my_new_item_names:

```
my_mfrm.rename_items_all(my_new_item_names)
```

12.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the second multiindex level of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename a person in object my_mfrm from Person_1 to my_new_person_name:

```
my_mfrm.rename_person('Person_1', 'my_new_person_name')
```

12.2.4 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the second multiindex level of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all persons in object my_mfrm with person names in a list stored as a variable my_new_person_names:

```
my_mfrm.rename_persons_all(my_new_person_names)
```

12.2.5 rename_rater

Description
Method to rename a single facet element (e.g. rater).

Usage
```
self.rename_rater(old, new)
```

Arguments

old	String: the old name for the facet element
new	String: the new name for the facet element

Returns
Replaces specified facet element name in the first multiindex level of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename a facet element in object my_mfrm from Rater_1 to my_new_rater_name:

```
my_mfrm.rename_rater('Rater_1', 'my_new_rater_name')
```

12.2.6 rename_raters_all

Description Method to rename all facet elements (e.g. raters).

Usage
```
self.rename_raters_all(new_names)
```

Arguments

new_names	List of new facet element names as strings

Returns
Replaces all facet element names in the first multiindex level of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename all facet elements in object my_mfrm with facet element names in a list stored as a variable my_new_rater_names:

```
my_rsm.rename_raters_all(my_new_rater_names)
```

12.2.7 drop_facet_elements

Description

Method to remove facet elements (e.g. raters) from the simulation: their responses and their generating parameters (self.facet_effects, plus self.item_effects/self.threshold_effects for bivector and the lambda_/omega parameters for the stretch models). The other facet elements' responses, and all item, threshold and person parameters, are unchanged. Aliased as self.drop_raters and, if a custom facet was passed, as self.drop_{facet_plural} (e.g. self.drop_judges).

Usage

```
self.drop_facet_elements(which, min_persons=3)
```

Arguments

which	String or List: 'thin' removes facet elements that rated fewer than min_persons persons (including those with no responses); 'extreme' removes those that used only the lowest or only the highest score category; a list removes the named facet elements (a ValueError is raised if any name is not in the data).
min_persons	Integer: threshold for which='thin'. Default is min_persons=3.

Returns

Removes the specified facet elements from self.responses, self.facet_names and the generating parameters, with a UserWarning listing those removed, and adds them to self.dropped_facet_elements.

Example

To remove Rater_3 from object my_mfrm_sim:

```
my_mfrm_sim.drop_raters(['Rater_3'])
```

13 class MFRM_Sim_Matrix

13.1 Generating an MFRM_Sim_Matrix simulation

13.1.1 MFRM_Sim_Matrix

Description

Creates an object of the class MFRM_Sim_Matrix. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the matrix representation (Elliott & Buttery, 2022a) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Usage

```
MFRM_Sim_Matrix(no_of_items, no_of_persons, max_score,
                no_of_facet_elements, item_range=2,
                facet_range=2, category_base=2, person_sd=1.5, max_disorder=0,
                offset=0, missing=0, shared_missing=True, manual_persons=None,
                manual_items=None, manual_thresholds=None, manual_facet_effects=None,
                manual_person_names=None, manual_item_names=None, manual_facet_names=None,
                facet=None, facet_plural=None, seed=None,
                missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
max_score	Integer: The maximum score on each item.
no_of_facet_elements	Integer or None: The number of facet elements (e.g. raters) in the simulation.
item_range	Float: The range (maximum - minimum) of item locations for randomly generated item locations, which are drawn from a uniform distribution. Default is item_range=2. Ignored if specified item locations are passed using the manual_items argument.
facet_range	Float: The range (maximum - minimum) of facet effects for randomly generated facet effects, which are drawn from a uniform distribution. Default is facet_range=2. Ignored if specified facet effects are passed using the manual_facet_effects argument.
category_base	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds, which are drawn from a uniform distribution between max_disorder (see below) and twice category_base minus max_disorder. Default is category_base=2.

Continued on next page...

person_sd	Float: The range standard deviation of person locations for randomly generated item locations, which are drawn from a normal distribution. Default is person_sd=1.5. Ignored if specified person locations are passed using the manual_persons argument.
max_disorder	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around category_base. Default is max_disorder=0, with no disordered thresholds.
offset	Float: The difference between the means of the person locations and item locations for randomly generated data. If positive, mean person location is higher than mean item location; if negative, mean person location is lower than mean item location. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data in the simulation. If missing is not zero, responses are removed according to a missing completely at random (MCAR) pattern, with each response having a probability of missing of being deleted. Since this process is randomly determined on a response-by-response basis, the proportion of missing data may not be exactly missing. Default is missing=0.
shared_missing	Boolean: If True, the same persons are missing across all facet elements (correlated missingness). If False, missingness is independent across facet elements. Default is shared_missing=True.
manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control simulation parameters. manual_persons=None, random locations are generated. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control simulation parameters. If manual_items=None, random locations are generated. Default is manual_items=None.
manual_thresholds	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control simulation parameters. If manual_items=None, random thresholds are generated. Default is manual_thresholds=None.
manual_facet_effects	List or 1-dimensional numpy array: Optional list of specified facet effects. Allows the user to control simulation parameters. If manual_facet_effects=None, random facet effects are generated. Default is manual_facet_effects=None.

Continued on next page...

manual_person_names	List: Optional list of person names specified by user. If manual_person_names=None, person names are generated automatically. Default is manual_person_names=None.
manual_item_names	List: Optional list of item names specified by user. If manual_item_names=None, item names are generated automatically. Default is manual_item_names=None.
manual_facet_names	List: Optional list of item names specified by user. If manual_facet_names=None, item names are generated automatically. Default is manual_facet_names=None.
facet	String: singular name for the second (non-item, non-person) facet of the simulated data – e.g. 'judge' if you want the simulated facet elements to be labelled as something other than raters (default generated names follow the format {Facet}_1 etc., e.g. Judge_1). Also aliases self.no_of_{facet_plural} and self.{facet_plural}_effects – note this does *not* alias self.rater_names/self.facet_names, which keep their fixed names regardless. If None, generated names and attribute aliases use 'rater' (Rater_1 etc.) and messages refer to 'facet element(s)', with '(e.g. rater)' at first mention. Default is facet=None.
facet_plural	String or None: plural form of facet, used for the pluralised attribute aliases described above. If None (default), facet + 's' is used. Default is facet_plural=None.
seed	Integer or None: seed for the random number generator. Pass an integer for a fully reproducible simulation; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing, per facet element when shared_missing=False, or in the one shared pattern when shared_missing=True. Default is missing_method='bernoulli'.

Format for manual_facet_effects **input**

Toy example of the nested dictionary input for manual_facet_effects, with two items, two persons and four categories. The outer dictionary has item names for values, which are also the keys for the inner dictionary. Values for the inner dictionary are numpy arrays of facet effects by threshold for the item key, with the index of the array index corresponding to each threshold (0 to self.max_score).

```
manual_facet_effects = {'Rater_1':  {'Item_1':  np.array([0, -1.5, -1, 0.5]),
                                     'Item_2':  np.array([0,    1,  0,  -1])},
                        'Rater_2':  {'Item_1':  np.array([0, -0.5,  0, 0.5]),
                                     'Item_2':  np.array([0,    2,  1,   0])}}
```

Returns

Object of class `MFRM_Sim_Matrix`. Several attributes of object `MFRM_Sim_Matrix` are automatically generated on its creation:

`self.responses`	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an `MFRM` object for analysis.
`self.no_of_items`	Integer: The number of items, specified by argument `no_of_items`.
`self.no_of_persons`	Integer: The number of persons, specified by argument `no_of_persons`.
`self.no_of_raters`	Integer: The number of facet elements, specified by argument `no_of_raters`.
`self.item_range`	Float: The generating range of item locations, specified by argument `item_range`.
`self.facet_range`	Float: The generating range of facet effects, specified by argument `facet_range`.
`self.max_score`	Integer: The shared maximum possible score on each item, specified by argument `max_score`.
`self.category_base`	The generating base category width for generating random Rasch-Andrich thresholds, specified by argument `category_base`.
`self.person_sd`	The generating standard deviation of person locations, specified by argument `person_sd`.
`self.offset`	The generating offset between the means of person location and item location, specified by argument `offset`.
`self.missing`	The generating proportion of missing data, specified by argument `missing`
`self.missing_method`	The method used to apply missing data, specified by argument `missing_method`
`self.persons`	pandas series: Person names as keys and person locations as values.
`self.items`	pandas series: keys are item names and values are item locations.
`self.facet_effects`	Nested dictionary: Outer keys are facet element names, inner keys are item names, as in the `manual_facet_effects` input example given above.
`self.thresholds`	pandas series: set of `self.max_score` Rasch-Andrich thresholds $\tau_1, \ldots, \tau_m$, indexed 1 to `self.max_score`. **Note (breaking change from earlier versions):** the leading sentinel $\tau_0 = 0$ has been removed.
`self.person_names`	List: Person names. If no person names are passed using the `manual_person_names` argument, default person names are generated in the format `Person_1` etc.

Continued on next page...

self.item_names	List: Item names. If no item names are passed using the manual_item_names argument, default item names are generated in the format Item_1 etc.
self.rater_names	List: Facet element names. If no facet element names are passed using the manual_facet_names argument, default facet element names are generated in the format Rater_1 etc.
self.dropped_facet_elements	List: facet elements removed with drop_facet_elements (empty on creation). Aliased as self.dropped_raters and, if a custom facet was passed, as self.dropped_{facet_plural}.

Example

To create an MFRM_Sim_Matrix object called my_mfrm_sim with a maximum score of 5, randomly generated person locations for 5,000 persons, randomly generated item locations for 10 items and randomly generated facet effects for 12 facet elements, specifying an item range of 4 logits, a facet element range of 3 logits, a base category width of 1.5 logits, a person SD of 2 logits, a maximum permitted threshold disorder of 0.5 logits, an offset of 1 logit and a proportion of 0.3 of missing data:

```
my_mfrm_sim = MFRM_Sim_Matrix(10, 5000, 12, 5, item_range=4, facet_range=3,
                              category_base=1.5, person_sd=2,
                              max_disorder=0.5, offset=1, missing=0.3)
```

To create an MFRM_Sim_Matrix object called my_mfrm_sim with a maximum score of 5, specified person locations for 100 persons saved as a variable named my_person_abils, specified item locations for 12 items saved as a variable named my_item_diffs, specified facet effects for 6 facet elements saved as a variable named my_facet_effects, a specified set of Rasch-Andrich thresholds saved as a variable named my_thresholds, specified person names saved as a variable named my_person_names, specified item names saved as a variable named my_item_names and specified facet element names saved as a variable named my_rater_names, with no missing data:

```
my_mfrm_sim = MFRM_Sim_Matrix(12, 100, 6, 5, manual_persons=my_person_abils,
                              manual_items=my_item_diffs,
                              manual_facet_effects=my_facet_effects,
                              manual_thresholds=my_thresholds,
                              manual_person_names=my_person_names,
                              manual_item_names=my_item_names,
                              manual_facet_names=my_rater_names)
```

13.2 Customising an MFRM_Sim_Matrix simulation

13.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_mfrm from Item_1 to my_new_item_name:

```
my_mfrm.rename_item('Item_1', 'my_new_item_name')
```

13.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_mfrm with item names in a list stored as a variable my_new_item_names:

```
my_mfrm.rename_items_all(my_new_item_names)
```

13.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the second multiindex level of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename a person in object my_mfrm from Person_1 to my_new_person_name:

```
my_mfrm.rename_person('Person_1', 'my_new_person_name')
```

13.2.4 rename_persons_all

Description Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the second multiindex level of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all persons in object my_mfrm with person names in a list stored as a variable my_new_person_names:

```
my_mfrm.rename_persons_all(my_new_person_names)
```

13.2.5 rename_rater

Description
Method to rename a single facet element (e.g. rater).

Usage
```
self.rename_rater(old, new)
```

Arguments

old	String: the old name for the facet element
new	String: the new name for the facet element

Returns
Replaces specified facet element name in the first multiindex level of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename a facet element in object my_mfrm from Rater_1 to my_new_rater_name:

```
my_mfrm.rename_rater('Rater_1', 'my_new_rater_name')
```

13.2.6 rename_raters_all

Description Method to rename all facet elements (e.g. raters).

Usage
```
self.rename_raters_all(new_names)
```

Arguments

new_names	List of new facet element names as strings

Returns
Replaces all facet element names in the first multiindex level of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename all facet elements in object my_mfrm with facet element names in a list stored as a variable my_new_rater_names:

```
my_mfrm.rename_raters_all(my_new_rater_names)
```

13.2.7 drop_facet_elements

Description

Method to remove facet elements (e.g. raters) from the simulation: their responses and their generating parameters (self.facet_effects, plus self.item_effects/self.threshold_effects for bivector and the lambda_/omega parameters for the stretch models). The other facet elements' responses, and all item, threshold and person parameters, are unchanged. Aliased as self.drop_raters and, if a custom facet was passed, as self.drop_{facet_plural} (e.g. self.drop_judges).

Usage

```
self.drop_facet_elements(which, min_persons=3)
```

Arguments

which	String or List: 'thin' removes facet elements that rated fewer than min_persons persons (including those with no responses); 'extreme' removes those that used only the lowest or only the highest score category; a list removes the named facet elements (a ValueError is raised if any name is not in the data).
min_persons	Integer: threshold for which='thin'. Default is min_persons=3.

Returns

Removes the specified facet elements from self.responses, self.facet_names and the generating parameters, with a UserWarning listing those removed, and adds them to self.dropped_facet_elements.

Example

To remove Rater_3 from object my_mfrm_sim:

```
my_mfrm_sim.drop_raters(['Rater_3'])
```

14 class MFRM_Sim_Bivector

14.1 Generating an MFRM_Sim_Bivector simulation

14.1.1 MFRM_Sim_Bivector

Description

Creates an object of the class MFRM_Sim_Bivector. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the bivector facet element (e.g. rater) representation (Elliott & Buttery, 2022b) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

In the bivector model, facet effect is decomposed into two additive components: a per-(facet element, item) item effect and a per-(facet element, threshold) threshold effect. The true facet effect for facet element r, item i and threshold k is:

$$\lambda_{rik} = \lambda_{ri.} + \lambda_{r.k} \tag{30}$$

The item effect carries the overall location of the facet element across items (free mean per facet element); the threshold effect is zero-sum per facet element across thresholds and carries only shape information. MFRM_Sim_Bivector is a thin convenience subclass of MFRM_Sim (model='bivector'); the two additive components are combined internally into the same facet element × item × threshold representation used by the matrix model, so all attributes of MFRM_Sim_Matrix are available on the bivector object, plus self.item_effects and self.threshold_effects.

Usage

```
MFRM_Sim_Bivector(no_of_items, no_of_persons, max_score, no_of_facet_elements, item_range=2,
                  item_facet_range=2, threshold_facet_range=1, category_base=2,
                  person_sd=1.5, max_disorder=0, offset=0, missing=0,
                  shared_missing=True, manual_persons=None, manual_items=None,
                  manual_thresholds=None, manual_item_effects=None,
                  manual_threshold_effects=None, manual_person_names=None,
                  manual_item_names=None, manual_facet_names=None, facet=None,
                  facet_plural=None, seed=None,
                  missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
max_score	Integer: The maximum score on each item.
no_of_facet_elements	Integer or None: The number of facet elements in the simulation.
max_score	Integer: The maximum score on each item.

Continued on next page...

item_range	Float: The range (maximum - minimum) of item locations for randomly generated item locations, drawn from a uniform distribution. Default is item_range=2. Ignored if manual_items is passed.
item_facet_range	Float: The total spread (maximum - minimum) of per-(facet element, item) facet effects across the full facet element × item matrix, in logits. Default is item_facet_range=2. Ignored if manual_item_effects is passed.
threshold_facet_range	Float: The total spread (maximum - minimum) of per-(facet element, threshold) facet effects across the full facet element × threshold matrix, in logits. Default is threshold_facet_range=1. Ignored if manual_threshold_effects is passed.
category_base	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds. Default is category_base=2.
person_sd	Float: The standard deviation of person locations for randomly generated locations, drawn from a normal distribution. Default is person_sd=1.5. Ignored if manual_persons is passed.
max_disorder	Float: If negative, Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width and limits the variation in category widths around category_base. Default is max_disorder=0, with no disordered thresholds.
offset	Float: The difference between the means of person locations and item locations. If positive, mean person location is higher than mean item location. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data in the simulation, introduced according to a missing completely at random (MCAR) pattern. Default is missing=0.
shared_missing	Boolean: If True, the same persons are missing across all facet elements (correlated missingness). If False, missingness is independent across facet elements. Default is shared_missing=True.
manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. Allows the user to control simulation parameters. manual_persons=None, random locations are generated. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified central item locations. Allows the user to control simulation parameters. If manual_items=None, random locations are generated. Default is manual_items=None.

Continued on next page...

manual_thresholds	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Allows the user to control simulation parameters. If manual_items=None, random thresholds are generated. Default is manual_thresholds=None.
manual_facet_effects	List or 1-dimensional numpy array: Optional list of specified facet effects. Allows the user to control simulation parameters. If manual_facet_effects=None, random facet effects are generated. Default is manual_facet_effects=None.
manual_item_effects	pandas DataFrame or None: Optional specified per-(facet element, item) facet effects. Rows are facet elements, columns are items. If None, random item effects are generated. Default is manual_item_effects=None. Details of the format are given below.
manual_threshold_effects	pandas DataFrame or None: Optional specified per-(facet element, threshold) facet effects. Rows are facet elements, columns are threshold positions (zero-sum per facet element). If None, random threshold effects are generated. Default is manual_threshold_effects=None. Details of the format are given below.
manual_person_names	List: Optional list of person names. If None, names are generated automatically in the format Person_1 etc. Default is manual_person_names=None.
manual_item_names	List: Optional list of item names. If None, names are generated automatically in the format Item_1 etc. Default is manual_item_names=None.
manual_facet_names	List: Optional list of facet element names. If None, names are generated automatically in the format Rater_1 etc. Default is manual_facet_names=None.
facet	String: singular name for the second (non-item, non-person) facet of the simulated data – e.g. 'judge' if you want the simulated facet elements to be labelled as something other than raters (default generated names follow the format {Facet}_1 etc., e.g. Judge_1). Also aliases self.no_of_{facet_plural} and self.{facet_plural}_effects – note this does *not* alias self.rater_names/self.facet_names, which keep their fixed names regardless. If None, generated names and attribute aliases use 'rater' (Rater_1 etc.) and messages refer to 'facet element(s)', with '(e.g. rater)' at first mention. Default is facet=None.
facet_plural	String or None: plural form of facet, used for the pluralised attribute aliases described above. If None (default), facet + 's' is used. Default is facet_plural=None.

Continued on next page...

seed	Integer or None: seed for the random number generator, shared by the item-effect and threshold-effect draws and every other random draw in the simulation, so the whole simulation is reproducible from one seed. Pass an integer for full reproducibility; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing, per facet element when shared_missing=False, or in the one shared pattern when shared_missing=True. Default is missing_method='bernoulli'.

Format for `manual_item_effects` **input**

A pandas DataFrame with facet element names as the row index and item names as column names. Each cell contains the facet effect for that facet element–item combination. Toy example with two facet elements and two items:

```
manual_item_effects = pd.DataFrame({'Item_1':  { 'Rater_1':  -0.5, 'Rater_2':   0.5},
                                    'Item_2':  { 'Rater_1':   0.3, 'Rater_2':  -0.3}})
```

Format for `manual_threshold_effects` **input**

A pandas DataFrame with facet element names as the row index and threshold positions (0 to max_score - 1) as columns. Each row must sum to zero. Toy example with two facet elements and a maximum score of 3:

```
manual_threshold_effects = pd.DataFrame({'Rater_1':  [-0.3,  0.0,  0.3],
                                         'Rater_2':  [ 0.2, -0.4,  0.2]}).T
```

Returns

Object of class MFRM_Sim_Bivector. Several attributes are automatically generated on its creation:

self.responses	pandas DataFrame: The multiindex dataframe of simulated responses, which may be saved to file or used to create an MFRM object for analysis.
self.no_of_items	Integer: The number of items, specified by argument no_of_items.
self.no_of_persons	Integer: The number of persons, specified by argument no_of_persons.
self.no_of_raters	Integer: The number of facet elements, specified by argument no_of_raters.
self.max_score	Integer: The shared maximum possible score on each item.
self.item_range	Float: The generating range of item locations.
self.category_base	Float: The generating base category width.
self.person_sd	Float: The generating standard deviation of person locations.

Continued on next page...

self.offset	Float: The generating offset between person location and item location means.
self.missing	Float: The generating proportion of missing data.
self.missing_method	The method used to apply missing data, specified by argument missing_method
self.persons	pandas series: Person names as keys and person locations as values.
self.items	pandas series: Item names as keys and item locations as values.
self.thresholds	pandas series: set of self.max_score Rasch-Andrich thresholds $\tau_1, \ldots, \tau_m$, indexed 1 to self.max_score. **Note (breaking change from earlier versions):** the leading sentinel $\tau_0 = 0$ has been removed.
self.facet_effects	pandas DataFrame with a MultiIndex (facet element, item) as the row index and threshold positions as columns: the reconstructed full facet effect matrix used internally by the MFRM probability machinery.
self.item_effects	pandas DataFrame: Per-(facet element, item) facet effects, with facet element names as the row index and item names as column names.
self.threshold_effects	pandas DataFrame: Per-(facet element, threshold) facet effects, with facet element names as the row index and threshold positions as column names. Each row sums to zero.
self.model	String: Always 'bivector'.
self.person_names	List: Person names.
self.rater_names	List: Facet element names.
self.dropped_facet_elements	List: facet elements removed with drop_facet_elements (empty on creation). Aliased as self.dropped_raters and, if a custom facet was passed, as self.dropped_{facet_plural}.

Examples

To create an MFRM_Sim_Bivector object called my_mfrm_sim with a maximum score of 5, randomly generated parameters for 5,000 persons, 10 items and 12 facet elements, specifying an item range of 4 logits, an item facet range of 3 logits, a threshold facet range of 1 logit, a base category width of 1.5 logits, a person SD of 2 logits and a proportion of 0.3 of missing data:

```
my_mfrm_sim = MFRM_Sim_Bivector(10, 5000, 12, 5, item_range=4,
                                item_facet_range=3, threshold_facet_range=1,
                                category_base=1.5, person_sd=2, missing=0.3)
```

To create an MFRM_Sim_Bivector object called my_mfrm_sim with specified item effects and threshold effects, together with specified person locations, item locations and thresholds:

```
my_mfrm_sim = MFRM_Sim_Bivector(12, 100, 6, 5,
                                manual_persons=my_person_abils,
```

```
manual_items=my_item_diffs,
manual_thresholds=my_thresholds,
manual_item_effects=my_item_effects,
manual_threshold_effects=my_threshold_effects,
manual_person_names=my_person_names,
manual_item_names=my_item_names,
manual_facet_names=my_rater_names)
```

14.2 Customising an MFRM_Sim_Bivector simulation

14.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_mfrm_sim from Item_1 to my_new_item_name:

```
my_mfrm_sim.rename_item('Item_1', 'my_new_item_name')
```

14.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_mfrm_sim with item names in a list stored as a variable my_new_item_names:

```
my_mfrm_sim.rename_items_all(my_new_item_names)
```

14.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the second multiindex level of `self.responses` with new name. Names are also replaced in the generating parameters (e.g. `self.items`, `self.persons`) and the response probabilities.

Example
To rename a person in object `my_mfrm_sim` from `Person_1` to `my_new_person_name`:

```
my_mfrm_sim.rename_person('Person_1', 'my_new_person_name')
```

14.2.4 rename_persons_all

Description
Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the second multiindex level of `self.responses` with new names. Names are also replaced in the generating parameters (e.g. `self.items`, `self.persons`) and the response probabilities.

Example
To rename all persons in object `my_mfrm_sim` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm_sim.rename_persons_all(my_new_person_names)
```

14.2.5 `rename_rater`

Description
Method to rename a single facet element (e.g. rater).

Usage
```
self.rename_rater(old, new)
```

Arguments

`old`	String: the old name for the facet element
`new`	String: the new name for the facet element

Returns
Replaces specified facet element name in the first multiindex level of `self.responses` with new name. Names are also replaced in the generating parameters (e.g. `self.facet_effects`) and the response probabilities.

Example
To rename a facet element in object `my_mfrm_sim` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm_sim.rename_rater('Rater_1', 'my_new_rater_name')
```

14.2.6 `rename_raters_all`

Description
Method to rename all facet elements (e.g. raters).

Usage
```
self.rename_raters_all(new_names)
```

Arguments

`new_names`	List of new facet element names as strings

Returns
Replaces all facet element names in the first multiindex level of `self.responses` with new names. Names

are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename all facet elements in object my_mfrm_sim with facet element names in a list stored as a variable my_new_rater_names:

```
my_mfrm_sim.rename_raters_all(my_new_rater_names)
```

14.2.7 drop_facet_elements

Description
Method to remove facet elements (e.g. raters) from the simulation: their responses and their generating parameters (self.facet_effects, plus self.item_effects/self.threshold_effects for bivector and the lambda_/omega parameters for the stretch models). The other facet elements' responses, and all item, threshold and person parameters, are unchanged. Aliased as self.drop_raters and, if a custom facet was passed, as self.drop_{facet_plural} (e.g. self.drop_judges).

Usage
```
self.drop_facet_elements(which, min_persons=3)
```

Arguments

which	String or List: 'thin' removes facet elements that rated fewer than min_persons persons (including those with no responses); 'extreme' removes those that used only the lowest or only the highest score category; a list removes the named facet elements (a ValueError is raised if any name is not in the data).
min_persons	Integer: threshold for which='thin'. Default is min_persons=3.

Returns
Removes the specified facet elements from self.responses, self.facet_names and the generating parameters, with a UserWarning listing those removed, and adds them to self.dropped_facet_elements.

Example
To remove Rater_3 from object my_mfrm_sim:

```
my_mfrm_sim.drop_raters(['Rater_3'])
```

15 class MFRM_Sim_Centrality

15.1 Generating an MFRM_Sim_Centrality simulation

15.1.1 MFRM_Sim_Centrality

Description

Creates an object of the class MFRM_Sim_Centrality. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the centrality/extremity model (Jin & Wang, 2018) from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Unlike MFRM_Sim_Thresholds, each facet element's (e.g. rater's) threshold-location profile is generated from just two true parameters: a location shift λ_r and a threshold-stretch ω_r:

$$\lambda_{rk} = \lambda_r + (\omega_r - 1)\tau_k \tag{31}$$

$\omega_r > 1$ spreads the reference thresholds τ_k apart for that facet element ('central' – a larger person-location gap is required to traverse to a more extreme category, so extreme categories are under-used); $0 < \omega_r < 1$ compresses them together ('extreme' – the opposite effect); $\omega_r = 1$ is neutral and collapses to the global representation.

Usage

```
MFRM_Sim_Centrality(no_of_items, no_of_persons, max_score, no_of_facet_elements,
                   global_range=2, stretch_range=1, manual_lambda=None, manual_omega=None,
                   item_range=2, category_base=2, person_sd=1.5, max_disorder=0,
                   offset=0, missing=0, shared_missing=True, manual_persons=None,
                   manual_items=None, manual_thresholds=None, manual_person_names=None,
                   manual_item_names=None, manual_facet_names=None, facet=None,
                   facet_plural=None, seed=None,
                   missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
no_of_facet_elements	Integer or None: The number of facet elements in the simulation.
max_score	Integer: The maximum score on each item.
global_range	Float: The total spread (maximum − minimum) of true λ_r (location shift) values across facet elements, analogous to facet_range in MFRM_Sim_Thresholds. Default is global_range=2. Ignored if manual_lambda is passed.

Continued on next page...

stretch_range	Float: The total spread of true ω_r (threshold-stretch) values around 1 – e.g. stretch_range=1 typically gives ω_r roughly in $[0.5, 1.5]$. Default is stretch_range=1. Ignored if manual_omega is passed.
manual_lambda	List or 1-dimensional numpy array: Optional list of specified true λ_r values, length no_of_raters. If None (default), drawn the same way MFRM_Sim's own facet_range scales locations.
manual_omega	List or 1-dimensional numpy array: Optional list of specified true ω_r values, length no_of_raters. If None (default), drawn as 1+ a stretch_range-scaled, zero-mean deviation.
item_range	Float: The range of item locations for the internally-generated preliminary MFRM_Sim_Thresholds simulation, drawn from a uniform distribution. Default is item_range=2. Ignored if manual_items is passed.
category_base	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds. Default is category_base=2.
person_sd	Float: The standard deviation of person locations for the preliminary simulation, drawn from a normal distribution. Default is person_sd=1.5. Ignored if manual_persons is passed.
max_disorder	Float: If negative, the preliminary simulation's Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width. Default is max_disorder=0.
offset	Float: The difference between the means of person locations and item locations in the preliminary simulation. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data, introduced according to a missing completely at random (MCAR) pattern. Default is missing=0.
shared_missing	Boolean: If True, the same persons are missing across all facet elements (correlated missingness). If False, missingness is independent across facet elements. Default is shared_missing=True.
manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. If None, random locations are generated. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified central item locations. If None, random locations are generated. Default is manual_items=None.
manual_thresholds	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. If None, random thresholds are generated. Default is manual_thresholds=None.

Continued on next page...

manual_person_names	List: Optional list of person names. If None, names are generated automatically in the format Person_1 etc. Default is manual_person_names=None.
manual_item_names	List: Optional list of item names. If None, names are generated automatically in the format Item_1 etc. Default is manual_item_names=None.
manual_facet_names	List: Optional list of facet element names. If None, names are generated automatically in the format Rater_1 etc. Default is manual_facet_names=None.
facet	String: singular name for the second (non-item, non-person) facet of the simulated data. If None, generated names and attribute aliases use 'rater' (Rater_1 etc.) and messages refer to 'facet element(s)', with '(e.g. rater)' at first mention. Default is facet=None.
facet_plural	String or None: plural form of facet. If None (default), facet + 's' is used.
seed	Integer or None: seed for the true λ_r/ω_r draws and every other random draw in the simulation (via both the preliminary and final internal MFRM_Sim calls), so the whole simulation is reproducible from one seed. Pass an integer for full reproducibility; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing, per facet element when shared_missing=False, or in the one shared pattern when shared_missing=True. Default is missing_method='bernoulli'.

Format for manual_lambda/manual_omega **input**

Simple lists or 1-dimensional numpy arrays, one true value per facet element, in facet element order. Toy example with two facet elements:

```
manual_lambda = [-0.5,  0.5]
manual_omega = [ 1.5,  0.6]
```

Returns

Object of class MFRM_Sim_Centrality. Several attributes are automatically generated on its creation:

self.responses	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an MFRM object for analysis.
self.no_of_items	Integer: The number of items.
self.no_of_persons	Integer: The number of persons.

Continued on next page...

self.no_of_raters	Integer: The number of facet elements.
self.max_score	Integer: The shared maximum possible score on each item.
self.persons	pandas series: Person names as keys and person locations as values.
self.items	pandas series: Item names as keys and item locations as values.
self.thresholds	pandas series: set of self.max_score Rasch-Andrich thresholds $\tau_1, \ldots, \tau_m$, indexed 1 to self.max_score.
self.lambda_	pandas series: True generating λ_r (location shift) per facet element.
self.omega	pandas series: True generating ω_r (threshold-stretch) per facet element.
self.facet_effects	pandas DataFrame: the implied facet element × threshold location table (Equation above), facet element names as the row index, threshold positions as columns – same format as self.facet_effects on MFRM_Sim_Thresholds.
self.model	String: Always 'centrality'.
self.person_names	List: Person names.
self.item_names	List: Item names.
self.rater_names	List: Facet element names.
self.dropped_facet_elements	List: facet elements removed with drop_facet_elements (empty on creation). Aliased as self.dropped_raters and, if a custom facet was passed, as self.dropped_{facet_plural}.

Examples

To create an MFRM_Sim_Centrality object called my_mfrm_sim with a maximum score of 5, randomly generated parameters for 200 persons, 12 items and 8 facet elements, a true λ_r spread of 2 logits and a true ω_r spread of 1 around neutral:

```
my_mfrm_sim = MFRM_Sim_Centrality(no_of_items=12, no_of_persons=200,
                                  no_of_raters=8, max_score=5, global_range=2,
                                  stretch_range=1, seed=42)
```

To create an MFRM_Sim_Centrality object with specified true λ_r/ω_r values for 5 facet elements, saved as variables named my_lambda and my_omega, together with specified item locations and thresholds:

```
my_mfrm_sim = MFRM_Sim_Centrality(no_of_items=6, no_of_persons=100,
                                  no_of_raters=5, max_score=5,
                                  manual_lambda=my_lambda, manual_omega=my_omega,
                                  manual_items=my_item_diffs,
                                  manual_thresholds=my_thresholds, seed=42)
```

15.2 Customising an MFRM_Sim_Centrality simulation

15.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_mfrm_sim from Item_1 to my_new_item_name:

```
my_mfrm_sim.rename_item('Item_1', 'my_new_item_name')
```

15.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_mfrm_sim with item names in a list stored as a variable my_new_item_names:

```
my_mfrm_sim.rename_items_all(my_new_item_names)
```

15.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the second multiindex level of `self.responses` with new name. Names are also replaced in the generating parameters (e.g. `self.items`, `self.persons`) and the response probabilities.

Example
To rename a person in object `my_mfrm_sim` from `Person_1` to `my_new_person_name`:

```
my_mfrm_sim.rename_person('Person_1', 'my_new_person_name')
```

15.2.4 rename_persons_all

Description
Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the second multiindex level of `self.responses` with new names. Names are also replaced in the generating parameters (e.g. `self.items`, `self.persons`) and the response probabilities.

Example
To rename all persons in object `my_mfrm_sim` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm_sim.rename_persons_all(my_new_person_names)
```

15.2.5 `rename_rater`

Description
Method to rename a single facet element (e.g. rater).

Usage
```
self.rename_rater(old, new)
```

Arguments

`old`	String: the old name for the facet element
`new`	String: the new name for the facet element

Returns
Replaces specified facet element name in the first multiindex level of `self.responses` with new name. Names are also replaced in the generating parameters (e.g. `self.facet_effects`) and the response probabilities.

Example
To rename a facet element in object `my_mfrm_sim` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm_sim.rename_rater('Rater_1', 'my_new_rater_name')
```

15.2.6 `rename_raters_all`

Description
Method to rename all facet elements (e.g. raters).

Usage
```
self.rename_raters_all(new_names)
```

Arguments

`new_names`	List of new facet element names as strings

Returns
Replaces all facet element names in the first multiindex level of `self.responses` with new names. Names

are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename all facet elements in object my_mfrm_sim with facet element names in a list stored as a variable my_new_rater_names:

```
my_mfrm_sim.rename_raters_all(my_new_rater_names)
```

15.2.7 drop_facet_elements

Description
Method to remove facet elements (e.g. raters) from the simulation: their responses and their generating parameters (self.facet_effects, plus self.item_effects/self.threshold_effects for bivector and the lambda_/omega parameters for the stretch models). The other facet elements' responses, and all item, threshold and person parameters, are unchanged. Aliased as self.drop_raters and, if a custom facet was passed, as self.drop_{facet_plural} (e.g. self.drop_judges).

Usage
```
self.drop_facet_elements(which, min_persons=3)
```

Arguments

which	String or List: 'thin' removes facet elements that rated fewer than min_persons persons (including those with no responses); 'extreme' removes those that used only the lowest or only the highest score category; a list removes the named facet elements (a ValueError is raised if any name is not in the data).
min_persons	Integer: threshold for which='thin'. Default is min_persons=3.

Returns
Removes the specified facet elements from self.responses, self.facet_names and the generating parameters, with a UserWarning listing those removed, and adds them to self.dropped_facet_elements.

Example
To remove Rater_3 from object my_mfrm_sim:

```
my_mfrm_sim.drop_raters(['Rater_3'])
```

16 class MFRM_Sim_PseudoHalo

16.1 Generating an MFRM_Sim_PseudoHalo simulation

16.1.1 MFRM_Sim_PseudoHalo

Description

Creates an object of the class MFRM_Sim_PseudoHalo. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the pseudo_halo facet element (e.g. rater) representation – the item-facet analogue of MFRM_Sim_Centrality – from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters.

Unlike MFRM_Sim_Items, each facet element's item-location profile is generated from just two true parameters: a location shift λ_r and an item-difficulty-stretch ω_r:

$$\mu_{ri} = \lambda_r + (\omega_r - 1)\delta_i \tag{32}$$

$\omega_r < 1$ compresses the spread of reference item locations δ_i toward zero for that facet element (the pseudo-halo signature – items look more similar in difficulty to that facet element than they really are); $\omega_r > 1$ exaggerates real item-difficulty differences instead; $\omega_r = 1$ is neutral and collapses to the global representation.

Usage

```
MFRM_Sim_PseudoHalo(no_of_items, no_of_persons, max_score, no_of_facet_elements,
                    global_range=2, stretch_range=1, manual_lambda=None, manual_omega=None,
                    item_range=2, category_base=2, person_sd=1.5, max_disorder=0,
                    offset=0, missing=0, shared_missing=True, manual_persons=None,
                    manual_items=None, manual_thresholds=None, manual_person_names=None,
                    manual_item_names=None, manual_facet_names=None, facet=None,
                    facet_plural=None, seed=None,
                    missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
no_of_facet_elements	Integer or None: The number of facet elements in the simulation.
max_score	Integer: The maximum score on each item.
global_range	Float: The total spread (maximum − minimum) of true λ_r (location shift) values across facet elements. Default is global_range=2. Ignored if manual_lambda is passed.
stretch_range	Float: The total spread of true ω_r (item-difficulty-stretch) values around 1. Default is stretch_range=1. Ignored if manual_omega is passed.

Continued on next page...

manual_lambda	List or 1-dimensional numpy array: Optional list of specified true λ_r values, length no_of_raters. If None (default), drawn the same way MFRM_Sim's own facet_range scales locations.
manual_omega	List or 1-dimensional numpy array: Optional list of specified true ω_r values, length no_of_raters. If None (default), drawn as 1+ a stretch_range-scaled, zero-mean deviation.
item_range	Float: The range of item locations for the internally-generated preliminary MFRM_Sim_Items simulation, drawn from a uniform distribution. Default is item_range=2. Ignored if manual_items is passed.
category_base	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds in the preliminary simulation. Default is category_base=2.
person_sd	Float: The standard deviation of person locations for the preliminary simulation, drawn from a normal distribution. Default is person_sd=1.5. Ignored if manual_persons is passed.
max_disorder	Float: If negative, the preliminary simulation's Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). A positive value creates a minimum category width. Default is max_disorder=0.
offset	Float: The difference between the means of person locations and item locations in the preliminary simulation. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data, introduced according to a missing completely at random (MCAR) pattern. Default is missing=0.
shared_missing	Boolean: If True, the same persons are missing across all facet elements (correlated missingness). If False, missingness is independent across facet elements. Default is shared_missing=True.
manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. If None, random locations are generated. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified central item locations. If None, random locations are generated. Default is manual_items=None.
manual_thresholds	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. If None, random thresholds are generated. Default is manual_thresholds=None.
manual_person_names	List: Optional list of person names. If None, names are generated automatically in the format Person_1 etc. Default is manual_person_names=None.

Continued on next page...

manual_item_names	List: Optional list of item names. If None, names are generated automatically in the format Item_1 etc. Default is manual_item_names=None.
manual_facet_names	List: Optional list of facet element names. If None, names are generated automatically in the format Rater_1 etc. Default is manual_facet_names=None.
facet	String: singular name for the second (non-item, non-person) facet of the simulated data. If None, generated names and attribute aliases use 'rater' (Rater_1 etc.) and messages refer to 'facet element(s)', with '(e.g. rater)' at first mention. Default is facet=None.
facet_plural	String or None: plural form of facet. If None (default), facet + 's' is used.
seed	Integer or None: seed for the true λ_r/ω_r draws and every other random draw in the simulation. Pass an integer for full reproducibility; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing, per facet element when shared_missing=False, or in the one shared pattern when shared_missing=True. Default is missing_method='bernoulli'.

Format for manual_lambda/manual_omega **input**

Simple lists or 1-dimensional numpy arrays, one true value per facet element, in facet element order. Toy example with two facet elements:

```
manual_lambda = [-0.5,  0.5]
manual_omega = [ 1.5,  0.6]
```

Returns

Object of class MFRM_Sim_PseudoHalo. Several attributes are automatically generated on its creation:

self.responses	pandas dataframe: The dataframe of responses, which may be saved to file or used to create an MFRM object for analysis.
self.no_of_items	Integer: The number of items.
self.no_of_persons	Integer: The number of persons.
self.no_of_raters	Integer: The number of facet elements.
self.max_score	Integer: The shared maximum possible score on each item.
self.persons	pandas series: Person names as keys and person locations as values.
self.items	pandas series: Item names as keys and item locations as values.

Continued on next page...

self.thresholds	pandas series: set of self.max_score Rasch-Andrich thresholds $\tau_1, \ldots, \tau_m$, indexed 1 to self.max_score.
self.lambda_	pandas series: True generating λ_r (location shift) per facet element.
self.omega	pandas series: True generating ω_r (item-difficulty-stretch) per facet element.
self.facet_effects	pandas DataFrame: the implied facet element × item location table (Equation above), facet element names as the row index, item names as columns – same format as self.facet_effects on MFRM_Sim_Items.
self.model	String: Always 'pseudo_halo'.
self.person_names	List: Person names.
self.item_names	List: Item names.
self.rater_names	List: Facet element names.
self.dropped_facet_elements	List: facet elements removed with drop_facet_elements (empty on creation). Aliased as self.dropped_raters and, if a custom facet was passed, as self.dropped_{facet_plural}.

Examples

To create an MFRM_Sim_PseudoHalo object called my_mfrm_sim with a maximum score of 5, randomly generated parameters for 200 persons, 12 items and 8 facet elements, a true λ_r spread of 2 logits and a true ω_r spread of 1 around neutral:

```
my_mfrm_sim = MFRM_Sim_PseudoHalo(no_of_items=12, no_of_persons=200,
                                  no_of_raters=8, max_score=5, global_range=2,
                                  stretch_range=1, seed=42)
```

To create an MFRM_Sim_PseudoHalo object with specified true λ_r/ω_r values for 5 facet elements, saved as variables named my_lambda and my_omega, together with specified item locations and thresholds:

```
my_mfrm_sim = MFRM_Sim_PseudoHalo(no_of_items=6, no_of_persons=100,
                                  no_of_raters=5, max_score=5,
                                  manual_lambda=my_lambda, manual_omega=my_omega,
                                  manual_items=my_item_diffs,
                                  manual_thresholds=my_thresholds, seed=42)
```

16.2 Customising an MFRM_Sim_PseudoHalo simulation

16.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_mfrm_sim from Item_1 to my_new_item_name:

```
my_mfrm_sim.rename_item('Item_1', 'my_new_item_name')
```

16.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_mfrm_sim with item names in a list stored as a variable my_new_item_names:

```
my_mfrm_sim.rename_items_all(my_new_item_names)
```

16.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the second multiindex level of `self.responses` with new name. Names are also replaced in the generating parameters (e.g. `self.items`, `self.persons`) and the response probabilities.

Example
To rename a person in object `my_mfrm_sim` from `Person_1` to `my_new_person_name`:

```
my_mfrm_sim.rename_person('Person_1', 'my_new_person_name')
```

16.2.4 rename_persons_all

Description
Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the second multiindex level of `self.responses` with new names. Names are also replaced in the generating parameters (e.g. `self.items`, `self.persons`) and the response probabilities.

Example
To rename all persons in object `my_mfrm_sim` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm_sim.rename_persons_all(my_new_person_names)
```

16.2.5 `rename_rater`

Description
Method to rename a single facet element (e.g. rater).

Usage
```
self.rename_rater(old, new)
```

Arguments

`old`	String: the old name for the facet element
`new`	String: the new name for the facet element

Returns
Replaces specified facet element name in the first multiindex level of `self.responses` with new name. Names are also replaced in the generating parameters (e.g. `self.facet_effects`) and the response probabilities.

Example
To rename a facet element in object `my_mfrm_sim` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm_sim.rename_rater('Rater_1', 'my_new_rater_name')
```

16.2.6 `rename_raters_all`

Description
Method to rename all facet elements (e.g. raters).

Usage
```
self.rename_raters_all(new_names)
```

Arguments

`new_names`	List of new facet element names as strings

Returns
Replaces all facet element names in the first multiindex level of `self.responses` with new names. Names

are also replaced in the generating parameters (e.g. self.facet_effects) and the response probabilities.

Example
To rename all facet elements in object my_mfrm_sim with facet element names in a list stored as a variable my_new_rater_names:

```
my_mfrm_sim.rename_raters_all(my_new_rater_names)
```

16.2.7 drop_facet_elements

Description
Method to remove facet elements (e.g. raters) from the simulation: their responses and their generating parameters (self.facet_effects, plus self.item_effects/self.threshold_effects for bivector and the lambda_/omega parameters for the stretch models). The other facet elements' responses, and all item, threshold and person parameters, are unchanged. Aliased as self.drop_raters and, if a custom facet was passed, as self.drop_{facet_plural} (e.g. self.drop_judges).

Usage
```
self.drop_facet_elements(which, min_persons=3)
```

Arguments

which	String or List: 'thin' removes facet elements that rated fewer than min_persons persons (including those with no responses); 'extreme' removes those that used only the lowest or only the highest score category; a list removes the named facet elements (a ValueError is raised if any name is not in the data).
min_persons	Integer: threshold for which='thin'. Default is min_persons=3.

Returns
Removes the specified facet elements from self.responses, self.facet_names and the generating parameters, with a UserWarning listing those removed, and adds them to self.dropped_facet_elements.

Example
To remove Rater_3 from object my_mfrm_sim:

```
my_mfrm_sim.drop_raters(['Rater_3'])
```

17 class MFRM_Sim_Bistretch

17.1 Generating an MFRM_Sim_Bistretch simulation

17.1.1 MFRM_Sim_Bistretch

Description

Creates an object of the class MFRM_Sim_Bistretch. Simulates response data fitting the rating scale model formulation of the many-facet Rasch model (MFRM) under the bistretch facet element (e.g. rater) representation from generating parameters, which may be explicitly specified or randomly generated according to specified generating parameters. Unlike MFRM_Sim_Bivector, each facet element's (item × threshold) location profile is generated from just three true parameters: a location shift λ_r, an item-difficulty-stretch ω_{rI} and a threshold-stretch ω_{rT} – combining both stretch axes at once.

Usage

```
MFRM_Sim_Bistretch(no_of_items, no_of_persons, max_score, no_of_facet_elements,
                   global_range=2, stretch_range=1, manual_lambda=None, manual_omega_items=None,
                   manual_omega_thresholds=None, item_range=2, category_base=2,
                   person_sd=1.5, max_disorder=0, offset=0, missing=0, shared_missing=True,
                   manual_persons=None, manual_items=None, manual_thresholds=None,
                   manual_person_names=None, manual_item_names=None, manual_facet_names=None,
                   facet=None, facet_plural=None, seed=None,
                   missing_method='bernoulli')
```

Arguments

no_of_items	Integer: The number of items in the simulation.
no_of_persons	Integer: The number of persons in the simulation.
no_of_facet_elements	Integer or None: The number of facet elements in the simulation.
max_score	Integer: The maximum score on each item.
global_range	Float: The total spread (maximum − minimum) of true λ_r (location shift) values across facet elements. Default is global_range=2. Ignored if manual_lambda is passed.
stretch_range	Float: The total spread of true $\omega_{\text{items},r}$ and $\omega_{\text{thresholds},r}$ values around 1 (use manual_omega_items/manual_omega_thresholds if you want them to differ). Default is stretch_range=1.
manual_lambda	List or 1-dimensional numpy array: Optional list of specified true λ_r values, length no_of_raters. If None (default), drawn the same way MFRM_Sim's own facet_range scales locations.

Continued on next page...

manual_omega_items	List or 1-dimensional numpy array: Optional list of specified true $\omega_{\text{items},r}$ (item-stretch) values, length no_of_raters. If None (default), drawn as 1+ a stretch_range-scaled, zero-mean deviation.
manual_omega_thresholds	List or 1-dimensional numpy array: Optional list of specified true $\omega_{\text{thresholds},r}$ (threshold-stretch) values, length no_of_raters. If None (default), drawn as 1+ a stretch_range-scaled, zero-mean deviation, independently of $\omega_{\text{items},r}$.
item_range	Float: The range of item locations for the internally-generated preliminary MFRM_Sim_Bivector simulation. Default is item_range=2. Ignored if manual_items is passed.
item_facet_range	Float: Spread of the preliminary MFRM_Sim_Bivector simulation's own free per-(facet element, item) effects, which are then overwritten by the true stretch structure. Default is item_facet_range=2.
threshold_facet_range	Float: Spread of the preliminary simulation's own free per-(facet element, threshold) effects, likewise overwritten. Default is threshold_facet_range=1.
category_base	Float: The base width of response categories for randomly generated Rasch-Andrich thresholds in the preliminary simulation. Default is category_base=2.
person_sd	Float: The standard deviation of person locations for the preliminary simulation. Default is person_sd=1.5. Ignored if manual_persons is passed.
max_disorder	Float: If negative, the preliminary simulation's Rasch-Andrich thresholds may be disordered (Andrich, 2010; Pallant & Tennant, 2007). Default is max_disorder=0.
offset	Float: The difference between the means of person locations and item locations in the preliminary simulation. Default is offset=0.
missing	Float between 0 and 1: The proportion of missing data (MCAR). Default is missing=0.
shared_missing	Boolean: If True, the same persons are missing across all facet elements. If False, missingness is independent across facet elements. Default is shared_missing=True.
manual_persons	List or 1-dimensional numpy array: Optional list of specified person locations. Default is manual_persons=None.
manual_items	List or 1-dimensional numpy array: Optional list of specified central item locations. Default is manual_items=None.
manual_thresholds	List or 1-dimensional numpy array: Optional list of specified Rasch-Andrich thresholds. Default is manual_thresholds=None.

Continued on next page...

manual_person_names	List: Optional list of person names. Default is manual_person_names=None.
manual_item_names	List: Optional list of item names. Default is manual_item_names=None.
manual_facet_names	List: Optional list of facet element names. Default is manual_facet_names=None.
facet	String: singular name for the second (non-item, non-person) facet of the simulated data. If None, generated names and attribute aliases use 'rater' (Rater_1 etc.) and messages refer to 'facet element(s)', with '(e.g. rater)' at first mention. Default is facet=None.
facet_plural	String or None: plural form of facet. If None (default), facet + 's' is used.
seed	Integer or None: seed for the true $\lambda_r/\omega_{\text{items},r}/\omega_{\text{thresholds},r}$ draws and every other random draw in the simulation. Pass an integer for full reproducibility; None (default) draws fresh entropy each run. See Section 1.
missing_method	String: how missing is applied. 'bernoulli' makes each response missing independently with probability missing; 'exact' makes exactly round(missing * cells) responses missing, per facet element when shared_missing=False, or in the one shared pattern when shared_missing=True. Default is missing_method='bernoulli'.

Format for manual_lambda/manual_omega_items/manual_omega_thresholds **input**
Simple lists or 1-dimensional numpy arrays, one true value per facet element, in facet element order. Toy example with two facet elements:

```
manual_lambda = [-0.5,  0.5]
manual_omega_items = [ 1.5,  0.6]
manual_omega_thresholds = [ 1.5,  0.6]
```

Returns
Object of class MFRM_Sim_Bistretch. Several attributes are automatically generated on its creation:

self.responses	pandas DataFrame: The multiindex dataframe of simulated responses, which may be saved to file or used to create an MFRM object for analysis.
self.no_of_items	Integer: The number of items.
self.no_of_persons	Integer: The number of persons.
self.no_of_raters	Integer: The number of facet elements.
self.max_score	Integer: The shared maximum possible score on each item.
self.persons	pandas series: Person names as keys and person locations as values.

Continued on next page...

self.items	pandas series: Item names as keys and item locations as values.
self.thresholds	pandas series: set of self.max_score Rasch-Andrich thresholds $\tau_1, \ldots, \tau_m$, indexed 1 to self.max_score.
self.lambda_	pandas series: True generating λ_r (location shift) per facet element.
self.omega_items	pandas series: True generating $\omega_{\text{items},r}$ (item-stretch) per facet element.
self.omega_thresholds	pandas series: True generating $\omega_{\text{thresholds},r}$ (threshold-stretch) per facet element.
self.item_effects	pandas DataFrame: the implied per-(facet element, item) effect table, facet element names as the row index, item names as columns – same format as self.item_effects on MFRM_Sim_Bivector.
self.threshold_effects	pandas DataFrame: the implied per-(facet element, threshold) effect table, facet element names as the row index, threshold positions as columns, zero-summed per facet element – same format as self.threshold_effects on MFRM_Sim_Bivector.
self.facet_effects	pandas DataFrame with a MultiIndex (facet element, item) as the row index and threshold positions as columns: the reconstructed full facet effect matrix (item effect plus threshold effect per cell), same format as self.facet_effects on MFRM_Sim_Bivector.
self.model	String: Always 'bistretch'.
self.person_names	List: Person names.
self.item_names	List: Item names.
self.rater_names	List: Facet element names.
self.dropped_facet_elements	List: facet elements removed with drop_facet_elements (empty on creation). Aliased as self.dropped_raters and, if a custom facet was passed, as self.dropped_{facet_plural}.

Examples

To create an MFRM_Sim_Bistretch object called my_mfrm_sim with a maximum score of 5, randomly generated parameters for 200 persons, 12 items and 8 facet elements, a true λ_r spread of 2 logits and a true $\omega_{\text{items},r}/\omega_{\text{thresholds},r}$ spread of 1.2 around neutral:

```
my_mfrm_sim = MFRM_Sim_Bistretch(no_of_items=12, no_of_persons=200,
                                 no_of_raters=8, max_score=5, global_range=2,
                                 stretch_range=1.2, seed=42)
```

To create an MFRM_Sim_Bistretch object with specified true $\lambda_r/\omega_{\text{items},r}/\omega_{\text{thresholds},r}$ values for 5 facet elements, saved as variables named my_lambda, my_omega_items and my_omega_thresholds, together with specified item locations and thresholds:

```
my_mfrm_sim = MFRM_Sim_Bistretch(no_of_items=4, no_of_persons=100,
                                 no_of_raters=5, max_score=5,
                                 manual_lambda=my_lambda,
                                 manual_omega_items=my_omega_items,
                                 manual_omega_thresholds=my_omega_thresholds,
                                 manual_items=my_item_diffs,
                                 manual_thresholds=my_thresholds, seed=42)
```

17.2 Customising an MFRM_Sim_Bistretch simulation

17.2.1 rename_item

Description
Method to rename a single item.

Usage
```
self.rename_item(old, new)
```

Arguments

old	String: the old name for the item
new	String: the new name for the item

Returns
Replaces specified item name in the relevant column of self.responses with new name. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename an item in object my_mfrm_sim from Item_1 to my_new_item_name:

```
my_mfrm_sim.rename_item('Item_1', 'my_new_item_name')
```

17.2.2 rename_items_all

Description
Method to rename all items.

Usage
```
self.rename_items_all(new_names)
```

Arguments

new_names	List of new item names as strings

Returns
Replaces all item names in the columns of self.responses with new names. Names are also replaced in the generating parameters (e.g. self.items, self.persons) and the response probabilities.

Example
To rename all items in object my_mfrm_sim with item names in a list stored as a variable my_new_item_names:

```
my_mfrm_sim.rename_items_all(my_new_item_names)
```

17.2.3 rename_person

Description
Method to rename a single person.

Usage
```
self.rename_person(old, new)
```

Arguments

old	String: the old name for the person
new	String: the new name for the person

Returns
Replaces specified person name in the second multiindex level of `self.responses` with new name. Names are also replaced in the generating parameters (e.g. `self.items`, `self.persons`) and the response probabilities.

Example
To rename a person in object `my_mfrm_sim` from `Person_1` to `my_new_person_name`:

```
my_mfrm_sim.rename_person('Person_1', 'my_new_person_name')
```

17.2.4 rename_persons_all

Description
Method to rename all persons.

Usage
```
self.rename_persons_all(new_names)
```

Arguments

new_names	List of new person names as strings

Returns
Replaces all person names in the second multiindex level of `self.responses` with new names. Names are also replaced in the generating parameters (e.g. `self.items`, `self.persons`) and the response probabilities.

Example
To rename all persons in object `my_mfrm_sim` with person names in a list stored as a variable `my_new_person_names`:

```
my_mfrm_sim.rename_persons_all(my_new_person_names)
```

17.2.5 `rename_rater`

Description
Method to rename a single facet element (e.g. rater).

Usage
```
self.rename_rater(old, new)
```

Arguments

`old`	String: the old name for the facet element
`new`	String: the new name for the facet element

Returns
Replaces specified facet element name in the first multiindex level of `self.responses` with new name. Names are also replaced in the generating parameters (e.g. `self.facet_effects`) and the response probabilities.

Example
To rename a facet element in object `my_mfrm_sim` from `Rater_1` to `my_new_rater_name`:

```
my_mfrm_sim.rename_rater('Rater_1', 'my_new_rater_name')
```

17.2.6 `rename_raters_all`

Description
Method to rename all facet elements (e.g. raters).

Usage
```
self.rename_raters_all(new_names)
```

Arguments

`new_names`	List of new facet element names as strings

Returns
Replaces all facet element names in the first multiindex level of `self.responses` with new names. Names

are also replaced in the generating parameters (e.g. `self.facet_effects`) and the response probabilities.

Example

To rename all facet elements in object `my_mfrm_sim` with facet element names in a list stored as a variable `my_new_rater_names`:

```
my_mfrm_sim.rename_raters_all(my_new_rater_names)
```

17.2.7 drop_facet_elements

Description

Method to remove facet elements (e.g. raters) from the simulation: their responses and their generating parameters (self.facet_effects, plus self.item_effects/self.threshold_effects for bivector and the lambda_/omega parameters for the stretch models). The other facet elements' responses, and all item, threshold and person parameters, are unchanged. Aliased as self.drop_raters and, if a custom facet was passed, as self.drop_{facet_plural} (e.g. self.drop_judges).

Usage

```
self.drop_facet_elements(which, min_persons=3)
```

Arguments

which	String or List: 'thin' removes facet elements that rated fewer than min_persons persons (including those with no responses); 'extreme' removes those that used only the lowest or only the highest score category; a list removes the named facet elements (a ValueError is raised if any name is not in the data).
min_persons	Integer: threshold for which='thin'. Default is min_persons=3.

Returns

Removes the specified facet elements from self.responses, self.facet_names and the generating parameters, with a UserWarning listing those removed, and adds them to self.dropped_facet_elements.

Example

To remove Rater_3 from object my_mfrm_sim:

```
my_mfrm_sim.drop_raters(['Rater_3'])
```

References

Andrich, D. (1978). A rating formulation for ordered response categories. *Psychometrika*, *43*(4), 561–573.

Andrich, D. (2010). Understanding the Response Structure and Process in the Polytomous Rasch Model. In *Handbook of polytomous item response theory models* (123–152).

Andrich, D., & Kreiner, S. (2010). Quantifying response dependence between two dichotomous items using the Rasch model. *Applied Psychological Measurement*, *34*(3), 181–192.

Benjamini, Y., & Hochberg, Y. (1995). Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society. Series B (Methodological)*, *57*(1), 289–300.

Bradley, R. A., & Terry, M. E. (1952). Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons. *Biometrika*, *39*(3/4), 324.

Choppin, B. (1968). Item bank using sample-free calibration. *Nature*, *219*(5156), 870–872.

Choppin, B. (1985). A fully conditional estimation procedure for Rasch model parameters. *Evaluation in Education*, *9*(1), 29–42.

Cochran, W. G. (1954). The Combination of Estimates from Different Experiments. *Biometrics*, *10*(1), 101–129.

Cronbach, L. J. (1951). Coefficient alpha and the internal structure of tests. *Psychometrika*, *16*(3), 297–334.

Elliott, M. (2025). *Extended many-facet Rasch models: Accounting for rater effects in automated essay scoring systems* [Doctoral dissertation, University of Cambridge].

Elliott, M. (2026). RaschPy: Python Rasch Analysis Package.

Elliott, M., & Buttery, P. J. (2022a). Extended rater representations in the many-facet Rasch model. *Journal of Applied Measurement*, *22*(1), 133–160.

Elliott, M., & Buttery, P. J. (2022b). Non-iterative conditional pairwise estimation for the rating scale model. *Educational and Psychological Measurement*, *82*(5), 989–1019.

Elliott, M., & Buttery, P. J. (2025). Addressing non-uniform rater effects with extended many-facet Rasch models: A case study. *Paper presented at the Nordic Educational Research Association Conference 2025, Helsinki, Finland, March 5–7*.

Garner, M., & Engelhard, G. (2002). An eigenvector method for estimating item parameters of the dichotomous and polytomous Rasch models. *Journal of Applied Measurement*, *3*(2), 107–128.

Garner, M., & Engelhard, G. (2009). Using paired comparison matrices to estimate parameters of the partial credit Rasch measurement model for rater-mediated assessments. *Journal of Applied Measurement*, *10*(1), 30–41.

Holland, P. W., & Wainer, H. (Eds.). (1993). *Differential Item Functioning*. Lawrence Erlbaum.

Huber, P. J. (1967). The Behavior of Maximum Likelihood Estimates Under Nonstandard Conditions. *Proceedings of the fifth Berkeley symposium on mathematical statistics and probability*, 221–233.

Iglewicz, B., & Hoaglin, D. C. (1993). *The ASQC Basic References in Quality Control: Statistical Techniques: How to Detect and Handle Outliers* (E. F. Mykytka, Ed.; Vol. 16). ASQC Quality Press.

Jin, K.-Y., & Wang, W.-C. (2018). A New Facets Model for Rater's Centrality/Extremity Response Style. *Journal of Educational Measurement*, *55*(4), 543–563.

Kauermann, G., & Carroll, R. J. (2001). A Note on the Efficiency of Sandwich Covariance Matrix Estimation. *Journal of the American Statistical Association*, *96*(456), 1387–1396.

Kornbrot, D. (2014). Point Biserial Correlation. In N. Balakrishnan, T. Colton, B. Everitt, W. Piegorsch, F. Ruggeri, & J. Teugels (Eds.), *Wiley statsref: Statistics reference online*. Wiley.

Kou, G., & Lin, C. (2014). A cosine maximization method for the priority vector derivation in AHP. *European Journal of Operational Research*, *235*(1), 225–232.

Linacre, J. M. (1994). *Many-Facet Rasch Measurement*. MESA Press.

Linacre, J. M. (2023). *Winsteps® Rasch measurement computer program User's Guide. Version 5.6.0*. Winsteps.com.

Marais, I. (2012). Local independence. In K. B. Christensen, S. Kreiner, & M. Mesbah (Eds.), *Rasch models in health* (111–130). John Wiley & Sons.

Masters, G. N. (1982). A Rasch model for partial credit scoring. *Psychometrika*, *47*(2), 149–174.

Pallant, J. F., & Tennant, A. (2007). An introduction to the Rasch measurement model: An example using the Hospital Anxiety and Depression Scale (HADS). *British Journal of Clinical Psychology*, *46*(1), 1–18.

Rasch, G. (1960). *Probabilistic models for some intelligence and attainment tests*. Danmarks Pædagogiske Institut.

Rasch, G. (1968). A Mathematical Theory of Objectivity and its Consequences for Model Construction. *Paper presented at the European Meeting on Statistics, Econometrics and Management Science, Amsterdam, September 2–7, 1968*.

Smith, E. V. (2002). Detecting and Evaluating the Impact of Multidimensionality Using Item Fit Statistics and Principal Component Analysis of Residuals. *Journal of Applied Measurement*, *3*(2), 205–231.

Tristan, A. (2006). An adjustment for sample size in DIF analysis. *Rasch Measurement Transactions*, *20*(3), 1070–1071.

Warm, T. A. (1989). Weighted likelihood estimation of ability in item response theory. *Psychometrika*, *54*(3), 427–450.

Waskom, M. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, *6*(60), 3021.

Wright, B. D. (1996). Comparing Rasch measurement and factor analysis. *Structural Equation Modeling*, *3*(1), 3–24.

Wright, B. D., & Masters, G. N. (1982). *Rating Scale Analysis*. MESA Press.

Zermelo, E. (1929). Die Bereehnung der Turnier-Ergebnisse als ein Maximumproblem der Wahrseheinliehkeit-sreclmung. *Mathematische Zeitschrift*, *29*(1), 436–460.

Zwick, R., Thayer, D. T., & Mazzeo, J. (1997, June). *DESCRIBING AND CATEGORIZING DIF IN POLYTOMOUS ITEMS* (tech. rep. No. 1). Educational Testing Service. John Wiley; Sons Inc.